Babet Manual

Contents

Babet — User manual **13**
- Getting started . . . 14
- Find a feature . . . 14
- Module reference . . . 16
- Structure of a module page . . . 18
- PDF generation . . . 18

Getting started **18**
- Download . . . 18
- Build from source . . . 19
- Three ways to run a Lua script . . . 19
  1. Folder mode . . . 19
  2. Embedded mode (single-file executable) . . . 19
  3. Embedded via PATH (folder + auto-detect) . . . 20
- Command-line invocation . . . 20
- First script . . . 21
- Where to look next . . . 21

Launch context and resources **21**
- Embedded resources . . . 22
- Strict globals . . . 22
- Multiple returns and arity . . . 22

Program exit **22**

Security model **23**
- What the hardening *does* protect against . . . 23
- What it does *not* protect against . . . 24
- Recommended pattern : least privilege . . . 24

Archive — secure ZIP and TAR operations with gzip, xz, bzip2, or zstd **25**
- Scope . . . 25
- Contents . . . 25
- `babet.archive.create` . . . 26
  - Output format . . . 26
  - Include and exclude filters . . . 27
- Inspection/extraction options and anti-bomb limits . . . 30
  - One example per limit . . . 31
- `babet.archive.list` . . . 32
  - Returned structure . . . 33
  - Fields shared by every entry . . . 34
  - ZIP-specific fields . . . 34
  - Binary names, unsafe paths, and normalisation . . . 35
  - Duplicates and collisions . . . 35
  - Usage examples . . . 35

What `list()` validates — and what it does not guarantee . . . 37
`babet.archive.read` . . . 37
Selection by raw name or index . . . 37
Unsafe names and rejected types . . . 38
The `max_size` memory limit . . . 39
Integrity and archive consumption . . . 39
Worker use . . . 40
`babet.archive.test` . . . 40
Technical verification . . . 41
Safety verification . . . 41
Returned structure . . . 41
Usage examples . . . 42
`babet.archive.extract` . . . 43
Safe-glob selection . . . 44
Read-only simulation with `dry_run` . . . 45
Validation, skipped entries, and integrity . . . 45
Return value . . . 46
Examples . . . 46
`babet.archive.extractFile` . . . 49
Path rules . . . 50
Destination protection . . . 50
Atomic publication and cleanup . . . 50
Permissions . . . 51
Supported entry types . . . 51
Errors and known limitations . . . 52

`babet.base64` — binary Base64 encoding and decoding **52**
Module contents . . . 53
API . . . 53
Standard encoding . . . 53
URL-safe alphabet . . . 54
Padding . . . 54
Producing output without = . . . 54
Accepting unpadded input . . . 54
Combined URL-safe, unpadded example . . . 55
Strict decoding . . . 55
ASCII whitespace . . . 56
Output limit . . . 56
Binary data and files . . . 56
Using the module in a worker . . . 57
Error contract . . . 57
Call errors: Lua exception . . . 57
Data errors: `nil, err` . . . 58
Limits and design choices . . . 58

Compression - standalone gzip, xz, bzip2, and zstd streams **58**
Scope . . . 58
API overview . . . 59
`compress(source, destination, format [, opts])` . . . 59
Example . . . 60
`decompress(source, destination [, opts])` . . . 60
Example . . . 60
Concatenated streams and integrity . . . 61
Filesystem and publication guarantees . . . 61
Binary data and memory use . . . 61
C++ exception boundary . . . 61
Workers . . . 62

Error examples . . . 62

CURSES — terminal user interfaces **62**
API overview . . . 62
Starting and stopping . . . 62
Screen operations . . . 63
Keyboard input . . . 63
Interactive child processes . . . 63
Workers . . . 64
Signals . . . 64
TERM and terminfo . . . 64
Deployment . . . 65

`argparse` — command-line arguments **65**
Module contents . . . 65
Loading and metadata . . . 65
API . . . 66
Declaring flags and options . . . 66
Declaring positionals . . . 66
The `opts` table . . . 67
Common fields . . . 67
Value-only fields . . . 67
Destination . . . 67
Argument source . . . 67
Global `arg` table . . . 67
Explicit array . . . 68
Recognized forms . . . 68
Defaults, choices, and conversion . . . 68
`parse` results . . . 69
Normal success . . . 69
Built-in help . . . 69
User-input error . . . 69
Programming error . . . 70
Complete example . . . 70
Text produced by `get_usage` . . . 70
Current limitations . . . 71

EXEC - external programs, capture, and streaming **71**
Module contents . . . 71
Essential conventions . . . 72
No implicit shell . . . 72
A non-zero exit is not a launch error . . . 72
Data strings are binary-safe . . . 72
API overview . . . 73
Signature and arguments . . . 73
`cmd` and program lookup . . . 73
`args` and shell-free execution . . . 74
Using a shell explicitly . . . 74
Options . . . 75
`opts.cwd` . . . 75
`opts.env` . . . 75
`opts.stdin` . . . 76
`opts.timeout` . . . 76
`opts.max_output` . . . 76
Result table . . . 77
Normal exit code . . . 77
Signal termination . . . 77

Timeout . . . 78
Truncation . . . 78
Complete examples . . . 78
Basic capture . . . 78
Arguments containing spaces . . . 78
Handling a non-zero code . . . 78
Capturing `stderr` . . . 79
Sending text or binary stdin . . . 79
Changing the environment . . . 79
Changing the working directory . . . 79
Bounding execution time . . . 80
Bounding captured output . . . 80
Building an application helper . . . 80
Controllable processes with `babet.spawn` . . . 81
Signature and options . . . 81
Redirection modes . . . 81
Process methods . . . 84
Reading stdout and stderr . . . 85
Writing stdin . . . 86
Waiting, terminating, and closing . . . 86
Combined example: launching a WebDriver or daemon . . . 86
Combined example: streaming loop . . . 86
Limits and deadlock prevention . . . 87
Error contract . . . 87
Raised Lua errors . . . 87
Returned `(nil, err)` errors . . . 88
Situations returning a table . . . 88
Processes, signals, and security . . . 88
Process group . . . 88
Child signal mask . . . 88
Interaction with `babet.signal` . . . 89
Memory . . . 89
Design and limitations . . . 89
Async completion — Babet 2.35.0 . . . 89

Process pipelines **89**
Shared command syntax . . . 89

`babet.pipeline()` — complete execution **90**
Signature . . . 90
Options . . . 90
Result . . . 91
Timeout, large streams and truncation . . . 91

`babet.spawnPipeline()` — streaming **92**
Signature . . . 92
Read methods . . . 93
Writing and closing stdin . . . 93
State and identifiers . . . 93
Waiting and final status . . . 94
Explicit termination . . . 94
`close()`, GC and Lua `<close>` . . . 94
Launch errors and reuse . . . 95

FS — files, directories, paths, searching, copying, permissions, and checksums **95**
Module contents . . . 95
General conventions . . . 96

Paths . . . 96
Symbolic links . . . 96
Result order . . . 97
Return values . . . 97
API overview . . . 97
Check and inspect . . . 97
Manipulate paths . . . 97
Act on the filesystem . . . 97
List and search . . . 98
Copy and move . . . 98
Change attributes . . . 98
Existence, type, and size . . . 98
`babet.fileExists(path)` . . . 98
`babet.isFile(path)` and `babet.isfile(path)` . . . 99
`babet.isDir(path)` and `babet.isdir(path)` . . . 99
`babet.fileSize(path)` . . . 99
Current working directory . . . 99
`babet.currentDir()` . . . 99
`babet.chdir(path)` . . . 99
Lexical path manipulation . . . 100
`babet.getBasename(path)` . . . 100
`babet.getFilename(path)` . . . 100
`babet.getExtension(path)` . . . 100
`babet.getPath(path)` . . . 100
`babet.joinPath(...)` . . . 101
Create, remove, rename, and link . . . 101
`babet.touch(path)` . . . 101
`babet.mkdir(path)` . . . 101
`babet.remove(path)` . . . 102
`babet.rmdir(path)` . . . 102
`babet.rmdirAll(path)` . . . 102
`babet.rename(source, destination)` . . . 103
`babet.link(target, linkpath)` . . . 103
List, iterate, and search . . . 103
`babet.listFiles(path, recursive?)` . . . 103
`babet.createFileIterator(path, recursive?)` . . . 104
`babet.find(path, opts?)` . . . 105
Copy and move . . . 108
`babet.copy(source, destination)` . . . 108
`babet.copyTree(source, destination, continue_on_error?)` . . . 108
`babet.moveTree(source, destination)` . . . 109
Unix permissions and attributes . . . 110
`babet.getMode(path)` and `babet.setMode(path, mode)` . . . 110
`babet.getAttributes(path)` . . . 111
`babet.setAttributes(path, uid, gid, mode?)` . . . 111
`babet.symlinkAttr(path, uid, gid)` and `babet.symlinkattr(...)` . . . 112
File checksums . . . 112
In-memory CRC-32 . . . 113
`babet.crc32(data)` . . . 113
Error contract . . . 113
Raised Lua errors . . . 113
Returned errors . . . 113
Partial operations . . . 113
Design decisions and limitations . . . 114
exists — Babet 2.35.0 . . . 114

GUI — optional GTK 4 desktop interface **114**

Availability and initialization . . . 114
Minimal widget API . . . 115
Label: wrapped display text . . . 116
Entry: single-line text input . . . 116
DrawingArea: native curves and click selection . . . 118
Refreshable containers: Box and ScrolledWindow . . . 119
SpinButton: numeric input . . . 119
Calendar: selecting and editing dates . . . 120
CSS styling . . . 121
Common widget properties . . . 121
Event loop and callbacks . . . 122
Lifetime rules . . . 122
`--create-exe` . . . 122
GTK4 additions — freely positioned notes (2.29.0) . . . 122
Native clickable progress bars and in-place list reordering . . . 123
Native GTK4 drag-and-drop for list rows . . . 123
Desktop-application primitives . . . 125
Split panes, menus and context popovers . . . 125
Generic pointer, keyboard, focus and geometry . . . 126
Desktop windows . . . 126
Pictures and file dialogs . . . 126
File pictures . . . 127
Asynchronous file dialogs . . . 127
File drops and TextView tags . . . 128
Files from the file manager — GTK 4.6 . . . 128
TextView tags — GTK 4.0 . . . 129
GUI loop and directory selection . . . 130
Timers . . . 130
Process completion . . . 131
Errors . . . 131
Size and closure . . . 131
Directory selection . . . 132

`writeFileAtomic` — atomic and durable file writing 132
Contents . . . 132
API . . . 132
Defaults . . . 133
Basic write . . . 133
Explicit replacement . . . 133
Permissions . . . 133
Durability . . . 134
Binary data . . . 134
Paths and symbolic links . . . 134
Atomicity and concurrency . . . 135
Using it in a worker . . . 135
Combined Selenium example . . . 135
Error contract . . . 136
Limitations . . . 136

lockFile — exclusive file locks 136
API . . . 136
One writer per directory . . . 137
Processes, workers and errors . . . 137
Limits . . . 138

HTTP — synchronous web requests, headers, bodies, TLS, and limits 138
Module contents . . . 138

Core conventions . . . 139
Synchronous client . . . 139
HTTP status is not a transport error . . . 139
Binary bodies . . . 139
Unknown option fields . . . 139
API overview . . . 140
`babet.http.request(opts)` . . . 140
`babet.http.get(url, opts?)` . . . 141
`babet.http.download(url, destination, opts?)` . . . 141
`babet.http.post` call forms . . . 142
URLs, fragments, and query . . . 142
HTTP methods . . . 143
Request headers . . . 143
Request body and `Content-Type` . . . 143
`opts.timeout` . . . 144
HTTPS, verification, and CA . . . 144
`follow_redirects` . . . 145
`max_body_size` . . . 145
`max_file_size` and file safety . . . 146
Response tables . . . 146
In-memory response . . . 146
Download result . . . 146
Complete examples . . . 147
GET JSON . . . 147
Authenticated JSON POST . . . 147
Binary PUT . . . 147
Download a release artifact . . . 147
Separate network and HTTP failures . . . 148
Parallel requests with WORKERS . . . 148
Error contract . . . 148
Security and limitations . . . 149

`babet.inotify` — filesystem watching **149**
Module contents . . . 149
API . . . 149
Creating a watcher . . . 150
Adding a watch . . . 150
`onlydir` option . . . 150
Reading events . . . 150
Queue overflow . . . 151
Signal interruption . . . 151
Removing and closing . . . 151
Example . . . 152
Error contract . . . 152
Limits . . . 152

`babet.json` — JSON encoding and decoding **153**
Module contents . . . 153
API . . . 153
Type mapping . . . 153
Lua to JSON . . . 153
JSON to Lua . . . 154
`null` . . . 154
Empty object, empty array, and `as_array` . . . 154
`encode` options . . . 155
Strings, UTF-8, and NUL bytes . . . 155
Numbers . . . 155

Object order and duplicate keys . . . 155
Error contract . . . 156
Limits . . . 156

`logging` — leveled logging **156**
Module contents . . . 156
Emitting a message . . . 157
No-propagation guarantee . . . 157
Output format . . . 157
Threshold . . . 157
Destination . . . 158
ANSI colors . . . 158
Module state . . . 158
Error contract . . . 159
Limitations . . . 159

SIGNAL — graceful shutdown, reloads, and POSIX signals **159**
Module contents . . . 159
Essential conventions . . . 160
Success returns one value . . . 160
Names are strict . . . 160
A callback receives no arguments . . . 160
API overview . . . 161
Supported signals . . . 161
`handle(name, fn)` — install or replace . . . 161
`handle(name, nil)` — remove a callback . . . 162
`ignore(name)` — ignore . . . 162
`default(name)` — restore the OS default . . . 162
Callback execution . . . 162
Main thread and safe context . . . 163
Dispatch order . . . 163
Occurrence coalescing . . . 163
Callback errors . . . 163
Interaction with blocking calls . . . 164
Interaction with `debug.sethook` . . . 164
Complete examples . . . 165
Graceful shutdown on `TERM` and `INT` . . . 165
Reload configuration with `HUP` . . . 165
Interrupt a socket wait . . . 166
Temporarily ignore `PIPE` . . . 166
Observe callback errors . . . 166
Error contract . . . 167
Raised Lua errors . . . 167
Returned system error . . . 167
Workers, concurrency, and limitations . . . 167
Features not provided . . . 168

SOCKET — TCP and Unix sockets, binary streams, lines, and timeouts **168**
Module contents . . . 169
Core conventions . . . 169
Synchronous streams . . . 169
Binary-safe data . . . 169
Timeout rules . . . 169
Return values . . . 169
API overview . . . 170
`babet.socket.connect(host, port, timeout?)` . . . 170
`babet.socket.listen(host, port, backlog?)` . . . 171

`babet.socket.connect_unix(path, timeout?)` . . . 171
`babet.socket.listen_unix(path, opts?)` . . . 171
`server:accept(timeout?)` . . . 172
`sock:set_timeout(seconds)` . . . 172
`sock:send(data)` . . . 172
`sock:recv(count, timeout?)` . . . 173
`sock:recv_line(timeout?)` . . . 173
`sock:recv_all(timeout?, max_bytes?)` . . . 173
Mixing `recv`, `recv_line`, and `recv_all` . . . 174
`sock:peer()` and `sock:sockname()` . . . 174
`sock:close()` and garbage collection . . . 174
Signals and interruption . . . 174
Complete examples . . . 175
Line-oriented echo server . . . 175
Length-prefixed binary client . . . 175
Response framed by EOF . . . 175
Error contract . . . 175
Security and limitations . . . 176

WEBSOCKET — RFC 6455 client for `ws://` and `wss://` 176
API overview . . . 176
`babet.websocket.connect(url, opts?)` . . . 177
Options . . . 177
Option examples . . . 178
Opening handshake . . . 179
Sending text . . . 179
Sending binary data . . . 179
Receiving messages . . . 180
Ping and Pong . . . 180
Closing handshake . . . 180
Protocol failures . . . 180
WebDriver BiDi example . . . 181
Concurrency and event loops . . . 181
Security and limits . . . 181

`babet.sqlite` — embedded SQL database 181
Module contents . . . 181
API . . . 182
Opening and closing . . . 182
Connection counters . . . 183
Coherent backups . . . 184
Direct execution . . . 186
Reusable prepared statements . . . 187
Managed transactions . . . 188
Nested savepoints . . . 191
SQL parameters . . . 193
Explicit NULL values . . . 194
Explicit BLOB values . . . 195
SQL text and NUL bytes . . . 195
Row reading and type mapping . . . 195
Lifetime . . . 195
Error contract . . . 196
Complete examples . . . 196
Prepared inserts, optional NULL, and BLOB values . . . 196
Reusable query and managed transaction . . . 197
Functions not exposed . . . 197

`babet` strings — splitting strings **198**
Module contents . . . 198
API . . . 198
Separator mode . . . 198
Limiting the number of cuts . . . 199
Byte mode . . . 199
Binary strings and NUL bytes . . . 199
Empty subject . . . 200
Error contract . . . 200
Limits and design choices . . . 200

SYS - process, host, environment, executables, version, and Lua memory **200**
Module contents . . . 201
General conventions . . . 201
Flat functions . . . 201
Strict strings . . . 201
Process-global state . . . 202
API overview . . . 202
Version constants . . . 202
Identify the process and host . . . 203
`babet.pid()` . . . 203
`babet.hostname()` . . . 203
`babet.uname()` . . . 203
Locate an executable . . . 204
`PATH` lookup . . . 204
Direct path . . . 205
Missing or non-executable program . . . 205
Environment variables . . . 205
`babet.env(name)` . . . 205
`babet.setenv(name, value)` . . . 206
Empty value versus missing variable . . . 206
Interaction with workers . . . 206
Lua VM memory . . . 207
`babet.getMemoryUsage()` . . . 207
`babet.getDetailedMemoryUsage()` . . . 207
What is and is not measured . . . 207
Error contract . . . 208
Design decisions and limits . . . 208

`babet` tables — merging and copying Lua tables **209**
Module contents . . . 209
API . . . 209
`babet.mergeTables(t1, t2, ...)` . . . 209
Positive integer keys: concatenation . . . 209
Every other key: last writer wins . . . 210
Shallow merge . . . 210
`babet.deepCopyTable(t)` . . . 210
Values and shared graph structure . . . 210
Keys are not copied . . . 211
Metatables and raw traversal . . . 211
Maximum depth . . . 211
Error contract . . . 211
Limits and design choices . . . 211
Not in v1 . . . 212

`babet` time - clocks, sleep, dates and durations **212**
Module contents . . . 212

API . . . 212
`monotonic()` and `now()` . . . 213
`sleep(amount [, unit])` . . . 213
`babet.time.iso([ts])` . . . 213
`babet.time.parse_iso(text)` . . . 214
`babet.time.parse_duration(text)` . . . 214
`babet.time.format_duration(seconds)` . . . 215
Error contract summary . . . 215
Limits and design choices . . . 215

TLS — direct encryption, STARTTLS, certificates, and SNI **216**
Module contents . . . 216
Core conventions . . . 216
Verification is on by default . . . 216
Same stream API after handshake . . . 216
SNI is not certificate verification . . . 217
API overview . . . 217
TLS options . . . 217
`babet.socket.connect_tls(host, port, opts?)` . . . 218
`sock:starttls(opts?)` . . . 218
Verification, hostname, and SNI . . . 218
Certificate authorities . . . 219
TLS versions . . . 219
Timeouts . . . 219
Socket state after failure . . . 219
Complete examples . . . 220
Raw HTTPS request . . . 220
IP connection with DNS identity . . . 220
Simplified SMTP STARTTLS . . . 220
Controlled self-signed test . . . 220
Error contract . . . 220
Security and limitations . . . 221

`babet.toml` - TOML decoding **221**
Module contents . . . 221
API . . . 221
Type mapping . . . 222
Integers and floats . . . 222
Strings, UTF-8, and keys . . . 222
Arrays and tables . . . 223
Empty-container ambiguity . . . 223
Dates and times . . . 223
Ordering and discarded information . . . 224
Error contract . . . 224
Limits . . . 224
Not exposed in v1 . . . 224

USER - look up system accounts by name or UID **225**
Module contents . . . 225
General conventions . . . 225
Dedicated subtable . . . 225
Name or UID, without implicit conversion . . . 225
Fields are always present . . . 226
API overview . . . 226
Returned user table . . . 226
Look up an account with `get` . . . 226
By name . . . 227

By UID . . . 227
Missing account . . . 227
Test existence with `exists` . . . 228
By name . . . 228
By UID . . . 228
When to prefer `get` . . . 228
Argument validation . . . 229
NSS, workers, and security . . . 229
Account sources . . . 229
Calls from workers . . . 229
Security-sensitive decisions . . . 230
Error contract . . . 230
Design decisions and limits . . . 230

WORKERS — OS threads, isolated Lua states, queues, and channels 231
Module contents . . . 231
Essential conventions . . . 232
Each worker owns a separate Lua state . . . 232
Results use a `pcall`-like convention . . . 232
The final result can be consumed once . . . 233
API overview . . . 233
Parent side . . . 233
Worker side . . . 234
`spawn(code, args?, opts?)` . . . 234
`code` . . . 235
`args` and `worker.args` . . . 235
Queue capacities . . . 235
Passing channels through `opts.channels` . . . 236
When `setenv`, `chdir` and `os.setlocale` become locked . . . 236
Transferable values . . . 237
Scalars and strings . . . 237
List and object tables . . . 237
Rejected values . . . 238
Copies, identity, depth, and budgets . . . 238
Final worker result . . . 239
`status()` — observe without consuming . . . 239
`done()` — non-consuming boolean test . . . 240
`join(timeout?)` — wait and consume . . . 240
`poll()` — check and consume . . . 241
Do not call `join()` after completed `poll()` . . . 241
Parent-to-worker messages . . . 241
Worker-to-parent messages . . . 242
`worker.send`, `worker.recv`, and `worker.cancelled` . . . 242
Direct shared channels . . . 243
Creating a channel and choosing capacity . . . 243
`send` and `recv` . . . 243
`close` and `is_closed` . . . 244
Concurrency, ordering, and lifetime . . . 244
Timeouts and failure reasons . . . 244
`close()` and queue lifecycle . . . 245
`cancel()` and cooperative cancellation . . . 245
Module loading and worker environment . . . 246
Available CPU count . . . 247
Reusable bounded pool . . . 247
Creation and options . . . 247
Submission and tasks . . . 248
State isolation and reuse . . . 248

Backpressure and timeouts . . . 248
Close, cancellation, and `join` . . . 249
Shared channels in tasks . . . 249
Complete examples . . . 250
Simple calculation with `join` . . . 250
Structured arguments . . . 250
Non-blocking progress with `poll` . . . 250
Persistent request/response worker . . . 251
Sending and receiving `nil` . . . 251
Close, then drain the outbox . . . 252
Backpressure and timeout . . . 252
Parent to worker through a channel . . . 253
Worker to worker without parent relay . . . 253
Multiple producers and consumers . . . 253
Native cooperative cancellation . . . 254
Native pool for many tasks . . . 255
Deadlocks to avoid . . . 255
Worker waits for inbox while parent calls `join()` . . . 255
Worker blocks on a full outbox . . . 255
Producers blocked on a full channel . . . 256
Forgetting that `poll()` consumes . . . 256
Potentially blocking GC . . . 256
Error contract . . . 256
Lua errors raised for invalid use . . . 256
Failures returned by `spawn` . . . 256
Final worker errors . . . 257
Message serialization . . . 257
Garbage collection and destruction . . . 257
Signals, environment, and concurrency . . . 257
Signals . . . 257
`os.execute` and `io.popen` inside a worker . . . 258
Environment and cwd . . . 258
External resources . . . 258
Cost . . . 258
Features not provided . . . 258

Cookbook **259**
Watch a directory and email new files . . . 259
Ensure a service user exists, then drop to it . . . 259
Fetch many URLs in parallel . . . 260
Use WebSocket as a WebDriver BiDi transport . . . 260
Graceful shutdown of a long-running script . . . 261
Read a TOML config, validate, persist to SQLite . . . 261
Cache with a human-readable TTL, expiry as ISO timestamps . . . 262
Pretty-print a Lua value . . . 262
Hello there . . . 262

Babet — User manual

> *Babet*, n.m. — a regional word from south-eastern France (Lyonnais, Forez, Dauphiné, Savoie, and neighbouring Swiss Romandy) meaning a pine cone. Small, light, full of seeds, and able to start a fire—like this binary.

Babet is a standalone Lua binary for Linux scripting and automation. This manual is organised by need and by module: the tables below describe both the technical chapter name and the features it contains.

Documentation for **Babet 2.35.0**.

Babet 2.35.0 adds GUI timers, process-completion callbacks and directory selection, callback tracebacks with `gui.onError`, window resize notifications and `requestClose`. It also adds launch context, embedded resources and strict globals, plus `babet.exists`. GTK 4 remains optional; the directory chooser requires GTK 4.10. Release archives include a runtime ABI report; the minimum glibc version still depends on the build environment.

Babet 2.34.0 adds `babet.lockFile` for cooperating processes that need one writer per data directory. Locks are exclusive and nonblocking, support in-place metadata writes, and are cleaned up on close, Lua state shutdown and process exit. GTK is not required.

Babet 2.33.0 adds file pictures and native file dialogs, file drops from other applications and TextView tags, and wrapped labels for note display. Drops report the action actually negotiated with the source application, including nested targets. GTK 4 remains optional and loaded only when requested.

Babet 2.32.0 expands the GTK4 desktop API with split panes, menus and context popovers, themed icons, pointer and keyboard events, geometry helpers and dialog-window controls. Native drag-and-drop gains COPY/MOVE/LINK, custom previews and lifecycle callbacks. GTK 4 remains optional and loaded only when requested.

Babet 2.31.0 adds native GTK4 drag-and-drop with `setDragData` and `onDrop`, handle-widget drag icons and protected callbacks. The GUI chapter covers nested targets and cleanup after a drop.

Babet 2.22.2 adds a native RFC 6455 client through `babet.websocket`, with `ws://` and verified `wss://`, strict handshakes, bounded framing, automatic Ping/Pong, fragmentation, close semantics, and worker registration. The self-test harness now contains 45 reachable suites.

Getting started

- `Getting started` — installation, first script, running a file or directory, creating an embedded executable, and using Babet through `PATH`.
- `Program exit` — `os.exit` status and cleanup, finalizers, workers, and the difference from stock Lua.
- `Security` — threat model, actual protections, limitations, and least-privilege rules.
- `Cookbook` — complete recipes combining several modules.
- `Changelog` — complete release notes and the history of earlier releases, migration, validation, and known limitations.

Find a feature

I need to…	Chapter	What it contains
identify launch mode, read embedded resources or check globals	`Runtime — context and resources`	`runtimeInfo`, `embedded.read/list`, `strict` and multiple-return composition
reserve a directory for one writer process	`lockFile — exclusive file locks`	`flock`, in-place metadata, cleanup and process inheritance
create, remove, list, search, copy, or move files	`FS — filesystem`	files, directories, paths, symlinks, `listFiles`, `find` with `xdev` confinement, `copyTree`, permissions, and checksums
atomically publish configuration or binary data	`writeFileAtomic — atomic writing`	default no-overwrite, permissions, durability, path confinement, concurrency, and workers

I need to…	Chapter	What it contains
parse script arguments	Argparse — command line	flags, options, positional arguments, defaults, choices, and conversion
build a terminal UI	CURSES — terminal user interfaces	UTF-8 screen drawing, symbolic keyboard input, resize/suspend, interactive child handoff, workers, and self-contained terminfo fallbacks
build an optional desktop GUI	GUI — optional GTK 4 desktop interface	lazy system GTK 4 loading, window/box/label/button/entry widgets, DrawingArea, refreshable containers, numeric/date inputs, CSS/classes, native drag-and-drop, callbacks, lifetime rules, event loop, and the `--create-exe` deployment exception
run an external program	Exec — processes	shell-free arguments, complete capture with `exec`, streaming with `spawn`, environment, cwd, and process control
chain several commands	Process pipelines	complete capture or streaming, separate stderr streams, per-stage statuses, and process-group cleanup
create or inspect/extract ZIP or TAR archives	Archive — secure multi-format archives	deterministic ZIP and plain/gzip/xz/bzip2/zstd TAR creation, listing, full testing, selective extraction, `dry_run` preview, anti-bomb limits, and atomic file publication
encode or decode binary data as Base64	Base64 — binary data	standard or URL-safe alphabets, optional padding, strict canonical decoding, controlled whitespace, and an output limit
compress or decompress one file	Compression — standalone streams	gzip, xz, bzip2, and zstd streams, content detection, output limits, concatenated members, integrity checks, and atomic publication
inspect the process, environment, machine, or Lua VM memory	SYS — process and machine	runtime version, PID, hostname, `uname`, `PATH`, `env`, `setenv`, and Lua-state memory
send an HTTP request	HTTP — web client	GET/POST and other methods, query, validated headers, binary bodies, atomic downloads, redirects, TLS, timeout, and size limits
open a TCP connection or local Unix socket	Socket — TCP and Unix	TCP/Unix clients and servers, accept, local permissions, binary streams, lines, EOF, timeouts, and cleanup

I need to…	Chapter	What it contains
speak WebSocket or WebDriver BiDi transport	WebSocket — RFC 6455 client	ws://, verified wss://, masking, fragmentation, Ping/Pong, close handshake, timeouts, and strict memory limits
encrypt TCP or perform STARTTLS	TLS — secure sockets	direct TLS, STARTTLS, verification, CA, hostname, SNI, versions, timeout, and fail-closed state
store SQL data locally	SQLite — embedded database	read/write or read-only opening, foreign-key enforcement, change counters, strict parameters, coherent atomic backups, explicit NULL and BLOB values, prepared statements, and assisted transactions
encode or decode JSON	JSON — structured data	scalars, arrays, objects, null, empty arrays, pretty printing, and errors
read TOML configuration	TOML — configuration	decoding, TOML types, arrays, sections, dates, and parse errors
watch a directory	Inotify — file events	watches, events, timeout reads, moves, cookies, and closing
handle Unix signals	SIGNAL — clean shutdown and reload	TERM/INT/HUP/USR1/USR2/PIPE, deferred callbacks, fixed order, coalescing, interruptible calls, and workers
parallelise Lua work	WORKERS — OS threads, messages, and channels	isolated Lua states, spawn, a persistent bounded pool, cpu_count, inbox/outbox, and direct MPMC channels
obtain or format time	Time — clocks and durations	realtime and monotonic clocks, sleep, ISO-8601, parsing, and duration formatting
look up a system account by name or UID	USER — system accounts	NSS, get, exists, UID, primary GID, GECOS, home, shell, and resolution errors
split or transform strings	Strings	split, separators, limits, character mode, and binary strings
copy, merge, or mark tables	Tables	mergeTables, deepCopyTable, cycles, and shared structures
produce logs	Logging	levels, threshold, destination, colours, and sink-error behaviour

Module reference

Each module has a standalone page under modules/. The expanded names below define each chapter's scope before you open it.

Module	Detailed scope
Archive — secure ZIP, TAR, gzip, xz, bzip2, and zstd operations	Create deterministic ZIP, TAR, gzip TAR, xz TAR, bzip2 TAR, or zstd TAR archives, then inspect, test, and selectively extract them with `dry_run` previews, bounded resources, confined paths, and atomic per-file publication.
Base64 — binary encoding and decoding	Encode or decode binary Lua strings with standard or URL-safe alphabets, controlled padding, canonical validation, optional whitespace, and a `max_output` ceiling.
Compression — standalone gzip, xz, bzip2, and zstd streams	Compress or decompress one regular file with automatic content detection, bounded expansion, integrity verification, symlink refusal, and atomic output publication.
Argparse — command-line arguments	Declare flags, options, and positional values; generate help; validate choices; convert values.
Exec — external programs and processes	Run without a shell using `exec`, or progressively drive stdin/stdout/stderr using `spawn`.
Process pipelines	Connect commands without a shell, using complete capture or streaming and per-stage statuses.
Runtime — launch context and embedded resources	Launch modes and paths, binary ZIP resources, strict globals and multiple-return composition.
FS — files, directories, paths, and attributes	Existence and types, creation/removal, paths, listings, search, iterators, tree copy/move, symlinks, Unix modes, and checksums.
writeFileAtomic — atomic and durable file writing	Publish a binary Lua string through a private temporary file and atomic rename, with explicit overwrite, exact permissions, `fsync`, path confinement, and worker support.
HTTP — web requests	URLs, methods, query, validated headers, binary bodies, responses, atomic file downloads, redirects, TLS verification, timeout, and limits.
Inotify — filesystem monitoring	Add/remove watches, read events, handle timeouts, moves, cookies, and closing.
JSON — encoding and decoding	Lua/JSON types, `null`, empty arrays, array marking, indentation, UTF-8, cycles, and limits.
Logging	Levels, filtering, destination, colours, and variadic calls.
SIGNAL — POSIX signals and clean shutdown	Install, replace, or remove callbacks; ignore/restore; dispatch order, coalescing, interruptions, and multithread restrictions.
Socket — TCP and Unix sockets	TCP/Unix connect and listen, accept, permissions and inode-sensitive cleanup, binary streams, block/line/EOF reads, addresses, and timeouts.
WebSocket — RFC 6455 client	`ws://`/`wss://` client connections, strict Upgrade validation, client masking, text/binary fragmentation, Ping/Pong, Close, TLS, timeouts, and resource ceilings.
SQLite — embedded database	Read/write and read-only connections, WAL/timeout, foreign keys, row/change counters, direct SQL, reusable prepared statements, explicit BLOBs, coherent WAL backups, iteration, and assisted transactions.

Module	Detailed scope
Strings	Splitting, character mode, separators, split limits, and binary content.
SYS — process, machine, environment, and Lua memory	Version constants, PID, hostname, `uname`, executable lookup, environment reads/writes, worker interaction, and Lua-state memory.
Tables	Deterministic merging, list/map keys, deep copy, cycles, and shared subtables.
Time — clocks, ISO, and durations	Realtime and monotonic clocks, sleep, ISO-8601 formatting/parsing, and duration parsing/rendering.
TLS — encrypted connections	Direct TLS, STARTTLS, verification, CA, hostname, SNI, versions, deadlines, and fail-closed behaviour.
TOML — configuration files	TOML decoding, scalars, arrays, tables, arrays of tables, dates/times, and diagnostics.
USER — NSS system users	Lookup by name or UID, existence, absence vs NSS errors, passwd fields, workers, and security limits.
WORKERS — OS threads, queues, and channels	Isolated Lua states, JSON transport, `status`, `done`, a persistent bounded pool, cooperative cancellation, inbox/outbox, direct MPMC channels, closing, GC, and deadlock traps.

Structure of a module page

Detailed pages are progressively aligned on this structure:

1. **Scope** — what the module covers and deliberately excludes;
2. **Internal contents** — direct access to each function group;
3. **API overview** — signatures and results;
4. **Detailed behaviour** — defaults, symlinks, recursion, limits, and side effects;
5. **Examples** — one example per important mode or option;
6. **Error contract** — raised errors and returned errors;
7. **Design decisions and limitations** — intentional choices and omissions.

The goal is to let a function be used without reading its source code or having to guess default values.

PDF generation

The whole manual can be exported as one PDF:

```
cd docs
./build_doc.sh
```

English chapter order is defined in `manual_order_en.txt`.

Getting started

Download

Prebuilt binaries for x86_64, aarch64 (RPi4), and armv6l (RPi0) are available on the releases page.

Build from source

Babet vendors all its dependencies (Lua, OpenSSL, SQLite, miniz, libarchive, zlib, XZ Utils/liblzma, bzip2/libbz2, Zstandard/libzstd, RE2, Abseil, nlohmann/json, cpp-httplib, tomlplusplus), so the only prerequisites on your system are :

- a C++23 compiler (recent GCC or Clang)
- CMake (≥ 3.22)
- `wget`, `unzip`, and `xz`

```
git clone https://github.com/Chipsterjulien/babet.git
cd babet
./build_local.sh        # downloads deps and compiles
                        # (~5 min on first run, faster afterwards)
./run_tests.sh          # offline harness — should finish with 0 FAIL
```

The build script downloads each dependency from a pinned upstream source, verifies its SHA256, then compiles it statically. Several dependencies have an explicit mirror or fallback URL. When a source has no mirror, the script prints the exact archive name to place manually in `downloads/`; the same SHA256 check still applies before extraction.

The resulting binary is at `test/babet`.

Three ways to run a Lua script

Babet supports three launch modes :

1. Folder mode

Point the binary at a directory containing `main.lua` :

```
echo 'print("hello from Babet")' > main.lua
./test/babet .
```

`require("X")` in `main.lua` looks for `X.lua` in the same directory.

2. Embedded mode (single-file executable)

Package a script and its modules into a self-contained binary :

```
./test/babet --create-exe . myapp
./myapp        # runs main.lua from the embedded ZIP
```

Version-control metadata directories `.git`, `.svn`, and `.hg` are automatically excluded at any depth.

Internally, Babet appends a ZIP to its own binary and reads `main.lua` and packaged Lua modules from it. The target machine does not need a separate Babet installation. Platform dependencies still apply; an application using the GUI also requires GTK4 at runtime. A generated application is a final artifact: invoking it with `--create-exe` or `-c` is rejected before its `main.lua` runs. Use the original Babet binary to create another executable.

Module sources. For a module that `require` has not already loaded, `package.preload` is checked before the ZIP. The embedded loader searches `name.lua`, then `name/init.lua`. If both entries are absent, the ordinary Lua loaders remain active: `package.path` can supply a disk Lua module and `package.cpath` a native module compatible with the Lua ABI. A read or compile error in a module found in the ZIP stops loading; it does not trigger this disk fallback. These rules also apply to workers in generated applications.

`package.loadlib` remains available to load a native library explicitly from disk. This Lua API is distinct from the Babet plugin API, which remains unavailable in generated applications. Packaging therefore does not guarantee that all loading comes exclusively from the ZIP and does not create a

sandbox. To distribute a self-contained application, include its Lua modules in the packaged project and check its external resource or library requirements.

Embedded `main.lua` and modules support the same initial UTF-8 BOM and `#` comment/shebang line as disk-loaded Lua files, including in workers. Source line numbers are preserved; Lua bytecode remains loadable with these prefixes. Modules are read from the running executable's inode: replacing or deleting its pathname does not mix the current application's modules with a new version.

Startup requires an accessible, readable `/proc/self/exe`. An image open or read failure stops Babet with status 1 and a diagnostic; it does not turn a generated application into the Babet CLI or builder.

The builder now writes a descriptor in its runtime copy identifying the output as an application and recording the ZIP offset and size. Startup reads this identity from the loaded image: incidental ZIP bytes in a bare runtime cannot change its mode. An application with a truncated ZIP, extra trailing data, unreadable payload or missing `main.lua` is rejected before processing arguments.

Rebuild applications with the new Babet binary to obtain this protection. Existing executables retain their bundled runtime. Use `--create-exe`: manually concatenating a ZIP to the new runtime no longer turns it into an application. If desired, run `strip` on the bare runtime **before** packaging, then preserve the generated application file.

Do not compress the runtime with UPX: this hides the descriptor in the file and prevents `--create-exe` from working. `build_and_deploy.sh` now preserves the built binary even when UPX is installed, and checks that it can create and run a small application before replacing the installed Babet binary. This check uses a temporary directory under `build/`, so it also works when `TMPDIR` is mounted `noexec`. Installation prepares a file with mode `0755` in the target directory, then replaces the target by atomic rename: an already running Babet keeps its old binary and a failed copy leaves the previous installation intact.

Each embedded Lua file (`*.lua`, including `main.lua` and `*/init.lua`) is limited to **16 MiB uncompressed**, inclusive. Packaging rejects larger scripts with their archive filename before publishing the executable and preserves any previous output. The completed ZIP is checked too, covering source growth during packaging. Non-Lua assets retain their existing size policy. The loader also enforces the limit on altered archives carrying a valid descriptor.

3. Embedded via PATH (folder + auto-detect)

If Babet is on `$PATH` and you `chmod +x main.lua` after adding a shebang line :

```
#!/usr/bin/env babet
print("hello")
```

```
chmod +x main.lua
./main.lua
```

Babet uses the script's own directory as the folder, so `require("helpers")` finds `helpers.lua` next to `main.lua`.

Command-line invocation

In addition to the three launch modes above, Babet accepts a few standard flags :

Flag	Effect
`-h`, `--help`	Print the help text and exit `0`.
`-V`, `--version`	Print `babet <version>` and exit `0`.
`-c <dir> <out>`, `--create-exe <dir> <out>`	Create a self-contained executable named `<out>` by embedding `<dir>` (which must contain `main.lua`).

Any other argument starting with `-` is treated as an **unknown option** : Babet prints `Unknown option: ...` + a hint to use `--help`, and exits 1 instead of trying to interpret it as a directory name. Folders whose name legitimately starts with `-` can still be passed via `./-dirname` (POSIX convention).

```
babet --version    # babet 2.35.0
babet --help       # full usage
babet --bogus      # Unknown option: --bogus
                      # Try 'babet --help' for more information.
```

The same version is also exposed to scripts as `babet.VERSION` (plus `babet.VERSION_MAJOR` / `VERSION_MINOR` / `VERSION_PATCH` as integers) — see `sys`.

First script

Once the binary is built, try this :

```lua
-- main.lua
print("Babet says hi")
print("PID:", babet.pid())
print("Host:", babet.hostname())

local r, err = babet.http.get("https://example.com/")
if r then
    print("Status:", r.status)
else
    print("HTTP failed:", err)
end
```

Run it with `./test/babet .` (or whichever mode you prefer).

Where to look next

- Each module has its own page under `modules/`.
- The `user` module is a small, complete example of the documentation pattern used throughout — read it as the canonical reference.
- For date and duration utilities (ISO 8601 timestamps, human durations like `"5m"` or `"2h30m"`), see `time` — the `babet.time.*` sub-table covers parsing and formatting.
- See `security.md` before exposing Babet scripts to anything that might receive untrusted input.

Launch context and resources

Available since Babet 2.35.0.

`babet.runtimeInfo()` returns `table, nil`. The table is an independent copy.

Field	Script	Directory	Generated application
`mode`	`"file"`	`"directory"`	`"embedded"`
`projectDir`	absolute script directory	absolute project directory	`nil`
`scriptPath`	absolute script path	absolute `main.lua` path	`"main.lua"`
`executablePath`	runtime path	runtime path	application path

Paths are captured at launch and do not follow working-directory changes. They are location information, not proof of identity after renaming. Workers inherit the context. C embedding hosts use `mode="host"` with unknown fields absent; their workers may know the directory configured by the host.

Embedded resources

- `babet.embedded.read(name)` → `bytes, nil` or `nil, error`.
- `babet.embedded.list()` → sorted names table and `nil`, or `nil, error`.

These APIs require an application created using `-c` / `--create-exe`; there is no filesystem fallback. Names are case-sensitive, relative to the ZIP root, and use `/`. Lists contain files, including `main.lua`, without directory entries. Missing resources return `nil, error`. Reads preserve all bytes: NUL, BOM, line endings and empty files. They use the actual running image via `/proc/self/exe`, even after its path is replaced.

Limits: 16 MiB uncompressed per file; 16 MiB for all listed names, including terminators. Packaging a large resource does not guarantee this API can read it.

```lua
local info = assert(babet.runtimeInfo())
local css
if info.mode == 'embedded' then
    css = assert(babet.embedded.read('assets/style.css'))
else
    local f = assert(io.open(info.projectDir .. '/assets/style.css', 'rb'))
    css = f:read('a')
    f:close()
end
```

Strict globals

`babet.strict()` or `babet.strict(true)` raises an error when reading an absent global. `babet.strict(false)` disables it. Returns `true, nil`, or `nil, error` if `_G` has a foreign metatable, which is never overwritten. Global assignments remain allowed; `nil` means absence. Use `rawget(_G, 'name')` for optional lookups. Each Lua state must opt in, including workers. This does not check other tables, analyze code statically or provide a security boundary. No new CLI switch is introduced.

Multiple returns and arity

Existing APIs retain strict arity. Lua forwards all results of a call used as the last argument, so `f(babet.getPath(x))` may pass two arguments. Use `f((babet.getPath(x)))` to pass only the first result. To handle errors, prefer `local path, err = babet.getPath(x)`, check `path`, then call `f(path)`. Extra `nil` arguments are not silently discarded.

Program exit

In the Babet CLI and generated executables, `os.exit([code [, close]])` terminates the process, including from a coroutine or inside `pcall`. Status arguments follow Lua: absent, `nil` or `true` means success; `false` means failure; an integer supplies the exit status. Invalid arguments raise a Lua error before cleanup begins.

Babet disables its GUI callbacks and restores its curses session before exit. If an interactive child still owns the terminal, Babet respects that ownership and does not take the TTY away from it. Wait for or explicitly terminate that child before exiting when an orderly terminal handback is required.

- Missing or false `close`: no Lua state closure, no Lua finalizers and no implicit worker join. Other threads terminate with the process.
- Lua-truthy `close`: the common Babet cleanup closes the Lua state, running its finalizers and the usual worker joins. This can wait for a worker stuck in a non-cooperative native call; it is neither forced cancellation nor a deadline guarantee.

Open C/Lua file buffers are flushed before exit. Flushing can itself wait for I/O; it does not provide an `fsync` durability guarantee. Automatic C++ objects on abandoned stacks are not destroyed.

Difference from stock Lua `os.exit`: following this explicit cleanup, Babet stops the process without running native `atexit` callbacks or C++ static destructors. This avoids tearing down global libraries while workers or native threads may still use them. Plugins must perform application cleanup explicitly before `os.exit`. Returning normally from the script retains the ordinary shutdown path and native process-exit callbacks.

An `os.exit` called by a finalizer during Babet state closure does not start a second close of the same VM; it exits with the new status. Remaining finalizers are then not guaranteed to run.

Workers still reject `os.exit` with a Lua error; use `return`. The embedding API does not replace stock `os.exit` in the host's main Lua state; this CLI policy is not installed there.

Security model

Babet runs trusted Lua scripts. It is not a sandbox.

Scripts execute with the full Lua standard library (`luaL_openlibs`), and Babet additionally exposes filesystem and process primitives such as `exec`, `remove`, `rmdirAll` and `chdir`. A script therefore has the same privileges as the process running it : it can read, modify or delete files and run arbitrary commands. This is by design — like `make`, a shell script, or any build tool, Babet is meant to run code you control.

Only run `main.lua` and `required` modules that you trust. Do **not** use Babet to execute Lua from untrusted sources ; it provides no isolation against hostile code, and none is intended. If you need to run untrusted scripts, use a purpose-built Lua sandbox instead.

What the hardening *does* protect against

The hardening work in Babet protects *legitimate* use against accidents and supply-chain tampering :

- **`copyTree` / `moveTree` confinement** : the destination root is opened once, then every create, copy and move is performed relative to that descriptor with `openat`/`mkdirat`/`renameat` and `O_NOFOLLOW`/`AT_SYMLINK_NOFOLLOW`. A pre-existing symlink, or one swapped during the operation, cannot redirect a write to an outside target. The `is_within()` guards additionally reject a destination resolving inside the source.
- **Confined archive creation and extraction**: `babet.archive` refuses symlinked creation-source and output/extraction-destination paths, selected unsupported source objects, non-UTF-8 selected source names, outputs inside the source tree, absolute archive-entry paths, `./..` components, backslashes, duplicates, ZIP symlinks, and special filesystem types. Input archives are opened once and must resolve to a regular file, excluding blocking FIFO sources. It walks the destination through descriptors with `O_NOFOLLOW`; each file is decompressed into a same-directory temporary, checked against anti-bomb limits, then published atomically. Creation-side `include`/`exclude` rules reuse the non-recursive safe-glob engine, prune excluded directories before opening, and enforce per-pattern, cumulative pattern-text, pattern-evaluation, matching-work, and pattern-count limits.
- **Standalone compression confinement**: `babet.compression` accepts only real regular source files, rejects symlink path components and same-inode destinations, pins and revalidates the source descriptor, bounds decompressed output, verifies codec integrity, rejects trailing junk, stages output beside the destination, and publishes atomically.
- **General atomic file writing**: `writeFileAtomic()` opens every parent without following symlinks, rejects `..`, a final symlink or non-regular destination, and implicit overwrite. Data is written to a private same-directory temporary file, permissions are applied, then the file and directory are synchronized around atomic publication.
- **Bounded process-group cleanup** in `babet.exec` also covers the `chdir`/`exec` launch phase, internal polling failures, and children that close all pipes while continuing to run. On timeout, TERM then KILL target the whole group; these error paths never end in an unbounded final `waitpid`.

- **Output limits** on `babet.exec` bound the amount of stdout and stderr captured into memory (default 10 MiB each, configurable), so a runaway subprocess can't OOM the Babet process.
- **Pinned local metadata updates**: `setAttributes` resolves its target once and keeps `chown`, `chmod`, and rollback attached to the same inode. `touch` creates with `O_EXCL`, never truncates an existing file, and pins an existing target before changing its timestamp.
- **Anonymous and pinned executable staging**: `--create-exe` unlinks its temporary ZIP immediately after `mkstemp` and lets miniz and the merge step reopen the still-live inode through `/proc/self/fd`, not through a replaceable temp path. The final merge pins the output directory and publishes with `renameat()`, so changing a parent symlink cannot redirect the executable.
- **Dependency checksums** : every vendored dependency is SHA256-pinned in `build_local.sh`. An empty hash refuses to build. A mismatch deletes the downloaded file and exits with an error. This protects against a compromised upstream or Wayback Machine archive (which is the fallback source).
- **TLS verification** is on by default (`verify=true`). Disabling it requires an explicit per-call `verify=false` — there is no silent fallback. TLS sockets accept extra trust through `ca_cert` or `ca_path`; HTTP exposes `ca_cert`. OpenSSL's `SSL_CERT_FILE` / `SSL_CERT_DIR` remain available. SNI is sent independently from verification when a DNS hostname is known.
- **HTTP line validation** rejects CR/LF in URLs and header values, and header names must follow the HTTP `token` grammar. User-controlled values cannot inject an extra header or request line through those fields.
- **WebSocket protocol hardening** keeps RFC 6455 client frames masked with a fresh cryptographic key, rejects masked/RSV/reserved server frames, validates UTF-8 and close codes, rejects control/space bytes in the URL authority, and checks advertised data-frame/message sizes before large allocations. RFC control frames retain their separate 125-byte ceiling. `wss://` verifies certificates by default and negotiates no compression extensions.
- **Network memory limits** bound `socket:recv_line`, `socket:recv_all`, and `http.max_body_size`. After a socket-read timeout, already consumed bytes stay in one shared pending buffer so switching receive methods cannot lose or reorder the stream.
- **Bounded filename matching** provides a non-recursive safe glob engine for `babet.find` and archive creation, plus RE2-backed regular expressions for `babet.find`. Regex patterns are capped at 4096 bytes, compiled with a 1 MiB memory budget, and cannot trigger catastrophic backtracking. Archive filter lists add cumulative count, byte, and matching-work ceilings. RE2-unsupported constructs fail before traversal.
- **Device-boundary traversal confinement** lets `babet.find(..., { xdev = true })` keep a foreign mount point observable while skipping its children. This prevents a local search from accidentally walking a mounted volume, network share, or pseudo-filesystem. The boundary is the `st_dev` value, as with `find -xdev`: a Btrfs subvolume may be pruned, while a bind mount of the same filesystem is not. It is not a sandbox: the script may still call `babet.find()` directly on that other path.

What it does *not* protect against

- A malicious script. Babet has no sandbox. `babet.exec` does not invoke a shell, but allowing untrusted input to choose the executable or its arguments can still permit arbitrary program execution or option injection. Passing untrusted text to `load` gives arbitrary Lua code execution.
- A determined attacker who has obtained code execution on the machine running Babet. The hardening makes accidental mistakes loud, not adversarial attacks impossible.
- Long-tail OS-level issues (kernel exploits, container escapes, privilege escalation). Babet is an ordinary userland binary.

Recommended pattern : least privilege

When running Babet as a service, treat it like any other unprivileged process :

```
# As root, create a dedicated service user with no shell.
useradd --system \
```

```
        --home-dir /var/lib/myapp \
        --create-home \
        --shell /usr/sbin/nologin \
        --comment "myapp Babet service" \
        myapp

# Use babet.user.exists("myapp") in your install script to
# verify the user is provisioned before launching the daemon.
```

Combine with `systemd` unit file options like `User=myapp`, `PrivateTmp=yes`, `ProtectSystem=strict`, `NoNewPrivileges=yes`, and `CapabilityBoundingSet=` (empty unless you genuinely need a capability). The kernel does much more than Babet can to contain a misbehaving script ; let it.

Archive — secure ZIP and TAR operations with gzip, xz, bzip2, or zstd

Scope

The `babet.archive` submodule creates, inspects, reads in memory, and extracts **ZIP**, **TAR**, and **gzip-, xz-, bzip2-, or zstd-compressed TAR** archives without calling an external command:

```
babet.archive.create(source_or_sources, archive [, opts])
babet.archive.list(archive [, opts])
babet.archive.read(archive, raw_name_or_index [, opts])
babet.archive.test(archive [, opts])
babet.archive.extract(archive, destination [, opts])
babet.archive.extractFile(archive, entry, destination [, opts])
```

ZIP creation, inspection, and extraction continue to use miniz. TAR creation, inspection, and extraction use the statically linked libarchive backend; gzip filtering is provided by the statically linked zlib backend, xz filtering uses the statically linked XZ Utils/liblzma backend, bzip2 filtering uses the statically linked bzip2/libbz2 backend, and zstd filtering uses the statically linked Zstandard/libzstd backend. Readers detect both the container and compression from the contents. For `create()`, `.tar` selects plain TAR, `.tar.gz` and `.tgz` select gzip-compressed TAR, `.tar.xz` or `.txz` select xz-compressed TAR, `.tar.bz2`, `.tbz2`, or `.tbz` select bzip2-compressed TAR, and `.tar.zst`, `.tar.zstd`, or `.tzst` select zstd-compressed TAR. `opts.format` can override the extension. Standalone compressed streams are handled separately by `babet.compression`.

All six functions use the usual result contract:

```
local result, err = babet.archive.list("backup.zip")
if not result then
    io.stderr:write(err, "\n")
end
```

They return `(result, nil)` on success and `(nil, message)` on failure. Wrong arity, a destination that is not a string, or a source that is neither a string nor a table raises a Lua error. Invalid source lists and option tables return `(nil, message)`.

Contents

- `babet.archive.create`
- Inspection/extraction options and anti-bomb limits
- `babet.archive.list`
- `babet.archive.read`
- `babet.archive.test`
- `babet.archive.extract`
- `babet.archive.extractFile`

- Path rules
- Destination protection
- Atomic publication and cleanup
- Permissions
- Supported entry types
- Errors and known limitations

babet.archive.create

```
local result, err = babet.archive.create(
    source_or_sources,
    archive
    [, opts]
)
```

The first argument supports two distinct contracts:

- a **string** preserves the historical behavior: it must name an existing directory, and Babet archives its contents without including the source directory name itself;
- a **non-empty dense array** (`1..n`, no holes or extra keys) explicitly selects regular files and/or directories, possibly from unrelated locations. Each source is rooted at its final component: `/tmp/report.txt` becomes `report.txt`, while `/opt/project/docs` becomes `docs/` with all descendants. Absolute sources are accepted, but host path prefixes are never exposed in the archive.

The explicit list is read through raw array access. `__len` and `__index` metamethods are not invoked, so only the paths actually stored at indices `1..n` are validated and archived.

In list mode, the final component must be stable: `.` and the filesystem root are rejected, as is every `..` component. Two sources with the same final component collide and fail, even when their descendants would differ. Repeating the same source therefore fails as well. Input list order does not affect the result: entries are always sorted by archive name.

In both modes traversal is descriptor-based and follows no symlink. A symlink in a top-level source path always fails. Within a traversed tree, a selected symlink, FIFO, socket, device, or other unsupported object fails the operation before the archive is published; an object removed by the filters is not opened. Entry names produced by `create()` must be valid UTF-8. Linux permits arbitrary bytes in filenames, but Babet refuses names that cannot be represented safely and consistently in both output formats.

Output format

Without `opts.format`, `create()` uses these rules:

- a destination ending in `.tar` (case-insensitive) creates an uncompressed POSIX pax TAR archive;
- `.tar.gz` and `.tgz` (case-insensitive) create a gzip-compressed POSIX pax TAR archive;
- `.tar.xz` and `.txz` (case-insensitive) create an xz-compressed POSIX pax TAR archive;
- `.tar.bz2`, `.tbz2`, and `.tbz` (case-insensitive) create a bzip2-compressed POSIX pax TAR archive;
- `.tar.zst`, `.tar.zstd`, and `.tzst` (case-insensitive) create a zstd-compressed POSIX pax TAR archive;
- every other destination keeps the pre-2.6 historical behaviour and creates a ZIP archive, regardless of its extension.

`opts.format = "zip"`, `"tar"`, `"tar.gz"`, `"tar.xz"`, `"tar.bz2"`, or `"tar.zst"` explicitly selects a backend. An explicit format always overrides the destination suffix. This means, for example, that `format = "zip"` may create a ZIP named `backup.tar.gz`, while `format = "tar"` may create an uncompressed TAR named `backup.tgz` or `backup.txz`, `backup.tbz2`, or `backup.tzst`. Format selection is based on the explicit option or content, not trust in the suffix.

Creation options:

Option	Default	Behaviour
`format`	inferred	strict string: `"zip"`, `"tar"`, `"tar.gz"`, `"tar.xz"`, `"tar.bz2"`, or `"tar.zst"`
`compression_level`	6	ZIP/gzip/xz: `0` through `9`; bzip2: `1` through `9`; zstd: `0` through `19`
`overwrite`	`false`	atomically replaces an existing regular archive
`deterministic`	`true`	stable entry order and fixed format-specific timestamps
`include_directories`	`true`	emits explicit directory entries, including empty directories
`include`	none	dense array of case-sensitive safe-glob patterns selecting archive entry paths
`exclude`	none	dense array of case-sensitive safe-glob patterns removed after inclusion; exclusion always wins
`max_entries`	`10000`	maximum output entries; hard ceiling `100000`
`max_file_size`	`256 * 1024 * 1024`	maximum source-file size; hard ceiling 8 GiB
`max_total_size`	`1024 * 1024 * 1024`	maximum source bytes; hard ceiling 64 GiB

For plain TAR, an explicitly supplied `compression_level` is rejected. For ZIP, gzip-compressed TAR, and xz-compressed TAR, level `0` is valid and level `9` is the maximum. bzip2 accepts levels `1` through `9`; level `0` is rejected because libbz2 block sizes start at 100 KiB. zstd accepts levels `0` through `19`; Babet deliberately excludes the ultra levels `20` through `22`.

All integer options require strict Lua integers. Boolean options require real Lua booleans. Unknown keys are rejected. `format` is case-sensitive and does not accept aliases.

Include and exclude filters

`include` and `exclude` are optional dense arrays of non-empty Lua strings. An absent or empty `include` list keeps the historical behaviour and initially selects every supported entry. A non-empty `include` list selects an entry when at least one pattern matches. `exclude` is then applied and always wins, even when the same path also matches `include`.

Patterns are matched against the complete path that will be stored in the archive, using `/` as the separator. The historical string form therefore matches paths relative to the source directory, while explicit-list sources include their final basename (`docs/readme.md`, `report.txt`, and so on). Matching is anchored, byte-oriented, and case-sensitive:

- `*` matches zero or more bytes except `/`;
- `**` matches zero or more bytes including `/`;
- `?` matches exactly one byte except `/`;
- `\x` quotes the following byte `x`.

Directories are tested both as `path` and `path/`. Consequently `build/**` selects or excludes the `build/` directory itself as well as its descendants. A directly excluded directory is pruned before it is opened, so nothing below it is inspected or archived. When a deep file is included, `include_directories = true` still emits every required parent directory even if those parents do not directly match `include`. A matched empty directory is preserved only when directory entries are enabled. With `include_directories = false`, only selected regular files are emitted and parents remain implicit.

Filtering defines the selected archive plan. A symlink, FIFO, socket, device, or invalid UTF-8 filename that is excluded or does not match a non-empty `include` list is ignored; the same object still fails creation when selected. The explicit paths supplied as top-level sources are always validated first and must themselves remain regular files or real directories reached without symlinks. It is valid for all filters to select nothing: Babet then creates a valid empty ZIP or TAR.

The safe-glob engine has no recursive backtracking. Each pattern is limited to 4096 bytes. `include` and `exclude` together are limited to 256 patterns and 256 KiB of pattern text, one million pattern evaluations, and a fixed 100,000,000-cell matching-work budget per creation call. Exceeding any limit fails before publication. Pattern order does not affect selection or deterministic archive bytes.

All source paths are validated and each non-pruned directory tree is scanned before the temporary output file is created. `max_entries`, `max_file_size`, and `max_total_size` apply only to entries selected by the filters. Each selected archive name is limited to 4096 bytes and their sum to 64 MiB. Traversal still has fixed ceilings of 100000 sources/filesystem objects and 256 directory levels, independent of `include_directories`; descendants of a pruned directory are not visited. Files are then reopened from their pinned source descriptor with `O_NOFOLLOW`. Device, inode, size, modification time, and change time are checked before and after streaming; a file modified during creation fails the operation and removes the temporary output.

The destination parent must already exist and may not contain symlink or `..` components. The destination itself must be absent unless `overwrite = true`, and it may never be a symlink or directory. The archive cannot be created inside the historical source directory or any explicitly selected directory, and cannot directly replace a selected regular file.

The result includes the selected format:

```
{
    files = 12,
    directories = 3,
    bytes = 987654,
    sources = 1,
    path = "backup.tar",
    format = "tar",
    compression = "none",
    compression_level = nil,
    deterministic = true,
    include_patterns = 0,
    exclude_patterns = 0,
}
```

`sources` is the number of supplied paths: `1` for the historical single source-directory contract, or the explicit list length. `include_patterns` and `exclude_patterns` report the number of compiled patterns supplied for that call. For ZIP and gzip-, xz-, bzip2-, or zstd-compressed TAR, `compression_level` is the effective integer level. For plain TAR it is `nil`. Archive-level `compression` is `"gzip"`, `"xz"`, `"bzip2"`, or `"zstd"` for compressed TAR and `"none"` otherwise. ZIP entry compression remains reported through the ZIP-specific per-entry metadata returned by `list()`.

With the default deterministic mode, identical directory contents, format, pinned dependency versions, and options produce byte-for-byte identical archives:

- ZIP entries use the fixed DOS timestamp 1980-01-01 00:00:00;
- TAR entries use Unix epoch zero, UID/GID `0`, empty owner/group names, mode `0644` for files, and mode `0755` for directories;

- gzip-compressed TAR additionally writes a gzip header timestamp of zero, so the outer stream does not inject wall-clock time;
- xz streams contain no wall-clock timestamp, and the selected LZMA2 preset is fixed by `compression_level`;
- bzip2 streams contain no wall-clock timestamp, and their block size is fixed by `compression_level`;
- zstd frames contain no wall-clock timestamp, enable their frame checksum, and use the compression level fixed by `compression_level`.

With `deterministic = false`, sorted entry order remains stable. ZIP uses each source modification time, which must fit the DOS range 1980 through 2107 in the local timezone. TAR preserves the source modification time with nanosecond precision when representable by the platform and is not limited to the ZIP DOS range.

Ownership, ACLs, extended attributes, and source permission bits are not copied. TAR uses libarchive's restricted POSIX pax writer: ordinary names use portable ustar headers where possible, while long paths or metadata that need extension records are represented with pax headers. ZIP64 output is enabled automatically when the preflight indicates that classic ZIP fields may be exceeded. Small ZIP archives remain classic ZIP files.

The preflight and file checks prevent symlink substitution and detect changes to every file that is actually archived. Creation is not a filesystem snapshot: a file added after its directory was scanned may be absent from the result; a previously enumerated file that is then removed or renamed normally makes its secure reopen fail. Concurrent directory renames are not reported as a transactional conflict.

When `include_directories = false`, empty directories cannot be represented and are therefore omitted. In an explicit list, regular files still retain the selected directory basename as their prefix. Required parents are recreated implicitly during extraction.

Examples:

```lua
local zip, err = babet.archive.create("project", "project.zip", {
    compression_level = 9,
    overwrite = true,
})
assert(zip, err)

local selected, list_err = babet.archive.create({
    "bin/babet",
    "README.md",
    "docs",
}, "release.tar.zst", {
    compression_level = 19,
})
assert(selected, list_err)
-- Entries: babet, README.md, docs/, docs/...

local filtered, filter_err = babet.archive.create("project", "source-only.tar.zst", {
    include = { "src/**", "README.md" },
    exclude = { "src/generated/**", "**/*.tmp" },
})
assert(filtered, filter_err)

local tar
tar, err = babet.archive.create("project", "project.tar", {
    deterministic = true,
})
assert(tar, err)
```

```lua
local compressed
compressed, err = babet.archive.create("project", "project.tar.gz", {
    compression_level = 9,
})
assert(compressed, err)

local xz
xz, err = babet.archive.create("project", "project.tar.xz", {
    compression_level = 9,
})
assert(xz, err)

local bzip2
bzip2, err = babet.archive.create("project", "project.tar.bz2", {
    compression_level = 9,
})
assert(bzip2, err)

local zstd
zstd, err = babet.archive.create("project", "project.tar.zst", {
    compression_level = 19,
})
assert(zstd, err)

local named
named, err = babet.archive.create("project", "snapshot.data", {
    format = "tar.zst",
})
assert(named, err)
```

Inspection/extraction options and anti-bomb limits

The following options are shared by `list()`, `read()`, `test()`, `extract()`, and `extractFile()`. They apply to the **whole archive**, even when `read()` or `extractFile()` requests only one entry: selecting a small file can never bypass global limits.

Option	Default	Accepted maximum	Effect
`max_entries`	`10000`	`100000`	maximum number of entries exposed by the archive
`max_entry_size`	`256 * 1024 * 1024`	`8 * 1024^3`	maximum expanded size of one non-directory entry
`max_total_size`	`1024 * 1024 * 1024`	`64 * 1024^3`	maximum sum of expanded non-directory entry sizes
`max_path_length`	`64 * 1024`	`1024 * 1024`	maximum raw entry-name length, in bytes, accepted for inspection

Option	Default	Accepted maximum	Effect
`max_total_name_bytes`	`64 * 1024 * 1024`	`64 * 1024 * 1024`	maximum byte sum of all entry names; this option can tighten the fixed ceiling
`max_compression_ratio`	`1000`	`1000000000`	per-entry ratio for ZIP; global expanded-file-bytes/archive-bytes ratio for gzip-, xz-, bzip2-, or zstd-compressed TAR; irrelevant for plain TAR

The five integer limits must be **real, strictly positive Lua integers**. Floating-point values, even `10.0`, booleans, numeric strings, zero, and values above the ceiling are rejected. `max_compression_ratio` must be finite and at least `1`; `NaN` and infinity are rejected. An unknown key is always an error.

`max_path_length` bounds metadata accepted by Babet; it does not make a long path extractable. Extraction path policy remains stricter: a name longer than **4096 bytes** is reported by `list()` with `safe_path = false`, even when its length remains below `max_path_length`. With the default value, a 4097-byte name can therefore be inspected and diagnosed without ever being eligible for extraction. Callers may lower `max_path_length` to reject the archive before the Lua result table is built.

`max_total_name_bytes` already equals its fixed hard ceiling by default. It can be lowered for an untrusted archive, but not increased. `list()` reports actual usage as `total_name_bytes`. Allocations made by the miniz ZIP reader are also capped at 128 MiB per archive, so an excessive central directory or ZIP comment cannot trigger arbitrary allocation before per-entry checks.

TAR inspection is streaming: Babet reads every entry data stream without keeping it in memory, both to reach the following header and to detect truncation. For gzip TAR, bounded streaming validation with zlib also checks each member's CRC and ISIZE; standard zero padding is accepted, while corrupt trailers or foreign trailing bytes are rejected. For zstd TAR, an independent bounded libzstd pass requires complete frames, accepts valid concatenated frames, and rejects corruption, truncation, or foreign trailing bytes. xz and bzip2 filters remain fully consumed by libarchive.

For complete extraction, the output tree is additionally capped internally at 100000 unique directories, including implicit parents, and 64 MiB of cumulative normalised directory paths. These fixed ceilings are not configurable.

A non-empty entry announcing a zero compressed size is rejected. Directories do not count towards `max_entry_size`, `max_total_size`, or the compression ratio, but they do count towards `max_entries` and name budgets.

One example per limit

Limit only the entry count:

```lua
local result, err = babet.archive.test("upload.zip", {
    max_entries = 2000,
})
assert(result, err)
```

Limit only one entry's expanded size:

```lua
local result, err = babet.archive.test("upload.zip", {
    max_entry_size = 64 * 1024 * 1024,
})
assert(result, err)
```

Limit only the announced total size:

```lua
local result, err = babet.archive.test("upload.zip", {
    max_total_size = 512 * 1024 * 1024,
})
assert(result, err)
```

Limit only raw name length:

```lua
local result, err = babet.archive.test("upload.zip", {
    max_path_length = 8192,
})
assert(result, err)
```

Limit only cumulative name bytes:

```lua
local result, err = babet.archive.test("upload.zip", {
    max_total_name_bytes = 2 * 1024 * 1024,
})
assert(result, err)
```

Limit only the compression ratio:

```lua
local result, err = babet.archive.test("upload.zip", {
    max_compression_ratio = 200,
})
assert(result, err)
```

Combine every protection for a third-party archive:

```lua
local result, err = babet.archive.test("upload.zip", {
    max_entries = 2000,
    max_entry_size = 64 * 1024 * 1024,
    max_total_size = 512 * 1024 * 1024,
    max_path_length = 8192,
    max_total_name_bytes = 2 * 1024 * 1024,
    max_compression_ratio = 200,
})
assert(result, err)
```

Creation or write-specific options such as `overwrite`, `preserve_permissions`, `format`, `dry_run`, `include`, or `exclude` are not accepted by `list()`, `read()`, or `test()`. `read()` additionally accepts `max_size`, documented in its section; the other five functions reject it.

babet.archive.list

```lua
local info, err = babet.archive.list(archive [, opts])
```

`archive` is the path to a ZIP, plain TAR, gzip-compressed TAR, xz-compressed TAR, bzip2-compressed TAR, or zstd-compressed TAR file. Format and compression are detected from **content**, never from the extension: a zstd TAR named `snapshot.bin` is recognised, while a file named `snapshot.tar.zst` containing something else is rejected. The usual `.tar`, `.tar.gz`, `.tgz`, `.tar.xz`, `.txz`, `.tar.bz2`, `.tbz`, `.tbz2`, `.tar.zst`, `.tar.zstd`, and `.tzst` suffixes are naming conventions only.

The source path may be a symlink to a regular file. Babet opens its target once and keeps that same descriptor for the whole scan. Directories, FIFOs, sockets, and devices are rejected before format parsing; non-blocking open notably prevents a FIFO with no writer from hanging the call.

Babet first tries the miniz ZIP reader. If the pinned file is not ZIP, the same descriptor is rewound and passed to libarchive's TAR reader, with only the built-in `none`, `gzip`, `xz`, `bzip2`, and `zstd` filters enabled. No external decompressor is executed. Split ZIP files and multi-volume archives are not supported. A standard empty archive is valid and returns `count = 0`.

`list()` creates **no file, directory, or temporary file**. It preserves the physical/logical entry order found in the archive: entries are not sorted, deduplicated, or rearranged. `entries[1]` is always the first encountered entry, `entries[2]` the second, and so on. This stable order lets `index`, `duplicate_of`, and `conflict_with` directly reference the returned array.

The function returns exactly two values: `(info, nil)` on success and `(nil, message)` on failure.

Returned structure

```
{
    format = "zip",           -- "zip" or "tar"
    compression = "none",    -- "none", "gzip", "xz", "bzip2", or "zstd"
    entries = {
        {
            index = 1,
            name = "docs/readme.txt", -- raw name, binary Lua string
            path = "docs/readme.txt", -- normalised path when safe_path == true
            valid_utf8 = true,
            type = "file",
            size = 1234,

            compressed_size = 530,     -- ZIP only, otherwise nil
            crc32 = 305419896,          -- ZIP only, otherwise nil
            compression_method = 8,    -- ZIP only, otherwise nil
            encrypted = false,
            supported = true,

            safe_path = true,
            extractable = true,
            reason = nil,

            unix_mode = 420,            -- 0644, or nil
            mtime = 1750000000,         -- Unix seconds, or nil
            mtime_nsec = nil,           -- TAR only, or nil
            uid = nil,                  -- TAR only, or nil
            gid = nil,                  -- TAR only, or nil
            sparse = false,
            link_target = nil,

            duplicate = false,
            duplicate_of = nil,
            conflict = false,
            conflict_with = nil,
            conflict_reason = nil,
        },
    },
    count = 1,
    total_size = 1234,
    archive_size = 666,
    total_name_bytes = 19,
    duplicates = 0,
    conflicts = 0,
```

```
    zip64 = false,              -- nil for TAR
}
```

`format` describes the container. For ZIP, `compression` is always `"none"` because compression is chosen per entry and reported by `compression_method`. For TAR, `compression` is `"none"`, `"gzip"`, `"xz"`, `"bzip2"`, or `"zstd"` according to the detected filter.

`count` is the number of elements in `entries`. `total_size` is the sum of announced non-directory entry sizes. `archive_size` is the size of the pinned archive file. `total_name_bytes` is the exact byte sum of all `name` fields. `zip64` is a boolean for ZIP and `nil` for TAR.

Fields shared by every entry

- `index` is the 1-based Lua index in archive order;
- `name` preserves the raw name as a binary Lua string;
- `path` is the normalised extraction form. A directory's trailing `/` is removed. It is usable as a path only when `safe_path == true`, and may be empty for an unsafe entry;
- `valid_utf8` says whether `name` is strict valid UTF-8. Babet never converts, replaces, or normalises the bytes;
- `type` is `file`, `directory`, `symlink`, `hardlink`, `fifo`, `character_device`, `block_device`, `socket`, or `unsupported`;
- `size` is the announced expanded size. It is normally `0` for directories or special objects, according to the actual metadata read;
- `safe_path` judges the name only: relative path, `/` separator, non-empty components, no `.` or `..`, no Windows drive prefix, no NUL byte, a trailing `/` consistent with the type, and the fixed 4096-byte extractable-name limit;
- `extractable` judges the entry in isolation: safe path, allowed type, no encryption, and a supported method. It deliberately does not include a duplicate or collision with another entry; also inspect `duplicate` and `conflict`;
- `reason` is `nil` for an individually extractable entry, otherwise a stable reason such as `symlink entries are refused` or `entry path contains '.' or '..'`;
- `unix_mode` contains bits `0000` through `7777` when available, otherwise `nil`. Decimal `420` is octal `0644`;
- `mtime` is modification time in seconds since the Unix epoch when available, otherwise `nil`;
- `mtime_nsec` is the nanosecond component of an available TAR timestamp. It is `nil` for ZIP;
- `uid` and `gid` are numeric TAR identifiers when explicitly present, otherwise `nil`. They are always `nil` for ZIP;
- `sparse` reports a sparse TAR file;
- `link_target` contains the raw target of a TAR symlink or hard link when exposed by libarchive, otherwise `nil`.

For ZIP, `mtime` comes from the entry's DOS timestamp converted by miniz to `time_t`. ZIP stores neither a time zone nor nanoseconds, so this is local calendar metadata and must be interpreted cautiously. For TAR, `mtime` may be negative or outside the DOS range, and `mtime_nsec` preserves available precision. Missing metadata is always represented by `nil`, never by an invented value.

ZIP-specific fields

`compressed_size`, `crc32`, and `compression_method` mirror ZIP central directory metadata. `encrypted` reports the encryption flag and `supported` says whether miniz understands the compression method. These fields are useful for inspection, but `crc32` is not proof of integrity: `list()` does not inflate every ZIP payload. A readable central directory may therefore coexist with compressed data whose CRC failure is only found during reading or extraction. For complete validation of local headers, payloads, and CRCs, use `archive.test()`.

For TAR, `compressed_size`, `crc32`, and `compression_method` are `nil`, `encrypted` is `false`, and `supported` means the header was parsed. Any outer compression applies to the whole TAR stream and is reported by the global `compression` field.

Binary names, unsafe paths, and normalisation

ZIP names are byte strings from the central directory. TAR names are native strings returned by libarchive after ustar prefix, GNU long-name, and pax `path` processing. Comparisons are exact, byte-oriented, and case-sensitive. No Unicode normalisation is applied: visually identical Unicode spellings remain distinct names. Invalid UTF-8 is preserved and reported by `valid_utf8 = false`; it is not automatically unsafe when all path rules pass.

Absolute paths, `.` or `..` components, doubled `/`, backslashes, `C:` drive prefixes, empty names, NUL bytes, and names longer than 4096 bytes are reported with `safe_path = false` and an explicit `reason`. `list()` continues with other entries while metadata limits remain satisfied. It never silently repairs an unsafe path.

Duplicates and collisions

Diagnostics are attached to the **second and subsequent occurrences**:

- `duplicate = true` means an identical byte-for-byte `name` was already seen. `duplicate_of` is the first occurrence's index;
- `conflict = true` means this safe normalised output path cannot coexist with an earlier entry. `conflict_with` is the selected earlier index and `conflict_reason` is currently `duplicate output path` or `file/directory path conflict`;
- `duplicates` counts additional exact occurrences;
- `conflicts` counts additional occurrences that would make an extraction plan ambiguous or impossible.

An exact duplicate with a safe path is therefore both a `duplicate` and a `conflict`. Different raw names such as `docs/` and `docs` are not duplicates, but collide after normalisation and the latter receives `conflict = true`. An unsafe entry may be reported as a raw duplicate, but does not participate in output-path collision analysis because it has no safe extraction path.

`list()` does not turn these diagnostics into an error: inspecting a suspicious archive is its purpose. `extract()` still rejects duplicates and collisions before any write.

Diagnostic example:

```
local info, err = babet.archive.list("upload.zip")
assert(info, err)

for _, entry in ipairs(info.entries) do
    if not entry.safe_path then
        print(entry.index, entry.name, entry.reason)
    elseif entry.duplicate then
        print(entry.index, "duplicates entry", entry.duplicate_of)
    elseif entry.conflict then
        print(entry.index, entry.conflict_reason,
            "with entry", entry.conflict_with)
    end
end
```

Usage examples

Print archive contents in stored order:

```
local info, err = babet.archive.list("backup.tar.zst")
assert(info, err)

print(info.format, info.compression, info.count)
for _, entry in ipairs(info.entries) do
```

```
    print(entry.index, entry.type, entry.path, entry.size)
end
```

Use optional metadata without assuming it exists:

```
local info, err = babet.archive.list("package.zip")
assert(info, err)

for _, entry in ipairs(info.entries) do
    if entry.mtime ~= nil then
        print(entry.name, os.date("%Y-%m-%d %H:%M:%S", entry.mtime))
    end
    if entry.unix_mode ~= nil then
        print(string.format("mode=%04o", entry.unix_mode))
    end
    if entry.uid ~= nil then
        print("uid/gid", entry.uid, entry.gid)
    end
end
```

Reject any ambiguous archive before further processing:

```
local info, err = babet.archive.list("upload.zip")
assert(info, err)

if info.duplicates ~= 0 or info.conflicts ~= 0 then
    error("ambiguous archive")
end

for _, entry in ipairs(info.entries) do
    if not entry.safe_path or not entry.extractable then
        error((entry.reason or "rejected entry") .. ": " .. entry.name)
    end
end
```

Handle an invalid UTF-8 name correctly:

```
local info, err = babet.archive.list("legacy.zip")
assert(info, err)

for _, entry in ipairs(info.entries) do
    if not entry.valid_utf8 then
        -- The string remains binary: display bytes instead of passing it to
        -- an interface that requires UTF-8 text.
        local bytes = {}
        for i = 1, #entry.name do
            bytes[#bytes + 1] = string.format("%02x", entry.name:byte(i))
        end
        print(entry.index, table.concat(bytes))
    end
end
```

Use `list()` in a worker:

```
local worker, err = babet.workers.spawn([[
local info, list_err = babet.archive.list(worker.args.archive, {
    max_entries = 5000,
})
if not info then error(list_err) end
return {
```

```
    format = info.format,
    compression = info.compression,
    count = info.count,
    duplicates = info.duplicates,
    conflicts = info.conflicts,
}
]], { archive = "backup.tar.zst" })
assert(worker, err)

local joined, result = worker:join()
assert(joined, result)
print(result.format, result.count)
```

The function and all fields behave identically in the main Lua state, folder mode, embedded mode, and workers.

What `list()` validates — and what it does not guarantee

For ZIP, Babet validates the structure required to open and read the central directory, announced counts and sizes, paths, and limits. It does not read every compressed file payload, so a payload CRC failure may remain invisible here.

For TAR and compressed TAR, reaching the next header requires consuming each entry's data; Babet therefore detects many truncations, invalid headers, and outer-stream errors. This complete read does not change the API's role: `list()` returns an inventory, whereas `archive.test()` turns full consumption and aggregate safety rules into a strict verdict.

A technically readable archive containing unsafe paths, links, or special objects is deliberately returned with diagnostics. An unknown format, an unreadable structure, an exceeded limit, or a truncated TAR stream returns `(nil, message)`.

`babet.archive.read`

```
local data, err = babet.archive.read(
    archive,
    raw_name_or_index
    [, opts]
)
```

`read()` loads **one regular file** into a binary Lua string without creating a destination, temporary file, or directory. It content-detects the same formats as `list()`: ZIP, plain TAR, gzip TAR, xz TAR, bzip2 TAR, and zstd TAR.

Success returns exactly `(data, nil)`. `data` may be empty and may contain NUL bytes or non-UTF-8 text. Failure returns exactly `(nil, message)`.

Selection by raw name or index

The second argument has two strict forms:

- a **string** matches the exact raw name byte for byte and case-sensitively;
- an **integer** selects the one-based index exposed as `list().entries[i].index`.

A float such as `2.0`, boolean, or table is never silently converted to an index. A name cannot contain a NUL byte. GNU long names and TAR pax `path` records are the decoded raw names already exposed by `list()`.

Direct lookup by name:

```
local manifest, err = babet.archive.read(
    "package.tar.zst",
```

```
    "manifest.json"
)
assert(manifest, err)
```

Lookup through an index without assuming archive order:

```
local info, err = babet.archive.list("package.zip")
assert(info, err)

local target
for _, entry in ipairs(info.entries) do
    if entry.name == "assets/logo.bin" then
        target = entry.index
        break
    end
end
assert(target, "assets/logo.bin is missing")

local logo
logo, err = babet.archive.read("package.zip", target)
assert(logo, err)
```

Name lookup is rejected when the same raw name occurs more than once: Babet never silently picks the first or last occurrence. After inspecting `duplicate` and `duplicate_of`, an explicit index can read the intended occurrence:

```
local info, err = babet.archive.list("duplicated.zip")
assert(info, err)

for _, entry in ipairs(info.entries) do
    if entry.name == "settings.json" then
        print(entry.index, entry.duplicate, entry.duplicate_of)
    end
end

-- Explicitly select the second occurrence.
local data
data, err = babet.archive.read("duplicated.zip", 7)
assert(data, err)
```

An index only disambiguates the entry being read. It never removes, sorts, or rewrites archive members.

Unsafe names and rejected types

Because `read()` never uses the name as a destination path, a regular file named `../config`, `/absolute`, or with a non-UTF-8 name can be read by its raw name or index. No normalisation or silent "repair" occurs. A script must inspect `valid_utf8`, `safe_path`, `duplicate`, and `conflict` through `list()` before reusing that name in another context:

```
local info, err = babet.archive.list("untrusted.zip")
assert(info, err)

local entry = assert(info.entries[1])
print(entry.name, entry.valid_utf8, entry.safe_path)

local data
data, err = babet.archive.read("untrusted.zip", entry.index)
assert(data, err)
```

This tolerance is safe only because no write occurs. The strict confinement rules for `extract()` and `extractFile()` remain unchanged.

Directories, symlinks, hard links, FIFOs, sockets, devices, unknown types, encrypted ZIP entries, unsupported ZIP methods, and sparse TAR files are rejected. `read()` never follows a link stored in an archive.

The `max_size` memory limit

`read()` adds one dedicated option:

Option	Default	Accepted maximum	Effect
`max_size`	`8 * 1024 * 1024`	`256 * 1024 * 1024`	maximum byte count returned in the Lua string

`max_size` must be a real Lua integer from `1` through `268435456`. The limit is inclusive: an entry producing exactly `max_size` bytes is accepted.

```lua
local config, err = babet.archive.read(
    "package.tar.gz",
    "config.json",
    { max_size = 1024 * 1024 }
)
assert(config, err)
```

Babet rejects an announced size above `max_size` early, then enforces the same limit in the callback receiving actually decompressed bytes. A dishonest ZIP that announces a small size but produces more is therefore stopped before the bounded buffer can be overrun. The count covers binary bytes, not Unicode characters.

`max_size` does not replace `max_entry_size`:

- `max_size` bounds only the requested Lua value;
- `max_entry_size`, `max_total_size`, `max_entries`, `max_path_length`, `max_total_name_bytes`, and `max_compression_ratio` still protect the whole archive.

Combined example for an untrusted archive:

```lua
local manifest, err = babet.archive.read(
    "upload.tar.xz",
    "manifest.json",
    {
        max_size = 512 * 1024,
        max_entries = 2000,
        max_entry_size = 64 * 1024 * 1024,
        max_total_size = 512 * 1024 * 1024,
        max_path_length = 8192,
        max_total_name_bytes = 2 * 1024 * 1024,
        max_compression_ratio = 200,
    }
)
assert(manifest, err)
```

Integrity and archive consumption

For ZIP, Babet scans archive metadata and then fully inflates the selected entry through miniz. Unsupported methods, decompression failures, produced-size errors, and CRC failures make the read fail.

For TAR, the first pass inspects and consumes the entire archive. The second pass compares each header against the inspected plan, forwards only the chosen file blocks into bounded memory, and consumes every other entry to the end. The additional gzip and zstd stream checks remain active. Corruption or truncation elsewhere in the TAR can therefore fail a request for a small file.

`read()` is not an alias for `test()`: it fully validates the selected data and the structures needed to read it, while `test()` inflates every ZIP payload and also applies the aggregate safety verdict to every entry.

No partial payload is ever returned. A missing or ambiguous selector, a refused entry type, a limit violation, or an integrity failure all return exactly `(nil, message)`. Retrying by index is appropriate only when a duplicate raw name made name lookup ambiguous; it never bypasses type, size, or integrity checks.

Worker use

The API is available in workers. Ordinary UTF-8 text can be returned directly. For binary data containing NUL or non-UTF-8 bytes, process it inside the worker or return a transferable result such as its size and CRC:

```
local worker, err = babet.workers.spawn([[
local data, read_err = babet.archive.read(
    worker.args.archive,
    worker.args.index,
    { max_size = worker.args.max_size }
)
if not data then error(read_err) end
return {
    size = #data,
    crc32 = babet.crc32(data),
}
]], {
    archive = "assets.tar.bz2",
    index = 3,
    max_size = 4 * 1024 * 1024,
})
assert(worker, err)

local joined, result = worker:join()
assert(joined, result)
print(result.size, result.crc32)
```

That return-value restriction belongs to worker serialisation, not to `archive.read()` itself: the read remains fully binary-safe.

`babet.archive.test`

```
local result, err = babet.archive.test(archive [, opts])
```

`test()` fully verifies an archive **without extracting, creating, or writing anything**. It accepts the same content-detected formats as `list()`: ZIP, plain TAR, gzip TAR, xz TAR, bzip2 TAR, and zstd TAR. It accepts exactly the same six anti-bomb limits documented above and rejects every other option.

The contract is deliberately strict: success means that the archive is both **technically valid** and **entirely safe under Babet's extraction rules**. A readable archive containing an unsafe path, duplicate, collision, symlink, hard link, sparse file, special object, encrypted entry, or unsupported ZIP method therefore returns `(nil, message)`. `test()` does not return a warnings table or `safe`

= false; use list() when a suspicious archive must be inventoried and diagnosed without being accepted.

The source path follows the same contract as list(): a symlink to a regular file is accepted, while directories, FIFOs, sockets, and devices are rejected without blocking. The opened file is pinned by descriptor for the whole test. The filename extension never chooses the backend.

Technical verification

For ZIP, Babet checks, among other things:

- EOCD and ZIP64 metadata, rejecting split and multi-volume archives;
- the central directory and declared limits;
- the local header of **every** entry, including empty files and directories;
- local/central filename identity, flags, method, sizes, CRC, local ZIP64 fields, and signed or unsigned data descriptors;
- every local range, overlap between local entries, and overlap with the central directory;
- complete decompression of every non-directory entry, the actual output size, and its CRC.

For TAR, the reader consumes all entry data and checks headers, available TAR checksums, sizes, truncation, padding, and trailing data. For gzip-, xz-, bzip2-, or zstd-compressed TAR, the compression stream is also consumed to its end; integrity errors, truncated members or frames, and foreign trailing bytes are rejected according to the codec and backend guarantees.

A correctly formed empty archive is valid. A read error, unknown format, ambiguous structure, exceeded limit, or disagreement between metadata and data returns (nil, message).

Safety verification

After technical validation, Babet applies the same rules as real extraction to entries in archive order:

- a non-empty relative path using only /, with no . or .., duplicate separators, backslash, drive prefix, or NUL byte;
- a maximum extractable path length of 4096 bytes;
- ordinary regular files and directories only;
- no symlink, hard link, FIFO, socket, device, unsupported object, or sparse file;
- no exact duplicate, output-path duplicate, or file/directory conflict;
- no encrypted ZIP entry or unsupported ZIP method.

The first diagnostic is deterministic: Babet keeps archive order and stops at the first failing check in the phases above. Success therefore guarantees that files + directories == entries.

Returned structure

```
{
    format = "tar",
    compression = "zstd",
    entries = 42,
    files = 35,
    directories = 7,
    total_size = 12345678,
    archive_size = 3456789,
    total_name_bytes = 812,
    zip64 = nil,
}
```

- format is "zip" or "tar";
- compression is "none", "gzip", "xz", "bzip2", or "zstd";
- entries, files, and directories count verified entries;
- total_size is the sum of expanded regular-file sizes;

- `archive_size` is the byte size of the pinned archive file;
- `total_name_bytes` is the sum of raw entry-name lengths;
- `zip64` is a boolean for ZIP and `nil` for TAR.

The function returns exactly `(result, nil)` on success and `(nil, message)` on failure.

Usage examples

Simply verify an archive before keeping it:

```
local result, err = babet.archive.test("upload.zip")
if not result then
    io.stderr:write("archive rejected: ", err, "\n")
    return 1
end

print(result.format, result.entries, result.total_size)
```

Verify a compressed TAR even when its extension is misleading:

```
local result, err = babet.archive.test("snapshot.bin")
assert(result, err)
assert(result.format == "tar")
assert(result.compression == "zstd")
```

Tighten only the entry count:

```
local result, err = babet.archive.test("upload.zip", {
    max_entries = 500,
})
assert(result, err)
```

Tighten only the per-file size:

```
local result, err = babet.archive.test("upload.zip", {
    max_entry_size = 16 * 1024 * 1024,
})
assert(result, err)
```

Tighten only the total size:

```
local result, err = babet.archive.test("upload.zip", {
    max_total_size = 128 * 1024 * 1024,
})
assert(result, err)
```

Tighten only entry-name length:

```
local result, err = babet.archive.test("upload.zip", {
    max_path_length = 4096,
})
assert(result, err)
```

Tighten only the cumulative name budget:

```
local result, err = babet.archive.test("upload.zip", {
    max_total_name_bytes = 1024 * 1024,
})
assert(result, err)
```

Tighten only the compression ratio:

```
local result, err = babet.archive.test("upload.zip", {
    max_compression_ratio = 100,
```

```
})
assert(result, err)
```

Combine all limits for an untrusted archive:

```
local result, err = babet.archive.test("upload.zip", {
    max_entries = 500,
    max_entry_size = 16 * 1024 * 1024,
    max_total_size = 128 * 1024 * 1024,
    max_path_length = 4096,
    max_total_name_bytes = 1024 * 1024,
    max_compression_ratio = 100,
})
assert(result, err)
```

Distinguish diagnostic inspection from a strict verdict:

```
local info, list_err = babet.archive.list("upload.zip")
assert(info, list_err)

local verified, test_err = babet.archive.test("upload.zip")
if not verified then
    print("contents visible but archive rejected:", test_err)
end
```

Use `test()` in a worker:

```
local worker, err = babet.workers.spawn([[
local result, test_err = babet.archive.test(worker.args.archive, {
    max_entries = 2000,
    max_total_size = 512 * 1024 * 1024,
})
if not result then error(test_err) end
return result
]], { archive = "backup.tar.zst" })
assert(worker, err)

local joined, result = worker:join()
assert(joined, result)
print(result.format, result.compression, result.entries)
```

The return contract and behaviour are identical in folder mode, embedded mode, embedded-via-`PATH` mode, and workers.

`babet.archive.extract`

```
local result, err = babet.archive.extract(
    archive,
    destination
    [, opts]
)
```

`archive` may be a ZIP, plain TAR, gzip-compressed TAR, xz-compressed TAR, bzip2-compressed TAR, or zstd-compressed TAR. Format and compression are detected from contents rather than the filename extension.

Accepted options:

Option	Default	Behaviour
`overwrite`	`false`	allows atomic replacement of an existing regular file
`dry_run`	`false`	previews extraction and verifies selected data without modifying the filesystem
`preserve_permissions`	`false`	preserves ordinary Unix `rwx` bits when available
`include`	none	dense array of safe globs selecting normalised internal paths
`exclude`	none	dense array of safe globs removed after inclusion; exclusion always wins
`max_entries`	`10000`	maximum entries scanned; hard ceiling `100000`
`max_entry_size`	`256 * 1024 * 1024`	maximum expanded size of one entry; hard ceiling 8 GiB
`max_total_size`	`1024 * 1024 * 1024`	maximum sum of announced sizes; hard ceiling 64 GiB
`max_path_length`	`64 * 1024`	maximum raw-name length; hard ceiling 1 MiB
`max_total_name_bytes`	`64 * 1024 * 1024`	maximum sum of name lengths; hard ceiling 64 MiB
`max_compression_ratio`	`1000`	maximum expanded/compressed ratio, from 1 through `1000000000`

Booleans must be real Lua booleans, integer limits must be strict positive Lua integers, and `max_compression_ratio` must be finite. Unknown keys, excess arguments, and creation-only options are rejected. `dry_run`, `include`, and `exclude` are accepted by `extract()` only, not by `list()`, `read()`, `test()`, or `extractFile()`.

Safe-glob selection

`include` and `exclude` reuse exactly the same bounded engine as `archive.create()`. There is no second matcher and no extraction-specific glob dialect.

An absent or empty `include` list preserves historical behaviour and initially selects every entry. A non-empty `include` list retains an entry when at least one pattern matches. `exclude` is applied afterwards and always wins, even when the same path also matches `include`.

Patterns are anchored against the entry's complete **normalised internal path**, with `/` as the separator. Matching is byte-oriented and case-sensitive:

- `*` matches zero or more bytes except `/`;
- `**` matches zero or more bytes including `/`;
- `?` matches exactly one byte except `/`;
- `\x` quotes the following byte `x`.

A directory is tested both as `path` and `path/`. Excluding `src/generated` or `src/generated/**` therefore excludes that directory and prunes its complete subtree, even when the archive has no explicit

directory entry. Only parents needed by retained files are created; an unselected explicit directory is not created. An empty directory is restored only when its explicit entry is selected.

The same bounds as creation apply: each pattern is limited to 4096 bytes; `include` and `exclude` together are capped at 256 patterns and 256 KiB of pattern text, one million pattern evaluations, and a fixed 100,000,000-cell matching-work budget per call. The normalised graph used to propagate directory exclusions is additionally capped at 100000 unique directory paths and 64 MiB of cumulative path text. Exceeding a limit fails before publication. Pattern order does not change selection.

Read-only simulation with `dry_run`

With `dry_run = true`, Babet builds the same plan as a real extraction but does not call any creation, write, chmod, rename, unlink, or publication operation. The archive is pinned and scanned, the same limits and filters are applied, selected entries are validated, and the destination is traversed read-only with `openat`/`fstatat` and `O_NOFOLLOW`. No destination root, implicit parent, temporary file, or final entry is created.

Destination checks retain the real extraction policy:

- a missing root is accepted and reported by `would_create_destination = true`;
- an existing root or parent that is a symlink or a non-directory object is rejected;
- a missing selected entry counts in `would_create`;
- an existing regular file counts in `would_overwrite` only with `overwrite = true`; with `overwrite = false`, the call fails exactly like real extraction;
- an explicitly selected directory that already exists counts in `would_skip`, because real extraction preserves it and does not chmod it;
- a file/directory conflict or a symlink final target remains an error.

`would_create`, `would_overwrite`, and `would_skip` count only **explicit selected archive entries**. Their sum therefore equals `entries`. The destination root and required implicit parents are not included; the root has its own boolean. When active filters select nothing, behaviour remains identical to real extraction: the destination is not inspected and `would_create_destination` is `false`. Conversely, an unfiltered empty archive previews creation of its root when that root is missing.

The simulation genuinely verifies the data that would be extracted. For ZIP, each selected regular file is inflated to a null consumer and its CRC is checked; unselected ZIP payloads remain untouched, as in real selective extraction. For TAR, the second pass consumes the complete plain or compressed stream and compares every header with the initial plan without forwarding bytes to the filesystem. `archive.test()` remains the API for checking every ZIP payload, including ones that extraction would skip.

The result is a **snapshot**. Another process may change the destination after `dry_run` returns, so the preview is never a guarantee that a later extraction will succeed or produce the same counters. Real extraction repeats all checks and pins the required descriptors before publication.

Validation, skipped entries, and integrity

Babet first pins the archive file and scans **every entry and the metadata needed for the extraction plan**, applying global limits before opening the destination. Selection is then computed on normalised names. Only retained entries participate in the output plan:

- a duplicate, file/directory collision, link, or special object found only outside the selection does not block extraction;
- a selected unsafe entry is still rejected before any write;
- with a non-empty `include` list, an unsafe path has no normalised form and cannot match, so it remains unselected;
- with `exclude` only, historical “select everything” behaviour remains active: an unsafe path cannot be sanitised by an exclusion pattern and still fails the call;
- existing destination collisions outside the selection are neither replaced nor modified.

This policy permits extraction of a safe part of a mixed archive without presenting `exclude` as a path-repair mechanism. Entry, size, name, and compression-ratio limits still apply to the complete scanned archive, not just the retained files.

For ZIP, skipped payloads are not inflated, so their CRC is not checked by selective extraction. For TAR, the reader must advance through the stream to reach later headers, including through an outer compressed stream, but `extract()` is not the complete verification API. Call `archive.test()` first when integrity of **every** entry, including skipped data, must be guaranteed.

Before publication, Babet rejects retained non-extractable entries, duplicate retained outputs, retained file/directory conflicts, and incompatible existing destination objects. Selected files are staged in the secure destination and then published atomically. For TAR, the same pinned file is reread from the beginning and every header is compared with the first-pass plan.

When active filters select nothing, the call succeeds without creating the destination. Without filters, an empty archive keeps historical behaviour and creates the destination root directory.

Return value

The function returns exactly `(result, nil)` or `(nil, message)`:

```
{
    entries = 28,     -- entries actually selected
    files = 24,       -- selected regular files
    directories = 4, -- selected explicit directory entries
    skipped = 15,     -- archive entries not selected
    bytes = 987654,   -- expanded bytes of selected files
    path = "restore",
}
```

With `dry_run = true`, the same table additionally contains:

```
{
    dry_run = true,
    would_create = 20,
    would_overwrite = 3,
    would_skip = 5,
    would_create_destination = false,
}
```

These additional fields are absent from a real extraction so its historical contract remains unchanged. `directories` does not count implicit parents created to reach a file. `entries == files + directories` after a successful extraction because selected links and special objects are rejected. In a successful simulation, `would_create + would_overwrite + would_skip == entries`. `skipped` is `0` when no filter is active.

Examples

Preview extraction to a missing destination:

```
local plan, err = babet.archive.extract("backup.zip", "restore", {
    dry_run = true,
})
assert(plan, err)
assert(plan.dry_run == true)
assert(plan.would_create_destination == true)
assert(not babet.fileExists("restore"))
```

Preview allowed replacements in an existing destination:

```lua
local plan, err = babet.archive.extract("backup.tar.zst", "restore", {
    dry_run = true,
    overwrite = true,
    preserve_permissions = true,
})
assert(plan, err)
print(plan.would_create, plan.would_overwrite, plan.would_skip)
```

Without `overwrite = true`, an existing final file remains an error:

```lua
local plan, err = babet.archive.extract("backup.zip", "restore", {
    dry_run = true,
})
assert(plan == nil and type(err) == "string")
```

Combine simulation, selection, and limits:

```lua
local plan, err = babet.archive.extract("project.tar.xz", "output", {
    dry_run = true,
    include = { "src/**", "docs/**", "README.md" },
    exclude = { "src/generated/**", "**/*.tmp" },
    overwrite = true,
    preserve_permissions = true,
    max_entries = 20000,
    max_total_size = 2 * 1024 * 1024 * 1024,
})
assert(plan, err)
assert(plan.would_create + plan.would_overwrite + plan.would_skip
    == plan.entries)
assert(not babet.fileExists("output"))
```

Extract only two precise areas:

```lua
local result, err = babet.archive.extract("project.tar.zst", "output", {
    include = {
        "src/**",
        "README.md",
    },
})
assert(result, err)
print(result.entries, result.skipped)
```

Exclude temporary files while retaining everything else:

```lua
local result, err = babet.archive.extract("backup.zip", "restore", {
    exclude = {
        "**/*.tmp",
        "cache/**",
    },
})
assert(result, err)
```

Combine inclusion and exclusion, with exclusion taking priority:

```lua
local result, err = babet.archive.extract("project.tar.xz", "output", {
    include = {
        "src/**",
        "docs/**",
        "README.md",
    },
    exclude = {
```

```
        "src/generated/**",
        "**/*.tmp",
    },
    overwrite = false,
    preserve_permissions = true,
    max_total_size = 2 * 1024 * 1024 * 1024,
})
assert(result, err)
```

Use `?` and quote a literal `*` byte:

```
local result, err = babet.archive.extract("assets.zip", "assets", {
    include = {
        "icons/icon-?.png", -- icon-a.png, but not icon-large.png
        "docs/file\\*.txt", -- a name that literally contains an asterisk
    },
})
assert(result, err)
```

Handle an empty selection without creating a directory:

```
local result, err = babet.archive.extract("package.zip", "unused", {
    include = { "does-not-exist/**" },
})
assert(result, err)
assert(result.entries == 0 and result.skipped > 0)
assert(not babet.fileExists("unused"))
```

Use filters inside a worker:

```
local worker, err = babet.workers.spawn([[
local result, extract_err = babet.archive.extract(
    worker.args.archive,
    worker.args.destination,
    {
        include = { "docs/**", "README.md" },
        exclude = { "docs/drafts/**" },
    }
)
if not result then error(extract_err) end
return result
]], {
    archive = "package.tar.gz",
    destination = "documentation",
})
assert(worker, err)

local joined, result = worker:join()
assert(joined, result)
print(result.files, result.skipped)
```

Preview inside a worker without creating the destination:

```
local worker, err = babet.workers.spawn([[
local plan, extract_err = babet.archive.extract(
    worker.args.archive, worker.args.destination, {
        dry_run = true,
        include = { "manifest.json", "docs/**" },
    })
if not plan then error(extract_err) end
```

```
return plan
]], {
    archive = "package.tar.gz",
    destination = "preview-only",
})
assert(worker, err)

local joined, plan = worker:join()
assert(joined, plan)
assert(plan.dry_run == true)
assert(not babet.fileExists("preview-only"))
```

The contract, limits, and selection are identical in folder mode, embedded mode, embedded-via-`PATH` mode, and workers.

`babet.archive.extractFile`

```
local result, err = babet.archive.extractFile(
    archive,
    entry,
    destination
    [, opts]
)
```

`archive` may be a ZIP, plain TAR, gzip-compressed TAR, xz-compressed TAR, bzip2-compressed TAR, or zstd-compressed TAR; detection uses the file contents, not the extension. ZIP stays on miniz. TAR uses the same pinned-source and two-pass libarchive reader as complete extraction.

`entry` is the exact **raw name** exposed by `list()`. Lookup is case-sensitive and fails when the name is absent or occurs more than once. For TAR, GNU long names and pax `path` records are matched after libarchive has decoded the TAR headers, exactly as they appear in `list().entries[i].name`.

Only a regular-file entry can be selected. Sparse TAR files, symlinks, hard links, directories, and special types are refused when selected. The entry path is not reproduced: the content is written exactly to the caller-supplied `destination` path.

```
local result, err = babet.archive.extractFile(
    "package.tar",
    "assets/logo.bin",
    "cache/current-logo.bin",
    { overwrite = true }
)
```

Result:

```
{
    bytes = 4096,
    path = "cache/current-logo.bin",
    entry = "assets/logo.bin",
}
```

Anti-bomb limits are still computed over the whole archive. For TAR, the first pass consumes and validates every data stream. The second pass compares every header with the inspected plan and consumes the archive to its end before the selected temporary file is published. Unselected regular-file data is discarded, and an unrelated unsafe path, special type, or sparse file does not prevent selecting a safe regular file. Malformed or truncated data anywhere in the archive still makes the operation fail without publishing the destination.

Path rules

For complete extraction, every entry name must satisfy all of these rules:

- between 1 and 4096 bytes;
- no NUL byte;
- no `\`;
- no leading `/`;
- no drive prefix such as `C:` in the first component;
- no empty component (`a//b`);
- no `.` or `..` component;
- a trailing `/` only for an actual directory entry.

Paths remain relative to the destination. Babet never converts a Windows name into a Linux path and never silently "cleans" traversal: the archive is rejected. With `extractFile()`, the same rules apply to the selected entry name; unselected unsafe names do not become destination paths.

`read()` is the deliberate exception: because it writes nothing, it can select a regular file with an unsafe name by raw name or index. It never normalises that name or uses it as a filesystem path.

Duplicates are detected after removing the trailing `/` from directory names. An archive containing two identical outputs, or `node` as a file followed by `node/child`, is rejected before extraction.

Destination protection

Security does not rely only on string comparison. Babet walks the destination component by component through directory descriptors and refuses to follow symlinks:

- a symlink in the destination root path is rejected;
- a symlinked parent below the destination is rejected;
- a symlink final target is rejected, even with `overwrite = true`;
- a file may only replace a regular file;
- a directory entry may only use an existing or newly created directory.

Existing directories are accepted when they are real directories and remain unchanged. Babet does not alter their permissions.

The destination path itself is trusted script input and may be absolute or relative. The `..` rules above apply to untrusted names stored inside the archive.

Atomic publication and cleanup

`create()` writes the complete ZIP, plain TAR, gzip-compressed TAR, or xz-compressed TAR to a unique mode-`0600` temporary file in the destination directory. Its short internal name does not repeat the output basename, so a valid destination close to the `NAME_MAX` limit remains creatable. After the miniz ZIP central directory or libarchive TAR stream is finalized, Babet flushes and `fsync`s the file, changes it to `0644`, publishes it atomically, and `fsync`s the parent directory. Without overwrite it uses a no-clobber hard link publication; with overwrite it uses same-directory `rename`. Every error before publication removes the temporary and leaves the previous destination unchanged. If the final parent-directory `fsync` itself fails, the new archive has already been atomically published but the call returns an error because crash durability could not be confirmed.

Each extracted file is progressively written to a unique temporary file in its final directory with mode `0600`. Temporary names are selected so they can never collide with an output declared by the archive. Writes are bounded by the announced size. For ZIP, miniz validates decompression and CRC. For TAR, libarchive provides streaming blocks and Babet validates their order, bounds, and final announced size. For gzip-wrapped TAR, Babet additionally validates every gzip member's CRC and ISIZE with zlib before publication. The temporary file then receives its final permissions.

All files of a complete extraction are prepared before publication starts. A read, decompression, or CRC error therefore publishes no file. `extractFile()` likewise keeps its selected file temporary

until the complete archive has passed the verified read. Temporary files are removed, as are newly created empty directories **below** the destination. Destination-root path components that had to be created may remain present.

Publication is atomic **per file**:

- without overwrite, Babet uses creation that fails if the target exists;
- with overwrite, the same-directory temporary atomically replaces the target regular file.

The complete operation is not a multi-file transaction. A very late system error during publication or final directory permission changes may happen after previous files have been published. Those files may remain, and an old file already replaced is not restored.

For extraction, atomic publication prevents a partially visible file but does not promise crash durability: extracted files and directories are not fully `fsynced`. `create()` follows the stronger file-and-parent synchronization sequence described above.

Permissions

Defaults:

```
files                                      0644
final directories                          0755
temporary files                            0600
archive directories during preparation    0700
created destination-path components        0755 (subject to umask)
```

With `preserve_permissions = true`, Babet preserves only the nine ordinary owner/group/other bits (`0777`) from Unix modes announced by the ZIP or TAR. setuid, setgid, and sticky bits are always stripped.

Permissions of a newly created directory are applied only after its contents have been extracted, so a restrictive final mode such as `0000` cannot block the current operation. Existing directories are never chmodded.

UIDs, GIDs, ACLs, extended attributes, and timestamps are not restored.

Supported entry types

Type	`create()`	`list()`	`read()`	`extract()`	`extractFile()`
regular file	supported	inspected	supported, except sparse TAR	supported, except sparse TAR	supported, except sparse TAR
directory	supported when enabled	inspected	not selectable	supported	not selectable
sparse TAR file	never created	identified in TAR	rejected	rejected	rejected if selected
symlink	rejected if selected	identified	rejected	rejected	rejected
hard link	never created	identified in TAR	rejected	rejected	rejected if selected
FIFO, socket, device, unknown type	rejected if selected	identified when exposed	rejected	rejected	rejected
encrypted entry	never created	identified in ZIP	rejected	rejected	rejected
unknown ZIP compression method	never created	identified	rejected	rejected	rejected

During creation, a symlink or special filesystem object that is removed by the filters is ignored rather than opened. Directory entries are emitted only when `include_directories = true`.

For regular-file creation, ZIP, gzip, and xz accept compression levels `0` through `9`; bzip2 accepts `1` through `9`; zstd accepts `0` through `19`; plain TAR stores the source bytes without compression.

A ZIP entry that resembles a hard link but is encoded as a regular file is treated as an independent regular file; no link is created.

Errors and known limitations

Common failures include:

- missing or unsafe source directory, source symlink, non-UTF-8 source name, unsupported source object, invalid creation format, ZIP-only option used for TAR, or out-of-range ZIP timestamp;
- output archive inside the source, unsafe destination parent, or destination symlink;
- missing, unreadable, non-regular, unsupported, or malformed archive file;
- truncated TAR data, non-TAR trailing data, a non-regular TAR entry carrying a payload, or a change detected between inspection and extraction;
- inconsistent metadata or invalid CRC;
- exceeded anti-bomb limit;
- `max_size` exceeded during an in-memory read;
- an ambiguous duplicate name passed to `read()` without an explicit index;
- absolute path, traversal, duplicate, or path conflict;
- encrypted entry, symlink, or unsupported type;
- existing destination without `overwrite = true`;
- symlink or unsafe type in the destination;
- insufficient disk space, permissions, or another I/O error.

The module is synchronous: a call returns only after creation, inspection, reading, or extraction has completed. It does not expose progress, cancellation, or a timeout yet.

All six public functions – `create()`, `list()`, `read()`, `test()`, `extract()`, and `extractFile()` – share the same C++ exception boundary. An internal allocation failure becomes `(nil, "archive: out of memory")`; another internal exception becomes `(nil, "archive: internal failure")` or `(nil, "archive: unknown internal failure")`. No C++ exception crosses the Lua boundary.

The archive file must not be modified concurrently. ZIP truncation or modification will normally cause a read, decompression, or CRC error. TAR is streamed through libarchive; concatenated TAR streams are scanned to their end, and truncated or non-TAR trailing data is rejected. Neither backend provides a concurrent snapshot. The destination must not be actively reorganised by another process either. Traversal rejects symlinks and rechecks types before publication, but an actor that continuously renames directories can make the operation fail and turn temporary-file cleanup into a best-effort action.

Plain, gzip-, xz-, bzip2-, and zstd-compressed TAR creation is supported. Standalone compression streams remain intentionally outside this archive API and are handled by `babet.compression`.

Multi-disk archives are not a supported target. ZIP64 is accepted when miniz can read it, while still being subject to the configured limits.

`babet.base64` — binary Base64 encoding and decoding

The `babet.base64` module implements the RFC 4648 Base64 alphabets without starting an external program. Inputs and outputs are **binary** Lua strings: NUL bytes, non-UTF-8 data, and every value from `0` through `255` are preserved exactly.

The decoder is deliberately strict. It validates the selected alphabet, padding placement, final-quantum length, and unused bits in the last symbol. A non-canonical representation is therefore not silently accepted.

Module contents

- API
- Standard encoding
- URL-safe alphabet
- Padding
- Strict decoding
- ASCII whitespace
- Output limit
- Binary data and files
- Using the module in a worker
- Error contract
- Limits and design choices

API

Function	Contract
`babet.base64.encode(data [, opts])`	Returns `text, nil` or `nil, err`
`babet.base64.decode(text [, opts])`	Returns `data, nil` or `nil, err`

`data` and `text` must be actual Lua strings. Numbers are never implicitly converted to text.

Encoding options:

```
{
    url_safe = false,
    padding = true,
}
```

Decoding options:

```
{
    url_safe = false,
    allow_unpadded = false,
    ignore_whitespace = false,
    max_output = nil,
}
```

Unknown options are rejected. Boolean options require actual Lua booleans; `max_output` must be a non-negative integer.

Standard encoding

Without options, `encode()` uses the standard Base64 alphabet: letters, digits, `+`, and `/`, with canonical `=` padding when the final group contains fewer than three bytes.

```
local encoded, err = babet.base64.encode("hello")
assert(encoded, err)
assert(encoded == "aGVsbG8=")
```

The classic RFC 4648 vectors are preserved:

```
assert(babet.base64.encode("")       == "")
assert(babet.base64.encode("f")      == "Zg==")
assert(babet.base64.encode("fo")     == "Zm8=")
assert(babet.base64.encode("foo")    == "Zm9v")
assert(babet.base64.encode("foob")   == "Zm9vYg==")
assert(babet.base64.encode("fooba")  == "Zm9vYmE=")
assert(babet.base64.encode("foobar") == "Zm9vYmFy")
```

`encode()` adds no line breaks and does not produce 76-column MIME Base64.

URL-safe alphabet

`url_safe = true` selects the RFC 4648 URL/file-name-safe alphabet: `-` replaces `+`, and `_` replaces `/`.

```
local binary = string.char(251, 255)
local encoded, err = babet.base64.encode(binary, {
    url_safe = true,
})
assert(encoded, err)
assert(encoded == "-_8=")
```

Decoding must select the same alphabet:

```
local decoded, err = babet.base64.decode("-_8=", {
    url_safe = true,
})
assert(decoded, err)
assert(decoded == string.char(251, 255))
```

The alphabets are not mixed automatically:

```
assert(babet.base64.decode("-_8=") == nil)
assert(babet.base64.decode("+/8=", { url_safe = true }) == nil)
```

This strict choice prevents the same input from being accepted in multiple forms when a protocol requires one precise alphabet.

Padding

Producing output without =

`padding = false` removes only trailing `=` characters. The useful content is unchanged.

```
local encoded, err = babet.base64.encode("hello", {
    padding = false,
})
assert(encoded, err)
assert(encoded == "aGVsbG8")
```

When the source length is a multiple of three, no padding is needed even with the default option:

```
assert(babet.base64.encode("foo") == "Zm9v")
```

Accepting unpadded input

The decoder rejects an incomplete final group without `=` by default:

```
local decoded, err = babet.base64.decode("aGVsbG8")
assert(decoded == nil)
assert(type(err) == "string")
```

Enable `allow_unpadded` explicitly when the protocol uses this form:

```lua
local decoded, err = babet.base64.decode("aGVsbG8", {
    allow_unpadded = true,
})
assert(decoded, err)
assert(decoded == "hello")
```

`allow_unpadded` does not relax other rules: a one-symbol final group remains truncated, and non-zero unused bits remain invalid.

Combined URL-safe, unpadded example

This form is common in protocol identifiers and some JSON fields:

```lua
local source = string.char(0, 1, 2, 251, 255)

local token, err = babet.base64.encode(source, {
    url_safe = true,
    padding = false,
})
assert(token, err)

local restored
restored, err = babet.base64.decode(token, {
    url_safe = true,
    allow_unpadded = true,
})
assert(restored, err)
assert(restored == source)
```

Strict decoding

The decoder checks, among other rules:

- every character belongs exactly to the selected alphabet;
- = appears only at the end;
- no more than two padding characters are present;
- padded length is a multiple of four;
- an unpadded tail contains two or three symbols, never one;
- unused bits in the final symbol are zero.

These inputs are therefore rejected:

```lua
local invalid = {
    "%%%",   -- characters outside the alphabet
    "AA=A",  -- padding in the middle
    "A===",  -- too much padding
    "Zg=",   -- invalid padded length
    "Zh==",  -- non-zero unused bits
    "Zm9=",  -- non-zero unused bits
}

for _, text in ipairs(invalid) do
    local value, err = babet.base64.decode(text)
    assert(value == nil)
    assert(type(err) == "string")
end
```

Positions reported by `invalid character`, `invalid padding`, or `non-zero trailing bits` errors are one-based byte positions in the original input string.

ASCII whitespace

Whitespace is an invalid character by default:

```
local value, err = babet.base64.decode("Zm9v\nYmFy")
assert(value == nil)
```

`ignore_whitespace = true` ignores the six classic ASCII whitespace bytes: space, tab, line feed, vertical tab, form feed, and carriage return.

```
local value, err = babet.base64.decode(" Zm9v\tYmFy\r\n", {
    ignore_whitespace = true,
})
assert(value, err)
assert(value == "foobar")
```

The option removes no other Unicode or control byte. It also does not permit bad padding or the wrong alphabet.

When whitespace is ignored, an error position still refers to the original string, including whitespace.

Output limit

`max_output` bounds the exact decoded result size. The limit is checked after complete lexical and canonical Base64 validation, then before the decoded output string is allocated.

```
local value, err = babet.base64.decode("Zm9v", {
    max_output = 2,
})
assert(value == nil)
assert(err == "base64: decoded output exceeds max_output")
```

The limit is inclusive:

```
local value, err = babet.base64.decode("Zm9v", {
    max_output = 3,
})
assert(value, err)
assert(value == "foo")
```

`max_output = 0` accepts only empty output:

```
assert(babet.base64.decode("", { max_output = 0 }) == "")
assert(babet.base64.decode("Zg==", { max_output = 0 }) == nil)
```

Set a protocol-appropriate limit for untrusted Base64 values:

```
local png, err = babet.base64.decode(response.value, {
    max_output = 32 * 1024 * 1024,
})
assert(png, err)
```

The limit applies to decoded output, not to Base64 text length.

Binary data and files

Base64 imposes no UTF-8 requirement. Lua strings containing arbitrary bytes are accepted:

```
local original = "\0\1\2abc\255"
local encoded = assert(babet.base64.encode(original))
local decoded = assert(babet.base64.decode(encoded))
assert(decoded == original)
```

To encode a file:

```lua
local file = assert(io.open("screenshot.png", "rb"))
local data = assert(file:read("*a"))
assert(file:close())

local text, err = babet.base64.encode(data)
assert(text, err)
```

To write a decoded value:

```lua
local data, err = babet.base64.decode(text, {
    max_output = 32 * 1024 * 1024,
})
assert(data, err)

local ok
ok, err = babet.writeFileAtomic("screenshot-restored.png", data, {
    overwrite = true,
    permissions = tonumber("600", 8),
})
assert(ok, err)
```

`babet.writeFileAtomic()` publishes the decoded bytes from a private same-directory temporary file and does not expose a partially written image.

Using the module in a worker

The submodule is registered in every worker Lua state:

```lua
local job = assert(babet.workers.spawn([[
    local encoded = assert(babet.base64.encode("hello"))
    local decoded = assert(babet.base64.decode(encoded))
    return decoded == "hello"
]]))

local ok, result = job:join()
assert(ok and result == true)
```

The current `worker.args`, result, and worker-queue transport still uses JSON and rejects strings containing a NUL byte. Base64 can turn binary data into transportable text, at a size cost of roughly one third.

```lua
local encoded = assert(babet.base64.encode(binary_data))
local job = assert(babet.workers.spawn([[
    return assert(babet.base64.decode(worker.args.encoded))
]], { encoded = encoded }))
```

If the decoded result contains a NUL, it cannot be returned directly through the current JSON worker transport. The worker must process it, write it to a file, or re-encode it before returning it.

Error contract

Call errors: Lua exception

Programming errors raise a Lua error:

- wrong argument count;
- non-string data or text;
- non-table options;
- non-string option key;

- unknown option;
- wrongly typed boolean option;
- non-integer or negative `max_output`.

```lua
local ok, err = pcall(function()
    babet.base64.decode("Zm9v", { max_output = -1 })
end)
assert(ok == false)
```

Data errors: `nil, err`

Invalid Base64 or an exceeded limit returns `nil, err`:

```lua
local value, err = babet.base64.decode("%%%")
assert(value == nil)
assert(type(err) == "string")
assert(err:find("base64:", 1, true) == 1)
```

The main message forms are:

- `base64: invalid character at byte N`;
- `base64: invalid padding at byte N`;
- `base64: truncated input`;
- `base64: non-zero trailing bits at byte N`;
- `base64: decoded output exceeds max_output`;
- `base64: out of memory`;
- `base64: encoded output too large` or `decoded output too large`.

The `internal ... mismatch` and `unexpected internal error` diagnostics are defensive guards. They should not occur for ordinary valid or invalid user input; report them as a Babet defect if they appear.

Only depend on documented reasons when code needs to distinguish a limit from invalid data. Allocation wording and positional detail may be enriched later.

Limits and design choices

The module processes a complete string in memory. It does not currently provide:

- streaming encoders or decoders;
- direct file input/output;
- MIME line wrapping;
- automatic standard-versus-URL-safe alphabet detection;
- a permissive mode accepting non-canonical padding or trailing bits.

These exclusions are deliberate. The API stays small, deterministic, and suited to Selenium screenshots, tokens, JSON payloads, small binary resources, and exchanges with external tools.

Compression - standalone gzip, xz, bzip2, and zstd streams

Scope

`babet.compression` compresses or decompresses the bytes of one regular file. It handles **standalone streams**, not archives:

- gzip (`.gz` by convention);
- xz (`.xz`);
- bzip2 (`.bz2`);
- Zstandard (`.zst`).

A standalone stream has no portable entry name, directory tree, permissions, or archive metadata. Use babet.archive for ZIP files or TAR archives, including .tar.gz, .tar.xz, .tar.bz2, and .tar.zst.

API overview

```
local ok, err = babet.compression.compress(
    source, destination, format [, opts]
)

local ok, err = babet.compression.decompress(
    source, destination [, opts]
)
```

Both functions return:

- true, nil on success;
- nil, "message" on an operational or option error.

Wrong required argument types, embedded NUL bytes, and incorrect arity raise a Lua error. Unknown option names are rejected.

compress(source, destination, format [, opts])

source, destination, and format are strict Lua strings. Paths must not be empty or contain an embedded NUL byte.

format must be exactly one of:

"gzip" "xz" "bzip2" "zstd"

The filename extension is not inspected or added automatically. This is valid:

```
local ok, err = babet.compression.compress(
    "payload.bin",
    "payload.data",
    "zstd"
)
```

The optional table accepts:

Field	Type	Default	Meaning
overwrite	boolean	false	Atomically replace an existing regular destination file.
level	Lua integer	format-specific	Explicitly select the compression level.

level is a strict Lua integer. Floats, numeric strings, and out-of-range values are rejected before the source file is opened. Accepted ranges and the values used when the field is absent are:

Format	Minimum	Maximum	Default
gzip	0	9	6
xz	0	9	6
bzip2	1	9	9
zstd	1	22	3

A higher level generally favours compression ratio at the cost of CPU time and sometimes additional memory. For gzip, level `0` creates a valid stream without DEFLATE compression. The highest xz presets can consume substantial memory and therefore must always be selected explicitly.

Example

```
local ok, err = babet.compression.compress(
    "database.dump",
    "database.dump.zst",
    "zstd",
    { overwrite = true, level = 10 }
)

if not ok then
    io.stderr:write(err, "\n")
    return 1
end
```

decompress(source, destination [, opts])

The compression format is detected from the input bytes, not from the extension. Renaming `file.gz` to `file.bin` therefore does not change how it is decoded. Uncompressed data and unsupported formats are rejected.

The optional table accepts:

Field	Type	Default	Meaning
`overwrite`	boolean	`false`	Atomically replace an existing regular destination file.
`max_output_size`	Lua integer	1 GiB	Maximum number of decompressed bytes accepted.

`max_output_size` must be between `1` and `68719476736` bytes (64 GiB). The limit is checked before each write. If it is exceeded, the temporary file is removed and the destination is not published.

Example

```
local ok, err = babet.compression.decompress(
    "database.dump.zst",
    "database.dump",
    {
        overwrite = true,
        max_output_size = 4 * 1024 * 1024 * 1024,
    }
)

if not ok then
    io.stderr:write(err, "\n")
    return 1
end
```

Concatenated streams and integrity

Valid concatenated gzip members, xz streams, bzip2 members, and zstd frames are decoded in order into the same destination file.

The decoders verify the mandatory integrity fields and any optional checksum present in the stream. Babet-generated zstd frames enable the optional content checksum. An external zstd frame without that field remains readable, but the format itself then cannot detect every possible payload bit flip. Babet rejects:

- truncated streams;
- corrupted checksums or frames;
- arbitrary trailing bytes after the final valid member/frame;
- inputs whose magic bytes do not identify a supported format.

Filesystem and publication guarantees

The module follows a fail-closed file policy:

- the source must be a real regular file;
- a symlink source is rejected;
- source and destination parent paths must already exist and are opened component by component with `O_NOFOLLOW`;
- `..` components in those parent paths are rejected;
- a destination symlink or non-regular destination is rejected;
- source and destination must not refer to the same inode, including through a hard link;
- the source descriptor is pinned and its size and timestamps are checked again before publication;
- output is streamed into a mode-`0600` temporary file in the destination directory;
- the completed file is synced, changed to mode `0644`, and atomically published;
- with `overwrite = false`, publication cannot replace a destination that appeared concurrently;
- temporary files are removed on failures before publication.

The module never uses a shell and never derives the destination filename from compressed metadata.

Binary data and memory use

Input and output are byte streams. NUL bytes and arbitrary binary content are preserved.

Compression and decompression use bounded 64 KiB working buffers. The full input or output is not loaded into the Lua state or into one C++ string. `max_output_size` additionally bounds decompression expansion. The xz decoder has a 256 MiB internal memory ceiling and the zstd decoder refuses windows above 128 MiB.

C++ exception boundary

Both public functions enter through the common Lua/C++ exception boundary. This does not change any ordinary result. It prevents an unexpected C++ exception from crossing Lua's C frames:

- allocation failure: `(nil, "compression: out of memory");`
- another standard exception: `(nil, "compression: internal failure");`
- unknown exception: `(nil, "compression: unknown internal failure")`.

The source descriptor and unpublished output remain owned by RAII guards while the operation runs. If such an exception occurs, those descriptors are closed and the same-directory temporary output is removed before the diagnostic is returned.

Workers

`babet.compression` is registered in every worker Lua state. Different workers may process different files concurrently. Normal filesystem collision rules still apply if several workers target the same destination.

Error examples

```
local ok, err = babet.compression.compress(
    "input.bin", "input.bin.gz", "zip"
)
-- nil, "compression: format must be 'gzip', 'xz', 'bzip2', or 'zstd'"

ok, err = babet.compression.decompress(
    "large.gz", "large.bin", { max_output_size = 1024 }
)
-- nil, "compression: decompressed output exceeds opts.max_output_size"
```

A failed operation should always be handled before using the destination.

CURSES — terminal user interfaces

`babet.curses` provides a small UTF-8 terminal UI API backed by statically linked **ncursesw**. It is intended for interactive, full-screen terminal programs while preserving Babet's one-file deployment model.

The first API deliberately stays small: one process-wide screen, no windows, panels, menus, forms, or exposed ncurses pointers/constants.

API overview

```
babet.curses.start()                -- true
babet.curses.stop()                 -- true (also safe when inactive)
babet.curses.clear()                -- true
babet.curses.refresh()              -- true
local rows, cols = babet.curses.size()
babet.curses.move(row, column)      -- true; coordinates are 1-based
babet.curses.write(text)            -- true; text must be valid UTF-8
local key, err = babet.curses.readKey([timeout_seconds])
```

All calls raise a Lua error for invalid arguments or terminal/curses failures. `readKey(timeout)` is the exception for an ordinary timeout: it returns `nil, "timeout"`. A blocking read interrupted without a delivered key may return `nil, "interrupted"`.

Starting and stopping

```
assert(babet.curses.start())
assert(babet.curses.clear())
assert(babet.curses.move(1, 1))
assert(babet.curses.write("Hello é Ω"))
assert(babet.curses.refresh())
local key = babet.curses.readKey()
assert(babet.curses.stop())
```

`start()` requires `stdin` and `stdout` to be the **same TTY**, a non-empty `TERM`, and a usable UTF-8 `LC_CTYPE` locale. It must run on Babet's main thread. Starting a second session raises an error.

A successful `start()` enables the shared signal/terminal Lua hook on demand on the main Lua thread and the calling coroutine. New coroutines inherit their creator's hook; other existing coroutines are not changed. Start curses before creating coroutines that must service events during pure Lua loops. The hook remains installed after `stop()`; see its contract and cost.

`stop()` is idempotent. If an interactive child currently owns the terminal, `stop()` ends the logical curses session **without taking the TTY back from the child**. The normal process handoff code later restores the parent terminal when that child exits or stops.

Normal completion, uncaught Lua errors, controlled C++ exceptions, and default `SIGINT`/`SIGTERM`/`SIGHUP` termination restore terminal mode before Babet exits. A fatal crash or `SIGKILL` cannot provide this guarantee; `reset` is the usual manual recovery command for a damaged terminal.

In the CLI and generated executables, `os.exit(...)` also restores curses, even without requesting Lua state closure. Exit status codes are preserved. See the exit contract for finalizers, active workers and native callbacks.

Screen operations

`size()` returns `(rows, columns)` as positive integers.

`move(row, column)` uses **1-based** coordinates. Positions outside the current screen raise an error.

`write(text)` writes valid UTF-8 at the current cursor position. Embedded NUL bytes and invalid UTF-8 are rejected. Drawing is not automatically flushed; call `refresh()` when the update should become visible.

`clear()` erases the standard screen and `refresh()` publishes pending changes.

Keyboard input

```lua
local key, err = babet.curses.readKey(0.5)
if not key and err == "timeout" then
    -- no input during 500 ms
elseif key == "up" then
    -- arrow key
elseif key == "resize" then
    local rows, cols = babet.curses.size()
end
```

Ordinary characters are returned as UTF-8 Lua strings. Special keys are stable symbolic names rather than ncurses numeric constants:

`up`, `down`, `left`, `right`, `home`, `end`, `page_up`, `page_down`, `backspace`, `delete`, `insert`, `enter`, `resize`, and `f1` through `f63`. An otherwise recognized ncurses special key is returned as `"special"`.

`SIGWINCH` is coalesced and handled on the main thread. After the terminal size has been applied, `readKey()` reports the symbolic `"resize"` event.

If a signal callback calls `curses.stop()` during `readKey`, the read returns `nil, "interrupted"`, including without a timeout. It no longer accesses the released screen, even if a key was read immediately before the callback.

Interactive child processes

Babet keeps **one interactive terminal owner at a time**. During a curses session this works transparently on the main thread:

```lua
assert(babet.curses.start())
```

```lua
local p = assert(babet.spawn("sh", {}, {
    stdin = "inherit",
    stdout = "inherit",
    stderr = "inherit",
}))
local result = assert(p:wait())
p:close()

-- curses has been restored on the main thread here
assert(babet.curses.write("child exited"))
assert(babet.curses.refresh())
assert(babet.curses.stop())
```

Before an interactive child receives the TTY, Babet saves curses program mode and leaves curses. The existing process terminal manager gives the foreground TTY to the child's process group. After the child exits or stops, the monitor thread performs only the POSIX reclaim; curses restoration itself happens later on the main thread.

A stopped interactive process can be resumed in the foreground with `process:resume(true)` and uses the same suspend/reclaim path.

Workers

Ncurses calls are main-thread-only. In a worker:

- every `babet.curses.*` operation is rejected;
- an interactive inherited-terminal `babet.spawn()` is rejected while the main curses session is active;
- non-interactive worker processes using pipes/files/null remain supported.

This avoids cross-thread ncurses access and ambiguous foreground ownership.

Signals

Babet remains responsible for signal policy while curses is active:

- `SIGWINCH` becomes a deferred `"resize"` event;
- Ctrl-Z (`SIGTSTP`) restores normal terminal mode before suspension, then re-enters curses after `SIGCONT`;
- `babet.signal` handlers for `INT`, `TERM`, and `HUP` remain authoritative;
- when those signals keep their OS default, Babet restores curses on the main thread before re-raising the signal with its default disposition.

No ncurses function is called from a fatal signal handler.

TERM and terminfo

Babet uses the system terminfo database first. Its statically linked ncursesw also embeds fallbacks for:

```
linux, vt100, xterm, xterm-256color,
screen, screen-256color, tmux, tmux-256color
```

This lets common terminals work even when a system terminfo tree is unavailable and keeps generated applications self-contained. Babet never silently changes an unknown `TERM` to another terminal type. Missing/unknown `TERM` or unusable terminfo raises a normal Lua error rather than allowing ncurses to terminate the process.

Deployment

ncursesw is linked statically into the normal Babet runtime. `--create-exe` therefore needs no compiler, linker, `libncursesw.so`, or external terminfo folder at application-build time or runtime. A generated application gets the same curses runtime as the original Babet binary.

`argparse` — command-line arguments

`argparse` is a pure-Lua module bundled with Babet. It builds a declarative parser for flags, valued options, and positional arguments:

```
local argparse = require("argparse")
local parser = argparse("my-script", "Optional description")
```

The module never calls `print` or `os.exit`. Help and errors are returned to the script, which decides how to display them and which exit code to use.

Module contents

- Loading and metadata
- API
- Declaring flags and options
- Declaring positionals
- The `opts` table
  - Common fields
  - Value-only fields
  - Destination
- Argument source
  - Global `arg` table
  - Explicit array
- Recognized forms
- Defaults, choices, and conversion
- `parse` results
  - Normal success
  - Built-in help
  - User-input error
  - Programming error
- Complete example
- Text produced by `get_usage`
- Current limitations

Loading and metadata

```
local argparse = require("argparse")

print(argparse._VERSION)      -- "babet argparse 1.1.0"
print(argparse._DESCRIPTION)  -- English description
```

`require("argparse")` returns a callable table. The constructor accepts zero, one, or two arguments:

```
argparse()                        -- displayed program: "prog"
argparse("tool")                  -- program name
argparse("tool", "Description")  -- name and description
```

`prog` and `description` must be strings when supplied. An extra argument or a wrong type raises a Lua error: this is a programming error, not a command-line error.

API

Method	Result	Purpose
`parser:flag(spec [, opts])`	`parser`	Declare a flag with no value.
`parser:option(spec [, opts])`	`parser`	Declare an option that consumes a value.
`parser:argument(name [, opts])`	`parser`	Declare a positional argument.
`parser:parse([src])`	`res, err`	Parse `src` or the global `arg` table.
`parser:get_usage()`	string	Generate the help text.

Builder methods are chainable:

```
local parser = argparse("copy")
    :flag("-v --verbose")
    :option("-o --output")
    :argument("source")
```

Arities are strict. For example, `parser:option("-o", {}, "extra")` and `parser:get_usage(true)` raise a Lua error.

Declaring flags and options

`spec` is one string containing one or more whitespace-separated names:

```
parser:flag("-v")
parser:flag("--verbose")
parser:flag("-v --verbose")
parser:option("-o --output")
```

Each name must:

- start with one or two dashes;
- contain at least one character after the dashes;
- contain no `=`;
- not be duplicated in the same spec or parser.

The following names are reserved and cannot be declared:

- `-`: treated as a literal positional;
- `--`: ends option parsing;
- `-h` and `--help`: built-in help.

A triple-dash name such as `---verbose` is rejected as well.

All validation happens before the parser is modified. If a builder error is caught with `pcall`, no partial option or alias remains registered.

Declaring positionals

```
parser:argument("input")
parser:argument("output", { required = false })
```

The name must be a non-empty string with no whitespace and no leading dash. Positionals are assigned strictly from left to right.

A positional is required by default. It becomes optional when:

- `required = false` is specified; or

- a `default` is supplied without an explicit `required` field.

All required positionals must precede optional ones. The builder therefore rejects this ambiguous declaration:

```
parser
    :argument("optional", { required = false })
    :argument("required") -- Lua error
```

The `opts` table

`opts` must be a table or `nil`. Unknown fields and fields with a wrong type raise a Lua error immediately. This catches a typo such as `hlep` instead of `help`.

Common fields

Field	Type	Effect
`help`	string	Text appended by `get_usage()`.
`default`	any Lua value	Value used when the item is absent.
`dest`	non-empty string	Key used in the result table.
`required`	boolean	Require the item to be present.

These four fields are accepted by `flag`, `option`, and `argument`.

Value-only fields

Field	Type	Effect
`choices`	dense array of strings	Allow only selected raw values.
`convert`	function	Convert the raw string to a Lua value.

`choices` and `convert` are accepted by `option` and `argument`, but not by `flag`, since a flag consumes no value.

The builder copies the `choices` array. Mutating the original table later does not change the parser.

Destination

Without `dest`, the destination is:

1. the first long name without `--`, when present;
2. otherwise the first short name without its first `-`.

```
parser:option("-o --output") -- destination: "output"
parser:flag("-v")            -- destination: "v"
```

A destination must be unique across the parser. `help` and `usage` are reserved for the built-in help result.

```
parser:flag("-q", { dest = "quiet" })
```

Argument source

Global `arg` table

With no argument, `parse()` reads the global `arg` table at indices `1`, `2`, and so on up to the first hole:

```lua
local result, err = parser:parse()
```

Index `0` and negative indices are ignored. `parse(nil)` is equivalent to `parse()`.

Every encountered value must be a string. If `arg` is not a table, the parser uses an empty argument list.

Explicit array

For tests or programmatically built input:

```lua
local result, err = parser:parse({ "-v", "input.txt" })
```

The table must be a dense array indexed from `1` to `n`, with no holes, no extra keys, and string values only. An invalid source is a parsing error and returns `(nil, err)`; it does not raise.

Recognized forms

A valued option accepts:

```
--long value
--long=value
-s value
-s=value
```

An empty inline value is valid:

```lua
parser:parse({ "--output=" }) -- output == ""
```

Options remain active after a positional until `--` is encountered:

```
input.txt --verbose
```

`--` ends option parsing. Every following token becomes positional, even if it starts with `-`:

```lua
parser:parse({ "--", "-5" })
```

Before `--`, a token such as `-5` is interpreted as an option name. A negative number used as an option value needs no delimiter:

```lua
parser:parse({ "--count", "-5", "input.txt" })
```

The token following a valued option is always consumed literally, even when it looks like an option or equals `--`:

```lua
parser:parse({ "--output", "--verbose" })
-- output == "--verbose"; verbose is not enabled
```

A lone `-` is positional. Repeated options are accepted and the last value wins. A repeated flag simply remains `true`.

The following forms are not supported:

- combined short options, such as `-abc` for three flags;
- attached short values, such as `-ofile.txt`;
- variadic positionals.

A literal declared name `-abc` is still a normal option when the spec contains that exact name; it is never split.

Defaults, choices, and conversion

A user-supplied value is processed in this order:

1. check `choices` against the raw string;
2. call `convert` with that string.

```lua
parser:option("-n --count", {
    choices = { "1", "2", "3" },
    convert = tonumber,
})
```

Every choice is a string. If the raw value is absent from the array, `parse()` returns `(nil, err)` without calling the converter.

A converter may:

- return any non-`nil` value, including `false`, to succeed;
- return `nil` to fail with the generic `invalid value` message;
- return `nil, "reason"` to supply a reason;
- raise, which is caught and reported as `conversion error`.

`default` values are already prepared Lua values: they are neither checked by `choices` nor passed through `convert`. They keep their type and, for a table, their identity. An option with `required = true` remains required even when a `default` exists.

For a flag:

- when absent, it equals `default` if supplied, otherwise `false`;
- when present, it always equals `true`.

An absent option or positional with a `nil` default has no observable key in the result table, as expected for Lua tables.

`parse` results

Normal success

```lua
local result, err = parser:parse()
-- result: table
-- err:    nil
```

The table is indexed by declared destinations.

Built-in help

Before `--`, `-h` or `--help` stops parsing immediately and returns:

```lua
{
    help = true,
    usage = "...",
}, nil
```

Required options, missing positionals, and later tokens are not validated. After `--`, `-h` and `--help` are ordinary positionals.

Help is a success and is not merged into a normal result table. Destinations `help` and `usage` are reserved to avoid ambiguity in calling code.

User-input error

Command-line errors return `(nil, message)`:

```
unknown option '--bad'
option '--output' requires a value
flag '-v' does not take a value
missing required option '--output'
missing required argument 'input'
argument 'mode': invalid choice 'other'
```

```
option '--count': invalid value
unexpected argument 'extra'
```

`parse()` catches errors raised by `convert`, so a user value never propagates the converter's exception.

Programming error

The constructor and builder raise a Lua error for:

- wrong arity;
- wrong type;
- an empty or unusable spec;
- a reserved name or destination;
- a duplicate name or destination;
- an unknown `opts` field;
- an invalid `choices` array;
- a required positional after an optional positional.

Wrong `parse` arity is also a programming error. In contrast, a malformed `src` table returns `(nil, err)`.

Complete example

```lua
local argparse = require("argparse")

local parser = argparse("convert", "Convert a file.")
    :flag("-v --verbose", { help = "Show more detail" })
    :option("-o --output", { default = "out.txt" })
    :option("-n --count", {
        choices = { "1", "2", "3" },
        convert = tonumber,
        default = 1,
    })
    :argument("input")

local args, err = parser:parse()
if args and args.help then
    print(args.usage)
    return
end
if not args then
    io.stderr:write("error: ", err, "\n")
    io.stderr:write(parser:get_usage(), "\n")
    os.exit(2)
end

print(args.input, args.output, args.count, args.verbose)
```

Text produced by `get_usage`

`get_usage()` returns a string and writes nothing. The section labels and the built-in help message are fixed in English: `Usage`, `Arguments`, `Options`, and `show this help`. There is no localization, column-width, or visual-group API yet.

The text lists positionals and declared help strings, but it does not automatically show defaults, choices, or the required status of options.

Current limitations

The module does not expose:

- subcommands;
- variadic positionals (`nargs`);
- mutually exclusive groups;
- repeatable options accumulated into an array;
- combined short options;
- help localization or customization;
- a way to disable built-in help.

EXEC - external programs, capture, and streaming

Babet provides two complementary APIs for starting an external program without an implicit shell:

- `babet.exec` optionally sends complete input, captures `stdout` and `stderr` in memory, then waits for completion;
- `babet.spawn` immediately returns a controllable process object so streams can be read and written progressively without automatic RAM accumulation.

EXEC covers:

- looking up a program through `PATH`;
- building `argv` safely, without shell parsing;
- selecting a working directory for the child process;
- merging a custom environment with Babet's environment;
- sending a binary string to standard input;
- capturing binary `stdout` and `stderr`;
- applying one timeout to launch, I/O, and final wait;
- enforcing a separate memory limit for each output stream;
- terminating the child's process group after a timeout.

`babet.exec` does not provide direct file redirection: it captures `stdout` and `stderr` in memory. For a controllable process, `babet.spawn` lets you configure `stdin`, `stdout`, and `stderr` independently as `pipe`, `inherit`, or `null`, and redirect `stdout` and `stderr` to a file.

`babet.exec` also does not provide an implicit command interpreter. To connect several programs without a shell, use `babet.pipeline()` or `babet.spawnPipeline()`.

Module contents

- Essential conventions
- API overview
- Signature and arguments
  - `cmd` and program lookup
  - `args` and shell-free execution
  - Using a shell explicitly
- Options
  - `cwd`
  - `env`
  - `stdin`
  - `timeout`
  - `max_output`
- Result table
  - Normal exit code
  - Signal termination
  - Timeout
  - Truncation

- Complete examples
  - Basic capture
  - Arguments containing spaces
  - Handling a non-zero code
  - Capturing `stderr`
  - Sending text or binary stdin
  - Changing the environment
  - Changing the working directory
  - Bounding execution time
  - Bounding captured output
  - Building an application helper
- Streaming processes with `babet.spawn`
  - Signature and options
  - Process methods
  - Reading stdout and stderr
  - Writing stdin
  - Waiting, terminating, and closing
  - Streaming loop example
  - Limits and deadlock prevention
- Error contract
- Processes, signals, and security
- Design and limitations

Essential conventions

No implicit shell

This call:

```lua
local result, err = babet.exec("printf", { "%s\n", "hello world" })
```

starts `printf` directly. Every `args` element becomes a separate `argv` slot. Spaces, `$`, `*`, `;`, `>`, quotes, and other shell characters are not interpreted.

This avoids most quoting mistakes and command-injection issues.

A non-zero exit is not a launch error

A program that starts successfully and exits with code 1, 2, or 127 still produces a result table. The caller decides which codes are acceptable.

```lua
local result, err = babet.exec("grep", { "needle", "data.txt" })
assert(result, err)

if result.code == 0 then
    print("match found")
elseif result.code == 1 then
    print("no match")
else
    error(result.stderr)
end
```

`assert(babet.exec(...))` only checks that Babet could launch and monitor the program. It does not check `result.code`.

Data strings are binary-safe

`opts.stdin`, `result.stdout`, and `result.stderr` are binary-safe Lua strings. They may contain NUL bytes and do not need to be UTF-8 text.

Strings passed to POSIX APIs that require C strings - `cmd`, `args` elements, `cwd`, and environment names and values - cannot contain a NUL byte.

API overview

```
local result, err = babet.exec(cmd, args?, opts?)
```

Element	Type	Default	Purpose
`cmd`	strict string	required	program name or path
`args`	string sequence	no arguments	`argv[1]..argv[n]`
`opts.cwd`	string	inherited current directory	child working directory
`opts.env`	string-to-string table	inherited environment	variables to add or replace
`opts.stdin`	binary string	stdin closed immediately	data sent to the child
`opts.timeout`	finite number > 0	no limit	global budget in seconds
`opts.max_output`	integer `1..2 GiB`	10 MiB	cap per captured stream

Main outcomes:

Situation	Return value
program launched and completed	`(result, nil)`
program launched then timed out	`(result, nil)` with `timed_out = true`
non-zero exit status	`(result, nil)` with `code ~= 0`
program not found or invalid `cwd`	`(nil, err)`
invalid `args` or `opts`	`(nil, err)`
missing or non-string `cmd`	raised Lua error

Signature and arguments

`cmd` and program lookup

`cmd` must be an actual Lua string.

```
local result, err = babet.exec("git", { "status", "--short" })
```

When `cmd` contains no `/`, Babet first builds the child's final environment in the parent, then searches with the exact `PATH` that will be transmitted. A `PATH` replacement in `opts.env` therefore controls both the initial lookup and commands started later by the child.

```
local result = assert(babet.exec("my-private-tool", {}, {
    env = { PATH = "/opt/private/bin:/usr/bin:/bin" },
}))
```

When the effective environment has no `PATH` variable, Babet uses the system's POSIX search path (`_CS_PATH`, falling back to `/bin:/usr/bin`). It does not add that default value to the child's environment.

Each empty `PATH` component denotes the child's effective working directory. A relative component is also resolved from that directory, after applying `opts.cwd` when present.

```
local result = assert(babet.exec("tool", {}, {
    cwd = "/opt/my-app",
    env = { PATH = "bin:/usr/bin:/bin" },
```

```
}))
-- first candidate: /opt/my-app/bin/tool
```

Lookup continues after `ENOENT` or `ENOTDIR`. A present but non-executable candidate (`EACCES`) does not hide a later valid candidate; if no candidate works and at least one was denied, the final error remains `EACCES` instead of being reduced to "not found".

When `cmd` contains `/`, `PATH` is not consulted. An absolute path is passed directly to `execve()`; a relative path is interpreted from the child's working directory.

```
local result = assert(babet.exec("./tool", {}, {
    cwd = "/opt/my-app/bin",
}))
```

The ordered candidate list, `argv`, and `envp` are prepared before `fork()`. The child performs no allocation or `PATH` parsing: it only tries those paths with `execve()`, continuing after `ENOENT`, `ENOTDIR`, or `EACCES`. A file may still be removed or replaced before one of those calls; Babet then returns `(nil, err)` with the system error it observed. An executable file with an unrecognized format also returns a launch error (`ENOEXEC`): Babet never retries it through an implicit shell. When migrating from Babet 2.16.x, add a shebang to executable scripts that do not have one, for example `#!/bin/sh`.

`args` and shell-free execution

`args` is optional. It must be a dense `1..n` sequence of strings.

```
local result = assert(babet.exec("cp", {
    "--",
    "file with spaces.txt",
    "destination/",
}))
```

Babet reads only the raw entries of the dense `1..n` sequence. The `__len` and `__index` metamethods are not invoked. For predictable behavior:

- use consecutive indexes starting at 1;
- do not leave holes;
- do not store metadata in the same table;
- use strings only.

```
-- Correct
local args = { "-c", "print('ok')" }

-- Invalid: numeric element
local result, err = babet.exec("lua", { "-e", 42 })
-- result == nil; err says all arguments must be strings
```

`argv[0]` is set to `cmd` automatically. The first `args` element becomes `argv[1]`.

Using a shell explicitly

Operators such as `|`, `>`, `&&`, `;`, variables such as `$NAME`, and globs are only interpreted when you explicitly start a shell.

```
local result = assert(babet.exec("sh", {
    "-c",
    "printf '%s\n' \"$HOME\" | sed 's#^#home=#'",
}))
```

Values embedded in shell source must then follow shell quoting rules. For untrusted data, pass values through `args`, `stdin`, or `env` instead of concatenating them into the `-c` script.

Options

The third argument is optional. When present, it must be a table. Unknown fields are currently ignored; a typo such as `timeot = 5` therefore does not raise an error. Use the exact names below.

opts.cwd

Sets the child's current directory without changing the Babet process's current directory.

```
local result, err = babet.exec("pwd", {}, {
    cwd = "/var/tmp",
})
assert(result, err)
print(result.stdout) -- /var/tmp
```

The child performs `chdir` before launching the program. It affects:

- relative paths used by the program;
- a relative `cmd` containing `/`;
- relative entries in the `PATH` used for lookup.

If the directory does not exist or is inaccessible, Babet returns `(nil, err)`. The parent current directory is unchanged.

opts.env

`env` is a table whose keys and values must be strings. It is **merged** with the Babet process's environment.

```
local result = assert(babet.exec("sh", {
    "-c", "printf '%s' \"$APP_MODE\"",
}, {
    env = { APP_MODE = "production" },
}))

assert(result.stdout == "production")
```

Rules:

- a supplied key replaces the inherited variable of the same name;
- an omitted key remains inherited;
- `""` creates or keeps a defined-but-empty variable;
- there is no option to remove a variable completely;
- a key must be non-empty and cannot contain `=`;
- keys and values reject NUL bytes;
- non-string keys and values are rejected.

Defined-but-empty is different from absent:

```
local result = assert(babet.exec("sh", { "-c", [[
    if [ "${TOKEN+x}" = x ] && [ -z "$TOKEN" ]; then
        printf 'defined-empty'
    fi
]] }, {
    env = { TOKEN = "" },
}))

assert(result.stdout == "defined-empty")
```

The environment is built in the parent before `fork`, avoiding `setenv` calls in a child of a multithreaded process.

opts.stdin

`stdin` is one string sent completely to the program's standard input.

```
local result = assert(babet.exec("sha256sum", { "-" }, {
    stdin = "hello world\n",
}))
print(result.stdout)
```

It is binary-safe:

```
local payload = "A\0B\0C"
local result = assert(babet.exec("cat", {}, { stdin = payload }))
assert(result.stdout == payload)
```

When `stdin` is absent or `nil`, Babet closes the input pipe immediately, so the child sees EOF. An empty string also produces EOF after the normal pipe setup.

Babet writes stdin while reading both output pipes, preventing deadlocks with a program that produces substantial output before consuming all its input.

opts.timeout

`timeout` is a finite, strictly positive number of seconds.

```
local result = assert(babet.exec("sleep", { "30" }, {
    timeout = 0.5,
}))

assert(result.timed_out)
```

The budget starts before launch preparation and covers:

1. environment preparation;
2. `fork`;
3. the child `chdir`;
4. the `exec` call;
5. stdin transfer;
6. stdout and stderr capture;
7. the final process wait.

Internal resolution is one millisecond. Values below `0.001` therefore act as an almost immediate deadline and should not be used for precise timing.

The value must fit within `INT_MAX` milliseconds, about 24.8 days. Zero, negative, NaN, infinite, and larger values return `(nil, err)`.

After expiry once the child has launched:

1. Babet sends `SIGTERM` to the process group;
2. it allows up to 2 seconds for a clean exit;
3. it sends `SIGKILL` if needed;
4. it continues draining output for a bounded window.

opts.max_output

`max_output` limits retained bytes **for each stream**.

```
local result = assert(babet.exec("yes", {}, {
    timeout = 0.2,
    max_output = 4096,
}))
```

```
assert(#result.stdout == 4096)
assert(result.stdout_truncated)
```

Values:

- default: `10 * 1024 * 1024`, or 10 MiB;
- minimum: 1 byte;
- maximum: `2 * 1024 * 1024 * 1024`, or 2 GiB;
- type: integral Lua number.

The cap is independent: up to 10 MiB of stdout **and** 10 MiB of stderr by default.

After reaching the cap, Babet keeps reading and discarding excess bytes. This is necessary to avoid filling the pipe and blocking the child. The first bytes are retained; the final bytes are lost.

Result table

After a successful launch, the first return value is always a table with six fields:

```
{
    stdout = "...",
    stderr = "...",
    code = 0,
    timed_out = false,
    stdout_truncated = false,
    stderr_truncated = false,
}
```

Field	Type	Meaning
`stdout`	binary string	first retained bytes from standard output
`stderr`	binary string	first retained bytes from standard error
`code`	integer	normal code, signal convention, or `-1` if unavailable
`timed_out`	boolean	Babet initiated termination after deadline expiry
`stdout_truncated`	boolean	stdout exceeded `max_output`
`stderr_truncated`	boolean	stderr exceeded `max_output`

Normal exit code

When the program calls `exit(n)` or returns from `main`, `code` is the POSIX status `0..255`.

```
local result = assert(babet.exec("sh", { "-c", "exit 7" }))
assert(result.code == 7)
assert(not result.timed_out)
```

Signal termination

When the process is terminated by a signal, Babet uses:

```
code = 128 + signal_number
```

For example, `SIGKILL` is normally 9 on Linux, giving `code == 137`. This is a useful convention but there is no separate result field containing the signal.

Timeout

`timed_out == true` means Babet reached its deadline and started termination. `code` depends on how the child eventually stopped:

- an application-defined code if it handles `SIGTERM` and exits cleanly;
- often `143` for `SIGTERM`;
- often `137` after `SIGKILL`;
- exceptionally `-1` if status could not be obtained within bounded cleanup.

Do not test only `code == 137` to detect a timeout; use `timed_out`.

Output produced before expiry is retained up to `max_output`.

Truncation

`stdout_truncated` and `stderr_truncated` only mean that the corresponding stream exceeded `max_output`.

```
if result.stdout_truncated then
    print("stdout is incomplete: increase max_output or redirect to a file")
end
```

In an extremely pathological kernel/process-group case, a descendant may keep a pipe open after `SIGKILL` and force Babet to abandon draining rather than block forever. This rare cause is not represented by the truncation flags, which are reserved for the memory cap.

Complete examples

Basic capture

```
local result, err = babet.exec("git", { "rev-parse", "HEAD" })
assert(result, err)

if result.code ~= 0 then
    error("git failed: " .. result.stderr)
end

local commit = result.stdout:gsub("%s+$", "")
print(commit)
```

Arguments containing spaces

```
local filename = "July 2026 report.txt"
local result = assert(babet.exec("printf", {
    "name=<%s>\n",
    filename,
}))

assert(result.stdout == "name=<July 2026 report.txt>\n")
```

No additional quoting is needed around `filename`: the Lua table already represents the argument boundary.

Handling a non-zero code

```
local result, err = babet.exec("diff", {
    "--brief",
    "config.old",
    "config.new",
```

```
})
assert(result, err)

if result.code == 0 then
    print("files are identical")
elseif result.code == 1 then
    print("files differ")
else
    error("diff failed: " .. result.stderr)
end
```

Capturing `stderr`

```
local result = assert(babet.exec("sh", {
    "-c", "printf out; printf err >&2",
}))

assert(result.stdout == "out")
assert(result.stderr == "err")
```

Sending text or binary stdin

```
local csv = "name,score\nAlice,18\nBob,15\n"
local result = assert(babet.exec("sort", {}, { stdin = csv }))
print(result.stdout)
```

```
local bytes = string.char(0x00, 0x01, 0xFE, 0xFF)
local result = assert(babet.exec("cat", {}, { stdin = bytes }))
assert(result.stdout == bytes)
```

Changing the environment

```
local result = assert(babet.exec("sh", { "-c", [[
    printf 'mode=%s lang=%s' "$APP_MODE" "$LANG"
]] }, {
    env = {
        APP_MODE = "test",
        LANG = "C",
    },
}))

print(result.stdout)
```

Only supplied keys are replaced. Other variables, including `HOME`, remain inherited.

Changing the working directory

```
local result = assert(babet.exec("find", {
    ".", "-maxdepth", "1", "-type", "f",
}, {
    cwd = "/var/log",
}))

print(result.stdout)
```

The main script's current directory is unchanged.

Bounding execution time

```
local result, err = babet.exec("sh", {
    "-c", "printf started; sleep 30",
}, {
    timeout = 1.0,
})
assert(result, err)

if result.timed_out then
    print("command exceeded its budget")
    print("available output:", result.stdout)
end
```

Bounding captured output

```
local result = assert(babet.exec("sh", {
    "-c", "yes log-line | head -n 100000",
}, {
    max_output = 64 * 1024,
}))

if result.stdout_truncated then
    print("only the first 64 KiB were retained")
end
```

For gigabytes, do not capture into memory. Explicitly start a shell and redirect to a carefully selected file:

```
local result = assert(babet.exec("sh", {
    "-c", "my-command > /var/tmp/my-command.log 2>&1",
}, {
    timeout = 600,
}))
```

Building an application helper

```
local function run_checked(cmd, args, opts)
    local result, err = babet.exec(cmd, args, opts)
    if not result then
        return nil, err
    end
    if result.timed_out then
        return nil, string.format("%s: timeout", cmd)
    end
    if result.code ~= 0 then
        local detail = result.stderr ~= "" and result.stderr or result.stdout
        return nil, string.format(
            "%s: exit code %d: %s",
            cmd,
            result.code,
            detail
        )
    end
    return result
end

local result, err = run_checked("git", { "status", "--short" }, {
```

```
    cwd = "/srv/project",
    timeout = 10,
})
assert(result, err)
print(result.stdout)
```

Controllable processes with `babet.spawn`

`babet.spawn` uses the same shell-free launch engine as `babet.exec`, but does not wait for program completion. It returns a userdata representing the child:

```
local process, err = babet.spawn("yt-dlp", {
    "--newline",
    "https://example.invalid/video",
})
assert(process, err)
```

By default, stdin, stdout, and stderr use three non-blocking pipes, preserving the historical behaviour exactly. Each stream may instead be inherited or connected to `/dev/null`; output streams may be redirected directly to files, and stderr may be merged into stdout.

Signature and options

```
local process, err = babet.spawn(command, args?, opts?)
```

Element	Type	Default	Purpose
`command`	strict string	required	program name or path
`args`	dense string array	no arguments	`argv[1]..argv[n]`
`opts.cwd`	string	inherited directory	child working directory
`opts.env`	string-to-string table	inherited environment	variables to add/replace
`opts.launch_timeout`	finite number > 0	no general deadline	bounds preparation, terminal handoff, `chdir`, and `exec`
`opts.stdin`	`"pipe"`, `"inherit"`, or `"null"`	`"pipe"`	standard-input source
`opts.stdout`	mode or file table	`"pipe"`	standard-output destination
`opts.stderr`	mode, `"stdout"`, or file table	`"pipe"`	standard-error destination

Unknown options are rejected. `spawn` does not accept the `timeout` and `max_output` options of `exec`: lifetime is controlled with `wait()`/`terminate()`, and piped outputs are read progressively.

`launch_timeout` covers preparation and launch only, until the new executable is established. It is not a total process-lifetime limit. A redirection-file open failure or program-resolution failure happens before `fork()`, so no child is created in that case.

`PATH` lookup, paths containing `/`, environment merging, and shell-free execution follow the same rules as `exec`.

Redirection modes

`"pipe"` The default mode creates a non-blocking pipe between parent and child:

```lua
local process = assert(babet.spawn("cat", {}, {
    stdin = "pipe",
    stdout = "pipe",
    stderr = "pipe",
}))

assert(process:write("hello\n"))
assert(process:close_stdin())
print(assert(process:read_stdout(64 * 1024, 1)))
```

`write`, `close_stdin`, `read_stdout`, and `read_stderr` are available only for streams configured as `"pipe"`.

`"inherit"` The child directly reuses Babet's corresponding standard descriptor:

```lua
local process = assert(babet.spawn("make", { "-j4" }, {
    stdin = "inherit",
    stdout = "inherit",
    stderr = "inherit",
}))
assert(process:wait())
```

Use this when a program should interact with the terminal or respect an external redirection applied to Babet itself.

When `stdin = "inherit"` refers to the controlling terminal and Babet owns its foreground, the child keeps its separate process group but Babet transfers the terminal before allowing it to execute the program. The handoff is synchronized, so the child cannot attempt its first read as a background group and be stopped by `SIGTTIN`. Direct interactive children can read normally and `Ctrl+C` is delivered to the foreground child group. The real Babet → Yaourt → Pacman workflow is also validated with Babet 2.15.0 and later.

Babet 2.16.0 adds a limited, explicit job-control contract:

- `wait()` and `state()` observe stopped and continued child states;
- when `Ctrl+Z` stops the child, Babet saves the child's current `termios`, restores the parent foreground group and parent `termios`, then `wait()` returns `(nil, "stopped")`;
- `is_running()` remains `true` while stopped because the child has not exited;
- `resume()` restores the saved child `termios`, returns the terminal to the child group, and sends `SIGCONT`;
- `resume(false)` sends `SIGCONT` without foregrounding the terminal. This is intended for pipe-oriented jobs; a background process that reads the terminal may be stopped again by the kernel.

```lua
local process = assert(babet.spawn("interactive-tool", {}, {
    stdin = "inherit",
    stdout = "inherit",
    stderr = "inherit",
}))

local result, err = process:wait()
if not result and err == "stopped" then
    assert(process:state() == "stopped")

    -- The parent owns the terminal here and may interact with the user.
    print("child suspended")

    -- Restore the child terminal state and foreground group, then SIGCONT.
    assert(process:resume())
    result, err = process:wait()
```

```
end
assert(result, err)
```

This is deliberately not a full shell job-control API. Babet does not maintain a jobs table, implement shell commands such as `fg`/`bg`, or transfer terminal ownership to `pipeline()` and `spawnPipeline()`.

Final-exit reclamation is asynchronous. Each interactive handoff has a native monitor that waits for the direct child to terminate without reaping it. On Linux it uses a pidfd when available and falls back to `waitid(..., WNOWAIT)`. The monitor owns a duplicated terminal descriptor; the serialized native handoff registry keeps the copied restoration state. The monitor does not call Lua and no `SIGCHLD` handler performs terminal work. Consequently the parent may read its terminal after the child exits even before calling `wait()`, `is_running()`, `state()`, or `close()`:

```
local process <close> = assert(babet.spawn("short-interactive-tool", {}, {
    stdin = "inherit",
    stdout = "inherit",
    stderr = "inherit",
}))

-- Other Lua work may happen here. Once the child exits, Babet's native
-- monitor restores the parent foreground group and termios automatically.
babet.sleep(0.5)
local answer = io.read("*l")

-- The final child status is still reaped and cached normally.
local result = assert(process:wait())
```

Terminal transfers are serialized inside Babet. If an interactive child exits and another interactive `spawn()` starts before the detached monitor is scheduled, the new launch detects that the previous direct child has already reached final exit with non-reaping `waitid(..., WNOHANG | WNOWAIT)`, restores the exact saved parent foreground group and `termios`, then performs the new handoff. An older monitor only restores while its own child group still owns the terminal, so it cannot take the terminal back from a later child.

An active reservation held by another thread is never awaited indefinitely. Babet waits for at most two seconds, or only the remaining `launch_timeout` when that deadline is shorter. If the reservation is still held, `spawn()` returns `(nil, "terminal handoff is busy")` before any `fork()`: no child is silently launched without terminal ownership. The original reservation continues its normal handoff or rollback; once released, a later interactive launch can acquire the terminal again with the saved parent `termios` intact.

The monitor follows the direct child, not an entire shell job. If that child exits after leaving descendants in its process group, Babet restores the parent terminal at the direct child's exit. Descendants that later attempt terminal input are background processes and may receive `SIGTTIN`. This limitation is intentional: Babet does not implement a shell jobs table or wait for arbitrary job descendants.

A `wait(timeout)` result of `"timeout"` still does not kill the child or take the terminal away from a running foreground child. Follow it with another `wait()`, `terminate()`, `kill()`, or `close()` as appropriate. A stopped child is different: `wait()` reports `"stopped"` immediately and Babet has already returned the terminal to the parent.

If inherited stdin is a pipe, file, or another non-terminal descriptor, no foreground-group transfer or asynchronous terminal monitor is created. stdout and stderr may still be captured or redirected independently.

`"null"` The stream is connected to `/dev/null`:

```
local process = assert(babet.spawn("quiet-program", {}, {
    stdin = "null",  -- immediate EOF in the child
    stdout = "null",
```

```
    stderr = "null",
}))
```

File redirection stdout and stderr accept a strict table:

```
{
    file = "/var/tmp/application.log", -- required, no NUL byte
    append = true,                     -- false by default: truncate
    permissions = tonumber("600", 8), -- 0600 by default, creation only
}
```

- `append = false` truncates an existing regular file;
- `append = true` writes with `O_APPEND`;
- `permissions` must be an integer from `0000` to `0777` and applies only when Babet creates the file;
- the final destination must not be a symbolic link;
- after opening, Babet uses `fstat()` to require a regular file; directories, FIFOs, sockets, and devices are rejected;
- the descriptor is prepared before `fork()` and retained only by the child.

stdout-only example:

```
local process = assert(babet.spawn("generate-report", {}, {
    stdout = {
        file = "report.log",
        permissions = tonumber("640", 8),
    },
}))
```

stderr-only append example:

```
local process = assert(babet.spawn("worker", {}, {
    stderr = {
        file = "errors.log",
        append = true,
    },
}))
```

Merging stderr into stdout `stderr = "stdout"` duplicates descriptor 2 onto the already configured stdout. It works with a pipe, inheritance, `/dev/null`, or a file:

```
local process = assert(babet.spawn("driver", {}, {
    stdout = {
        file = "driver.log",
        append = true,
    },
    stderr = "stdout",
}))
```

For one combined log file, this form is preferable to two independent file tables because it uses one descriptor and one write offset.

Process methods

Method	Return	Purpose
`read_stdout([max [, timeout]])`	`data, nil` or `nil, reason`	read piped stdout
`read_stderr([max [, timeout]])`	`data, nil` or `nil, reason`	read piped stderr

Method	Return	Purpose
`write(data [, timeout])`	`count, nil` or `nil, reason`	write piped stdin
`close_stdin()`	`true, nil` or `nil, reason`	close piped stdin
`is_running()`	boolean or `nil, err`	true until final exit
`state()`	state string or `nil, err`	inspect lifecycle state
`pid()`	integer	return the child PID
`wait([timeout])`	`result, nil` or `nil, reason`	wait, or report stop/timeout
`resume([foreground])`	`true, nil` or `nil, reason`	continue a stopped child
`terminate([grace])`	`result, nil` or `nil, err`	SIGTERM then bounded SIGKILL
`kill()`	`result, nil` or `nil, err`	SIGKILL the process group
`close()`	`true, nil`	finalize the process handle

Short reasons are:

- `"timeout"`: no progress before the deadline;
- `"stopped"`: `wait()` observed a suspended child and returned the terminal;
- `"closed"`: closed handle, closed pipe, or EOF depending on the method;
- `"interrupted"`: handled Babet interruption;
- `"not_piped"`: the method targets a stream not configured as `"pipe"`;
- `"not_stopped"`: `resume()` was requested for a running child;
- `"not_interactive"`: foreground resume was requested without a terminal handoff;
- `"exited"`: `resume()` was requested after final child exit.

Other diagnostics use a `process:` or `spawn:` prefix.

The table returned by `wait`, `terminate`, and `kill` contains:

```
{
    code = 0,           -- normal code, or 128 + signal
    exited = true,      -- normal exit/_exit termination
    signaled = false,   -- terminated by a signal
    signal = nil,       -- signal number when signaled == true
}
```

A second `wait()` after completion returns the same cached status.

Reading stdout and stderr

Pipe reads are binary-safe. By default they read at most 64 KiB and are non-blocking (`timeout = 0`). The maximum size per call is 16 MiB.

```
local chunk, err = process:read_stdout(64 * 1024, 0.25)

if chunk then
    io.write(chunk)
elseif err == "timeout" then
    -- nothing became available for 250 ms
elseif err == "closed" then
    -- final EOF on stdout
elseif err == "not_piped" then
    -- stdout is inherited, null, or redirected to a file
else
    error(err)
end
```

`"closed"` is EOF for that stream, not necessarily process completion. When stdout and stderr are both pipes, drain both: reading only one while the other fills can block the child. File, inherited, and null outputs do not fill a parent-side pipe.

Writing stdin

With `stdin = "pipe"`, `write` accepts a binary Lua string including NUL bytes. The non-blocking pipe may accept only part of the string, so resume from the returned byte count.

```
local function write_all(process, data)
    local offset = 1
    while offset <= #data do
        local written, err = process:write(data:sub(offset), 1)
        assert(written, err)
        offset = offset + written
    end
end

write_all(process, payload)
assert(process:close_stdin())
```

An empty string returns `(0, nil)`. After `close_stdin`, `write` returns `(nil, "closed")`. With `stdin = "inherit"` or `stdin = "null"`, `write` and `close_stdin` return `(nil, "not_piped")`.

Waiting, terminating, and closing

`wait()` without an argument waits for completion. `wait(timeout)` returns `(nil, "timeout")` without sending a signal or invalidating the object.

`terminate(grace_period)` sends SIGTERM to the whole group and then SIGKILL if needed. The default grace period is two seconds. `kill()` sends SIGKILL directly.

`close()` is idempotent. It terminates and reaps an active child with bounded waits, then closes every descriptor still owned by the parent. Lua `__close` and garbage collection perform the same cleanup.

Combined example: launching a WebDriver or daemon

```
local process, err = babet.spawn("bin/geckodriver", {
    "--port=4444",
}, {
    stdin = "null",
    stdout = {
        file = "logs/geckodriver.log",
        append = true,
        permissions = tonumber("600", 8),
    },
    stderr = "stdout",
})
assert(process, err)

local result, wait_err = process:wait(0)
if not result and wait_err == "timeout" then
    print("geckodriver is running with PID", process:pid())
end
```

No shell is involved, arguments remain separate, stdin cannot wait for input, and logs cannot fill a parent-side pipe.

Combined example: streaming loop

```
local process = assert(babet.spawn("yt-dlp", {
    "--newline",
```

```
    "-f", "bestvideo+bestaudio/best",
    url,
}))

local stdout_open, stderr_open = true, true
while stdout_open or stderr_open do
    if stdout_open then
        local data, read_err = process:read_stdout(64 * 1024, 0.1)
        if data then
            io.write(data)
            io.flush()
        elseif read_err == "closed" then
            stdout_open = false
        elseif read_err ~= "timeout" then
            error(read_err)
        end
    end

    if stderr_open then
        local data, read_err = process:read_stderr(64 * 1024, 0)
        if data then
            io.stderr:write(data)
            io.stderr:flush()
        elseif read_err == "closed" then
            stderr_open = false
        elseif read_err ~= "timeout" then
            error(read_err)
        end
    end
end

local result, wait_err = process:wait(5)
assert(result, wait_err)
process:close()
```

Limits and deadlock prevention

- `wait()` does not drain outputs configured as `"pipe"`. For verbose children, drain the pipes, redirect logs to a file, use `"inherit"`/`"null"`, or use `babet.exec` for small bounded output.
- Large simultaneous input and output require alternating writes and reads.
- A `babet.spawn()` object cannot be connected after launch to another process; use `babet.spawnPipeline()` for a linear pipeline.
- File tables are not supported for stdin in this version.
- Babet does not create parent directories for redirection files.
- Redirection files are opened before `fork()`. If a later `fork()` or launch error occurs, a file may already have been created or truncated even though no usable process is returned.
- `close()` and `terminate()` target the launch-time process group. A descendant that deliberately changes group or session may escape that control.

Error contract

Raised Lua errors

Only the first argument's signature raises:

```
babet.exec()   -- raises: missing command
babet.exec(42) -- raises: cmd must be a string
```

Use `pcall` only when an invalid signature can come from uncontrolled data.

Returned `(nil, err)` errors

The following validations return `(nil, err)` rather than raising:

- `args` is neither a table nor `nil`;
- an `args` element is not a string;
- `opts` is neither a table nor `nil`;
- `cwd`, `stdin`, `timeout`, `max_output`, or `env` has the wrong type;
- an environment key is empty, contains `=`, or contains NUL;
- an environment value contains NUL;
- timeout is zero, negative, non-finite, or too large;
- `max_output` is zero, negative, fractional, or above 2 GiB;
- program not found or not executable;
- interactive terminal reservation held for too long;
- invalid `cwd`;
- `pipe`, `fork`, `poll`, `waitpid`, or another internal system failure.

After a successful `fork()`, Babet retains ownership of descriptors and of the process group until ownership is transferred to a `spawn()` object or final reaping completes for `exec()`. If an internal C++ exception occurs in that window—for example while allocating an output buffer—the emergency path closes descriptors, immediately sends `SIGKILL` to the group, and attempts to reap the child within a short bounded window before returning `nil, err`. Exception cleanup does not use `terminate()`'s grace period, which is reserved for explicitly requested shutdown.

```lua
local result, err = babet.exec("echo", "hello")
assert(result == nil)
print(err) -- args must be a table
```

Situations returning a table

These are not API errors:

- non-zero program exit;
- output written to `stderr`;
- timeout during or after launch;
- output exceeding `max_output`.

Processes, signals, and security

Process group

The child creates its own process group. On timeout, Babet signals the group, not only the direct PID. This normally covers descendants started by a shell or by the program.

A descendant that deliberately calls `setsid` or changes process group can escape this strategy. `babet.exec` is not a sandbox and does not replace systemd, cgroups, namespaces, or resource limits.

Child signal mask

`exec`, `spawn`, `pipeline` and `spawnPipeline` start external programs with an empty signal mask, from both the main thread and workers. The worker's internal mask therefore does not block `SIGTERM`, `SIGINT` or `SIGPIPE` in the child. The calling thread's mask stays unchanged.

This does not change POSIX inheritance of signal dispositions: an explicitly ignored signal may remain ignored after `execve`. This policy applies to the four Babet APIs, not the standard Lua `os.execute` and `io.popen` functions.

Interaction with `babet.signal`

`babet.exec` is not signal-aware. A handled signal received while the call is running does not return `(nil, "interrupted")`. The C handler records a pending flag; the Lua callback can only be dispatched after control returns to the Lua VM, normally through SIGNAL's instruction hook.

Use `opts.timeout` to bound a long-running command. For externally requested cancellation, design another protocol or run the command under a supervisor.

Memory

The script already holds `stdin` in memory, and stdout/stderr are captured in memory up to the selected limits. The defaults permit about 20 MiB of captured output in total, in addition to application and Lua-string allocations.

Design and limitations

- Linux/POSIX only: the implementation uses `fork`, `execve`, `poll`, process groups, and signals.
- No implicit shell.
- One `exec` call still represents one command; native pipelines use `babet.pipeline()` or `babet.spawnPipeline()`.
- One complete stdin string, no streaming producer.
- Complete capture returned at the end, no line callback.
- No native stdin/stdout/stderr redirection to files or descriptors.
- No separate `signal` field in the result table.
- No full environment replacement or unset operation: `env` merges and an empty string remains a defined variable.
- Unknown `opts` fields are ignored.
- The ordered `PATH` candidate list, `argv`, and `envp` are prepared in the parent; after `fork()`, the child only traverses that list with `execve()`.
- `babet.signal` callbacks are not dispatched during the call.

See `Security` for the general security model.

Async completion — Babet 2.35.0

`proc:onExit(fn_or_nil)` observes a `babet.spawn` process through `gui.run()`. See the GUI loop for cancellation, results and lifetime. Pipes are not drained automatically: read incrementally, inherit streams or redirect them.

Process pipelines

Babet provides two complementary APIs for connecting programs through POSIX pipes without invoking a shell:

- `babet.pipeline()` runs the whole pipeline and captures its streams;
- `babet.spawnPipeline()` immediately returns a streaming process object.

In both cases, each stage's stdout is connected directly to the next stage's stdin. The parent only owns the first stage's stdin, the last stage's stdout, and one separate stderr stream per stage.

Shared command syntax

```
local commands = {
    { "printf", { "orange\napple\norange\n" } },
    { "grep", { "orange" } },
```

```
    { "sort", { "-u" } },
}
```

`commands` must be a dense array containing 2 to 32 stages. Each stage has one of these forms:

```
{ command }
{ command, args }
{ command, args, opts }
```

To provide `opts` without arguments, explicitly use `{ command, {}, opts }`; sparse arrays are rejected.

- `command` is a non-empty string without a NUL byte;
- `args` is a dense array of strings without NUL bytes;
- `opts.cwd` sets this stage's working directory;
- `opts.env` adds or replaces environment variables for this stage.

No string is parsed by a shell: globbing, redirection, variables, quoting and operators such as `|`, `&&` or `>` have no special meaning.

Global `cwd` and `env` options provide defaults. A local `cwd` replaces the global one. Local environment entries are merged last and win on duplicate names. Every command is validated before the first `fork()`.

As with `exec()` and `spawn()`, when a command contains no `/`, lookup uses the stage's final environment after merging the inherited environment, global options, and local options. An empty or relative `PATH` component is interpreted from that stage's effective `cwd`. Every ordered candidate list, `argv`, and `envp` is prepared before the first `fork()`; each child then traverses only that list with `execve()`, without parsing `PATH` or allocating memory.

`babet.pipeline()` — complete execution

Signature

```
result, err = babet.pipeline(commands [, opts])
```

Example:

```
local result, err = babet.pipeline({
    { "printf", { "orange\napple\norange\n" } },
    { "grep", { "orange" } },
    { "sort", { "-u" } },
}, {
    timeout = 10,
    max_output = 16 * 1024 * 1024,
})

assert(result, err)
print(result.stdout)
```

Options

Option	Type	Description
`cwd`	string	Default working directory for every stage.
`env`	string-to-string table	Variables merged with Babet's environment.
`stdin`	binary string	Data progressively written to the first stage, then stdin is closed.

Option	Type	Description
`timeout`	finite positive number, at most `10^12`	Global timeout in seconds, including launch, I/O and final wait.
`max_output`	integer from 1 to 2 GiB	Separate limit for final stdout and each stderr; default: 10 MiB.

Unknown keys are rejected. `stdin`, stdout and stderr are binary-safe and may contain NUL bytes. Timeout resolution is one millisecond; values above `10^12` seconds are rejected.

Result

Once every stage has launched, a non-zero exit or signal is a process result, not an API error. Babet returns:

```
{
    stdout = "...",
    stderr = { "stage 1 stderr", "stage 2 stderr", ... },
    stages = {
        {
            launched = true,
            code = 0,
            exited = true,
            signaled = false,
            signal = nil,
        },
        -- ...
    },
    code = 0,
    all_succeeded = true,
    failed_index = nil,
    timed_out = false,
    stdout_truncated = false,
    stderr_truncated = { false, false, ... },
}
```

`code` is the last stage's code, matching Unix pipeline convention. It is the normal exit status or `128 + signal_number`. In the pathological case where a status remains unavailable after bounded timeout cleanup, it is `-1` for that stage. `all_succeeded` is true only when every stage succeeded. `failed_index` identifies the first non-zero stage even when the last one succeeds.

A non-zero intermediate stage does not artificially stop the other programs. Normal POSIX pipe behaviour applies; an upstream producer may receive `SIGPIPE` after a downstream consumer closes its input.

Timeout, large streams and truncation

`pipeline()`'s timeout covers launch, stdin writes, stdout/stderr draining and the final wait. If it expires before every stage completes `chdir` and `exec`, the call returns `nil, err` with the message `"pipeline: launch timed out"` after rolling back stages already created. After a complete launch, expiry returns a result table with `timed_out == true`: Babet closes stdin, sends `SIGTERM` to every stage process group, then `SIGKILL` to survivors, and reaps direct children.

Descriptors are non-blocking and driven with `poll()`. The first stdin, final stdout and all stderr streams progress concurrently, preventing the usual large-volume deadlocks. Once `max_output`

is reached, extra bytes are discarded but the stream continues to be drained, and the matching truncation flag is set.

`babet.spawnPipeline()` — streaming

Signature

```
pipeline, err = babet.spawnPipeline(commands [, opts])
```

Global options:

Option	Type	Description
`cwd`	string	Default working directory for every stage.
`env`	string-to-string table	Variables merged with Babet's environment.
`launch_timeout`	finite positive number	Deadline covering only creation and complete launch of all stages, at most `INT_MAX` milliseconds (about 24.8 days).

`spawnPipeline()` accepts no `stdin`, `timeout`, or `max_output` option: the script streams data and applies its own limits. Success returns a userdata and `nil`; validation or launch failure returns `nil, err` after cleaning up any stage that was already created.

This API is intended for commands connected by pipes, not interactive programs. The first stage's standard input is always Babet's managed pipe, and no foreground-terminal handoff occurs. A stage that explicitly opens `/dev/tty` and tries to read from it may therefore be stopped by POSIX background-process-group rules. Use `babet.spawn()` with `stdin`, `stdout`, and `stderr` set to `"inherit"` for a tool that must interact directly with the user.

```
local pipeline, err = babet.spawnPipeline({
    { "cat" },
    { "tr", { "a-z", "A-Z" } },
})
assert(pipeline, err)

local written, write_err = pipeline:write("hello\n")
assert(written, write_err)
assert(pipeline:close_stdin())

local chunks = {}
while true do
    local data, read_err = pipeline:read_stdout(64 * 1024, 1)
    if data then
        chunks[#chunks + 1] = data
    elseif read_err == "closed" then
        break
    elseif read_err ~= "timeout" then
        error(read_err)
    end
end

local status, wait_err = pipeline:wait(5)
assert(status, wait_err)
```

```
assert(status.code == 0)
print(table.concat(chunks))
```

Read methods

```
data, err = pipeline:read_stdout([max_bytes [, timeout]])
data, err = pipeline:read_stderr(stage [, max_bytes [, timeout]])
```

- `max_bytes` defaults to 64 KiB and must be between 1 and 16 MiB;
- `timeout` is in seconds, defaults to zero, and must be finite and non-negative;
- `stage` ranges from 1 to the number of stages;
- invalid arity, stage index, or `max_bytes` raises a Lua error, matching `babet.spawn()` methods;
- an invalid timeout returns `nil, err`;
- success returns a non-empty binary string and `nil`;
- no immediately available data returns `nil, "timeout"`;
- EOF, or reading an already closed stream, returns `nil, "closed"`.

Each call reads at most one chunk. There is no hidden capture: the script must retain, limit, or discard received data itself. Stderr streams remain independent, so every stream capable of filling its pipe must be drained.

Writing and closing stdin

```
count, err = pipeline:write(data [, timeout])
ok, err = pipeline:close_stdin()
```

`write()` accepts a binary string and may write only part of it; `count` is the number of bytes actually transmitted. An empty string returns `0, nil`. A timeout leaves the object reusable. Stdin closed by the script or downstream process returns `nil, "closed"` without delivering `SIGPIPE` to Babet itself.

`close_stdin()` closes the first stage's input and is idempotent. Call it after the last byte, otherwise a program waiting for EOF may never terminate.

Complete-write pattern:

```
local offset = 1
while offset <= #data do
    local count, err = pipeline:write(data:sub(offset), 1)
    if count then
        offset = offset + count
    elseif err ~= "timeout" then
        error(err)
    end
    -- Drain stdout and stderr during large transfers as well.
end
assert(pipeline:close_stdin())
```

State and identifiers

```
running = pipeline:is_running([stage])
pids = pipeline:pids()
```

Without an index, `is_running()` is true while at least one direct stage has not been reaped. With an index, it describes only that stage. It refreshes statuses through `waitpid(..., WNOHANG)` without consuming the final result: `wait()` remains available afterwards.

`pids()` returns direct PIDs in stage order. They are diagnostic values; scripts must not assume that a PID remains valid after the pipeline exits.

Waiting and final status

```
status, err = pipeline:wait([timeout])
```

Without a timeout, `wait()` waits for every direct child. With a finite non-negative timeout, `nil, "timeout"` leaves the pipeline intact and reusable. Once every status is known, subsequent calls are idempotent and return the same result:

```
{
    stages = {
        { launched = true, code = 0, exited = true,
          signaled = false, signal = nil },
        -- ...
    },
    code = 0,
    all_succeeded = true,
    failed_index = nil,
}
```

`wait()` **does not drain** stdout or stderr. A pipeline that fills a pipe may therefore block before exiting. Streams must be read while it runs, exactly as with `babet.spawn()`.

Explicit termination

```
status, err = pipeline:terminate([grace_period])
status, err = pipeline:kill()
```

`terminate()` sends `SIGTERM` to every process group, gives processes `grace_period` seconds—2 seconds by default—to exit, then sends `SIGKILL` to survivors. The grace period must fit within `INT_MAX` milliseconds, about 24.8 days. `kill()` sends `SIGKILL` immediately. Both close stdin and return the same status table as `wait()` after direct children are reaped.

Each stage owns its own process group. Before reaping the direct process, Babet removes descendants that still remain in that group, including normal completion where a command exits without waiting for a background child. The leader PID therefore remains reserved until signalling and cannot be recycled for an unrelated process.

For `terminate()`, every group member receives `SIGTERM` at the beginning of the grace period. Direct leaders that have already exited deliberately remain unreaped for the whole period: their PID, which is also used as the PGID, cannot be recycled before the optional final `SIGKILL` sent to the group. A stage that deliberately creates a new session or process group escapes this POSIX guarantee. A script that knows its processes are cooperative may select a shorter grace period, for example `pipeline:terminate(0.05)`.

`close()`, GC and Lua `<close>`

```
ok, err = pipeline:close()
```

`close()` is idempotent. It closes descriptors, attempts `SIGTERM`, uses `SIGKILL` when necessary, and reaps direct children. After closure, reads and writes return `nil, "closed"`, and `is_running()` returns `false`. A fully collected status remains available from `wait()`.

The same cleanup is used by `__gc` and `__close`. Recommended use:

```
local pipeline <close> = assert(babet.spawnPipeline({
    { "producer" },
    { "consumer" },
}))
```

GC is a safety net, not a synchronization mechanism. Use `wait()`, `terminate()`, or `close()` explicitly when cleanup timing matters. When the pipeline is already known to be finished, call `wait()` (or

observe `is_running() == false`) before `close()` if immediate closure matters: this lets Babet use the completed-pipeline fast path. Calling `close()` without having reaped a merely-finished pipeline may still spend the normal 0.5 s grace period protecting process-group cleanup.

Launch errors and reuse

Validation, pipe, `fork`, `chdir`, `exec`, and `launch_timeout` failures return `nil, err`. When a stage fails during launch, every already-created stage is terminated and reaped; Lua never receives a partially valid object. The launcher itself enforces the public 32-stage limit independently of the Lua binding's validation.

After the first `fork()`, and throughout synchronous `pipeline()` capture, Babet retains an emergency owner for descriptors and direct children. If an internal C++ exception occurs, emergency cleanup closes streams, immediately sends `SIGKILL` only to groups whose leader has not already been reaped, and then performs a bounded reap attempt. `terminate()`'s grace period is not used during stack unwinding. For `spawnPipeline()`, resources are transferred to the userdata only after launch has completed successfully.

Non-destructive `"timeout"` and `"interrupted"` errors allow another call. `"closed"` means the relevant stream or object can no longer perform that operation. Other system errors are returned as text and should be handled as such.

FS — files, directories, paths, searching, copying, permissions, and checksums

The functions in this chapter are exposed directly on the global `babet` table. There is no `babet.fs` subtable: this flat namespace is historical and is now part of the stable API.

The FS module covers:

- checking whether a path exists and what type it has;
- creating, atomically writing, and removing files and directories;
- changing the current working directory;
- lexical path manipulation;
- listing, iterating, and recursively searching a tree;
- copying and moving directory trees;
- symbolic links;
- Unix permissions, owners, and groups;
- CRC-32, MD5, SHA, and BLAKE2 checksums.

Module contents

- General conventions
- API overview
- Existence, type, and size
  - `fileExists`
  - `isFile` / `isfile`
  - `isDir` / `isdir`
  - `fileSize`
- Current working directory
  - `currentDir`
  - `chdir`
- Lexical path manipulation
  - `getBasename`
  - `getFilename`
  - `getExtension`
  - `getPath`

  - joinPath
- Create, remove, rename, and link
  - touch
  - lockFile — exclusive file locks
  - writeFileAtomic
  - mkdir
  - remove
  - rmdir
  - rmdirAll
  - rename
  - link
- List, iterate, and search
  - listFiles
  - createFileIterator
  - find
- Copy and move
  - copy
  - copyTree
  - moveTree
- Unix permissions and attributes
  - getMode / setMode
  - getAttributes / setAttributes
  - symlinkAttr / symlinkattr
- File checksums
- In-memory CRC-32
- Error contract
- Design decisions and limitations

General conventions

Paths

Paths are Lua strings and may be relative or absolute.

Babet does not automatically perform:

- ~ expansion;
- variable expansion such as `$HOME`;
- glob expansion such as `*.lua`;
- general `.` and `..` normalization in lexical path helpers.

For example:

```
local home = assert(babet.env("HOME"))
local config = babet.joinPath(home, ".config", "my-app")
```

A NUL byte in a path is always rejected. Depending on the function, a bad signature raises a Lua error, while a filesystem failure is generally returned as `(nil, err)`.

Symbolic links

There is no single symlink rule that applies to every function:

- `isFile`, `isDir`, `fileSize`, `getMode`, `getAttributes`, and checksum functions follow a valid link to its target;
- `remove` removes the link itself, including a dangling link;
- `rmdir` and `rmdirAll` reject a symlink root;
- `symlinkAttr` changes the owner and group of the link itself;
- `listFiles`, `createFileIterator`, and `find` do not recurse into a symlink to a directory;

- `copyTree` and `moveTree` reject a symlink source or destination root, but handle links found inside the tree.

Each function documents its exact behavior below.

Result order

`listFiles`, `createFileIterator`, and `find` preserve the order supplied by the filesystem. That order is not sorted and must not be treated as stable.

Sort explicitly when order matters:

```lua
local files = assert(babet.listFiles("documents", true))
table.sort(files)
```

Return values

The most common convention is:

```lua
local value, err = function_name(...)
if not value then
    print(err)
end
```

A successful action generally returns `(true, nil)`. A few pure functions and iterator methods use a different shape; those exceptions are called out in their sections.

API overview

Check and inspect

Function	Successful result
`babet.fileExists(path)`	`(boolean, nil)` — true only for a regular file
`babet.isFile(path)`	`(boolean, nil)` — same functional predicate as `fileExists`
`babet.isDir(path)`	`(boolean, nil)` — true for a directory
`babet.fileSize(path)`	`(integer, nil)` — regular-file size in bytes
`babet.currentDir()`	`(string, nil)` — current working directory
`babet.getMode(path)`	`(integer, nil)` — Unix mode `0000..07777`
`babet.getAttributes(path)`	`(table, nil)` — `{mode, owner, group}`

Manipulate paths

Function	Successful result
`babet.getBasename(path)`	`(string, nil)`
`babet.getFilename(path)`	`(string, nil)`
`babet.getExtension(path)`	`(string, nil)`
`babet.getPath(path)`	`(string, nil)`
`babet.joinPath(a, b, ...)`	`string`
`babet.joinPath({a, b, ...})`	`string`

Act on the filesystem

Function	Successful result
`babet.touch(path)`	`(true, nil)`
`babet.writeFileAtomic(path, data, opts?)`	`(true, nil)` — atomic and durable binary publication; detailed chapter
`babet.mkdir(path)`	`(true, nil)` — recursive and idempotent
`babet.remove(path)`	`(true, nil)` — regular file or symlink
`babet.rmdir(path)`	`(true, nil)` — real, empty directory
`babet.rmdirAll(path)`	`(true, nil)` — real directory, recursive removal
`babet.rename(source, destination)`	`(true, nil)`
`babet.link(target, linkpath)`	`(true, nil)` — creates a symlink
`babet.chdir(path)`	`(true, nil)`

List and search

Function	Successful result
`babet.listFiles(path, recursive?)`	`(table, nil)`
`babet.createFileIterator(path, recursive?)`	`(iterator, nil)`
`babet.find(path, opts?)`	`(table, nil)`

Copy and move

Function	Successful result
`babet.copy(source, destination)`	`(true, nil)` — one file
`babet.copyTree(source, destination, continue_on_error?)`	`(true, nil)` — one tree
`babet.moveTree(source, destination)`	`(true, nil)` — one tree

Change attributes

Function	Successful result
`babet.setMode(path, mode)`	`(true, nil)`
`babet.setAttributes(path, uid, gid, mode?)`	`(true, nil)`
`babet.symlinkAttr(path, uid, gid)`	`(true, nil)`

Existence, type, and size

`babet.fileExists(path)`

Checks whether `path` designates a **regular file**.

```
local exists, err = babet.fileExists("report.txt")
```

Results:

- regular file: `(true, nil)`;
- directory, dangling link, or missing path: `(false, nil)`;
- valid link to a regular file: `(true, nil)`;
- genuine inspection error: `(nil, err)`.

Despite its historical name, `fileExists` is not true for every kind of path. Use `isDir` for a directory.

```
local is_file = assert(babet.fileExists("archive.tar"))
local is_dir  = assert(babet.isDir("backups"))
```

`babet.isFile(path)` and `babet.isfile(path)`

`isFile` is the canonical name. `isfile` is a deprecated compatibility alias.

It tests the same kind of path as `fileExists`: a regular file, following a valid symlink if present.

```
local regular, err = babet.isFile("program.lua")
if err then
    print("Cannot inspect path:", err)
elseif regular then
    print("This is a regular file")
end
```

A missing path returns `(false, nil)`, not an error.

`babet.isDir(path)` and `babet.isdir(path)`

`isDir` is the canonical name. `isdir` is a deprecated alias.

```
local directory, err = babet.isDir("assets")
```

Results:

- real directory: `(true, nil)`;
- valid link to a directory: `(true, nil)`;
- file, dangling link, or missing path: `(false, nil)`;
- access or inspection error: `(nil, err)`.

`babet.fileSize(path)`

Returns the size of a regular file in bytes.

```
local size, err = babet.fileSize("video.mp4")
if not size then
    print("Cannot read size:", err)
else
    print(size, "bytes")
end
```

The function follows a valid symlink to a file. Directories and other file types are rejected.

```
local size, err = babet.fileSize("documents")
-- size == nil; err says the path is not a regular file
```

Current working directory

`babet.currentDir()`

Returns the process current working directory.

```
local cwd, err = babet.currentDir()
assert(cwd, err)
print(cwd)
```

The result is the path supplied by the filesystem and is normally absolute.

`babet.chdir(path)`

Changes the current working directory of the entire process.

```
local previous = assert(babet.currentDir())
assert(babet.chdir("/tmp"))
```

```
print(assert(babet.currentDir()))
assert(babet.chdir(previous))
```

The path is resolved to its canonical form: `.` and `..` components and symlinks are resolved before changing directory.

The working directory is shared by all threads. Consequently, `chdir` is permanently rejected after the first `babet.workers.spawn()`, even after that worker has completed. The first GTK loading attempt through `gui.available()` or `gui.init()` triggers the same rule, even on failure. Configure the directory before those calls; `babet.currentDir()` remains available as a query. See the GUI contract.

```
assert(babet.chdir("/srv/my-app")) -- do this before any spawn
local worker = assert(babet.workers.spawn("return true"))

local ok, err = babet.chdir("/tmp")
-- ok == nil; err explains that chdir is forbidden after workers.spawn
```

Lexical path manipulation

These helpers only process strings. They do not check whether the path exists.

babet.getBasename(path)

Returns the last component, including its extension.

```
local name = assert(babet.getBasename("/var/log/app.log"))
print(name) -- app.log
```

A path with no usable last component, such as `/`, returns `(nil, err)`.

babet.getFilename(path)

Returns the last component without its final extension.

```
print(assert(babet.getFilename("report.tar.gz"))) -- report.tar
print(assert(babet.getFilename("README")))        -- README
```

babet.getExtension(path)

Returns the final extension, including the dot.

```
print(assert(babet.getExtension("report.tar.gz"))) -- .gz
```

A name without an extension returns `(nil, err)`, not an empty string.

```
local ext, err = babet.getExtension("README")
-- ext == nil
```

babet.getPath(path)

Returns the lexical parent directory.

```
print(assert(babet.getPath("/var/log/app.log"))) -- /var/log
```

A simple name with no parent component, such as `app.log`, returns `(nil, err)` rather than `"."`.

`babet.joinPath(...)`

Two forms are accepted:

```
local path = babet.joinPath("var", "lib", "my-app", "data.db")
```

```
local path = babet.joinPath({ "var", "lib", "my-app", "data.db" })
```

At least two non-empty segments are required. On success, `joinPath` returns **one value**, the joined string. On failure it returns `(nil, err)`.

In the table form, Babet reads the stored array entries directly in raw `1..n` order. `__len` and `__index` metamethods are not invoked and cannot manufacture path segments. A missing raw entry is therefore rejected instead of being supplied through `__index`. Non-array keys are not path segments and are ignored.

The function only adjusts the `/` separator at segment boundaries:

```
print(babet.joinPath("/var/", "/log", "app.log"))
-- /var/log/app.log
```

It does not normalize `.` or `..`, and a later absolute-looking segment does not reset the path:

```
print(babet.joinPath("base", "..", "other")) -- base/../other
print(babet.joinPath("base", "/other"))       -- base/other
```

Do not mix the table form with separate arguments. When the first argument is a table, that table is the complete segment list.

Create, remove, rename, and link

`babet.touch(path)`

Creates an empty file when the path does not exist, or updates the last-write time of an existing path.

```
assert(babet.touch("cache/ready.flag"))
```

The parent directory must already exist; `touch` does not create it.

```
local ok, err = babet.touch("missing/nested/file.txt")
-- ok == nil if missing/nested does not exist
```

Create parents first when needed:

```
assert(babet.mkdir("cache/images"))
assert(babet.touch("cache/images/index.dat"))
```

For an existing path, the function updates its timestamp; like the Unix `touch` command, it can therefore also touch a directory. A valid final symlink is followed to its target.

`touch` never opens an existing path with a truncating mode. The parent and existing target are pinned before the operation, so a file that appears during a concurrent create race is reused without losing its contents. A dangling final symlink is refused: Babet does not create the missing target through a mutable symlink path.

`babet.mkdir(path)`

Creates a directory with `mkdir -p` semantics:

- creation is **always recursive**;
- missing parents are created;
- calling it again on an existing directory succeeds;
- a file or another non-directory blocking the path returns `(nil, err)`.

```lua
assert(babet.mkdir("data/cache/images"))
-- creates data, data/cache, and data/cache/images as needed

assert(babet.mkdir("data/cache/images"))
-- also succeeds when the directory already exists
```

There is **no option to disable recursion**. To require an already-existing parent, check it before calling `mkdir`:

```lua
local parent = "data/cache"
assert(babet.isDir(parent), "parent must already exist")
assert(babet.mkdir(babet.joinPath(parent, "images")))
```

New directories use mode `0777` filtered through the process umask. No source directory mode is involved.

`babet.remove(path)`

Removes only:

- a regular file;
- a symbolic link, valid or dangling.

```lua
assert(babet.remove("temp.txt"))

-- the link is removed, not its target
assert(babet.link("important-file.txt", "shortcut"))
assert(babet.remove("shortcut"))
```

Directories, FIFOs, sockets, devices, and other special types are rejected.

```lua
local ok, err = babet.remove("my-directory")
-- ok == nil; use rmdir or rmdirAll
```

`babet.rmdir(path)`

Removes a **real empty directory**.

```lua
assert(babet.mkdir("empty-cache"))
assert(babet.rmdir("empty-cache"))
```

It rejects:

- a non-empty directory;
- a file;
- a symlink, even when it points to a directory or ends in `/` or `/.`;
- a missing path.

```lua
local ok, err = babet.rmdir("non-empty-directory")
```

`babet.rmdirAll(path)`

Recursively removes a **real directory root** and all of its contents.

```lua
assert(babet.rmdirAll("temporary-build"))
```

The root must exist and be a real directory. A file or a symlink to a directory is rejected, including paths ending in `/`, `/.` or `/./`.

Symlinks *inside* the removed directory are deleted as directory entries; their external targets are not traversed.

```lua
local is_dir = babet.isDir("temporary-build")
if is_dir then
```

```
    assert(babet.rmdirAll("temporary-build"))
end
```

babet.rename(source, destination)

Renames or moves an entry on the same filesystem.

```
assert(babet.rename("old.txt", "new.txt"))
```

Files, directories, and symlinks are accepted, including dangling links. The destination parent directory is not created.

On Linux, the function follows `rename(2)` semantics: an existing destination may be replaced when the entry types are compatible. Crossing filesystems fails; `rename` does not perform a copy-and-delete fallback.

```
local ok, err = babet.rename("/tmp/a.txt", "/other-mount/a.txt")
-- may fail with cross-device link
```

Use `moveTree` to move a directory tree with a cross-filesystem fallback.

babet.link(target, linkpath)

Always creates a **symbolic link**, never a hard link.

The target is stored exactly as supplied:

```
assert(babet.link("../shared/config.toml", "app/config.toml"))
```

A relative target stays relative, and a missing target is allowed:

```
assert(babet.link("future-file.txt", "dangling-link"))
```

Unlike `touch` and `copy`, `link` recursively creates explicit parent directories of `linkpath`:

```
assert(babet.link("../../data.db", "a/b/c/database"))
-- creates a/, a/b/, and a/b/c/ as needed
```

The link path must not already be occupied.

List, iterate, and search

The three APIs do not return paths in the same form:

Function	Returned path form
`listFiles(root, ...)`	relative to `root`
`createFileIterator(root, ...)`	as produced from `root`: prefixed by the supplied root path
`find(root, ...)`	as produced from `root`: prefixed by the supplied root path

babet.listFiles(path, recursive?)

Returns an array containing regular files only. Directories are never included.

Non-recursive listing — the default

```
local files, err = babet.listFiles("assets")
assert(files, err)

for _, relative_path in ipairs(files) do
```

```
    print(relative_path)
end
```

Only files directly contained in `assets` are returned.

Recursive listing

```lua
local files, err = babet.listFiles("assets", true)
assert(files, err)

table.sort(files)
for _, relative_path in ipairs(files) do
    print(relative_path)
end
```

Paths are relative to the requested root:

```
logo.png
icons/open.png
icons/close.png
```

not:

```
assets/logo.png
assets/icons/open.png
```

Symlinks

- a valid symlink to a regular file is included under the link name;
- a dangling link is skipped;
- a symlink to a directory is neither included as a file nor traversed;
- directory-symlink loops therefore cannot cause infinite recursion.

```lua
local files = assert(babet.listFiles("collection", true))
-- collection/external -> /other/directory is not followed
```

The second argument is optional and defaults to `false`. When present, it must be a strict Lua boolean; numbers and strings are not coerced. The function accepts exactly one or two arguments.

`babet.createFileIterator(path, recursive?)`

Creates a genuinely lazy userdata with two methods:

- `iterator:next()` — `(path, nil)`, `(nil, err)`, or `(nil, nil)` at end;
- `iterator:close()` — explicitly releases the native iterator.

Only the directory iterator itself is created initially. The tree is advanced one entry at a time by `next()`, so a large hierarchy is not copied into a `std::vector` before the first result and memory use no longer grows with the complete number of files.

Non-recursive

```lua
local iterator, err = babet.createFileIterator("assets")
assert(iterator, err)

while true do
    local path, next_err = iterator:next()
    assert(not next_err, next_err)
    if path == nil then
        break
    end
```

```
    print(path) -- assets/logo.png, etc.
end

iterator:close()
```

Recursive

```
local iterator = assert(babet.createFileIterator("assets", true))
while true do
    local path, next_err = iterator:next()
    assert(not next_err, next_err)
    if not path then break end
    print(path)
end
iterator:close()
```

Opening the root can still fail immediately and returns `(nil, err)`. Errors encountered later while inspecting or advancing the directory are deferred to the corresponding `next()` call as `(nil, err)`. End of iteration is the distinct result `(nil, nil)`.

Symlink behavior:

- valid link to a regular file: included under the link path;
- dangling link, loop, or inaccessible target: skipped;
- link to a directory: not traversed;
- genuine entry-inspection or traversal-increment error: reported by `next()`.

Existing code that reads only the first return value remains compatible, but it cannot distinguish end of iteration from a deferred traversal error; new code should always inspect the second result. Calling `next()` after `close()` raises a Lua error. Omitting `close()` is not fatal: the garbage collector eventually releases the object. The optional `recursive` argument must be a strict Lua boolean, and the constructor accepts exactly one or two arguments.

`babet.find(path, opts?)`

Recursively searches the entries of a directory and returns an array of paths.

`path` must be a strict Lua string. `opts` must be a table or `nil`, and the function accepts exactly one or two arguments. Option fields are also strict: depths are Lua integers, every name/path/regex/glob field is a Lua string, and `xdev` is a boolean.

These calls are equivalent:

```
local entries = assert(babet.find("src"))
local entries = assert(babet.find("src", nil))
```

Without options, both files **and** directories are included. The root itself is not included.

Options

Option	Type	Default	Effect
`type`	string	no filter	`"f"` for files, `"d"` for directories
`glob`	string	absent	bounded glob on the complete basename, case-sensitive
`iglob`	string	absent	bounded glob on the complete basename, ASCII case-insensitive

Option	Type	Default	Effect
`path_glob`	string	absent	bounded glob on the complete returned path, case-sensitive
`path_iglob`	string	absent	bounded glob on the complete returned path, ASCII case-insensitive
`name`	string	absent	RE2 regex on the complete basename, case-sensitive
`iname`	string	absent	RE2 regex on the complete basename, case-insensitive
`path`	string	absent	RE2 regex searched within the full path
`mindepth`	integer	`0`	minimum included depth
`maxdepth`	integer	practically unlimited	maximum included depth
`xdev`	boolean	`false`	does not descend into a directory whose `st_dev` differs from the root

All supplied options are combined with logical **AND**. An empty pattern acts like an absent filter, matching the historical regex behavior.

Safe glob filters Use the glob fields for ordinary filename filtering, especially when a pattern may come from an untrusted user:

```
local lua_files = assert(babet.find("src", {
    type = "f",
    glob = "*.lua",
}))

local images = assert(babet.find("assets", {
    type = "f",
    iglob = "*.jpg",
}))
```

The glob language is deliberately small:

- `*`: zero or more bytes except `/`;
- `**`: zero or more bytes, including `/`;
- `?`: exactly one byte except `/`;
- `\x`: literal byte `x`.

Matching is anchored to the complete basename or path. Brackets, braces, and shell extglob syntax have no special meaning and are matched literally. Each pattern is limited to 4096 bytes. The matcher uses no recursive backtracking; its work is bounded polynomially by the pattern and candidate lengths.

`path_glob` and `path_iglob` match the complete returned path with `/` as the separator. Use ** when a wildcard must cross directories:

```
local tests = assert(babet.find(".", {
    type = "f",
```

```
    path_glob = "**/tests/**/*.lua",
}))
```

The insensitive variants fold ASCII letters only. UTF-8 bytes outside ASCII are compared exactly, which keeps behavior independent of the process locale.

Bounded RE2 regular expressions The historical `name`, `iname`, and `path` fields now use the statically linked RE2 engine. `name` and `iname` match the complete basename:

```
local images = assert(babet.find("assets", {
    type = "f",
    iname = ".*\\.(png|jpg|jpeg)$",
}))
```

`path` performs a partial regex search within the complete path:

```
local tests = assert(babet.find(".", {
    type = "f",
    path = "/tests/",
}))
```

Each regex is limited to 4096 bytes and compiled with a 1 MiB RE2 memory budget. Matching is linear-time; when the DFA cache reaches its budget, RE2 falls back to its bounded NFA instead of backtracking exponentially. Babet uses RE2's Latin-1 mode so every Linux pathname byte remains matchable, including names that are not valid UTF-8. `iname` disables case sensitivity through RE2; no process locale is consulted.

RE2 deliberately rejects constructs that require backtracking or non-regular state, notably backreferences and look-around assertions. Such a pattern makes the whole call fail with `(nil, err)`. Use the safe glob fields for simple filename filters and RE2 when grouping, alternation, character classes, or bounded repetitions are genuinely useful.

Limit depth Depth `0` is the root's immediate children:

```
local direct_children = assert(babet.find("src", {
    maxdepth = 0,
}))
```

Children of a subdirectory are at depth `1`:

```
local one_level_below = assert(babet.find("src", {
    mindepth = 1,
    maxdepth = 1,
}))
```

`mindepth` filters results only; shallower directories are still traversed to reach requested depths.

Live trees and disappearing directories `babet.find()` traverses with an explicit stack of directory iterators. Before opening a child directory, Babet advances the parent iterator to the next sibling. If the child directory disappears in that interval, its `ENOENT` is local to that vanished subtree: the search skips it and continues from the already-positioned parent.

The same rule applies if a directory currently being enumerated disappears: that exhausted frame is discarded and traversal resumes in its parent. This behavior applies with or without `xdev`, preserves pre-order results for entries that still exist, and does not turn other failures into partial success. Permission errors, I/O errors, symlink loops, and every error other than `ENOENT` still fail the complete call with `(nil, err)`.

Stay on the root filesystem `xdev = true` records the Linux `st_dev` of the directory actually traversed, then prevents recursion into any directory on another device. The option is a strict boolean and defaults to `false`, so omitting it preserves every historical traversal.

```
local local_entries = assert(babet.find("/srv/application", {
    xdev = true,
}))
```

As with `find -xdev`, the foreign mount point itself remains visible and may match `type`, `name`, `path`, glob filters, and depth limits. Only its children are pruned. For example, this search may return the mounted directory `/srv/application/cache`, but no file stored below that mount:

```
local local_logs = assert(babet.find("/srv/application", {
    xdev = true,
    type = "f",
    path_iglob = "**/*.log",
    maxdepth = 8,
}))
```

The comparison uses the target device of a valid root-directory symlink. Directory symlinks encountered below the root remain non-followed. The check is strictly based on `st_dev`, like `find -xdev`: a Btrfs subvolume may therefore be pruned, while a bind mount of the same filesystem keeps the same `st_dev` and is not pruned.

If a candidate disappears during the additional `xdev` inspection, the same `ENOENT`-only live-tree rule applies and the vanished subtree is skipped. An error while inspecting the root, or any other error while inspecting a candidate descent point, fails the entire call with `(nil, err)`.

Symlinks and errors Traversal does not recurse into directory symlinks. Type checks do follow a valid target, however: a symlink to a file may match `type = "f"`, and a symlink to a directory may appear with `type = "d"` without its contents being traversed.

An invalid or oversized regex, invalid or oversized glob, or traversal error fails the entire call with `(nil, err)`. There is no “continue after errors” mode for `find`.

Copy and move

`babet.copy(source, destination)`

Copies one file using `std::filesystem::copy_file` and overwrites an existing file destination.

```
assert(babet.copy("config/default.toml", "config/local.toml"))
```

The destination parent directory must already exist:

```
assert(babet.mkdir("backup/config"))
assert(babet.copy("config/app.toml", "backup/config/app.toml"))
```

Important behavior:

- a valid source symlink is followed and produces a new regular file;
- a valid destination symlink is followed: the link target is overwritten;
- source and destination directories are rejected;
- ordinary source permissions are copied according to filesystem semantics; owner, group, and timestamps are not guaranteed;
- this operation does not use `copyTree`’s hardened destination confinement.

Use `copyTree` for a tree or when destination symlink redirection must be prevented.

`babet.copyTree(source, destination, continue_on_error?)`

Recursively copies a source directory to a destination directory.

```lua
assert(babet.copyTree("site", "backup/site"))
```

Default value of `continue_on_error` The optional third argument defaults to **`true`**. When present it must be a strict Lua boolean; the function accepts exactly two or three arguments.

For an error limited to one entry:

1. a warning is written to `stderr`;
2. copying continues with other entries when possible;
3. the final call returns `(nil, "completed with warnings ...")`.

A partial success is therefore never reported as a full success.

```lua
local ok, err = babet.copyTree("source", "destination")
if not ok then
    print(err) -- may report warnings after a partial copy
end
```

Strict mode Pass `false` explicitly to stop at the first error:

```lua
local ok, err = babet.copyTree("source", "destination", false)
assert(ok, err)
```

Pass `true` explicitly when you want the continuation choice to be visible:

```lua
local ok, err = babet.copyTree("source", "destination", true)
```

Destination and merging

- the destination is recursively created when absent;
- existing destinations are merged;
- existing regular files are replaced;
- a pre-existing destination symlink, either as an intermediate component or final file, is rejected;
- the trees must be disjoint: the destination must not be the source itself, one of its descendants, or one of its ancestors;
- source and destination roots must not be symlinks.

Metadata When a new file is created:

- ordinary source `rwx` bits are preserved;
- setuid, setgid, and sticky bits are removed;
- the new inode's owner and group are determined by the system for the caller;
- timestamps are not preserved.

Created directories use `0777` filtered through the umask. Source directory mode, owner, group, and timestamps are not reproduced.

Internal symlinks Links inside the source tree are recreated:

- relative targets stay unchanged;
- dangling links stay dangling;
- an absolute target inside the source is rewritten to its corresponding destination path;
- an absolute external target stays unchanged.

Directory symlinks are never traversed as directories.

`babet.moveTree(source, destination)`

Recursively moves a directory tree.

```lua
assert(babet.moveTree("staging/site", "public/site"))
```

Fast path When the destination does not exist, the move remains on one filesystem, and no internal absolute link needs rewriting, Babet renames the whole tree. This is fast and preserves inodes and their metadata.

Merge or cross-filesystem fallback When the destination exists, the move crosses a filesystem, or an internal link needs rewriting, Babet uses an entry-by-entry fallback:

- scan the complete source before the first modification;
- create destination directories;
- create destination symlinks;
- move other entries;
- remove only scanned symlinks, then empty directories from deepest to root.

Final cleanup never recursively deletes the remaining contents. A file added after the scan stays in the source: its non-empty directory makes the move fail even if other entries have already been transferred. Link and directory identities are checked again before removal; this does not make the overall operation transactional against concurrent mutations.

In this fallback, a file copied to a new inode preserves ordinary `rwx` bits but not special bits, owner, group, or timestamps. Created directories follow the umask.

Across filesystems, only regular files support the copy-and-remove fallback. A FIFO is rejected without waiting for a writer. Renaming a FIFO within one filesystem remains supported.

Before opening a copy source for reading, Babet inspects its inode through a Linux `O_PATH` descriptor, which does not open devices for I/O. Only a regular file is then opened through `/proc/self/fd`, retaining that inode even if its pathname has been replaced. This step, shared by `copyTree` and `moveTree`'s fallback, requires procfs; failure rejects the copy without publishing its destination file.

After an EXDEV copy, `moveTree` keeps the source inode pinned and compares its identity with the source pathname before removal. A detected replacement returns an error, preserving the replacement and the already published copy. This does not make `lstat` followed by `unlink` atomic: avoid concurrent changes to the source tree during the move.

Safety checks

- source must be a real directory, not a symlink;
- destination root must not be a symlink;
- destination cannot be the source, a descendant, or an ancestor of the source, including after path resolution;
- a pre-existing symlink in a merge destination is rejected;
- internal links use the same rewriting rules as `copyTree`.

`moveTree` reduces partial-state risk by scanning first and preparing symlinks before moving files. It is not a general transaction, however: an error after several file moves may leave some entries in the source and others in the destination. Such an error requires checking both trees manually.

Unix permissions and attributes

Modes are integers. Lua has no `0o755` literal; use an octal string where the API accepts it, or `tonumber("755", 8)`.

```
local mode_755 = tonumber("755", 8)
```

`babet.getMode(path)` and `babet.setMode(path, mode)`

`getMode` returns ordinary permissions and special bits in the range `0000..07777`.

```
local mode, err = babet.getMode("run.sh")
assert(mode, err)
print(string.format("%04o", mode))
```

`setMode` accepts two forms:

```
assert(babet.setMode("run.sh", "755"))

assert(babet.setMode("run.sh", tonumber("755", 8)))
```

A string is interpreted in base 8. A number is taken as the direct integer value. Non-integers, negative values, and values above `07777` are rejected.

Special bits may be requested explicitly:

```
assert(babet.setMode("tool", "4755"))
local mode = assert(babet.getMode("tool"))
assert(mode == tonumber("4755", 8))
```

Both functions follow a symlink to its target.

`babet.getAttributes(path)`

Returns exactly these three fields:

```
local attrs, err = babet.getAttributes("file.txt")
assert(attrs, err)

print(attrs.mode)  -- integer 0000..07777
print(attrs.owner) -- UID
print(attrs.group) -- GID
```

The table contains no size, mtime, or file type.

The function follows a symlink to its target.

`babet.setAttributes(path, uid, gid, mode?)`

Changes owner and group, then optionally the mode.

Without a mode:

```
local attrs = assert(babet.getAttributes("file.txt"))
assert(babet.setAttributes(
    "file.txt",
    attrs.owner,
    attrs.group
))
```

With a mode:

```
assert(babet.setAttributes(
    "file.txt",
    1000,
    1000,
    tonumber("640", 8)
))
```

Unlike `setMode`, the fourth argument must be an **integer**; a string such as `"640"` is not accepted.

Validation before mutation:

- UID and GID must be non-negative integers in the `uid_t` and `gid_t` range;
- mode must be between `0` and `07777`.

POSIX provides no atomic operation combining `chown` and `chmod`. Babet resolves the path once, pins that target, and performs the initial metadata read, `chown`, optional `chmod`, and any rollback on the same inode. Replacing the pathname during the call therefore cannot redirect the later phase to

a new target. If `chmod` fails after a successful `chown`, Babet attempts to restore the original owner, group, and mode; an incomplete rollback is reported in the error.

The call follows a final symlink, but the target selected when the operation starts remains pinned even if the link is changed concurrently. Required privileges depend on the system and process identity; changing owner normally requires root privileges.

`babet.symlinkAttr(path, uid, gid)` and `babet.symlinkattr(...)`

`symlinkAttr` is the canonical name. `symlinkattr` is a deprecated alias.

The function uses `lchown` and therefore does not follow a symbolic link:

```
assert(babet.symlinkAttr("shortcut", 1000, 1000))
```

It changes the UID and GID of the link itself, not of its target. It does not change mode.

UID and GID are validated as in `setAttributes`.

File checksums

All functions below stream the file and return lowercase hexadecimal text:

Function	Algorithm	Hex length	Recommended use
`babet.crc32sum(path)`	CRC-32 IEEE	8	accidental-error detection only
`babet.md5sum(path)`	MD5	32	legacy compatibility, not security
`babet.sha1sum(path)`	SHA-1	40	legacy compatibility, not security
`babet.sha256sum(path)`	SHA-256	64	common cryptographic integrity
`babet.sha384sum(path)`	SHA-384	96	cryptographic integrity
`babet.sha512sum(path)`	SHA-512	128	cryptographic integrity
`babet.sha3_256sum(path)`	SHA3-256	64	cryptographic integrity
`babet.sha3_384sum(path)`	SHA3-384	96	cryptographic integrity
`babet.sha3_512sum(path)`	SHA3-512	128	cryptographic integrity
`babet.blake2b512sum(path)`	BLAKE2b-512	128	cryptographic integrity
`babet.blake2s256sum(path)`	BLAKE2s-256	64	cryptographic integrity

Example:

```
local digest, err = babet.sha256sum("release.tar.gz")
assert(digest, err)
print(digest)
```

Check against an expected value:

```
local expected = "..."
local actual = assert(babet.sha256sum("release.tar.gz"))
assert(actual == expected, "checksum mismatch")
```

Common contract:

- regular files only;
- a valid symlink to a regular file is followed;
- directories, FIFOs, sockets, devices, and non-regular pseudo-files are rejected;
- a read error returns `(nil, err)` and never a misleading digest of partial data;
- the complete content is not loaded into memory.

CRC-32, MD5, and SHA-1 must not authenticate data against an attacker. Prefer SHA-256, SHA-3, or BLAKE2 for new integrity checks.

In-memory CRC-32

`babet.crc32(data)`

Computes CRC-32 for a Lua string already in memory. Lua strings are binary-safe, so embedded NUL bytes are accepted.

```
print(babet.crc32("abc")) -- 352441c2
print(babet.crc32(""))    -- 00000000
```

```
local bytes = "a\0b\0c"
local digest = babet.crc32(bytes)
```

On success this function returns one string, without a second `nil` value. A bad argument count or type raises a Lua error.

Error contract

Raised Lua errors

A bad signature or incompatible type generally raises a Lua error:

```
babet.fileSize({})       -- Lua error
babet.mkdir("a", true)  -- Lua error: mkdir accepts exactly one argument
```

Paths containing a NUL byte are rejected before any system call.

Returned errors

Filesystem failures are normally returned:

```
local ok, err = babet.remove("missing.txt")
if not ok then
    print(err)
end
```

The `fileExists`, `isFile`, and `isDir` predicates treat a simply missing path as the normal result `(false, nil)`. A genuine inspection failure remains `(nil, err)`.

Partial operations

- `copyTree` in continuation mode may copy some entries and then return a summary error reporting warnings;
- `moveTree` may leave state split between source and destination when a late error occurs in its entry-by-entry fallback;
- `setAttributes` attempts rollback if `chmod` fails after `chown`, and explicitly reports an incomplete rollback.

Design decisions and limitations

- **Historical flat namespace**: `babet.fileExists`, not `babet.fs.fileExists`.
- **Canonical names**: `isFile`, `isDir`, and `symlinkAttr`. The `isfile`, `isdir`, and `symlinkattr` forms are deprecated.
- **`mkdir` is always recursive**: no non-recursive API variant exists.
- **`copy` for one file, `copyTree` for a tree**: no implicit overload based on source type.
- **Glob is a `find` filter, not a standalone function**: use `glob`, `iglob`, `path_glob`, or `path_iglob` in `babet.find`.
- **No automatic sorting**: call `table.sort` when stable ordering matters.
- **No complete metadata preservation during copy**: regular file permission bits are handled, but owners, groups, timestamps, and directory modes are not preserved.
- **No asynchronous I/O**: use workers to parallelize independent work.

exists — Babet 2.35.0

`babet.exists(path)` → `bool, nil` or `nil, error`. Accepts any existing type (file, directory, FIFO, device) without opening its contents. Follows symlinks; dangling links return `false, nil`. `fileExists` remains restricted to regular files. Access errors return `nil, error`.

GUI — optional GTK 4 desktop interface

`babet.gui` is Babet's optional desktop-GUI API. It is deliberately different from `babet.curses`: the GUI toolkit is **not** linked into Babet. On Linux the initial and only supported backend is GTK 4, loaded lazily from the target system when GUI use is requested.

A script that never uses `babet.gui` keeps Babet's normal deployment contract. A generated GUI application is still exactly one file, but GTK 4 must already be installed on the target machine.

Availability and initialization

```
local gui = babet.gui

local available, why = gui.available()
if not available then
    print(why)
    return
end

local ok, err = gui.init()
assert(ok, err)
```

`available()` loads and validates the narrow GTK/GLib/Cairo symbol surface but does not initialize a display. Once a GTK DSO has been opened successfully, it is kept resident until process exit, including when validation of a required symbol fails. The load result is memoized, so later calls do not reopen GTK. `init()` disables GTK's process-global locale change and then uses the recoverable GTK initialization path. Both calls are restricted to Babet's main OS thread.

The first GTK loading attempt, through `available()` or `init()`, permanently freezes process-wide mutations: `babet.setenv` and `babet.chdir` return `nil, err`, and `os.setlocale(locale, category)` raises when `locale` is not `nil`. Configure the environment, current directory and locale **before** these calls. `babet.env`, `babet.currentDir` and `os.setlocale(nil, category)` remain available as queries.

The freeze precedes `dlopen`: loading and initializing GTK can execute native code and create threads independently of Babet workers. It remains even if GTK is absent, a symbol is missing, or display initialization fails, and after windows close or an embedding context is recreated. A call rejected before loading (invalid arguments or wrong thread) does not trigger this freeze.

The guard covers Babet's Lua entry points. Hosts and native plugins must also coordinate their own mutations and threads; Babet does not intercept direct libc calls. See GLib's threading and global-state guidance.

If GTK 4 is absent, the diagnostic identifies the missing runtime and includes installation examples for common Debian/Ubuntu, Arch Linux, and Fedora systems. If GTK is installed but no graphical display can be initialized, `init()` returns a display-specific diagnostic instead of terminating the process.

Minimal widget API

```
local gui = babet.gui
assert(gui.init())

local window = gui.window {
    title = "Babet GUI",
    width = 480,
    height = 240,
}

local column = gui.box {
    orientation = "vertical", -- or "horizontal"
    spacing = 8,
}

local label = gui.label("Ready")
local button = gui.button("Run")

assert(window:add(column))
assert(column:add(label))
assert(column:add(button))

assert(button:onClick(function()
    assert(label:setText("Clicked"))
    assert(gui.quit())
end))

assert(window:show())
assert(gui.run())
assert(window:close())
```

The initial surface is intentionally small:

- `babet.gui.available()`
- `babet.gui.init()`
- `babet.gui.setCss(css_or_nil)`
- `babet.gui.window([options])`
- `babet.gui.box([options])`
- `babet.gui.label(text)`
- `babet.gui.button(text)`
- `babet.gui.entry([options])`
- `babet.gui.drawingArea([options])`
- `babet.gui.scrolledWindow()`
- `babet.gui.spinButton([options])`
- `babet.gui.calendar([options])`
- `container:add(child)` for windows, boxes and ScrolledWindow
- `box:remove(child)` / `box:clear()`
- `scrolledWindow:remove(child)` / `scrolledWindow:clear()`

- `label:setText(text)` / `button:setText(text)` / `entry:setText(text)`
- label layout: `setWrap(boolean)`, `setWrapMode("word|char|word-char")`, `setXAlign(0..1)`
- `button:onClick(function)`
- `drawingArea:onClick(function_or_nil)`
- `spinButton:getValue()` / `spinButton:setValue(number)` / `spinButton:onChanged(function_or_nil)`
- `calendar:getDate()` / `calendar:setDate(year, month, day)` / `calendar:onChanged(function_or_nil)`
- `calendar:markDay(day)` / `calendar:unmarkDay(day)` / `calendar:clearMarks()` / `calendar:onViewChanged(function_or_nil)`
- common properties: `setMargins`, `setHExpand`, `setVExpand`, `setVisible`, `setSensitive`, `addClass`, `removeClass`
- `window:show()` / `window:close()`
- `babet.gui.run()` / `babet.gui.quit()`

`window` options currently accept `title`, positive integer `width`, and positive integer `height`. `box` options accept `orientation = "vertical"|"horizontal"` and a non-negative integer `spacing`. GTK-facing strings reject embedded NUL bytes because the toolkit APIs consume C strings.

Label: wrapped display text

`gui.label()` is the preferred widget for read-only note text that must size its height from an imposed width. Keep `gui.textView()` for the note currently being edited. GTK's wrapped `GtkLabel` participates in height-for-width layout, which avoids the transient one-pixel-wide measurement that can make an editable text view appear vertically exploded before its first allocation.

```lua
local note = assert(gui.label("A longer note that can wrap across several lines."))
assert(note:setWrap(true))
assert(note:setWrapMode("word-char"))
assert(note:setXAlign(0)) -- wrapped lines start at the left edge
assert(note:setSizeRequest(300, -1))
```

Method	Contract
`label:setWrap(boolean)`	Enables or disables automatic line wrapping.
`label:setWrapMode("word"|"char"|"word-char")`	Selects the Pango line-breaking strategy used while wrapping is enabled.
`label:setXAlign(number)`	Aligns label text inside its allocated width; the number must be finite and between `0` and `1` (`0` = left, `1` = right).

`label:setWrapMode()` deliberately has a different accepted set from `textView:setWrapMode()`: a label disables wrapping with `setWrap(false)`, while its mode maps to `PangoWrapMode`. `textView:setWrapMode()` continues to accept `"none"|"char"|"word"|"word-char"` and maps to `GtkWrapMode`. This distinction is important because the two native enums do not share the same numeric values.

Calling `label:setText()` changes only the text: the wrap flag, wrap mode and `xalign` setting remain in force. Common CSS classes and application CSS continue to style the label normally, including color, font size/weight, underline and strikethrough. Invalid arguments raise a Lua error using the same conventions as the other GUI setters.

Entry: single-line text input

```lua
local entry = assert(gui.entry {
    text = "",              -- default: empty
    placeholder = "Name", -- default: empty
    editable = true,        -- default: true
})
```

```lua
assert(column:add(entry))

assert(entry:onChanged(function()
    print("Current text:", entry:getText())
end))
assert(entry:onActivate(function()
    print("Submitted:", entry:getText())
end))
```

Create the entry after `gui.init()` and keep its window alive while running the event loop. `entry()` and `entry(nil)` use the defaults. Options are read from the table itself, without invoking `__index`.

Method	Contract
`entry:getText()`	Returns one Lua string: a snapshot of the current text.
`entry:setText(text)`	Replaces the text. If the final value differs, invokes `onChanged` exactly once synchronously with the final text; identical text invokes no callback.
`entry:setPlaceholder(text)`	Sets the hint for an empty, unfocused entry; `""` clears it.
`entry:setEditable(boolean)`	Enables/disables user editing; programmatic `setText` remains allowed.
`entry:onChanged(function_or_nil)`	Replaces the text-change callback; explicit `nil` removes it.
`entry:onActivate(function_or_nil)`	Replaces the activation callback, normally triggered by Enter; explicit `nil` removes it.

Setters and callback registrations return `true, nil`. Creation returns a widget, or `nil, diagnostic` on a controlled runtime failure. Invalid arguments or a destroyed/wrong widget raise a Lua error. Text and placeholder must be strings containing UTF-8 suitable for GTK, without embedded NUL bytes; numbers are not converted to strings. `editable` requires an actual boolean. These operations do not parse numbers or validate application data.

Callbacks receive no arguments: capture the entry and call `getText()` when needed. The two callbacks are independent, and registering one does not invoke it. For programmatic `setText()`, Babet deliberately hides GTK's internal delete/insert notification burst and emits at most one `onChanged` after GTK returns from the native setter: exactly one when the final text differs from the initial snapshot, none when it is identical. The callback therefore observes the final value, never an intermediate empty string caused by GTK's implementation. That synthetic notification still runs before `setText()` returns, on the calling Lua thread (including a coroutine resumed on the main OS thread). User edits continue to follow GTK's ordinary `changed` events, including intermediate states: for example, typing over a selection may briefly expose an empty string. Only `entry:setText()` applies the coalescing described above. Events processed by `gui.run()` use the main Lua thread. Callbacks cannot yield across the GTK boundary. Errors, including an attempted yield, are reported and contained. A callback may replace/remove itself or close its window; native and Lua state remain valid until an active setter has returned. Changing text from `onChanged` can trigger another notification: avoid unconditional recursive updates, or temporarily remove the handler.

See examples/gui_entry/main.lua for a complete example with live text, Enter submission and a read-only toggle:

```
babet examples/gui_entry
babet --create-exe examples/gui_entry ./gui-entry-app
./gui-entry-app
```

DrawingArea: native curves and click selection

`DrawingArea` can draw through Cairo and receive user clicks without an external charting library.

```
local area = assert(gui.drawingArea {width = 640, height = 300})
assert(column:add(area))
assert(area:onDraw(function(ctx, width, height)
    assert(ctx:setSourceRGB(1, 1, 1))
    assert(ctx:rectangle(0, 0, width, height))
    assert(ctx:fill())
    assert(ctx:setSourceRGB(0.1, 0.4, 0.8))
    assert(ctx:setLineWidth(2))
    assert(ctx:moveTo(20, height - 20))
    assert(ctx:lineTo(width - 20, 20))
    assert(ctx:stroke())
    assert(ctx:setFontSize(14))
    assert(ctx:text(20, 22, "Native curve"))
end))
-- From a button/Entry callback, after changing the data:
assert(area:queueDraw())

assert(area:onClick(function(x, y, button, n_press)
    print("click", x, y, button, n_press)
end))
-- nil removes only the click callback:
assert(area:onClick(nil))
```

`drawingArea()` and `drawingArea(nil)` use 320 x 200 logical pixels. Options `width` and `height` must be positive integers no greater than `INT_MAX`; they request content size, not a fixed or maximum allocation. Options use raw table access. GTK passes the actual allocation to `onDraw(ctx, width, height)`.

`area:onDraw(function_or_nil)` replaces the handler; explicit `nil` removes it. Registration/removal schedules a redraw. `area:queueDraw()` requests a later redraw; it does not call the handler synchronously. GTK may combine several requests. Both methods return `true, nil` and are only for DrawingArea handles.

`area:onClick(function_or_nil)` replaces or removes the click callback. The callback receives exactly `x`, `y`, `button`, `n_press`: the first two are Lua numbers in widget allocation coordinates, `button` is an integer (1 for the primary button), and `n_press` is GTK's consecutive press count (1 for a single click, 2 for the second press of a double-click, and so on). Existing Lua functions that declare only three parameters remain valid because Lua ignores extra arguments. Unlike `onDraw`, this is an ordinary event callback: it may mutate widgets and call `queueDraw()`. Errors are protected and reported to stderr like the other GUI callbacks.

The context is borrowed and valid **only during that invocation of `onDraw`**. A saved context raises a Lua error after return, failure or attempted yield. Drawing runs on the main Lua thread; a synchronously resumed main-OS-thread coroutine may use the context while that invocation is active. Workers cannot. Callback argument allocation and user code are both protected against Lua errors, including out-of-memory failures. Errors go to stderr; the loop survives.

During `onDraw`, draw from existing data. Widget creation, mutation, callback registration and `queueDraw()` are rejected, even on other widgets. Prepare data and request updates from an input callback instead. `entry:getText()` and `gui.quit()` remain allowed; `quit()` only asks the loop to stop after GTK returns. Widget finalizers triggered during drawing are deferred until GTK returns to Babet. These rules respect GTK's drawing-stage restrictions.

All methods below return `true, nil`. Invalid types, arity, ranges, expired contexts and Cairo failures raise a Lua error. Numeric parameters must be actual, finite Lua numbers; numeric strings, NaN and infinity are rejected.

Context method	Meaning
`ctx:newPath()`	Clear the current path.
`ctx:moveTo(x, y)`	Start a subpath at the given point.
`ctx:lineTo(x, y)`	Add a straight segment.
`ctx:closePath()`	Join the current subpath back to its start.
`ctx:rectangle(x, y, width, height)`	Add a rectangle; signed dimensions are allowed.
`ctx:arc(x, y, radius, start, finish)`	Add an arc; non-negative radius, angles in radians, increasing angle direction.
`ctx:stroke()`	Draw and clear the path using the current line width and color.
`ctx:fill()`	Fill and clear the path using the current color.
`ctx:setLineWidth(width)`	Set a strictly positive line width.
`ctx:setSourceRGB(r, g, b)`	Opaque color; each component must be in [0, 1].
`ctx:setSourceRGBA(r, g, b, a)`	Color with alpha; every component must be in [0, 1].
`ctx:setFontSize(size)`	Set a strictly positive font size.
`ctx:text(x, y, text)`	Draw at a baseline position; valid UTF-8 string without NUL.

Coordinates are GTK logical pixels, with the origin at the top left and Y increasing downwards. Cairo preserves GTK's clipping and scale. Babet saves and restores Cairo graphics state around the callback and clears the drawing path. `arc` may connect to the current point; call `newPath()` for an independent arc. `text` uses Cairo's minimal text API and changes the current point: finish a line path before labeling it. It suits chart labels, not rich text, multiline layout or complex script shaping. It does not expose a native pointer, retained surface, image exporter or general Cairo binding.

See `examples/gui_drawing/main.lua`: an illustrative weight curve with unequal date intervals, automatic Y bounds and a button that changes the data. It is a drawing demo, without persistence.

```
babet examples/gui_drawing
babet --create-exe examples/gui_drawing ./gui-drawing-app
./gui-drawing-app
```

Refreshable containers: Box and ScrolledWindow

A `Box` can now remove one child or clear all children:

```
assert(column:remove(old_widget))
assert(column:clear())
```

`remove(child)` requires the child to belong to that container. `clear()` removes all children. In both cases, a child whose Lua handle is still held remains valid and may be added to another `Box` or `ScrolledWindow`; the old parent stops rooting it in Lua. This makes refreshable histories/lists possible without rebuilding the whole window. `Window` deliberately does not expose `remove()`/`clear()` in this surface.

`gui.scrolledWindow()` creates a native GTK scrolling container. It accepts one logical child through `add(child)`; adding a second child before removing or clearing the first is an error. Put a Box inside it for an arbitrary number of rows.

SpinButton: numeric input

```
local weight = assert(gui.spinButton {
    min = 30, max = 250, step = 0.1, value = 91.4, digits = 1,
})
```

```
assert(weight:onChanged(function()
    print("Weight:", weight:getValue())
end))
assert(weight:setValue(90.8))
```

Defaults are `min = 0`, `max = 100`, `step = 1`, `value = 0`, `digits = 0`. With a custom range and no explicit `value`, the initial value is `min`, matching GTK's native range constructor. Numeric options must be finite Lua numbers; `step` must be strictly positive and an explicitly supplied initial value must be inside the range. `digits` is an integer from 0 to 20. Options use raw table access. The SpinButton is configured for numeric input.

`getValue()` returns one Lua number. `setValue(number)` passes the value to GTK, which constrains it to the SpinButton range. `onChanged(function_or_nil)` replaces or removes the handler. A programmatic value change may call it synchronously before `setValue()` returns. Native and logical state are pinned across that call, including when the callback closes its window.

Calendar: selecting and editing dates

```
local date = assert(gui.calendar {year = 2026, month = 10, day = 7})
assert(date:onChanged(function()
    local year, month, day = date:getDate()
    print(year, month, day)
end))
local daysByMonth = { ["2026-10"] = {7, 18}, ["2025-12"] = {31} }
local function refreshMarks()
    local year, month = date:getDate() -- only the displayed year/month
    assert(date:clearMarks())
    local key = string.format("%04d-%02d", year, month)
    for _, day in ipairs(daysByMonth[key] or {}) do
        assert(date:markDay(day))
    end
end
assert(date:onViewChanged(refreshMarks))
refreshMarks() -- initial display
assert(date:setDate(2025, 12, 31)) -- past dates are allowed
refreshMarks() -- setDate() does not invoke onViewChanged()
```

`calendar()` and `calendar(nil)` keep GTK's default selected date. To set a date at construction, `year`, `month` and `day` must all be present, must be integers, and must form a valid Gregorian date with year 1..9999. `getDate()` returns exactly three values: `year, month, day`. `setDate(year, month, day)` also accepts past dates. `onChanged(function_or_nil)` has the same protected, replaceable callback contract as SpinButton and Entry.

Since Babet **2.30.0**, Calendar also exposes `markDay`, `unmarkDay`, `clearMarks` and `onViewChanged`.

`markDay(day)` places GTK's native visual marker on a day number; `unmarkDay(day)` removes it and `clearMarks()` removes all markers. `day` must be an integer from 1 through 31. Markers remain attached to **day numbers** when the displayed month changes: marking day 5 also marks day 5 in subsequent months. For dated data, clear and rebuild the markers for each displayed month.

`onViewChanged(function_or_nil)` runs on native user navigation to the previous/next month or year (header arrows, scrolling, keyboard or selection of a greyed-out day in an adjacent month). **`setDate()` does not invoke it**, even when changing the month or year. After a successful `setDate()`, call the same marker-refresh function explicitly, as above; otherwise the previous markers remain on the new month.

GTK navigation can also adjust the selected day **without `onChanged()`**. For example, with GTK 4.14, moving forward from 31 January 2024 selects 29 February and emits only `onViewChanged`. Selecting a greyed-out day can produce two stages: `onViewChanged` first sees the new month with

an intermediate day (29 February in this example), then `onChanged` sees the selected day (e.g. 1 February).

Use only the **year and month** from `getDate()` when rebuilding markers in `onViewChanged`; its day is not necessarily the final user selection. If the application tracks the complete selected date, read `getDate()` in both callbacks and again when confirming an operation. Do not assume that `onChanged` alone reports every date change, or that every navigation is followed by `onChanged`. Babet forwards these native GTK signals without synthesizing an extra selection notification.

The binding uses the Calendar API available since early GTK 4 instead of requiring much newer setters, preserving compatibility with GTK 4 runtimes older than 4.20.

CSS styling

Babet can install one application-level GTK CSS provider and attach CSS classes to individual widgets:

```
assert(gui.setCss([[
.card { background: #ffffff; border-radius: 12px; padding: 12px; }
.primary { background: #2563eb; color: white; }
]]))

local panel = assert(gui.box())
assert(panel:addClass("card"))
local button = assert(gui.button("Save"))
assert(button:addClass("primary"))
assert(button:removeClass("primary"))
assert(gui.setCss(nil))
```

`gui.setCss(css_or_nil)` replaces the application stylesheet for the current Lua GUI owner. `nil` removes it. `widget:addClass(name)` and `widget:removeClass(name)` work on every live widget; pass class names without a leading dot. CSS parsing diagnostics are reported by GTK. On GTK 4.12 and newer Babet uses `gtk_css_provider_load_from_string()`; on GTK 4.0–4.10 it falls back to the older `gtk_css_provider_load_from_data()` API. Neither symbol is a direct link-time dependency.

Common widget properties

Every live widget exposes these methods:

Method	Effect
`widget:setMargins(margin)`	Use the same non-negative integer margin on all four sides.
`widget:setMargins(top, end_, bottom, start_)`	Set the four GTK margins separately, in this order.
`widget:setHExpand(boolean)`	Enable/disable horizontal expansion.
`widget:setVExpand(boolean)`	Enable/disable vertical expansion.
`widget:setVisible(boolean)`	Show or hide the widget.
`widget:setSensitive(boolean)`	Enable or disable user interaction with the widget.
`widget:addClass(name)`	Add a GTK CSS class to the widget.
`widget:removeClass(name)`	Remove a GTK CSS class from the widget.

Margins must be integers from 0 through `INT_MAX`; the other methods require actual Lua booleans. They return `true, nil`. As with other GTK mutations, these methods are rejected during `onDraw`. Read-only `entry:getText()`, `spinButton:getValue()` and `calendar:getDate()` remain non-mutating operations.

Event loop and callbacks

GTK operations are main-thread only. A live GUI session and an active `babet.curses` session are mutually exclusive. `gui.run()` owns the interactive main loop until all windows are destroyed and no file dialog is pending, or `gui.quit()` is called. `gui.run()` must be entered from the main Lua thread itself; calling it from a Lua coroutine is rejected even when that coroutine is resumed on Babet's main OS thread.

Babet inserts a small bounded GLib wake source so its deferred Unix-signal callbacks continue to be serviced while GTK owns the loop. Workers may continue to perform non-GUI work, but they must not call GUI methods directly.

Lua button, Entry, SpinButton and Calendar callbacks execute under `lua_pcall`. An error is reported to stderr with a `babet.gui callback error:` prefix and does not unwind across GTK's C stack; the event loop remains usable.

Lifetime rules

Lua handles are validated on every method call. Parent Lua handles keep child handles alive, while GTK owns a child widget after `add()`. `remove()` and `clear()` acquire a native reference before GTK unparents the child, then drop the parent's Lua root; a child handle still held by the script therefore remains valid and reusable. GTK's `destroy` signal invalidates the corresponding Babet handle. Any later operation on that handle raises a controlled Lua error rather than dereferencing a stale native pointer.

Dropping an unparented widget handle releases its construction reference. Dropping a top-level window handle destroys that window. During Lua-state shutdown, GUI callback ownership is neutralized before `lua_close()`.

Callbacks belong to their Lua widget handle. Capturing that same widget in its callback does not permanently root it: an unreachable widget/callback cycle is collectable. A live parent still keeps its child handles and callbacks alive.

`--create-exe`

No special packaging command is needed:

```
babet --create-exe ./my-gui-project my-gui-app
```

The result is one application file. Unlike a non-GUI Babet application, that file is intentionally dependent on the GTK 4 runtime installed on its target Linux system. No `babet-gtk.so` helper, extracted temporary DSO, compiler, or linker is required at application launch or `--create-exe` time.

This GUI-only exception does not change the autonomy of scripts and generated applications that do not use `babet.gui`.

GTK4 additions — freely positioned notes (2.29.0)

These are **general-purpose** GUI components, with no built-in task database. The application is responsible for storing note contents and positions.

- `gui.fixed()` is a free-position canvas: `canvas:put(child, x, y)`, `canvas:move(child, x, y)`, `canvas:remove(child)`, `canvas:clear()`.
- `gui.textView {text="...", editable=true}` supports `getText()`, `setText()`, `setEditable(boolean)`, `onChanged(callback or nil)`.
- `gui.checkButton {label="...", active=false}` supports `getActive()`, `setActive(boolean)`, `setText(label)`, `onChanged(callback or nil)`.
- `gui.progressBar()` supports `setFraction(0..1)`, `getFraction()`, `setShowText(boolean)`, `setText(label)`, `onClick(callback or nil)`.

- `gui.expander {label="...", expanded=false}` supports `add(child)` (one child), `remove(child)`, `clear()`, `setExpanded(boolean)`, `getExpanded()`, `setText(label)`.
- On a non-interactive label, box, drawingArea, fixed, progressBar or picture, `onDrag(function(phase, x, y) ... end or nil)` installs a drag gesture. `"begin"` sends widget-local start coordinates; `"update"` and `"end"` send offsets from the drag start in stable GTK surface coordinates, even when the handle moves. The application moves the desired widget using `canvas:move(...)`. Install on a title/drag handle instead of on interactive controls (button, entry, checkButton, textView, expander, etc.).
- All widgets support `setSizeRequest(width, height)`; `-1` means automatic sizing. Canvas coordinates must be finite and between ±10,000,000.

A child removed from the fixed canvas remains usable. For a scrolling area, put `gui.fixed()` inside `gui.scrolledWindow()` and request sufficient canvas size. GTK4 does not automatically arrange children inside `GtkFixed`.

Working example: examples/gui_free_notes/main.lua.

Native clickable progress bars and in-place list reordering

Babet exposes the real GTK4 `GtkProgressBar` and `gtk_box_reorder_child_after()`; no custom drawing or remove/recreate workarounds are needed.

- `progressBar:onClick(function_or_nil)` installs or clears a protected callback receiving `(x, y, button, n_press)` on **primary-button press**. Coordinates are widget-local. `n_press` is GTK's consecutive-click counter, **not** the total number of application steps. The application maintains its own step count and calls `setFraction(0..1)`. Automatic calculated progress remains fully supported. The label to the right is a separate `gui.label()`. A progress bar is not a keyboard-activatable button: when progression is an action rather than display-only feedback, provide an equivalent real button or another keyboard-accessible control as well.
- `box:reorder(child, previousChild_or_nil)` moves an **existing** child of the same `gui.box` natively and in place. The second argument is required: pass `nil` explicitly for first position; otherwise the widget goes immediately after `previousChild`. Ownership and callbacks are unchanged, and Babet's Lua child order stays in sync. Invalid children are rejected without mutation. A no-op succeeds.

```lua
local bar, text = assert(gui.progressBar()), assert(gui.label("The graphics"))
local row = assert(gui.box {orientation = "horizontal", spacing = 8})
assert(row:add(bar)); assert(row:add(text))
assert(bar:setHExpand(true))
local count = 0
assert(bar:onClick(function(x, y, button, n_press)
    count = math.min(4, count + 1)
    assert(bar:setFraction(count / 4))
    if count == 4 then assert(text:addClass("completed")) end
end))
-- Move the last checklist row immediately after the first one:
assert(list:reorder(last, first))
```

For native text strikethrough styling apply GTK CSS such as: `label.completed, checkbutton.completed label { text-decoration-line: line-through; }`. A check button's `onChanged()` handler can add/remove its `completed` class.

Native GTK4 drag-and-drop for list rows

Since Babet **2.31.0**, native DND is available alongside `onDrag()`:

- `handle:setDragData(string_or_nil)` installs a real **`GtkDragSource`** on a non-interactive widget (`label`, `box`, `drawingArea`, `fixed`, `progressBar` or `picture` since 2.33). The UTF-8 string is the application payload; `nil` disables the source without rebuilding the widget. The string must

be valid UTF-8 and embedded NUL bytes are rejected. MOVE remains the default action and the default drag icon shows the handle widget.

- `target:onDrop(function_or_nil)` installs a real **`GtkDropTarget`** on any non-window widget. The protected callback receives `(phase, payload, x, y, action, kind)` for `"enter"`, `"motion"`, `"drop"` and `"leave"`. `action` is the currently preferred `"copy"`, `"move"` or `"link"`; `leave` receives nil payload/coordinates/action. During hover, a backend may not have preloaded the string yet, so `payload` can be nil on `enter`/`motion`; it is present on `drop`.
- Callbacks accept by default (`nil` or `true`). `false` or an error refuses the current notification. Returning `"copy"`, `"move"` or `"link"` accepts and explicitly prefers that enabled action **during `enter`/`motion`**. In `drop`, the action can no longer be changed: return `nil`, `true`, or the same action received by the callback to accept, and `false` to refuse. A different action string raises a contained callback error and rejects the transfer. For a transfer between Babet widgets, the source's `end` callback receives `delete_data=false` after a refusal. Validate the received action **before modifying the model**: refusing a transfer does not undo Lua-side changes. Refusing `enter`/`motion` **does not cancel the gesture**: further motion, `leave` and possibly `drop` can follow. Revalidate the ID and position in `drop`; return `false` if the operation is forbidden.
- `onDrop(nil)` disables the **native target** as well as its Lua callback and cancels an ongoing hover/preload. A disabled child no longer intercepts drops for a parent target or the widget's built-in text handling. Calling `onDrop(function)` enables the same controller again.
- Disabling inside a callback rejects the current transfer for that target. Re-enabling affects the next gesture, even when immediate. Existing model changes are not undone.
- The transfer owns its payload: changing or clearing `setDragData()` during a gesture affects subsequent gestures, not the ID already being transferred.
- MOVE is the compatibility default. `setDragActions(action_or_array)` and `setDropActions(action_or_arr` enable any combination of `copy`, `move` and `link`. Ctrl prefers COPY, Shift prefers MOVE, and Ctrl+Shift prefers LINK when the target allows it; the callback can override the preference by returning an enabled action string during `enter`/`motion`. The `action` argument only proposes an action that the source also allows, including when the drag comes from another application. During `drop`, it is the action actually sent to the source by the last hover. GTK keeps the first target that accepts: on enter, the parent comes before the child; during `motion`, the child comes before the parent. The action received at drop time can therefore differ from the one the target returned, for example when the button is released right after entering the window. If the target receiving the drop does not allow that action, `action` is `nil` and the drop is refused whatever the callback returns. Babet transports only the stable ID: the application still owns its model and performs the actual copy/move/link.
- `setDragIcon(widget_or_nil)` selects a custom live widget as the native drag preview. At each drag start, an unmapped, hidden, detached or destroyed preview falls back to the source handle. A standalone `gui.label("Preview")` is therefore not rendered off-screen; use a widget displayed in a window for a custom image. `nil` restores the handle. Babet keeps the chosen handle alive without reparenting it. `onDragState(function_or_nil)` reports `begin`, `end` and `cancel` as `(phase, detail, action)`; for `end`, `detail` is the boolean telling the source whether its data should be deleted, while `cancel` receives the native cancellation reason. After a refused local drop, GTK may still report the previously selected action and omit `cancel`: use the `end` boolean to decide whether to delete source data, not the action name alone. Babet vetoes this boolean when its local target refused the transfer, including callback errors.

Use a dedicated non-interactive handle. Avoid combining `onDrag()` and `setDragData()` on the same widget: `onDrag` can start first, then lose the sequence when native DND takes over at its drag threshold.

With nested targets, the parent also receives `enter`/`motion` while hovering its child. If the parent accepts the hover, the child can still receive `drop` even after returning `false` during hover; its `action` is then the one kept by GTK, here the parent's. **Revalidate every final drop.** Returning `false` from the child's `drop` does not forward it to the parent; only disabling the child with `onDrop(nil)` makes it transparent.

Do not wait for `leave` after a drop. GTK may finish an accepted or refused drop without emitting `leave`, including on the parent. In every `drop` handler, clear the indicators on all participating

targets before validating or updating the model. Keep `leave` cleanup for a pointer exit/cancelled gesture:

```
local rows = { ["task-42"] = rowA }
local function clearDropIndicators()
    for _, widget in ipairs({list, rowA, rowB}) do
        assert(widget:removeClass("drop-target"))
    end
end
assert(handleA:setDragData("task-42"))
assert(rowB:onDrop(function(phase, payload, x, y)
    if phase == "enter" or phase == "motion" then
        assert(rowB:addClass("drop-target"))
    elseif phase == "leave" then
        assert(rowB:removeClass("drop-target"))
    elseif phase == "drop" then
        clearDropIndicators() -- shared by every drop handler, including list's
        local moved = rows[payload]
        if not moved or moved == rowB then return false end
        assert(list:reorder(moved, rowB)) -- move immediately after rowB
    end
end))
```

If the parent `list` also has an `onDrop` handler, call the same cleanup function in its `drop` branch, including when rejecting the drop.

To insert before a target, pass the target's previous row from the application model to `reorder()` (or explicit `nil` for first position). The insertion indicator and persistence remain application concerns, but the gesture/drop path itself is native `GtkDragSource`/`GtkDropTarget`, not an `onDrag()` simulation.

Desktop-application primitives

Available since Babet **2.32.0**.

This layer is intentionally generic. It provides the GTK4 mechanics required by rich desktop applications without adding note, task, tree or persistence models to Babet.

Split panes, menus and context popovers

- `gui.paned{orientation="horizontal"|"vertical", position=..., resizeStart=..., resizeEnd=..., shrinkStart=..., shrinkEnd=..., wideHandle=...}` wraps `GtkPaned`. It accepts exactly two children through `add()`. Use `getPosition()`, `setPosition(px)` and `onPositionChanged(function_or_nil)` to persist a sidebar width. Defaults: `orientation="horizontal"`, `resizeStart=false`, `resizeEnd=true`, `shrinkStart=true`, `shrinkEnd=true`, `wideHandle=false`; an omitted `position` leaves the initial divider placement to GTK. In particular, Babet defaults `resizeStart` to false (GTK itself defaults it to true).
- `gui.separator([orientation])` exposes a native `GtkSeparator`.
- `gui.menuButton{label="..."}` or `gui.menuButton{icon="..."}` plus `gui.popover{hasArrow=..., autohide=..., position="left|right|top|bottom"}` build menu bars and mark menus from ordinary Babet widgets. A popover has one content child. `menuButton:setPopover(popover_or_nil)`, `popup()` and `popdown()` manage an anchored menu. Avoid calling `popup()` immediately followed by `popdown()` in the same callback with `autohide=true`: GTK may swallow the next key event. Open the menu and close it in response to a later event.
- For a pointer-positioned context menu, call `popover:attachTo(widget)`, `setPointingTo(x,y[,width,height])` then `popup()`. `attachTo(nil)` detaches the popover so it can be reused elsewhere. Direct attachment is deliberately limited to popovers; it is not a generic escape hatch for arbitrary GTK parenting. Destroying the parent detaches its context popovers; a retained popover

handle can be attached to another live widget. Parenting cycles are rejected before mutation, including paths through menus and context popovers.

A top menu/toolbar can therefore remain ordinary GTK widgets: a horizontal `gui.box()` containing `menuButton`, `button`, `image` and `separator` children. `gui.image(iconName)` and `setIconName(iconName)` on images/buttons/menu buttons use the target desktop's icon theme.

Generic pointer, keyboard, focus and geometry

`widget:onPointer(function_or_nil)` installs native click and motion controllers. The callback receives `(phase, x, y, button, n_press, modifiers)` where `phase` is `press`, `release`, `enter`, `motion` or `leave`; `leave` has nil coordinates. Unlike `onClick`, all mouse buttons are reported, so button 3 can open a context menu. `widget:onKey(function_or_nil)` receives `(phase, key_name_or_nil, keyval, keycode, modifiers)` for `press`/`release`; a truthy return from `press` consumes the event. On a window, the controller runs in GTK **capture** phase so application shortcuts such as `Ctrl+1` or `Escape` are seen before a focused `TextView`/`Entry` can consume them. On an `Entry` or `TextView` itself, GTK can consume text input before `onKey` receives it. Do not use that callback to collect typed characters: use `onChanged` for text and the window's `onKey` for application shortcuts.

Modifier masks are exported as `gui.MOD_SHIFT`, `gui.MOD_CONTROL`, `gui.MOD_ALT` and `gui.MOD_SUPER`. Test masks with Lua 5.5 bitwise operators, e.g. `(mods & gui.MOD_CONTROL) ~= 0`.

Common desktop helpers are:

- `setCursor(name_or_nil)` and `setTooltipText(text_or_nil)`;
- `grabFocus()` / `hasFocus()`;
- `getSize()` for the current allocated `(width,height)`;
- `translatePoint(target,x,y)` to express one widget point in another widget's coordinate system; it returns `nil` if GTK cannot translate between them;
- `pick(x,y)`, which performs native GTK hit-testing and walks back to the nearest Babet widget handle. This lets a free canvas distinguish a background click from a click on a child note;
- `setHAlign("fill|start|end|center|baseline")` and the equivalent `setVAlign`;
- `textView:setWrapMode("none|char|word|word-char")`; `wrap=` accepts the same values at construction time.

These primitives are sufficient for hover-only resize/move handles and precise insertion indicators. The application decides hit zones and visual CSS; Babet does not infer a note layout.

`getSize`, `translatePoint` and `pick` reflect the last completed GTK layout. Immediately after `add`, `remove` or another layout change in the same callback, their results may still describe the previous arrangement. Query them after GTK has processed the next layout; returning from a callback alone does not force it.

Desktop windows

`window:setTitle(text)`, `window:setIconName(iconName)`, `window:setTransientFor(parent_or_nil)` and `window:setModal(boolean)` support properties/dialog windows. `window:onCloseRequest(function_or_nil)` runs a protected callback before a user close; returning `true` blocks that close. Use this only for policy such as unsaved-change confirmation—programmatic `window:close()` remains explicit. If the close-request callback raises an error, Babet reports it and allows the user-requested close to continue.

Pictures and file dialogs

These additions arrive with **Babet 2.33.0**. `gui.image(iconName)` remains for themed icons; `gui.picture` displays a file image. New symbols are optional: older GTK versions retain the existing API.

API	Minimum GTK
Empty `gui.picture()`, `setFile(nil)`, `setCanShrink`, `getIntrinsicSize`, `setAlternativeText`	4.0
`gui.picture{file=...}`, `picture:setFile(path)`	4.6
Explicit `contentFit` option, `picture:setContentFit(...)`	4.8
`gui.openFile`, `gui.saveFile` and their options	4.10

File pictures

```
local picture, err = gui.picture {
    file = "/home/julien/Pictures/photo.jpg", -- optional
    contentFit = "contain",                   -- optional; explicit: GTK 4.8
    canShrink = true,                          -- default
}
assert(picture, err)
assert(picture:setAlternativeText("Garden photograph"))
assert(picture:setHExpand(true))
```

Without `file`, the widget is empty. Omitting `contentFit` keeps the native aspect-preserving default (`contain`), including on GTK 4.0–4.6. Explicitly passing `contentFit = "contain"` still requires GTK 4.8.

Method	Result
`picture:setFile(path_or_nil)`	`true, nil` or `nil, err`
`picture:setContentFit("fill" / "contain" / "cover" / "scale-down")`	`true, nil` or `nil, err`
`picture:setCanShrink(bool)`	`true, nil`
`picture:getIntrinsicSize()`	width, height in pixels; `0, 0` when empty
`picture:setAlternativeText(text_or_nil)`	`true, nil`

`setFile` loads the replacement first: an unreadable file or unsupported format leaves the previous image intact. `nil` clears the image and releases its texture reference. Loading is synchronous. PNG and JPEG are supported; additional formats such as WebP, GIF or SVG depend on GTK and installed loaders. Pictures are static; animations are not exposed. Native decoding is not isolated in a separate process.

`fill` stretches; `contain` preserves the whole image and its aspect ratio; `cover` fills the allocation with cropping; `scale-down` avoids upscaling. With the default fit, in a width-constrained vertical Box without vertical expansion or a forced height, the requested height follows the image's aspect ratio. `getIntrinsicSize()` reports file dimensions; `getSize()` reports the most recent widget allocation.

Common methods (CSS classes, margins, sizing, visibility, sensitivity, tooltips, `onPointer`, `pick`, etc.) are available. A `picture` also qualifies as a non-interactive `onDrag` or `setDragData` handle. String DND keeps its existing contract.

Asynchronous file dialogs

```
local started, err = gui.openFile({
    title = "Choose an image",
    parent = window,                    -- optional live Babet window
    folder = "/home/julien/Pictures", -- optional
    multiple = false,                   -- openFile only
```

```
    filters = {
        {name = "Images", patterns = {"*.png", "*.jpg"}, mimeTypes = {"image/*"}},
    },
}, function(path, error_message)
    if error_message then
        print(error_message)
    elseif path then
        local ok, load_error = picture:setFile(path)
        if not ok then print(load_error) end
    end
end)
if not started then print(err) end
```

`gui.saveFile(opts, callback)` accepts the same options except `multiple`, which is rejected. It accepts `name = "image.png"` to suggest a filename. `openFile` rejects `name`. The options table and callback are mandatory; individual options are optional. Each filter accepts `name`, `patterns` and `mimeTypes`, using dense arrays. UI text and filters use UTF-8; paths use native filesystem bytes. Wrong arity, wrong types, unknown options, NUL and invalid UTF-8 text raise Lua errors.

Both functions immediately return `true, nil` on start, or `nil, err` on failure, including `"babet.gui: requires GTK 4.10"`. No deprecated GtkFileChooser fallback is used. The callback receives:

- `callback(absolute_path)` for a single selection;
- `callback({path1, path2, ...})` with `multiple = true`, in GTK model order;
- `callback(nil)` when the user cancels;
- `callback(nil, err)` on failure. A selection without a local path gives `"babet.gui: file is not local"`; a multiple selection then fails as a whole.

The chooser is always modal: while it is open, GTK blocks input to every Babet window, with or without `parent`. `parent` attaches the chooser to that window and cancels it when the window is destroyed. Without a parent, `gui.run()` may start with only a pending chooser. The loop continues while there is a window or pending selection, unless `gui.quit()` is called. Callbacks run at most once on the main Lua thread, including when started from a coroutine. They may modify widgets but cannot yield. Errors, including argument construction failures, are contained and reported to stderr.

Destroying the parent, leaving `gui.run()` or closing the Lua state cancels corresponding pending requests **without an application callback**. Babet releases its native references; late GIO results are consumed and freed without accessing the former Lua state. User cancellation remains distinct and invokes `callback(nil)`.

`saveFile` only selects a path: **Babet writes no file**. GTK confirms selection of an existing file; the application remains responsible for the write, errors and its replacement policy. Full example: `examples/gui_picture/main.lua`.

File drops and TextView tags

Files from the file manager — GTK 4.6

`target:setDropTypes(types)` accepts a dense array containing `"text"`, `"files"`, or both (once each). The default is `{"text"}`. This configures the formats; install the handler with `onDrop`. It returns `true, nil`, or `nil, err` if file-list support is unavailable (`"babet.gui: requires GTK 4.6 (file drops)"`). Invalid arguments raise Lua errors. Changing types during `drop` returns `nil, err`; changing them during hover rejects that transfer and applies to the next one. The setting survives `onDrop(nil)` and re-enabling the handler.

The protected `onDrop` callback now receives six arguments: `(phase, payload, x, y, action, kind)`:

- `kind == "text"`: the payload is the existing UTF-8 string.

- `kind == "files"`: the payload is a dense array of absolute local file paths, in the order provided by GTK. Paths retain their native filesystem bytes. Non-local URI entries are ignored; a list with no local path is refused without calling the application with an empty list.
- During `enter`/`motion`, the payload may still be `nil`. The kind is the only configured type, or `nil` for a mixed target until GTK provides the value. During `drop`, both the kind and payload are known.
- A mixed target asks for files first: a list from a file manager arrives with `kind == "files"`, even when the source also offers its text form. Browsers often offer a link as a URI list: if it contains no local path, a mixed target receives it with `kind == "text"` (one URI per line). A `{"files"}`-only target refuses such a link.
- `leave` still has nil payload, coordinates and action. Its kind follows the configured types (nil for a mixed target); do not rely on it to remember the previous transfer. Keep any needed hover state in your application.

Existing fixed-argument callbacks continue to work. Vararg callbacks now see a sixth argument. The rules for actions, nested targets, refusal, callback errors and indicator cleanup remain the same. File lists are copied before Lua is invoked, so callbacks may retain them or destroy widgets safely.

For importing files, explicitly select COPY. MOVE remains the compatibility default; accepting an external MOVE can ask the source application to delete its originals. Babet passes paths only and performs no file copy, move or link. Validate and import the data in `drop` before returning acceptance.

```
assert(target:setDropTypes({"files"}))
assert(target:setDropActions("copy"))
assert(target:onDrop(function(phase, paths, x, y, action, kind)
    if phase == "drop" then
        if kind ~= "files" or action ~= "copy" then return false end
        for _, path in ipairs(paths) do print("Selected:", path) end
        -- Perform the application's import here; return false on failure.
    end
    return true
end))
```

TextView tags — GTK 4.0

Tags style ranges in the native GTK text buffer. They do not parse markup or launch URLs. `createTag`, `applyTag`, `removeTag` and `removeAllTags` return `true, nil` or `nil, err`. All new TextView methods require the optional GTK symbol group; if unavailable, they return `nil, "babet.gui: requires GTK 4.0 (TextView tags)"`.

`tv:createTag(name, props)` defines a non-empty UTF-8 tag name. Supported properties (all optional) are:

Property	Value
`foreground`, `background`	Color string `"#rrggbb"`
`underline`, `strikethrough`	Boolean
`weight`	Integer 100–900
`style`	`"normal"` or `"italic"`
`scale`	Positive finite number; 1 preserves the current size

A duplicate name or unknown property returns `nil, err`. Wrong arity, types, empty names, NUL, invalid UTF-8 names or invalid property values raise Lua errors. Unspecified properties do not override other styling. Overlapping tags follow GTK priority rules; tags defined later have higher priority.

Positions are **1-based, inclusive Lua byte indices**, as returned by `string.find`. `first` must point to the first byte of a UTF-8 character; `last` must point to its final byte. For a single-position argument, use the first byte of a character. Out-of-range positions or a split UTF-8 sequence return `nil, err`; reversed/empty ranges are rejected. An empty buffer has no valid character index. Indices refer to the current text: recompute them after edits.

Method	Result / effect
`tv:applyTag(name, first, last)`	Apply a defined tag to that range
`tv:removeTag(name, first, last)`	Remove only that tag from the range; it may split a segment
`tv:removeAllTags()`	Remove all applications, retaining definitions
`tv:tagsAt(index)`	Array of covering tag names, in definition order; empty when none
`tv:tagRange(name, index)`	Inclusive byte bounds of the contiguous tagged segment containing the character; one `nil` if absent
`tv:offsetAt(x, y)`	First byte index of the character under the point; one `nil` outside text

Unknown names return `nil, err`. `setText()` removes tag applications but keeps their definitions, even when the replacement text is unchanged. User edits follow GTK's native tag behavior. `offsetAt` takes TextView-local coordinates, as supplied to its `onPointer`; it converts to buffer coordinates, including scrolling. It uses the latest layout, so call it after allocation, not directly after a layout change. It works for non-editable and insensitive TextViews. An insensitive widget itself does not receive pointer events: hit-test it from another suitable event source after translating coordinates. Negative/outside coordinates return nil; nonnumeric or nonfinite coordinates raise Lua errors.

```
local text = "Café : https://example.org"
local tv = assert(gui.textView {text = text, editable = false})
assert(tv:createTag("link", {foreground = "#1565c0", underline = true}))
local first, last = text:find("https://example.org", 1, true)
assert(tv:applyTag("link", first, last))
assert(tv:onPointer(function(phase, x, y, button)
    if phase ~= "release" or button ~= 1 then return end
    local index = tv:offsetAt(x, y)
    if not index then return end
    local a, b = tv:tagRange("link", index)
    if a then print(tv:getText():sub(a, b)) end
end))
```

Tag mutations are forbidden inside `onDraw`, like other widget mutations; queries are permitted. Tags add no asynchronous state. See `examples/gui_rich_text/main.lua` for a complete window with both features.

GUI loop and directory selection

Available since Babet 2.35.0. GTK 4.0 is sufficient except for directory selection, which requires GTK 4.10.

Timers

- `gui.timeout(ms, fn)` → `id, nil`: run once after at least `ms` milliseconds (integer from 0 to 2147483647).
- `gui.idle(fn)` → `id, nil`: run once on a later event-loop turn.
- `gui.cancel(id)` → `bool, nil`: true when removed; false if already executed, cancelled or unknown.

Babet owns these identifiers and never reuses them within the process. Callbacks run protected, with no arguments, on the main Lua thread and may modify widgets. `onDraw` may schedule an `idle` for changes after drawing. To repeat a timer, explicitly schedule another one in its callback.

Deadlines are checked at loop service points, with a bounded 25 ms wake interval outside blocking work. This is not real-time scheduling: slow callbacks delay others. Ordering among ready callbacks is unspecified. A newly created callback waits for another turn. `gui.run()` requires a live window or pending dialog; callbacks do not keep a windowless application running. All pending callbacks are cancelled on `run()` exit (including `quit()` and errors) or Lua state close.

```
local pending
entry:onChanged(function()
    if pending then gui.cancel(pending) end
    pending = assert(gui.timeout(700, function()
        pending = nil
        rebuildPreview()
    end))
end)
```

Process completion

`proc:onExit(fn) → id, nil` observes a `babet.spawn` process from `gui.run()`. `fn(result, error)` receives the same values as `proc:wait(0)` exactly once, even when the process already exited. `proc:onExit(nil)` removes observation and returns `true, nil`. A new registration replaces the previous one; `gui.cancel(id)` also cancels it. The status remains available through `wait()`. Stopped processes are not completed. Closed handles may report `nil, 'closed'` if no status is available. No `waitpid(-1)` is used.

An observer retains its handle. Cancellation does not kill a process whose handle the application still retains; an unreachable handle keeps its existing `close()` finalization behavior. Output pipes are not automatically drained: they may fill and block the child. Inherit streams, redirect to files, or read pipes incrementally. This API does not apply to `spawnPipeline` handles.

Errors

GUI and signal callback errors print a traceback to stderr by default, prefixed with `babet.gui callback error:` for a GUI callback and with `babet.signal callback error:` for a signal callback. Without a terminal, consult the launcher's journal. `gui.onError(fn)` installs a handler for this Lua state receiving the error text; `gui.onError(nil)` restores default reporting. Both return `true, nil`. Handler failure prints both errors to stderr without infinite recursion. Errors never cross GTK. Under memory exhaustion, tracebacks may be incomplete. Signal callbacks outside GUI are also covered; workers do not inherit the handler.

Size and closure

`window:onResize(fn_or_nil) → true, nil`. `fn(width, height)` receives the first positive size of a mapped window and subsequent changes after layout processing. Intermediate changes may be coalesced. Widget mutations are allowed; avoid resize feedback loops. `nil` removes the callback.

`window:requestClose()` asks GTK to close the window; `onCloseRequest` may veto by returning true. GTK ignores the request when the window was never shown with `show()`, or when it arrives during that same window's `onCloseRequest`: nothing is emitted and the window stays open. Its `true, nil` result means the request was handed to GTK, not that the callback ran or that the window was destroyed. `window:close()` retains explicit destruction without `onCloseRequest`. Errors in `onCloseRequest` are reported and allow closure.

Directory selection

`gui.selectFolder({title=?, parent=?, folder=?}, callback)` → `true, nil` when an asynchronous dialog starts, otherwise `nil, error`. `parent` is a Babet window and `folder` the initial directory. Filters, filenames and multiple selection are unsupported; unknown options raise Lua errors. The protected callback receives an absolute local path, `nil` for user cancellation, or `nil, error` for failure/non-local selection. The dialog is modal relative to its parent. Parent destruction, leaving `run()` or closing Lua cancels without late callbacks. GTK < 4.10 returns `nil, error`, while other widgets remain usable.

`writeFileAtomic` — atomic and durable file writing

`babet.writeFileAtomic()` publishes a binary Lua string to a regular file without exposing partially written content under the final name. Data is first written to a private same-directory temporary file, final permissions are applied, then the temporary file is published atomically.

It is intended for configuration, tokens, JSON state, small binary files, and screenshots already held in memory. It does not replace streaming APIs such as `babet.http.download()` for very large payloads.

Contents

- API
- Defaults
- Basic write
- Explicit replacement
- Permissions
- Durability
- Binary data
- Paths and symbolic links
- Atomicity and concurrency
- Using it in a worker
- Combined Selenium example
- Error contract
- Limitations

API

```
local ok, err = babet.writeFileAtomic(path, data, opts?)
```

Argument	Type	Purpose
`path`	string	relative or absolute destination, with no NUL byte
`data`	string	complete binary content; embedded NUL bytes are allowed
`opts`	table or `nil`	strict options

Results:

- success: `true, nil`;
- filesystem or publication failure: `nil, err`;
- bad arity, bad type, or unknown option: raised Lua error.

Defaults

```
{
    overwrite = false,
    permissions = tonumber("644", 8),
    durable = true,
}
```

Option	Default	Effect
`overwrite`	`false`	rejects an existing destination
`permissions`	`0644`	exact permissions for the new final file
`durable`	`true`	synchronizes the temporary file and parent directory

Unknown and non-string keys are rejected. Booleans are not coerced from `0`, `1`, or strings. `permissions` must be an integer from `0` through `0777`.

Basic write

```
local ok, err = babet.writeFileAtomic("state.json", [[{"ready":true}]])
assert(ok, err)
```

The destination becomes visible only after the temporary file is complete. If `state.json` already exists, the call fails and preserves its contents.

Parent directories are not created automatically:

```
assert(babet.mkdir("runtime"))
assert(babet.writeFileAtomic("runtime/state.json", "{}"))
```

Explicit replacement

```
local ok, err = babet.writeFileAtomic("state.json", new_state, {
    overwrite = true,
})
assert(ok, err)
```

With `overwrite = true`:

- an absent destination is created;
- an existing regular file is replaced by a same-directory rename;
- a symbolic link, directory, FIFO, socket, or device is rejected;
- the old file remains intact after any failure before publication.

Without overwrite, Babet uses a publication operation that atomically fails if another task wins the race and creates the destination first.

Permissions

```
local ok, err = babet.writeFileAtomic("token.bin", token, {
    permissions = tonumber("600", 8),
})
assert(ok, err)
```

The temporary file is always created with private mode `0600`, or more restrictive because of the process umask. Before publication Babet applies the requested permissions exactly using `fchmod()`. Setuid, setgid, and sticky bits cannot be requested: the accepted range is `0000..0777`.

On replacement, the new file gets the requested permissions rather than those of the previous file.

Durability

With default `durable = true`, the sequence is:

1. create a unique temporary file in the destination directory;
2. write all bytes while handling `EINTR` and partial writes;
3. apply final permissions;
4. `fsync()` the temporary file;
5. publish atomically;
6. `fsync()` the parent directory.

This aims to preserve the update after a crash, subject to filesystem and hardware guarantees.

For a rebuildable cache where speed matters more:

```
local ok, err = babet.writeFileAtomic("cache/index.bin", cache, {
    overwrite = true,
    durable = false,
})
assert(ok, err)
```

`durable = false` retains atomic publication but does not confirm persistence after a crash or power loss.

If publication succeeds and parent-directory synchronization then fails, the call returns an explicit error stating that the destination has already been published atomically but its persistence could not be confirmed.

Binary data

`data` is a binary Lua string. No UTF-8 validation is performed:

```
local payload = "\0\1\2PNG\255\254"
assert(babet.writeFileAtomic("capture.bin", payload))
```

An empty string creates a zero-length regular file.

All data must already be in memory. Prefer a streaming API for downloads or transformations measured in gigabytes.

Paths and symbolic links

Paths may be relative or absolute. They are interpreted literally: there is no ~, `$HOME`, shell, or glob expansion.

To confine the write:

- every parent component is opened with `openat()` and `O_NOFOLLOW`;
- symlink parent components are rejected;
- `..` is rejected;
- a final symbolic link is rejected;
- missing parent directories are not created;
- the destination must be absent or a regular file when `overwrite = true`.

```
local ok, err = babet.writeFileAtomic("link/config.json", "{}", {
    overwrite = true,
})
assert(not ok)
print(err)
```

A process that can itself write the parent directory remains inside the same trust boundary: POSIX has no portable conditional rename tied to one exact inode. Babet never follows the final link and reinspects the destination immediately before publication, but directory permissions remain the security boundary against a hostile concurrent actor.

Atomicity and concurrency

Readers never observe a mixture of old and new data: they see one complete file or the other.

With `overwrite = false`, several producers may attempt the same creation:

```
local ok, err = babet.writeFileAtomic("once.dat", value)
if not ok and err:find("already exists", 1, true) then
    -- Another producer won.
end
```

Exactly one call publishes the destination. Others fail without replacing it.

With `overwrite = true`, the last successful rename determines the final contents. Ordering between concurrent writers is unspecified.

Using it in a worker

The function is available in every worker Lua state:

```
local job = assert(babet.workers.spawn([[
    local ok, err = babet.writeFileAtomic(worker.args.path, worker.args.data, {
        overwrite = true,
        permissions = tonumber("600", 8),
    })
    assert(ok, err)
    return true
]], {
    path = "runtime/result.bin",
    data = "worker-result",
}))

assert(job:join())
```

Values carried in `worker.args` remain subject to the workers JSON serializer. A binary string containing NUL cannot therefore be passed directly through `worker.args`; generate it in the worker, retain it as Base64, or read it from another source.

Combined Selenium example

Decode a WebDriver screenshot and publish it without a partial final file:

```
local png, err = babet.base64.decode(response.value, {
    max_output = 32 * 1024 * 1024,
})
assert(png, err)

assert(babet.mkdir("screenshots"))
local ok
ok, err = babet.writeFileAtomic("screenshots/latest.png", png, {
    overwrite = true,
    permissions = tonumber("640", 8),
    durable = true,
})
assert(ok, err)
```

Readers of `latest.png` see either the complete previous screenshot or the complete new one, never a partially decoded or written image.

Error contract

Raised Lua errors:

```
babet.writeFileAtomic()                       -- arity
babet.writeFileAtomic(42, "data")             -- path is not a string
babet.writeFileAtomic("x", {})                -- data is not a string
babet.writeFileAtomic("x", "data", 42)        -- opts is not a table
babet.writeFileAtomic("x", "data", {
    permissions = "600",
})
babet.writeFileAtomic("x", "data", {
    unknown = true,
})
```

Failures returned as `nil, err` include:

- absent, inaccessible, symlink, or non-directory parent;
- existing destination without `overwrite`;
- final symlink or non-regular destination;
- temporary creation, write, `fchmod`, `fsync`, close, or publication failure;
- inability to allocate a unique temporary name.

After a failure before publication, the old destination remains intact and the temporary file is removed on a best-effort basis.

Limitations

The first version does not provide:

- automatic parent-directory creation;
- append mode;
- input from an `io.file` object or numeric descriptor;
- a streaming API;
- application-level locking between writers;
- automatic rotation or size limits;
- atomic publication of several files as one transaction.

For multiple related files, publish a new version directory and switch an application pointer, or use SQLite when a transaction is required.

lockFile — exclusive file locks

Available since Babet 2.34.0. This API is not available in Babet 2.33.0 or earlier.

`babet.lockFile(path)` lets cooperating processes reserve a resource, such as a data directory with one writer process. This Linux primitive requires no GTK and is available in the runtime, workers and embedding. It uses `flock(LOCK_EX | LOCK_NB)`, not POSIX `fcntl(F_SETLK)` locks.

API

Call	Result
`babet.lockFile(path)`	`handle, nil`, or `nil, "locked"` for contention, or `nil, message` for a system error

Call	Result
`handle:write(text)`	`true, nil` or `nil, message`; replaces the contents on the same inode
`handle:close()`	`true, nil` or `nil, message`; closing an already closed handle succeeds

Every call returns exactly two values. Invalid arity/types, an empty path or a path containing NUL raise a Lua error. `write` accepts binary strings, including empty strings and NUL bytes. Writing to a closed handle fails.

The path must identify a regular file. If absent, Babet creates it with mode `0600`, possibly reduced by umask. Acquiring an existing file preserves its contents and permissions, including on contention. Symlinks to existing files and hard links refer to the same lock. Dangling symlinks are rejected without creating their target. Directories, FIFOs and devices are rejected before opening them for I/O.

`lockFile` does not wait for a held lock to become available, though filesystem operations may still block. A handle lasts until explicit close, `<close>` scope exit, Lua garbage collection or Lua state destruction. Keep a reference, preferably a `<close>` variable, throughout directory use. Leaving `gui.run()` does not automatically close a file lock.

One writer per directory

```lua
local directory = "/path/to/my-data"
local path = directory .. "/.basket.lock"
local lock, err = babet.lockFile(path)
if not lock then
    if err == "locked" then
        -- Reading a PID is optional: metadata may be empty, unreadable or
        -- being updated. The actual lock is authoritative.
        io.stderr:write("This directory is already open.\n")
    else
        io.stderr:write(tostring(err), "\n")
    end
    return
end
local guard <close> = lock
assert(lock:write(tostring(babet.pid()) .. "\n"))

-- Open the database and run the application only after acquisition.
-- Keep guard in scope during writes, backup/restore and database shutdown.
-- Multiple windows in this process may share this handle.
```

Babet writes no PID automatically. `write` lets the application set metadata without reopening the file. Opening and closing a separate descriptor for the same inode does not release this `flock` lock. A PID read from the file is informational: it may be stale, absent, partial or reused. Never infer lock availability from it or automatically kill that process.

Keep the lock file permanently. Never unlink, rename or replace it, even after closing it. Exclude it from restore, cleanup and atomic replacement operations. Otherwise two processes could lock distinct inodes under the same filename. Different files have independent locks.

Processes, workers and errors

- Descriptors use `O_CLOEXEC`, so an executed program cannot retain the lock. After libc `fork()`, a child handler closes inherited descriptors using `close()`, never `LOCK_UN`, and invalidates copied

handles. The parent's lock remains active. The parent can resume before the handler, so temporary contention is possible until the handler runs. Raw fork syscalls and Lua execution after `vfork()` are outside this contract.
- Babet workers are threads with separate Lua states. Their independent lock attempts contend with the main thread. Closing a worker releases its own handles without releasing another state's handles.
- Process exit, including `kill -9`, releases its descriptors. The lock file and metadata remain on disk. Embedding releases handles on Lua state cleanup.
- A failed `write`, including a partial write, **keeps the lock held**. Interrupted calls and short writes are handled; truncation to the new size occurs after writing. Unlocked readers can see partial data. This is neither atomic publication nor durable storage (`fsync`): use it for lock metadata, not for saving application data.

`write` runs synchronously in its owning Lua state, without holding the process-wide registry gate: other states can acquire locks and other threads can fork during a slow write. Acquisition and close still take that gate to synchronize descriptor registration with fork cleanup; slow filesystem calls there can delay other acquisitions, closes and forks.

Limits

This is an advisory lock: every writer must cooperate using the same file. The contract targets local Linux filesystems; NFS/SMB semantics depend on the server and mount. `locked` specifically means contention reported by `flock`; permission failures and other errors have different diagnostics.

To validate an existing file without opening a device for I/O, Babet pins the inode with `O_PATH`, then reopens it through `/proc/self/fd`. Accessible procfs is therefore required, including in an embedding host. The application must control the data directory and keep it from being replaced while in use.

HTTP — synchronous web requests, headers, bodies, TLS, and limits

`babet.http` is a synchronous HTTP/HTTPS client built on cpp-httplib and OpenSSL. It is intended for APIs, webhooks, simple REST calls, bounded in-memory responses, and direct-to-file downloads.

It supports:

- HTTP and HTTPS;
- GET, HEAD, OPTIONS, POST, PUT, PATCH, and DELETE;
- encoded and normalised query parameters;
- request headers;
- text or binary request bodies;
- binary responses;
- repeated response headers;
- optional redirects;
- TLS verification by default;
- connection and global request timeouts;
- a maximum in-memory response size;
- streamed GET downloads committed atomically to a destination file.

It does not expose streaming uploads, response callbacks, persistent sessions, a cookie jar, multipart/form-data, form helpers, proxies, an HTTP server, WebSockets, HTTP/2, or HTTP/3.

Module contents

- Core conventions
- API overview

- `request(opts)`
- `get(url, opts?)`
- `download(url, destination, opts?)`
- `post` call forms
- URLs, fragments, and query
- HTTP methods
- Request headers
- Request body and Content-Type
- Timeouts
- TLS and CA
- Redirects
- Maximum response size
- Download size and file safety
- Response tables
- Complete examples
- Error contract
- Security and limitations

Core conventions

Synchronous client

The call blocks the current thread until response or error. Use independent `workers` for parallel requests.

HTTP status is not a transport error

A received 404 or 500 is a normal response table:

```
local response, err = babet.http.get(url, { timeout = 10 })
assert(response, err)

if response.status >= 200 and response.status < 300 then
    print(response.body)
else
    io.stderr:write("HTTP ", response.status, "\n", response.body)
end
```

DNS, TCP, TLS, timeout, and body-limit failures return `(nil, err)`.

Binary bodies

Request bodies and `response.body` may contain NUL bytes:

```
local response = assert(babet.http.post(url, "AB\0CD", {
    timeout = 10,
}))
```

URLs, methods, header names, CA paths, and other C-facing strings reject NUL. URLs and headers also reject CR/LF to prevent HTTP-line injection.

Unknown option fields

Unknown fields are currently ignored. A typo such as `timeot = 5` does not activate a timeout and does not fail. Use the documented names exactly.

API overview

```
local response, err = babet.http.request(opts)
local response, err = babet.http.get(url, opts?)
local result, err = babet.http.download(url, destination, opts?)
local response, err = babet.http.post(url)
local response, err = babet.http.post(url, body)
local response, err = babet.http.post(url, opts)
local response, err = babet.http.post(url, body, opts)
local response, err = babet.http.post(url, nil, opts)
```

Field	Type	Default	Meaning
`url`	string	required by `request`	absolute `http://` or `https://` URL
`method`	string	`"GET"`	supported method, case-insensitive
`headers`	table	`{}`	string names; string/number values
`body`	binary string	absent	body for body-capable methods
`query`	table	absent	string keys; string/number values
`timeout`	finite number `> 0`	library defaults	seconds, connect plus global budget
`verify`	strict boolean	`true`	HTTPS certificate verification
`ca_cert`	string	default OpenSSL authorities	PEM file used instead of the default authorities for this request
`follow_redirects`	strict boolean	`false`	follow redirects automatically
`max_body_size`	integer `1..2 GiB`	`64 MiB`	in-memory cap for `request`/`get`/`post`
`max_file_size`	positive integer	`8 GiB`	received-file cap for `download`

`get` and `post` always overwrite `url` and `method` after shallow-copying options.

`babet.http.request(opts)`

General form for every method:

```
local response, err = babet.http.request({
    url = "https://api.example.com/items/42",
    method = "DELETE",
    headers = {
        ["Authorization"] = "Bearer " .. token,
    },
    timeout = 10,
})
```

The first argument must be a table. `opts.url` must be a strict string. Invalid option fields return `(nil, err)`; a wrong first-argument type raises.

babet.http.get(url, opts?)

The wrapper shallow-copies `opts`, then forces:

```
opts.url = url
opts.method = "GET"
```

```
local response, err = babet.http.get(
    "https://api.example.com/health",
    { timeout = 5 }
)
```

The second argument must be absent, `nil`, or a table. GET rejects `opts.body` instead of silently dropping it.

babet.http.download(url, destination, opts?)

Downloads a GET response directly to a file without accumulating the body in Lua or in a C++ response string:

```
local result, err = babet.http.download(
    "https://example.com/archive.tar.gz",
    "downloads/archive.tar.gz",
    {
        timeout = 120,
        follow_redirects = true,
        max_file_size = 4 * 1024 * 1024 * 1024,
    }
)
assert(result, err)
assert(result.saved, "HTTP " .. result.status)
print(result.path, result.bytes)
```

The call is synchronous, but received chunks are written progressively to a same-directory temporary file. The final destination is replaced with one `renameat` only after the complete final response has been received and the temporary file has been synced and closed.

The wrapper always forces GET. It accepts the common `headers`, `query`, `timeout`, `verify`, `ca_cert`, and `follow_redirects` options. `body` is rejected. `max_file_size` replaces `max_body_size` for this call:

- strict positive Lua integer;
- default: 8 GiB;
- configurable up to `math.maxinteger`;
- enforced on the actual bytes delivered to the file receiver.

Only a final 2xx response is committed. A received 1xx, unfollowed 3xx, 4xx, or 5xx response is not a transport failure: `download` returns a result table with `saved = false`, removes the temporary file, and leaves any existing destination unchanged.

The destination contract is intentionally strict:

- the parent directory must already exist;
- Babet does not create parent directories;
- `..` path components are rejected;
- parent-directory symlink components are rejected;
- a symlink at the final destination is replaced as an inode; its target is never followed or modified;
- a temporary file is created in the same directory with `O_EXCL` and is removed on every failure;
- a pre-existing destination remains untouched on DNS, TCP, TLS, timeout, size-limit, HTTP non-2xx, or disk-write failure.

The atomic rename prevents readers from observing a partially downloaded final path. It does not provide resume support, progress callbacks, crash-proof parent-directory durability, or preservation of metadata from an older file. The new file is created with mode `0666` filtered by the process umask.

`babet.http.post` call forms

No explicit body:

```
babet.http.post(url)
babet.http.post(url, { timeout = 5 })
babet.http.post(url, nil, { timeout = 5 })
```

Positional body:

```
babet.http.post(url, "payload", { timeout = 5 })
```

Body in options:

```
babet.http.post(url, {
    body = "payload",
    timeout = 5,
})
```

A table in the second position is interpreted as `opts`. Any other non-string, non-nil second value raises. When both a positional body and `opts.body` exist, the positional body wins.

URLs, fragments, and query

Only absolute HTTP/HTTPS URLs are accepted. A non-empty host is required; bracketed IPv6 literals are supported:

```
local r, err = babet.http.get("http://[::1]:8080/", { timeout = 2 })
```

Fragments are removed and never sent.

Add query parameters with a table:

```
local response = assert(babet.http.get("https://example.com/search", {
    query = {
        q = "lua socket",
        page = 2,
    },
    timeout = 10,
}))
```

Keys must be strings; values may be strings or numbers. Babet protects query delimiters and non-ASCII bytes first, then cpp-httplib 0.45.0 normalises the query before it is sent. The effective wire format therefore follows these conventions:

- spaces become `+`;
- `+` becomes `%2B`;
- `/` and `?` remain literal inside a value;
- non-ASCII UTF-8 bytes are percent-encoded as `%HH`;
- `&`, `=`, and `%` are percent-encoded when they belong to a key or value.

For example, `q = "lua socket"` becomes `q=lua+socket`, while `path = "a/b"` remains `path=a/b`.

When the URL already contains `?a=b`, new parameters are appended with `&`. The existing query is normalised as well: `%20` may become `+` and `%2F` may become `/`. The `#...` fragment is removed before transmission.

Lua table iteration order is not stable, so do not build canonical signatures by assuming the order produced from `query`. For canonical signing, construct an already ordered URL yourself and account for the normalisation described above.

NUL bytes in query keys/values are sent as `%00`; make sure the server accepts that semantic.

HTTP methods

Supported methods:

- GET;
- HEAD;
- OPTIONS;
- POST;
- PUT;
- PATCH;
- DELETE.

Method names are uppercased internally:

```
local response = assert(babet.http.request({
    url = "https://api.example.com/items/42",
    method = "patch",
    body = '{"enabled":true}',
    headers = { ["Content-Type"] = "application/json" },
    timeout = 10,
}))
```

GET, HEAD, and OPTIONS reject a body. POST, PUT, PATCH, and DELETE accept an absent, empty, or non-empty body.

There are no `put`, `patch`, or `delete` shortcuts; use `request`.

Request headers

```
headers = {
    ["Accept"] = "application/json",
    ["Authorization"] = "Bearer " .. token,
    ["X-Retry"] = 3,
}
```

Header names must be non-empty HTTP `token` strings: alphanumeric plus `!#$%&'*+-.^_`|~`. Spaces, tabs, colon, CR/LF, NUL, and non-string keys are rejected.

Values may be strings or numbers. Numbers are converted using Lua's textual conversion. NUL, CR/LF, booleans, tables, and other types are rejected.

A Lua table cannot represent repeated exact request-header names. The binding does not expose a multi-value request-header form. Combine values only when the specific header grammar permits it.

Names are sent with the provided case; HTTP matching is case-insensitive. `Content-Type` detection is case-insensitive.

Request body and `Content-Type`

Absent and explicitly empty bodies differ:

```
babet.http.post(url, { timeout = 5 }) -- no body field
babet.http.post(url, "", { timeout = 5 }) -- present empty body
```

When a body exists on POST/PUT/PATCH/DELETE and no Content-Type is supplied, Babet uses `application/octet-stream`. No default Content-Type is added when the body is absent.

JSON example:

```
local payload = assert(babet.json.encode({ name = "babet" }))
local response = assert(babet.http.post(url, payload, {
    headers = {
        ["Content-Type"] = "application/json",
        ["Accept"] = "application/json",
    },
    timeout = 10,
}))
```

There is no `form` helper. Encode URL-encoded form data yourself and set `application/x-www-form-urlencoded`. The `query` table changes the URL only; it does not build a form body.

No multipart builder or streaming upload is exposed. For large/complex uploads, use a specialised tool through `babet.exec` without unsafe shell concatenation.

`opts.timeout`

```
local response, err = babet.http.get(url, { timeout = 10 })
```

The value must be finite and strictly positive. Positive values below 1 ms are rounded up. Babet configures both a connection timeout and cpp-httplib's global maximum request timeout; the first limit reached wins.

Synchronous DNS may still happen outside the controllable library budget and can make wall-clock duration exceed the requested timeout.

Without an explicit timeout, cpp-httplib/system defaults apply. Always set one for remote services.

HTTPS, verification, and CA

Verification defaults to true:

```
local response = assert(babet.http.get("https://example.com/", {
    timeout = 10,
}))
```

Use a private CA file:

```
local response = assert(babet.http.get("https://internal.example/", {
    ca_cert = "/etc/myapp/internal-ca.pem",
    timeout = 10,
}))
```

With a non-empty `ca_cert`, HTTP uses that file's authorities **instead of the default authorities** for the request. When `ca_cert` is absent or `""`, it loads OpenSSL defaults, honoring `SSL_CERT_FILE` and `SSL_CERT_DIR`. An explicit file is not added to default trust. This also applies to `http.download`.

This differs from socket.connect_tls and starttls, and websocket.connect, which **add** custom authorities to those already loaded. HTTP does not expose `ca_path` or perform their additional probing of known distribution CA locations.

For example, if A is a default authority and the supplied file contains only B, HTTP accepts a server signed by B and rejects one signed only by A. Socket TLS and WSS continue to accept both A and B. In every case, `verify = true` also checks server identity. A call's configuration does not make B trusted for subsequent calls.

Selecting a CA file is not leaf-certificate or public-key pinning: multiple certificates signed by an authority in the file can be accepted, subject to the other TLS checks.

```lua
local response = assert(babet.http.get("https://127.0.0.1:8443/", {
    verify = false,
    timeout = 3,
}))
```

Use `verify = false` only in controlled tests. HTTP has no separate hostname override; the URL host is used.

follow_redirects

Redirects are visible by default:

```lua
local r = assert(babet.http.get(url, { timeout = 10 }))
if r.status == 302 then
    print(r.headers.location)
end
```

Enable automatic following:

```lua
local r = assert(babet.http.get(url, {
    follow_redirects = true,
    timeout = 10,
}))
```

The result describes the final response. Redirect history and `max_redirects` are not exposed; cpp-httplib's internal limit applies.

When a redirect changes scheme, host, or port, cpp-httplib 0.45 removes `Host`, `Authorization`, and `Proxy-Authorization` before issuing the next request. A same-origin redirect keeps `Authorization`. Other script-provided headers, notably `Cookie` or custom secrets, are not covered by a general stripping rule, so do not follow an untrusted destination with such headers.

For untrusted URLs, also consider HTTPS-to-HTTP downgrade, internal or cloud metadata addresses, and the final destinations explicitly allowed by the application. The local regression suite verifies that `Authorization` is not forwarded across origins.

max_body_size

The entire response is buffered in memory:

```lua
local response, err = babet.http.get(url, {
    timeout = 30,
    max_body_size = 8 * 1024 * 1024,
})
```

- default: 64 MiB;
- minimum: 1 byte;
- maximum: 2 GiB;
- strict positive Lua integer.

When actual received chunks would exceed the cap, Babet aborts and returns:

```lua
nil, "http: response body exceeds max_body_size"
```

No partial body is exposed. The guard applies to actual received bytes, not only Content-Length, including chunked or misleading responses.

Babet supports the three HTTP/1.1 response-body framings relevant here: `Content-Length`, `Transfer-Encoding: chunked`, and a body terminated by connection close. Babet 2.9.1 specifically fixes the latter two with cpp-httplib 0.45.0; limits still apply to bytes actually delivered to Babet's receiver.

For large GET payloads, use `babet.http.download` instead of raising this in-memory limit.

`max_file_size` and file safety

`max_file_size` applies only to `download`. It limits received body bytes, not the value advertised by `Content-Length`. If the stream would exceed the cap, Babet aborts, removes the temporary file, preserves an existing destination, and returns:

```
nil, "http: response body exceeds max_file_size"
```

The default 8 GiB is a safety guard, not a recommendation to accept every file of that size. Set the smallest limit compatible with the expected artifact. Free-space exhaustion and filesystem errors are reported as `(nil, err)` with the same cleanup guarantee.

Response tables

In-memory response

```
{
    status = 200,
    body = "...",
    headers = {
        ["content-type"] = "application/json",
        ["set-cookie"] = "b=2",
    },
    headers_multi = {
        ["content-type"] = { "application/json" },
        ["set-cookie"] = { "a=1", "b=2" },
    },
}
```

- `status`: integer HTTP status;
- `body`: complete binary string, possibly empty; HEAD returns an empty body;
- `headers`: lowercase names, last occurrence wins;
- `headers_multi`: lowercase names, every occurrence in an array, even one.

Use `headers_multi` for repeated fields such as Set-Cookie:

```
for _, cookie in ipairs(response.headers_multi["set-cookie"] or {}) do
    print(cookie)
end
```

Download result

```
{
    status = 200,
    saved = true,
    bytes = 123456,
    path = "downloads/archive.tar.gz",
    headers = { ... },
    headers_multi = { ... },
}
```

- `status`, `headers`, and `headers_multi` have the same meaning as above;
- `saved` is true only when the final status is 2xx and the atomic commit succeeded;
- `bytes` is the number of received body bytes, including for a non-2xx result;
- `path` exists only when `saved` is true;
- there is deliberately no `body` field.

Complete examples

GET JSON

```
local response, err = babet.http.get("https://api.example.com/v1/status", {
    headers = { ["Accept"] = "application/json" },
    query = { verbose = 1 },
    timeout = 10,
    max_body_size = 1024 * 1024,
})
assert(response, err)
assert(response.status == 200, response.body)
local data = assert(babet.json.decode(response.body))
```

Authenticated JSON POST

```
local body = assert(babet.json.encode({ title = "hello" }))
local response, err = babet.http.post(
    "https://api.example.com/v1/items",
    body,
    {
        headers = {
            ["Authorization"] = "Bearer " .. token,
            ["Content-Type"] = "application/json",
        },
        timeout = 15,
        max_body_size = 2 * 1024 * 1024,
    }
)
assert(response, err)
```

Binary PUT

```
local file = assert(io.open("image.bin", "rb"))
local payload = file:read("a")
file:close()

local response = assert(babet.http.request({
    url = "https://upload.example.com/blob/42",
    method = "PUT",
    body = payload,
    headers = { ["Content-Type"] = "application/octet-stream" },
    timeout = 30,
    max_body_size = 1024 * 1024,
}))
```

This loads the file into memory and is not suitable for very large uploads.

Download a release artifact

```
assert(babet.mkdir("downloads"))
local result, err = babet.http.download(
    "https://example.com/releases/tool.tar.gz",
    "downloads/tool.tar.gz",
    {
        timeout = 120,
        follow_redirects = true,
        max_file_size = 512 * 1024 * 1024,
```

```
    }
)
assert(result, err)
if not result.saved then
    error("unexpected HTTP status " .. result.status)
end
print("saved", result.bytes, "bytes to", result.path)
```

Separate network and HTTP failures

```
local response, err = babet.http.get(url, { timeout = 5 })
if not response then
    io.stderr:write("transport failure: ", err, "\n")
elseif response.status == 404 then
    print("not found")
elseif response.status >= 400 then
    io.stderr:write("HTTP ", response.status, "\n")
else
    print(response.body)
end
```

Parallel requests with WORKERS

```
local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = assert(babet.workers.spawn([[
        local r, err = babet.http.get(worker.args.url, {
            timeout = 10,
            max_body_size = 1024 * 1024,
        })
        if not r then error(err) end
        return { status = r.status, size = #r.body }
    ]], { url = url }))
end

for i, job in ipairs(jobs) do
    local ok_result, value = job:join()
    if ok_result then
        print(urls[i], value.status, value.size)
    else
        io.stderr:write(urls[i], ": ", value, "\n")
    end
end
```

Error contract

Wrong positional types raise:

```
babet.http.request("not a table")
babet.http.get(42)
babet.http.get(url, "not a table")
babet.http.post(url, 42)
babet.http.download(url, 42)
babet.http.download(url, "file", "not a table")
```

Invalid values and runtime failures return `(nil, err)`: missing/bad URL, unsupported method, forbidden body, invalid options/query/headers, DNS/TCP/TLS, timeout, response/file limit, destination-

path or filesystem failure, and converted internal exceptions. Allocation failure returns `(nil, "http: out of memory")`; other internal exceptions return `(nil, "http: internal failure")`.

Any received HTTP status, including 3xx/4xx/5xx, returns a normal response. For `download`, only 2xx is saved; all other statuses return `saved = false`.

Internal network operations, including TLS handshake and shutdown, are protected against `SIGPIPE` without replacing the application's signal handler. Transport failures surface as HTTP errors; a best-effort shutdown failure does not discard an already received response.

Security and limitations

- Validate untrusted URLs against SSRF to loopback, LAN, cloud metadata, and administrative services.
- Treat redirects as new destinations and protect sensitive headers.
- Keep TLS verification enabled; use private CA trust instead of disabling it.
- Set the smallest practical `max_body_size` or `max_file_size`.
- `request`, `get`, and `post` keep response bodies in memory. `download` streams only GET responses to a file; uploads are still in memory.
- Atomic replacement protects the final path from partial content, but callers remain responsible for validating expected status, size, and cryptographic checksum.
- Every call constructs a client; no exposed cookie jar or connection pool.
- No form/multipart helper, proxy configuration, HTTP/2/3, WebSocket, or server.
- DNS is synchronous and may exceed the requested timeout.

`babet.inotify` — filesystem watching

`babet.inotify` provides a small interface to Linux `inotify(7)`. Notifications come from the kernel, so watched directories do not need to be scanned periodically.

The module is **Linux-specific** and watching is not recursive.

Module contents

- API
- Creating a watcher
- Adding a watch
  - `onlydir` option
- Reading events
  - Queue overflow
  - Signal interruption
- Removing and closing
- Example
- Error contract
- Limits

API

Function	Result
`babet.inotify.new()`	`watcher | (nil, err)`
`w:add(path, events [, opts])`	integer `wd` | `(nil, err)`
`w:read([timeout])`	event array | `(nil, "timeout")` | `(nil, "interrupted")` | `(nil, err)`
`w:remove(wd)`	`(true, nil) | (nil, err)`
`w:close()`	`(true, nil)`

Signatures are strict: extra arguments raise a Lua error. `close()` is idempotent, and garbage collection also closes a forgotten watcher.

Creating a watcher

```lua
local watcher, err = babet.inotify.new()
if not watcher then
    error(err)
end
```

Each call creates an independent instance. The inotify descriptor uses `IN_NONBLOCK` and `IN_CLOEXEC`: `read()` controls waiting itself, and the descriptor is not inherited by programs launched through `exec`.

Adding a watch

```lua
local wd, err = watcher:add(path, events [, opts])
```

`path` must be a string without an embedded NUL byte. It may refer to an existing directory or file.

`events` is a strict, non-empty `1..N` array with no holes or extra keys. Every element must be one of these names:

Name	Requested event
`"access"`	file read or accessed
`"modify"`	content modified
`"attrib"`	attributes or metadata modified
`"close_write"`	write-open descriptor closed
`"close_nowrite"`	non-write descriptor closed
`"close"`	combination of `close_write` and `close_nowrite`
`"open"`	file opened
`"moved_from"`	entry moved out of the watched directory
`"moved_to"`	entry moved into the watched directory
`"move"`	combination of `moved_from` and `moved_to`
`"create"`	entry created in the watched directory
`"delete"`	entry deleted from the watched directory
`"delete_self"`	watched target deleted
`"move_self"`	watched target moved

`onlydir` option

```lua
local wd = assert(watcher:add(path, { "create" }, {
    onlydir = true,
}))
```

When present, `onlydir` must be a boolean:

- `true` adds Linux's `IN_ONLYDIR` mask and therefore rejects a target that is not a directory;
- `false` keeps the normal behavior;
- other fields in `opts` are currently ignored.

Adding a watch again for the same target follows native inotify semantics: the existing mask is replaced because `IN_MASK_ADD` is not used.

Reading events

```lua
local events, err = watcher:read([timeout])
```

The timeout is in seconds and may be fractional:

- omitted or `nil`: wait indefinitely;
- `0`: non-blocking read of events already available;
- positive value: bounded wait.

A negative, NaN, infinite, or excessively large timeout returns `(nil, err)`. A value of the wrong type raises a Lua error.

A successful read returns an array of tables:

```
{
    {
        wd = 1,
        name = "photo.jpg",
        events = { create = true, close_write = true },
        is_dir = false,
        cookie = 0,
    },
}
```

- `wd` is the watch descriptor returned by `add`;
- `name` is the affected entry name, or `""` for an event on the watched target itself;
- `events` contains a true boolean for every received bit;
- `is_dir` reflects `IN_ISDIR`;
- `cookie` pairs `moved_from` with `moved_to`; outside moves it is usually `0`.

Besides events requestable through `add`, the kernel may produce:

- `events.ignored` when a watch is removed, automatically or through `remove`;
- `events.unmount` when the watched filesystem is unmounted.

One call may return several events because the module reads and decodes a whole batch from the kernel queue.

Queue overflow

If the inotify queue overflows, events have been lost. The module surfaces this explicitly:

```
{
    wd = -1,
    name = "",
    events = { overflow = true },
    is_dir = false,
    cookie = 0,
}
```

The program must then rescan watched paths to rebuild reliable state.

Signal interruption

If a signal handled by `babet.signal` arrives while waiting, its Lua callback runs and `read()` then returns `(nil, "interrupted")`.

Removing and closing

```
assert(watcher:remove(wd))
assert(watcher:close())
```

After `remove(wd)`, the kernel normally queues an `ignored` event. Removing an invalid descriptor returns `(nil, err)`.

After `close`, `add`, `read`, and `remove` return an error saying the watcher is closed. `close` may be called more than once.

Example

```
local watcher = assert(babet.inotify.new())
local wd = assert(watcher:add("/srv/incoming", {
    "close_write",
    "moved_to",
}, {
    onlydir = true,
}))

while true do
    local events, err = watcher:read()
    if not events then
        if err == "interrupted" then
            break
        end
        error(err)
    end

    for _, event in ipairs(events) do
        if event.events.overflow then
            rescan_directory()
        elseif event.wd == wd then
            process_entry(event.name)
        end
    end
end

watcher:close()
```

Error contract

- wrong argument count or type: Lua error;
- invalid value, system error, or closed watcher: `(nil, err)`;
- no event before the deadline: `(nil, "timeout")`;
- handled signal during `read`: `(nil, "interrupted")`;
- successful `remove` and `close`: `(true, nil)`;
- unexpected C++ exception: `(nil, "inotify: out of memory")`, `(nil, "inotify: internal failure")`, or `(nil, "inotify: unknown internal failure")`.

System errors are prefixed with `"inotify: "` and retain the description provided by the operating system.

The factory and every userdata method enter through this boundary. `__gc` uses a dedicated non-throwing catch-all path: garbage-collector cleanup never tries to allocate or return an error while handling an exception.

Limits

- Linux only;
- no recursive watching;
- no `IN_DONT_FOLLOW`, `IN_ONESHOT`, or `IN_MASK_ADD`;
- one instance may hold several watches, and several independent instances may coexist in one process.

`babet.json` — JSON encoding and decoding

The `babet.json` module is backed by **nlohmann/json 3.11.3**. It converts Lua values to JSON text and back, with two sentinels that represent JSON `null` and the empty array `[]` without ambiguity.

Module contents

- API
- Type mapping
  - Lua to JSON
  - JSON to Lua
- `null`
- Empty object, empty array, and `as_array`
- `encode` options
- Strings, UTF-8, and NUL bytes
- Numbers
- Object order and duplicate keys
- Error contract
- Limits

API

Element	Contract
`babet.json.encode(value [, opts])`	Returns `text, nil` or `nil, err`
`babet.json.decode(text)`	Returns `value, nil` or `nil, err`
`babet.json.null`	Sentinel representing JSON `null`
`babet.json.empty_array`	Constant sentinel that encodes as `[]`
`babet.json.as_array(t)`	Tags `t` as a JSON array and returns the same table

`encode` accepts exactly one or two arguments. `decode` requires exactly **one Lua string**: a number is not implicitly converted to text. `as_array` requires exactly one table.

Type mapping

Lua to JSON

Lua	JSON
`nil` passed directly to `encode`	`null`
`babet.json.null`	`null`
`babet.json.empty_array`	`[]`
`babet.sqlite.NULL`	rejected; use `babet.json.null`
boolean	boolean
Lua integer	JSON integer number
finite Lua float	JSON floating-point number
valid UTF-8 string	JSON string
untagged empty table	empty object `{}`
integer keys exactly `1..n`	JSON array
string keys only	JSON object

Inside a Lua table, assigning `nil` removes the key before encoding ever starts. Use `babet.json.null` to retain a key whose JSON value must explicitly be `null`.

`babet.sqlite.NULL` is intentionally not an alias for JSON null. It is a SQLite-only lightuserdata bind sentinel, so `encode` returns an error naming `babet.sqlite.NULL` explicitly. Use `babet.json.null` at a JSON boundary.

A table mixing string and integer keys, a sparse array, or a table with an unrepresentable key (`0`, a negative integer, a float, a boolean, a table, and so on) cannot be converted: `encode` returns `nil, err`.

JSON to Lua

JSON	Lua
`null`	`babet.json.null`
boolean	boolean
signed integer in the Lua range	Lua integer
unsigned integer above `math.maxinteger`	Lua float
JSON floating-point number	Lua float
string	Lua string
array	sequential table `1..n`
object	string-keyed table

A JSON integer too large for `lua_Integer` is converted to a float instead of silently wrapping to a negative value. As with any IEEE 754 conversion, precision may be lost.

`null`

Lua cannot retain a `nil` value in a table. The shared `babet.json.null` sentinel fills that gap:

```lua
local J = babet.json

local value = assert(J.decode([[{"answer":null}]]))
assert(value.answer == J.null)

local text = assert(J.encode({ answer = J.null }))
-- text represents {"answer":null}
```

Compare the sentinel by identity with `==`. It is implemented as a special table, but must not be used as a data table.

Empty object, empty array, and `as_array`

An empty Lua table is ambiguous. Babet chooses a JSON object by default:

```lua
local J = babet.json

assert(J.encode({}) == "{}")
assert(J.encode(J.empty_array) == "[]")
assert(J.encode(J.as_array({})) == "[]")
```

`empty_array` is a shared sentinel for values that are not meant to be modified. `as_array(t)` is preferable for a dynamically built array: it tags the table with an internal metatable, leaves it mutable, and returns that exact same table.

```lua
local tags = {}
-- table.insert(tags, ...) may never be called

local text = assert(J.encode({ tags = J.as_array(tags) }))
-- {"tags":[]}
```

Important details:

- `as_array` is idempotent;
- the tag **replaces any existing metatable** on the table;
- the shape is validated only when `encode` runs: a tagged table with string keys or holes returns `nil, err`;
- `as_array(J.null)` and `as_array(J.empty_array)` raise a Lua error;
- only a decoded **empty** JSON array receives the tag automatically. A decoded non-empty array that is later emptied becomes an ordinary empty table and therefore re-encodes as `{}`; call `as_array(t)` to retain `[]` in that case;
- ordinary writes to the sentinels are rejected. As with every Lua table, `rawset` bypasses metamethods: do not mutate the sentinels.

`encode` options

The optional table has one interpreted option:

```lua
local compact = assert(babet.json.encode({ a = 1 }))
local pretty  = assert(babet.json.encode({ a = 1 }, { indent = 4 }))
```

- `indent = 0..256` enables formatted output with line breaks;
- `indent = 0` adds no indentation spaces but keeps formatted-output line breaks;
- omitted, `nil`, or a negative integer: compact output;
- a non-integer value or a value above `256` raises a Lua error;
- other option fields are ignored. In particular, the previously and incorrectly documented `pretty` option does not exist.

Strings, UTF-8, and NUL bytes

JSON strings must contain valid UTF-8. Unicode characters are preserved in output; Babet does not force them into `\uXXXX` escapes.

Lua strings may contain NUL bytes. They remain binary-safe on the Lua side and are escaped according to JSON rules during encoding. The same applies to object keys:

```lua
local J = babet.json
local original = "a\0b"
local text = assert(J.encode({ [original] = original }))
local back = assert(J.decode(text))
assert(back[original] == original)
```

A string or key containing invalid UTF-8 makes `encode` return `nil, err`. JSON text containing an invalid UTF-8 string is likewise rejected by `decode`.

Numbers

Babet preserves the Lua numeric subtype when JSON permits it: `42` decodes as a Lua integer, whereas `42.0` and `1e3` decode as floats. When encoding, a Lua float such as `3.0` remains a JSON floating-point number.

`NaN`, `+Inf`, and `-Inf` are not part of the JSON standard and return `nil, err`.

Object order and duplicate keys

JSON object member order has no semantic meaning and must not be treated as an API contract. The current backend generally emits members in lexicographic key order, but callers should compare decoded data rather than raw text whenever an external format does not prescribe ordering.

When decoding a JSON object that repeats the same key, the last value replaces earlier ones.

Error contract

Conversion and parsing failures are returned and do not raise a Lua exception:

```
local value, err = babet.json.decode("{ invalid")
if not value then
    print(err) -- prefixed with "json:"
end
```

This includes, among other cases:

- invalid JSON, trailing data after the document, or invalid UTF-8;
- mixed tables, sparse arrays, or unrepresentable keys;
- a function, userdata (including `babet.sqlite.NULL`), or thread that cannot be encoded;
- `NaN` or infinity;
- a cycle or excessive nesting depth.

API misuse does raise a Lua error:

- wrong argument count;
- a non-string argument to `decode`;
- an `encode` second argument that is neither a table nor `nil`;
- a non-integer `opts.indent` or one above `256`;
- an invalid argument to `as_array`.

Parsing errors produced by nlohmann/json include the failure position, usually as a line and column.

Limits

Conversion depth is capped at **1000 levels** in both directions. This also acts as the guard against cyclic Lua tables. The module builds the whole document in memory and does not provide a streaming API.

`logging` — leveled logging

`logging` is a pure-Lua module bundled with Babet. Load it with `require("logging")`; it is not a member of the `babet` table.

```
local log = require("logging")
```

It exposes five levels:

Level	Value
`log.TRACE`	10
`log.DEBUG`	20
`log.INFO`	30
`log.WARN`	40
`log.ERROR`	50

There is no `fatal` level.

Module contents

- Emitting a message
  - No-propagation guarantee
- Output format
- Threshold
- Destination

- ANSI colors
- Module state
- Error contract
- Limitations

Emitting a message

```
log.trace(...)
log.debug(...)
log.info(...)
log.warn(...)
log.error(...)
```

Each function accepts zero or more values and returns no values. A message is emitted when the function's level is greater than or equal to the current threshold.

The initial threshold is `log.INFO`, so `trace` and `debug` are filtered by default.

Arguments are converted with `tostring` and joined with one space:

```
log.info("user=", 42, "active=", true)
-- ... [INFO ] user= 42 active= true
```

Calling a log function with no arguments emits a line with an empty message. Embedded NUL bytes and newlines are neither removed nor escaped. A multiline message therefore produces several physical lines, but only the start of the call has the logger prefix.

No-propagation guarantee

The five emission functions never raise. The whole pipeline is protected:

- argument conversion through `tostring`;
- timestamp creation through `os.date`;
- the sink's `write` call.

If any of these operations fails, the message is silently lost and the script continues. Arguments of a filtered message are not passed to `tostring`.

Output format

Without colors, each line uses this format:

`YYYY-MM-DD HH:MM:SS [LEVEL] message\n`

Example:

`2026-07-13 21:45:02 [WARN ] attempt 3 of 5`

The timestamp uses the machine's **local time**. It includes neither a time zone nor milliseconds. Labels occupy five characters: `[TRACE]`, `[DEBUG]`, `[INFO ]`, `[WARN ]`, `[ERROR]`.

Threshold

```
log.set_level(level)
local level = log.get_level()
```

`set_level` requires exactly one argument and returns no values. `get_level` accepts no arguments and returns the current threshold.

The level may be:

- one of `"trace"`, `"debug"`, `"info"`, `"warn"`, `"error"`, case-insensitively;
- any finite number, allowing an intermediate threshold such as `25.5`.

```lua
log.set_level("DEBUG")
log.set_level(log.WARN)
log.set_level(25.5)
```

Numeric strings are not coerced and whitespace is not trimmed: `"30"` and `" info "` are rejected. `NaN`, `+inf`, and `-inf` are rejected as well.

Destination

```lua
log.set_output(out)
local out = log.get_output()
```

The initial destination is `io.stderr`.

`set_output` requires exactly one table or userdata exposing a callable `write` method. A method supplied through `__index` is accepted. The method is then called as `out:write(line)`.

```lua
local file, err = io.open("app.log", "a")
if not file then
    log.error("cannot open log file:", err)
else
    log.set_output(file)
end
```

The module:

- performs no write during `set_output`;
- never closes the destination;
- never calls `flush`;
- ignores the value returned by `write`;
- swallows write errors during emission.

A closed file still exposes a `write` method and can therefore be installed; subsequent writes will fail silently. Sink lifetime management belongs to the script.

`get_output` accepts no arguments and returns the exact installed object.

ANSI colors

```lua
log.set_color(true)
local enabled = log.get_color()
```

Colors are disabled by default. `set_color` requires exactly one boolean and returns no values. `get_color` accepts no arguments and returns a boolean.

When colors are enabled:

- `trace`: ANSI dim;
- `debug`: cyan;
- `info`: no color;
- `warn`: yellow;
- `error`: red.

The ANSI reset is written before the `\n`. The module does not detect whether the sink is a terminal; enabling colors is always explicit.

Module state

`require("logging")` returns the same table until `package.loaded.logging` is removed. The threshold, sink, and color setting are therefore shared by every user of the module within **one Lua state**.

Each Babet worker owns a separate Lua state, so its logging settings are independent from the main thread and from other workers.

The initial values are:

```
log.set_level(log.INFO)
log.set_output(io.stderr)
log.set_color(false)
```

The module also exposes:

```
log._VERSION      -- "babet logging 1.1.0"
log._DESCRIPTION  -- textual description
```

Error contract

Setters and getters raise a Lua error on wrong arity or an invalid value. They do not return `(nil, err)`.

`trace`, `debug`, `info`, `warn`, and `error` never raise and return no values.

Limitations

The module does not provide:

- named or hierarchical loggers;
- per-module thresholds;
- multiple destinations;
- custom or JSON formatting;
- UTC timestamps or sub-second precision;
- file rotation;
- buffered or asynchronous writes;
- protection against a recursive sink that calls the logger again;
- automatic terminal detection.

Use an external tool such as `logrotate` for rotation. For more advanced needs, wrap the module or replace its functions in the script.

SIGNAL — graceful shutdown, reloads, and POSIX signals

`babet.signal` lets the main script install a Lua callback for a small set of POSIX signals, ignore a signal, or restore its operating-system default.

The module covers:

- graceful shutdown on `TERM` and `INT`;
- configuration reload requests on `HUP`;
- two application-defined signals, `USR1` and `USR2`;
- explicitly ignoring or restoring a signal;
- interruption of selected blocking Babet calls;
- deferred, safe callback dispatch in the main Lua thread.

It does not send signals, expose a pending-signal queue, configure POSIX masks, or allow handlers to be installed from a worker.

Module contents

- Essential conventions
- API overview
- Supported signals

- `handle(name, fn)` — install or replace
- `handle(name, nil)` — remove a callback
- `ignore(name)` — ignore
- `default(name)` — restore the OS default
- Callback execution
  - Main thread and safe context
  - Dispatch order
  - Occurrence coalescing
  - Callback errors
- Interaction with blocking calls
- Interaction with `debug.sethook`
- Complete examples
  - Graceful shutdown on `TERM` and `INT`
  - Reload configuration with `HUP`
  - Interrupt a socket wait
  - Temporarily ignore `PIPE`
  - Observe callback errors
- Error contract
- Workers, concurrency, and limitations
- Features not provided

Essential conventions

Success returns one value

All three functions return only the boolean `true` on success. They do not return a second `nil`.

```lua
local ok = babet.signal.ignore("PIPE")
assert(ok == true)
```

A rare `sigaction(2)` failure instead returns two values:

```lua
local ok, err = babet.signal.ignore("PIPE")
if not ok then
    io.stderr:write(err, "\n")
end
```

Names are strict

A signal name must be an actual Lua string, with no NUL byte, in the exact case shown by this documentation.

```lua
assert(babet.signal.handle("TERM", function() end))
```

These forms are invalid:

```lua
babet.signal.handle("term", function() end) -- wrong case
babet.signal.handle(15, function() end)     -- number, not string
babet.signal.handle("TERM\0X", function() end)
```

A callback receives no arguments

The callback is invoked with no parameters. Install a distinct function for each signal when code needs to know which signal arrived.

```lua
babet.signal.handle("USR1", function()
    print("USR1")
end)
```

API overview

```
local ok, err = babet.signal.handle(name, callback_or_nil)
local ok, err = babet.signal.ignore(name)
local ok, err = babet.signal.default(name)
```

Function	Success	Rare system failure	Invalid call
`handle(name, fn)`	`true`	`(nil, err)`	raises a Lua error
`handle(name, nil)`	`true`	`(nil, err)`	raises a Lua error
`ignore(name)`	`true`	`(nil, err)`	raises a Lua error
`default(name)`	`true`	`(nil, err)`	raises a Lua error

`handle(name)` without a second argument is intentionally rejected. Write `handle(name, nil)` explicitly to remove a callback.

Supported signals

Babet name	POSIX signal	Common use	Usual OS default
`"TERM"`	`SIGTERM`	graceful shutdown request, notably from systemd	terminate the process
`"INT"`	`SIGINT`	keyboard interruption, usually Ctrl-C	terminate the process
`"HUP"`	`SIGHUP`	configuration reload request	often terminate the process
`"USR1"`	`SIGUSR1`	free application event	terminate the process
`"USR2"`	`SIGUSR2`	second free application event	terminate the process
`"PIPE"`	`SIGPIPE`	writing to a closed pipe or socket	terminate the process

`KILL` and `STOP` cannot be exposed: POSIX does not allow them to be caught or ignored.

Dangerous signals or signals reserved for other internal mechanisms — such as `SEGV`, `BUS`, `FPE`, `ILL`, `CHLD`, and `ALRM` — are not in the allowlist.

`handle(name, fn)` — install or replace

```
local ok, err = babet.signal.handle(name, fn)
```

- `name` is one of the six supported names;
- `fn` must be a Lua function;
- the previous POSIX disposition is replaced;
- an existing Lua callback for the same signal is replaced;
- the callback is retained in the Lua registry until removed.

```
local stopping = false

assert(babet.signal.handle("TERM", function()
    stopping = true
end))
```

Replacing a callback:

```lua
assert(babet.signal.handle("USR1", function()
    print("first version")
end))

assert(babet.signal.handle("USR1", function()
    print("new version")
end))
```

After the second call, only the new function can run.

`handle(name, nil)` — remove a callback

```lua
local ok, err = babet.signal.handle(name, nil)
```

This form:

1. removes the associated Lua callback;
2. restores `SIG_DFL`, the operating-system default.

```lua
assert(babet.signal.handle("HUP", reload_config))

-- Later: explicit removal.
assert(babet.signal.handle("HUP", nil))
```

`handle("TERM")` without a second argument is **not** shorthand: it raises an error, preventing accidental removal caused by a missing argument.

`ignore(name)` — ignore

```lua
local ok, err = babet.signal.ignore(name)
```

`ignore` installs `SIG_IGN` and removes any previously associated Lua callback. The kernel then ignores that signal.

```lua
assert(babet.signal.ignore("PIPE"))
```

This is useful when an application prefers normal write errors over process termination caused by `SIGPIPE`.

The ignore disposition remains active until a later `handle`, `default`, or `handle(name, nil)` call for the same signal.

`default(name)` — restore the OS default

```lua
local ok, err = babet.signal.default(name)
```

`default` installs `SIG_DFL` and removes any Lua callback. For `TERM`, `INT`, `HUP`, `USR1`, `USR2`, and usually `PIPE`, the next matching signal terminates the process according to operating-system rules.

```lua
assert(babet.signal.ignore("PIPE"))
-- ... section where SIGPIPE should be ignored ...
assert(babet.signal.default("PIPE"))
```

`handle(name, nil)` and `default(name)` have the same system effect. `default` is more explicit when no callback is involved.

Callback execution

The real POSIX handler never invokes Lua. It only sets a `sig_atomic_t` flag, which is safe in asynchronous signal context.

The Lua callback runs later from a safe point:

1. a supported signal arrives;
2. the C handler marks that signal pending;
3. the main thread notices the flag from the instruction hook or when an interrupted blocking call returns;
4. Babet clears the flag;
5. Babet invokes the Lua callback with zero arguments.

Main thread and safe context

The callback runs in the active Lua coroutine on the main OS thread, never in the POSIX handler or a worker. Callbacks share the registry of that Lua state and can use ordinary Lua and Babet APIs.

```lua
babet.signal.handle("USR1", function()
    -- Runs in normal Lua context.
    local now = babet.time.now()
    print("USR1 at", now)
end)
```

That freedom does not mean a callback should be long. A short function that sets a flag or schedules an action remains easier to reason about.

Dispatch order

When several distinct signals are pending at the same dispatch point, Babet processes them in the fixed order of its allowlist:

1. `TERM`;
2. `INT`;
3. `HUP`;
4. `USR1`;
5. `USR2`;
6. `PIPE`.

This is neither callback registration order nor a guarantee of exact kernel arrival order.

For example, if `USR1` and then `TERM` arrive before the next dispatch, the `TERM` callback runs before the `USR1` callback.

Occurrence coalescing

Babet stores one flag per signal type, not a counter or queue. Multiple identical occurrences received before the next dispatch are therefore coalesced into one callback invocation.

```
USR1, USR1, USR1 before dispatch -> one USR1 callback invocation
```

A new identical signal received **while** its callback runs can set the flag again and be processed during a later dispatch.

Do not use this module as a reliable event counter. Use a queue, pipe, socket, or another message mechanism when every business event must be retained.

Callback errors

Since Babet 2.35.0, callback errors are contained and reported with a traceback on stderr, prefixed with `babet.signal callback error:` (or passed to this state's `gui.onError`). They do not propagate to the interrupted code. Published 2.34.0 discarded them.

```lua
babet.signal.handle("USR1", function()
    error("invisible to the caller")
end)
```

The body may also handle errors locally:

```
babet.signal.handle("USR1", function()
    local ok, err = pcall(function()
        refresh_metrics()
    end)
    if not ok then
        io.stderr:write("USR1: ", tostring(err), "\n")
    end
end)
```

Interaction with blocking calls

Babet installs handlers without `SA_RESTART`. Selected internal blocking calls can therefore be interrupted by a handled signal. When the main thread sees a pending Babet signal, it dispatches callbacks and generally returns a typed `"interrupted"` error.

Integrated call families currently include, among others:

- `babet.sleep(...)`;
- Inotify `watcher:read(...)`;
- TCP connection, accept, and receive waits;
- TLS handshakes and waits.

Example:

```
local data, err = peer:recv(4096, 30)
if not data then
    if err == "interrupted" then
        -- A callback has already run: re-check application state.
    elseif err == "timeout" then
        -- No bytes arrived before the deadline.
    else
        -- Close or network error.
    end
end
```

Not every Babet function is interruptible through this mechanism. `babet.exec`, for example, applies its own timeout and does not expose an `"interrupted"` result for handled signals.

A syscall can also receive `EINTR` while no Babet-managed signal is pending. In that case internal loops resume waiting instead of reporting a false `"interrupted"`.

Interaction with `debug.sethook`

Outside integrated blocking functions, Babet uses a Lua count hook, triggered about every 10,000 instructions, to inspect pending flags and terminal events. It is installed on demand by a successful `signal.handle(name, fn)` or `curses.start()`. At startup, `debug.gethook()` remains `nil`, including in a recreated embedding context. Scripts using neither function therefore avoid the hook cost. An active count hook also slows interpretation between callback invocations; merely increasing its interval does not remove that cost.

New coroutines inherit their creator's current hook. Enable signals or curses before creating coroutines that must process events during pure Lua loops. Existing coroutines do not receive the hook retroactively: calling `signal.handle(name, fn)` from such a coroutine equips both that coroutine and the main Lua thread. Integrated blocking calls retain their signal dispatch points. Workers do not install this hook reserved for the main OS thread.

Lua supports only one active hook per Lua thread. Consequently:

- `signal.handle(name, fn)` checks and, if necessary, restores Babet's hook on both the main Lua thread and the calling coroutine, replacing a custom hook;
- a later `debug.sethook(...)` replaces Babet's hook on its target coroutine; instruction-based signal/terminal dispatch stops there until restored, although integrated blocking calls may still dispatch signals;
- hooks on other existing coroutines are not changed by that restoration; a coroutine with a deliberately replaced hook must restore it through `signal.handle(name, fn)` from that coroutine if automatic dispatch is wanted;
- removing callbacks with `handle(name, nil)`, `ignore` or `default` leaves the runtime hook active if already installed and does not restore a previous user hook; these calls do not install an absent hook. `curses.stop()` does not remove the shared hook either, so its cost can persist.

Avoid combining `babet.signal` and a custom debug hook unless this interaction is deliberate and understood. The runtime hook also serves curses events even when no signal callback has been registered.

Complete examples

Graceful shutdown on `TERM` and `INT`

```lua
local running = true

local function request_stop()
    running = false
end

assert(babet.signal.handle("TERM", request_stop))
assert(babet.signal.handle("INT", request_stop))

while running do
    do_one_iteration()

    local slept, err = babet.sleep(250, "ms")
    if not slept and err ~= "interrupted" then
        io.stderr:write(tostring(err), "\n")
        break
    end
    -- On "interrupted", the callback has already run and the loop
    -- re-checks running.
end

close_resources()
```

For a network loop, use the socket's direct `"interrupted"` result instead of `pcall`.

Reload configuration with `HUP`

It is often better for the callback to set a flag only. Full loading then happens in the normal loop.

```lua
local running = true
local reload_requested = false

assert(babet.signal.handle("TERM", function()
    running = false
end))

assert(babet.signal.handle("HUP", function()
    reload_requested = true
```

```
end))

while running do
    if reload_requested then
        reload_requested = false
        local new_config, err = load_config()
        if new_config then
            config = new_config
        else
            io.stderr:write("reload: ", err, "\n")
        end
    end

    do_one_iteration(config)
end
```

Interrupt a socket wait

```
local running = true

assert(babet.signal.handle("TERM", function()
    running = false
end))

while running do
    local client, err = server:accept(60)
    if client then
        serve(client)
    elseif err == "interrupted" then
        -- The TERM callback may have set running to false.
    elseif err ~= "timeout" then
        io.stderr:write("accept: ", err, "\n")
    end
end

server:close()
```

Temporarily ignore `PIPE`

```
assert(babet.signal.ignore("PIPE"))

local sent, err = socket:send(payload)
if not sent then
    io.stderr:write("send: ", err, "\n")
end

assert(babet.signal.default("PIPE"))
```

Warning: restoring default `SIGPIPE` behavior means a later write to a closed pipe may terminate the process.

Observe callback errors

```
local function safe_handler(label, fn)
    return function()
        local ok, err = xpcall(fn, debug.traceback)
        if not ok then
```

```
            io.stderr:write(label, ": ", tostring(err), "\n")
        end
    end
end

assert(babet.signal.handle("USR2", safe_handler("USR2", function()
    rotate_logs()
end)))
```

Error contract

Raised Lua errors

These situations immediately raise a Lua error:

- call from a worker;
- missing, non-string, or NUL-containing `name`;
- unsupported name or wrong case;
- `handle` without a second argument;
- handler other than a function or `nil`.

```
local ok, err = pcall(function()
    babet.signal.handle("TERM")
end)
assert(not ok)
```

An unsupported name also raises; it does not return `(nil, err)`.

```
local ok, err = pcall(function()
    babet.signal.ignore("QUIT")
end)
```

Returned system error

If `sigaction(2)` fails, the function returns:

```
nil, "signal: sigaction failed: ..."
```

This is rare with the six allowed signals, but callers may retain the `(ok, err)` convention when they need to handle it.

Workers, concurrency, and limitations

Signal dispositions are process-wide, not thread-local. Babet therefore enforces these rules:

- `handle`, `ignore`, and `default` are restricted to the main thread;
- using them from a worker raises an explicit error;
- workers block all six supported signals so they remain owned by the main thread;
- workers cannot observe or consume the parent's pending flags.

Control a worker through message queues, not `babet.signal`:

```
local job = assert(babet.workers.spawn([[
    while true do
        local ok, message = worker.recv()
        if not ok then
            return "closed"
        end
        if message == "stop" then
            return "stopped"
        end
```

```
    end
]]))

-- The main handler converts the signal into an application message.
babet.signal.handle("TERM", function()
    job:send("stop", 0)
end)
```

Keep this callback short. A full queue can make the non-blocking send fail; a robust application also keeps a fallback flag.

Features not provided

The module currently does not provide:

- `kill(pid, name)` to send a signal;
- `list()` to retrieve supported names;
- `is_pending()` or an occurrence counter;
- `sigprocmask` or per-thread masks;
- `signalfd`;
- signals beyond the six listed names;
- a debug hook that can be chained with the user's hook.

An external program can send a signal from Lua:

```
local result, err = babet.exec("kill", {
    "-TERM",
    tostring(pid),
})
assert(result, err)
assert(result.code == 0, result.stderr)
```

Reception remains subject to ordinary POSIX rules and the dispatch model described on this page.

SOCKET — TCP and Unix sockets, binary streams, lines, and timeouts

`babet.socket` provides synchronous stream sockets over TCP or the Unix domain (`AF_UNIX`) for custom protocols: IRC, plain SMTP before STARTTLS, line-oriented services, binary streams, and small local daemons.

It covers:

- connecting to a hostname or IP address;
- connecting locally through a Unix socket pathname;
- listening on a network interface or Unix pathname;
- accepting clients;
- sending a complete binary Lua string;
- receiving chunks, lines, or everything until remote EOF;
- per-socket and per-call deadlines;
- local and peer addresses;
- explicit close and garbage-collector cleanup;
- interruption by signals handled through Babet.

It does not expose UDP, the Linux abstract Unix namespace, asynchronous I/O, `select`/`poll` loops, or an application protocol. Use HTTP for web requests and TLS for direct TLS or STARTTLS.

Module contents

- Core conventions
- API overview
- Connecting a client
- Creating a TCP server
- Connecting to a Unix socket
- Creating a Unix server
- Accepting a client
- Default timeout
- Sending data
- Receiving at most N bytes
- Receiving one line
- Receiving until EOF
- Mixing receive methods
- Local and peer addresses
- Closing and GC
- Signals and interruption
- Complete examples
- Error contract
- Security and limitations

Core conventions

Synchronous streams

Every call runs in the current thread. Without a timeout, `connect`, `connect_unix`, `accept`, `send`, and receive methods can wait indefinitely. Use `workers` for independent concurrent connections; Babet does not expose a Lua event loop.

Binary-safe data

Payload strings may contain NUL bytes:

```
assert(sock:send("AB\0CD") == 5)
local data = assert(sock:recv(5, 2))
assert(#data == 5 and data:byte(3) == 0)
```

Hostnames and Unix pathnames are passed to C/POSIX APIs and may not contain NUL.

Timeout rules

Timeouts are seconds and may be fractional:

- omitted or `nil`: use the socket default;
- `0`: wait forever;
- positive finite number: one deadline for the whole call;
- positive values below 1 ms: rounded up to 1 ms;
- negative, NaN, infinity, or excessively large: error.

The deadline is not restarted for every byte or internal retry.

Return values

Constructors and reads return a value or `(nil, err)`. `set_timeout`, `starttls`, and `close` return exactly `(true, nil)` on success.

Wrong positional types raise a Lua error. Invalid values and runtime failures normally return `(nil, err)`.

API overview

```
local sock, err = babet.socket.connect(host, port, timeout?)
local server, err = babet.socket.listen(host, port, backlog?)
local local_sock, err = babet.socket.connect_unix(path, timeout?)
local local_server, err = babet.socket.listen_unix(path, opts?)

local count, err = sock:send(data)
local data, err = sock:recv(count, timeout?)
local line, err, partial = sock:recv_line(timeout?)
local data, err = sock:recv_all(timeout?, max_bytes?)
local ok, err = sock:set_timeout(seconds)
local addr, err = sock:peer()
local addr, err = sock:sockname()
local ok, err = sock:close()

local client, err = server:accept(timeout?)
```

Argument	Type	Default	Meaning
client `host`	strict non-empty string	required	DNS name or IP address
listen `host`	strict string	required	interface; `""` means all interfaces
`port`	strict Lua integer `0..65535`	required	TCP port
connect `timeout`	finite number `>= 0`	`0`	connection deadline in seconds
`backlog`	strict Lua integer `1..INT_MAX`	`16`	requested listen queue size

Numeric strings such as `"443"` and floats such as `443.0` are rejected for integer fields.

`listen_unix` accepts these strict options:

Option	Default	Contract
`backlog`	`16`	integer `1..INT_MAX`
`permissions`	`0600`	exact final Unix mode `0000..0777`, applied after `bind()`
`unlink_on_close`	`true`	remove the created pathname on close/GC

A listening socket rejects stream operations and `peer`; a connected socket rejects `accept`.

babet.socket.connect(host, port, timeout?)

```
local sock, err = babet.socket.connect("example.net", 9000, 5)
assert(sock, err)
```

Babet calls `getaddrinfo()` and tries returned IPv4/IPv6 addresses in order. All connection attempts share the same deadline. Synchronous DNS resolution itself is outside that deadline and may add delay.

Use a numeric address to avoid DNS:

```
local sock = assert(babet.socket.connect("127.0.0.1", 9000, 2))
```

The constructor timeout only covers connection establishment. The new socket's default I/O timeout is still zero:

```
local sock = assert(babet.socket.connect(host, port, 3))
assert(sock:set_timeout(10))
```

Typical results:

- timeout: `(nil, "timeout")`;
- handled signal: `(nil, "interrupted")` after callback dispatch;
- DNS/TCP failure: `(nil, "socket: …")`;
- empty host or out-of-range port: `(nil, err)`;
- wrong positional type: Lua error.

babet.socket.listen(host, port, backlog?)

Creates a TCP listening socket with `SO_REUSEADDR`.

Loopback only:

```
local server = assert(babet.socket.listen("127.0.0.1", 9000))
```

All interfaces:

```
local server = assert(babet.socket.listen("", 9000))
```

An empty host uses `AI_PASSIVE`. Use `"0.0.0.0"` or `"::"` to request a specific address family. Listening on all interfaces exposes the service to the network; add authentication and firewall rules.

Ask the kernel for a free test port:

```
local server = assert(babet.socket.listen("127.0.0.1", 0))
local addr = assert(server:sockname())
print(addr.host, addr.port)
```

Backlog is a kernel request, not a maximum number of active clients:

```
local server = assert(babet.socket.listen("127.0.0.1", 9000, 128))
```

babet.socket.connect_unix(path, timeout?)

Connects to a pathname-based Unix stream socket:

```
local sock, err = babet.socket.connect_unix("/run/my-service.sock", 2)
assert(sock, err)
```

The pathname must be a strict non-empty string without NUL and must fit in `sockaddr_un.sun_path` (107 useful bytes on Linux). The timeout uses the same global monotonic contract as TCP connect; zero means infinite. It is not kept as the socket's later I/O timeout.

Linux abstract sockets are deliberately not exposed. `peer()` returns `{ path = ... }`; `sockname()` on an unbound Unix client normally returns `{ path = "" }`.

babet.socket.listen_unix(path, opts?)

Creates an `AF_UNIX`/`SOCK_STREAM` listener:

```
local server = assert(babet.socket.listen_unix("/run/my-service.sock", {
    backlog = 32,
    permissions = tonumber("660", 8),
    unlink_on_close = true,
}))
```

Babet refuses **every pre-existing pathname**, including stale sockets, regular files, symlinks, FIFOs, and directories. It never silently removes an existing entry; stale sockets must be explicitly removed by the application after validation.

After `bind()`, the final permissions are applied exactly without following a symlink. The final secure default is `0600`. Between `bind()` and that mode change, the pathname briefly has the kernel-created mode filtered by the process `umask`. Babet does not change that `umask` because it is process-global and the runtime is multithreaded. Sensitive services should therefore place the socket in a private parent directory, for example mode `0700`. The parent directory must already exist.

With `unlink_on_close = true`, close and GC remove the pathname only when it is still the same socket inode created by this listener. A replacement file is never deleted. With `false`, the stale pathname remains for explicit cleanup.

`server:accept(timeout?)`

```
local client, err = server:accept(10)
```

The positional timeout overrides `server:set_timeout()` for this call only. The accepted socket has a default I/O timeout of zero; it does not inherit the server timeout:

```
local client = assert(server:accept(10))
assert(client:set_timeout(30))
```

Accepted descriptors are close-on-exec.

`sock:set_timeout(seconds)`

Sets the default for `send`, `recv`, `recv_line`, `recv_all`, and `accept`:

```
assert(sock:set_timeout(3.5))
assert(sock:set_timeout(0)) -- infinite again
```

It does not apply to a completed constructor and does not control STARTTLS; `starttls` uses `opts.timeout`.

Per-call overrides:

```
assert(sock:set_timeout(10))
local a, err = sock:recv(4096, 0.2) -- 200 ms
local b, err = sock:recv(4096)      -- 10 s
local c, err = sock:recv(4096, 0)   -- infinite
```

`sock:send(data)`

Sends the whole string or fails:

```
local count, err = sock:send("PING\r\n")
assert(count, err)
assert(count == 6)
```

Babet loops over partial OS/OpenSSL writes, so a successful count equals `#data`. Empty and binary strings are accepted. `send` has no positional timeout and uses the socket default.

If a transport error happens after some bytes were written, no partial count is returned. Protocols needing safe retries must use application-level IDs or acknowledgements.

On a TLS socket, a failure after the first `SSL_write` attempt closes the connection, even if OpenSSL has not yet reported any application bytes written. The original call keeps its diagnostic (`timeout`, `interrupted` or TLS error), then subsequent I/O reports a closed socket. A timeout before any TLS write attempt leaves the connection usable.

sock:recv(count, timeout?)

Reads **at most** `count` bytes and may return fewer bytes while the connection remains open:

```
local chunk, err = sock:recv(4096, 5)
```

`count` must be a strict Lua integer from 1 through 16 MiB. This is not “read exactly N”. A helper can accumulate:

```
local function recv_exact(sock, wanted, timeout)
    local chunks, total = {}, 0
    while total < wanted do
        local chunk, err = sock:recv(wanted - total, timeout)
        if not chunk then return nil, err end
        chunks[#chunks + 1] = chunk
        total = total + #chunk
    end
    return table.concat(chunks)
end
```

Here the timeout is per `recv` call. Compute a monotonic application deadline when one budget must cover the whole helper.

EOF before data returns `(nil, "closed")`; expiration returns `(nil, "timeout")`; a handled signal returns `(nil, "interrupted")`.

sock:recv_line(timeout?)

Reads through LF and returns the line without LF. A CR immediately before LF is also stripped:

```
hello\n   -> "hello"
hello\r\n -> "hello"
```

Complete lines already buffered by `recv_all` are handled before polling the network. Each call extracts one line and keeps all following bytes for `recv_line`, `recv` or `recv_all`.

The line limit is 8 MiB before removing an optional CR, excluding LF. It applies to buffered lines too. Oversized lines return a `line too long` error and the protocol should generally be considered desynchronised. If the complete oversized line was already buffered, it is discarded through LF and following data is preserved.

EOF before LF returns three values:

```
local line, err, partial = sock:recv_line(5)
if not line and err == "closed" then
    print(partial) -- always a string, possibly empty
end
```

On timeout/interruption, bytes already consumed are retained internally and are delivered first by the next `recv`, `recv_line`, or `recv_all`.

sock:recv_all(timeout?, max_bytes?)

Reads until remote EOF; EOF is the successful terminator:

```
local body, err = sock:recv_all(10)
```

Use this only when connection close frames the message. A persistent peer that keeps the connection open makes this call wait until timeout.

Memory limit:

- default: 64 MiB;
- strict positive integer;

- maximum: 2 GiB.

```
local body, err = sock:recv_all(10, 4 * 1024 * 1024)
```

Limit, timeout, and interruption failures expose no partial result, but already consumed bytes remain buffered. A later call can resume with a larger limit or new deadline:

```
local body, err = sock:recv_all(2, 1024)
if not body and err:find("max_bytes", 1, true) then
    body = assert(sock:recv_all(2, 4096))
end
```

Mixing `recv`, `recv_line`, and `recv_all`

All three methods share one pending-byte buffer. Switching methods after a timeout does not lose or reorder the TCP stream:

```
-- Peer sent "abc" without LF.
local line, err = sock:recv_line(0.1)
assert(line == nil and err == "timeout")
local prefix = assert(sock:recv(3, 1))
assert(prefix == "abc")
```

Still prefer one framing strategy per protocol phase. Mixing helpers is useful for a line header followed by a fixed-size body, but the application must know the exact boundary.

Pending plaintext must be consumed before `starttls`; Babet rejects the upgrade otherwise.

`sock:peer()` and `sock:sockname()`

Both return numeric address tables:

```
{ host = "127.0.0.1", port = 9000 }
```

- `peer()` returns the remote endpoint of a connected socket;
- `sockname()` returns the local endpoint and also works on listeners.

No reverse DNS is performed. `peer()` is invalid on a listening socket.

`sock:close()` and garbage collection

`close()` is idempotent and returns `(true, nil)`:

```
assert(sock:close())
assert(sock:close())
```

Other methods then report a closed socket. The userdata owns its FD and SSL object, so GC eventually cleans forgotten sockets, but GC timing is not a lifecycle guarantee. Close explicitly.

Created and accepted sockets are close-on-exec, so `babet.exec` children do not silently inherit them.

Signals and interruption

A wait interrupted by a signal registered through `babet.signal` may return `(nil, "interrupted")` after the Lua callback is dispatched. Pending bytes from line/all reads are retained.

```
local stopping = false
babet.signal.handle("TERM", function() stopping = true end)
while not stopping do
    local client, err = server:accept(1)
    if client then
        client:close()
```

```
    elseif err ~= "timeout" and err ~= "interrupted" then
        error(err)
    end
end
```

Complete examples

Line-oriented echo server

```
local S = babet.socket
local server = assert(S.listen("127.0.0.1", 9000, 64))
assert(server:set_timeout(1))

while true do
    local client, err = server:accept()
    if client then
        assert(client:set_timeout(30))
        local line, read_err = client:recv_line()
        if line then
            assert(client:send("echo: " .. line .. "\n"))
        elseif read_err ~= "closed" then
            io.stderr:write(read_err, "\n")
        end
        client:close()
    elseif err ~= "timeout" then
        error(err)
    end
end
```

Length-prefixed binary client

```
local sock = assert(babet.socket.connect("127.0.0.1", 9000, 3))
assert(sock:set_timeout(5))
assert(sock:send(string.pack(">I4", 5) .. "hello"))
local raw_len = assert(recv_exact(sock, 4, 5))
local len = string.unpack(">I4", raw_len)
local payload = assert(recv_exact(sock, len, 5))
print(payload)
sock:close()
```

Response framed by EOF

```
local sock = assert(babet.socket.connect("127.0.0.1", 9001, 3))
assert(sock:send("request\n"))
local response = assert(sock:recv_all(10, 8 * 1024 * 1024))
print(#response)
```

Error contract

Wrong positional types raise:

```
babet.socket.connect("localhost", "9000")
sock:send(42)
sock:recv(4096.0)
sock:set_timeout("5")
```

Invalid ranges and runtime failures return `(nil, err)`: invalid ports, backlog, timeouts, receive sizes, incompatible methods, closed sockets, DNS/TCP errors, and the stable typed states `"timeout"`, `"closed"`, and `"interrupted"`.

Only compare complete strings for stable typed errors. System messages may vary across libc and kernel versions.

Internal C++ exceptions never cross into Lua. Allocation failure is converted to `(nil, "socket: out of memory")`; other internal exceptions receive a fixed literal diagnostic. This boundary covers every public `babet.socket` function and method.

Security and limitations

- Raw TCP has no encryption or peer authentication.
- Set deadlines for peer-controlled input.
- Bind loopback unless remote access is required.
- TCP is a byte stream; `send` boundaries are not preserved.
- DNS is synchronous and outside the controlled connection deadline.
- No half-close, configurable keepalive, UDP, or Lua event loop.
- Unix support is pathname stream sockets only: no abstract namespace, datagrams, `SCM_RIGHTS`, or credentials.
- The requested final mode is applied after `bind()`. Before that mode change, the pathname briefly inherits the process-`umask`-filtered mode. Babet does not modify this process-global state in a multithreaded runtime; put sensitive sockets in a private parent directory, ideally mode `0700`. Socket mode alone does not secure a writable directory.
- STARTTLS is TCP-only and is rejected on Unix streams without closing them.
- One userdata should not be used concurrently; socket userdata cannot cross a WORKERS message boundary.
- `connect`, `listen`, `accept`, and `connect_tls` create the Lua userdata owner before acquiring an FD or OpenSSL object, so a Lua memory-error longjmp cannot abandon an unowned resource. `starttls` uses a separate guard and closes the stream if an exception occurs after the handshake has started.
- OOM exception safety is established by structural review, strict compilation, and static analysis rather than deterministic `bad_alloc` injection; the integration suite covers observable lifecycle and failure paths.

WEBSOCKET — RFC 6455 client for `ws://` and `wss://`

`babet.websocket` is a synchronous WebSocket **client**. It implements the RFC 6455 opening handshake and framing directly in Babet and is designed for protocols that need full-duplex message transport, including WebDriver BiDi.

Babet deliberately keeps the layer generic: it knows WebSocket frames, TLS, timeouts and close semantics, but it does not know Selenium, WebDriver BiDi, JSON-RPC, browser events, or application-level command identifiers.

API overview

```
local ws, err = babet.websocket.connect(url, opts?)

local n, err = ws:send_text(data, timeout?)
local n, err = ws:send_binary(data, timeout?)
local message, err = ws:recv(timeout?)
local ok, err = ws:ping(data?, timeout?)
local ok, err = ws:set_timeout(seconds)
local ok, err = ws:close(code?, reason?, timeout?)
```

`recv()` returns one complete application message:

```
{ type = "text",   data = "..." }
{ type = "binary", data = "..." }
{ type = "close",  code = 1000, reason = "..." }
```

Ping frames are answered automatically with Pong and are not returned as application messages. Pong frames are consumed internally.

`babet.websocket.connect(url, opts?)`

Accepted URL schemes are `ws://` and `wss://`; URI scheme matching is ASCII case-insensitive.

```
local ws = assert(babet.websocket.connect(
    "ws://127.0.0.1:9222/session/abc",
    { timeout = 5 }
))
```

Secure connection:

```
local ws = assert(babet.websocket.connect(
    "wss://example.net/events",
    { timeout = 5 }
))
```

The URL parser accepts DNS names, IPv4, bracketed IPv6, explicit ports, paths, and query strings. Userinfo and fragments are rejected. Spaces and control bytes in the HTTP target must be percent-encoded by the caller.

Options

`opts` is a raw strict table: metamethods are not consulted, option names must be strings, and unknown fields are rejected.

Option	Default	Contract
`timeout`	`0`	finite seconds `>= 0`; one deadline for TCP, TLS and HTTP Upgrade
`verify`	`true`	verify the certificate chain and reference identity for `wss://`
`ca_cert`	none	PEM CA file added to default authorities for this connection
`ca_path`	none	OpenSSL CA directory added to default authorities for this connection
`hostname`	URL host	TLS identity/SNI override
`min_version`	`"1.2"`	`"1.2"` or `"1.3"`
`max_message_bytes`	16 MiB	complete reassembled message ceiling
`max_frame_bytes`	16 MiB	application data-frame ceiling; must not exceed message limit; RFC control frames retain their independent 125-byte ceiling

Both size limits are strict integers in `1..2147483648`.

The constructor timeout is also stored as the WebSocket's default I/O timeout. Use `set_timeout()` to change it after connection.

Option examples

Bound the complete TCP/TLS/Upgrade connection sequence after DNS resolution:

```
local ws = assert(babet.websocket.connect("ws://127.0.0.1:9222/events", {
    timeout = 2,
}))
```

Use a private CA file for a local or internal `wss://` service:

```
local ws = assert(babet.websocket.connect("wss://automation.internal/events", {
    ca_cert = "/etc/myapp/automation-ca.pem",
}))
```

For WSS, `ca_cert` and `ca_path` **add** authorities to those already loaded from OpenSSL defaults, its environment variables, and recognized distribution locations. They do not restrict trust to the supplied file or directory. An empty string adds nothing; a call's custom authorities are not retained by subsequent connections.

This matches `socket TLS` and differs from `HTTP`, where a non-empty `ca_cert` file replaces the default authorities. This is not leaf-certificate or public-key pinning: several certificates issued by one CA can be accepted. With `verify = true`, the server's identity is still checked.

Use a directory of OpenSSL-compatible CA certificates:

```
local ws = assert(babet.websocket.connect("wss://automation.internal/events", {
    ca_path = "/etc/myapp/certs",
}))
```

Override the TLS reference identity and SNI while connecting to a different address. This is useful for controlled local routing or tests; the certificate is still checked against `automation.internal`:

```
local ws = assert(babet.websocket.connect("wss://127.0.0.1:9443/events", {
    hostname = "automation.internal",
    ca_cert = "/etc/myapp/automation-ca.pem",
}))
```

Require TLS 1.3 instead of the default TLS 1.2 minimum:

```
local ws = assert(babet.websocket.connect("wss://example.net/events", {
    min_version = "1.3",
}))
```

Disable certificate verification only for a deliberately controlled test environment:

```
local ws = assert(babet.websocket.connect("wss://127.0.0.1:9443/events", {
    verify = false,
}))
```

`verify = false` disables both chain and reference-identity verification. Do not use it for untrusted networks or production endpoints.

Bound complete messages and individual application data frames independently:

```
local ws = assert(babet.websocket.connect("ws://127.0.0.1:9222/events", {
    max_message_bytes = 8 * 1024 * 1024,
    max_frame_bytes = 1 * 1024 * 1024,
}))
```

A deliberately small `max_frame_bytes` does not reduce the RFC control-frame ceiling: Ping, Pong and Close can still use payloads up to 125 bytes.

Combined secure configuration:

```
local ws = assert(babet.websocket.connect("wss://127.0.0.1:9443/bidi", {
    timeout = 5,
    ca_cert = "/etc/myapp/automation-ca.pem",
    hostname = "automation.internal",
    min_version = "1.3",
    max_message_bytes = 8 * 1024 * 1024,
    max_frame_bytes = 512 * 1024,
}))
```

Opening handshake

The client sends an HTTP/1.1 Upgrade request with WebSocket version 13 and a fresh random `Sec-WebSocket-Key`. The response is accepted only when:

- the status is `101 Switching Protocols`;
- `Upgrade` is exactly `websocket` case-insensitively;
- `Connection` contains the `Upgrade` token;
- `Sec-WebSocket-Accept` exactly matches SHA-1 + Base64 of the RFC 6455 GUID;
- the server did not negotiate an extension Babet did not request;
- the server did not select an unsolicited subprotocol.

The handshake header block is capped at 64 KiB.

Babet 2.22.0 does not yet expose custom handshake headers, cookies, subprotocol negotiation, HTTP proxies, or WebSocket extensions such as `permessage-deflate`.

Sending text

```
local n, err = ws:send_text('{"id":1,"method":"session.status","params":{}}')
assert(n, err)
```

Text must be valid UTF-8. The return value is the number of application bytes, not the number of bytes placed on the wire.

Large messages are automatically fragmented into continuation frames. Every client frame is masked with a fresh unpredictable 32-bit key obtained from OpenSSL's cryptographic RNG, including frames sent over TLS.

Once sending a frame has been attempted, abandoning it because of a timeout, interruption or transport error closes the connection. This also covers fragmented messages, Ping and automatic Pong replies during `recv`. The first call keeps its error; subsequent operations return `closed` and `close()` stays idempotent. Closure happens before any signal callback is dispatched.

Validation errors before transmission (UTF-8, message size, arguments) leave the connection usable. A receive timeout that does not abandon a write keeps the receive state and remains resumable.

Sending binary data

```
local n = assert(ws:send_binary("A\0B\255"))
assert(n == 4)
```

Binary messages are arbitrary Lua strings and may contain NUL or invalid UTF-8.

Receiving messages

```
local message, err = ws:recv(2)
assert(message, err)

if message.type == "text" then
    print(message.data)
elseif message.type == "binary" then
    -- binary-safe string
elseif message.type == "close" then
    print(message.code, message.reason)
end
```

`recv()` reassembles fragmented text/binary messages and permits control frames between fragments. Text is validated as UTF-8 only after the complete message has been reassembled.

Incoming server frames must be unmasked. Reserved RSV bits, reserved opcodes, non-minimal length encodings, fragmented control frames, and malformed close payloads are protocol errors.

The advertised payload length of application data frames is checked against `max_frame_bytes` **before** the payload string is allocated or read. RFC control frames instead retain their fixed 125-byte ceiling, so a deliberately small data-frame cap cannot prevent Ping/Pong or Close handling. Reassembled data is independently bounded by `max_message_bytes`.

Ping and Pong

```
assert(ws:ping("health"))
```

Ping payloads are limited to 125 bytes as required for control frames. Received Ping frames are immediately echoed as Pong by `recv()` before it continues waiting for the next application message.

Closing handshake

```
assert(ws:close(1000, "done", 2))
```

`close()` validates the status code and UTF-8 reason, sends a masked Close frame, then waits for the peer Close using the same absolute deadline. The reason is limited to 123 bytes so code + reason fit in the RFC 6455 125-byte control-frame ceiling.

If the peer sends Close first, `recv()` validates it, automatically echoes it when necessary, closes the transport, and returns a `type = "close"` message.

The GC is intentionally best-effort: forgotten userdata closes the underlying TCP/TLS resources but does not perform a potentially blocking WebSocket close handshake.

Protocol failures

Babet fails closed:

- framing/protocol error -> Close `1002` when possible;
- invalid UTF-8 -> Close `1007`;
- configured frame/message limit exceeded -> Close `1009`.

Transport failures and timeouts remain `(nil, err)` results. A handled POSIX signal interrupts the current wait, dispatches the pending callback, then returns `(nil, "interrupted")`.

For `wss://`, internal TLS writes during handshake, send and receive are protected against `SIGPIPE`. A transport failure does not terminate the program or replace the application's signal handler.

WebDriver BiDi example

WebDriver BiDi uses WebSocket as its transport. The WebDriver session returns a `webSocketUrl`; Babet only needs that URL:

```
local ws = assert(babet.websocket.connect(webSocketUrl, {
    timeout = 10,
    max_message_bytes = 8 * 1024 * 1024,
}))

local command = assert(babet.json.encode({
    id = 1,
    method = "session.status",
    params = {},
}))
assert(ws:send_text(command))

local message = assert(ws:recv())
assert(message.type == "text")
local decoded = assert(babet.json.decode(message.data))
print(decoded.id)
```

A Selenium binding should keep command IDs, pending replies, subscriptions, and event callbacks in Lua. `babet.websocket` remains transport only.

Concurrency and event loops

The API is synchronous. Calling `recv()` without a timeout on the main thread of a GUI/game loop will block that loop. Use a Babet worker, a dedicated threading strategy in the host application, or short bounded polls when appropriate.

Each worker has its own Lua state and can create its own WebSocket connection. A WebSocket userdata itself is not serialized between workers.

Security and limits

- `wss://` verifies certificates by default and requires TLS 1.2 or newer.
- Client masking uses cryptographically strong random keys for every frame.
- No compression extension is negotiated, avoiding decompression amplification and extension-specific state complexity.
- Frame and message limits prevent peer-controlled length fields from causing unbounded allocations.
- Server frames that violate RFC 6455 are rejected rather than normalized.
- DNS resolution is synchronous and is not itself bounded by the socket deadline; use a numeric address where this matters.

`babet.sqlite` — embedded SQL database

`babet.sqlite` embeds SQLite 3.53.1 and exposes a compact but complete Lua API: read/write or read-only connections, per-connection foreign keys, change counters, direct SQL execution, lazy iterators, reusable prepared statements, managed transactions, nested savepoints, coherent online backups, and explicit BLOB and `NULL` binding.

Module contents

- API
  - Opening and closing

  - Connection counters
  - Coherent backups
  - Direct execution
  - Reusable prepared statements
  - Managed transactions
  - Nested savepoints
  - SQL parameters
  - Explicit NULL values
  - Explicit BLOB values
  - SQL text and NUL bytes
- Row reading and type mapping
- Lifetime
- Error contract
- Complete examples
- Functions not exposed

API

Opening and closing

Function	Returns
`babet.sqlite.open(path, opts?)`	`db` userdata | `(nil, err)`
`babet.sqlite.blob(data)`	opaque BLOB wrapper
`babet.sqlite.NULL`	exact lightuserdata sentinel used to bind SQL `NULL`
`db:close()`	`(true, nil)` — idempotent
`db:in_transaction()`	boolean | `(nil, err)`
`db:transaction(callback, mode?)`	`true, ...callback_results` | `(nil, err)`
`db:savepoint(callback)`	`true, ...callback_results` | `(nil, err)`
`db:last_insert_rowid()`	integer | `(nil, err)`
`db:changes()`	integer | `(nil, err)`
`db:total_changes()`	integer | `(nil, err)`
`db:backup(path, opts?)`	`(true, nil)` | `(nil, err)`

By default, `open` opens or creates a read/write database. With `readonly = true`, it opens an existing database without creating it and SQLite rejects writes. The special `":memory:"` path creates a temporary in-memory database. The path must be a string without a NUL byte.

Optional `opts` fields:

Field	Type	Default
`wal`	boolean — requests `PRAGMA journal_mode=WAL`	`false`
`busy_timeout`	integer from `0` to `3600000` ms	`0`
`readonly`	boolean — opens without creation or writes	`false`
`foreign_keys`	boolean — enforces foreign-key constraints	`false`

`wal = true` requests WAL mode, but SQLite may keep another journal mode when WAL is not applicable, notably for `":memory:"`.

`busy_timeout` asks SQLite to retry for the requested duration when the database is locked. Zero reports `SQLITE_BUSY` immediately.

`readonly = true` uses native `SQLITE_OPEN_READONLY`. A missing database returns `(nil, err)` without creating a file. Reads remain available; a write statement returns `(nil, err)`. It combines with `busy_timeout` and `foreign_keys`, but not with `wal = true`: that contradictory combination raises a Lua error before opening the connection. This refusal applies only to a request to **switch** the database to WAL mode. It does not prevent `readonly = true` from opening a database that already uses WAL when `wal = true` is omitted. SQLite must then be able to use the companion `-wal` and `-shm` files; a fully read-only filesystem can fail when the required `-shm` file does not already exist or cannot be used.

`foreign_keys = true` enables referential-integrity enforcement before the connection's first statement. The setting belongs to each connection and does not retroactively validate rows already stored. The `false` default preserves Babet's historical SQLite behaviour.

Each option is useful independently. Set only a lock wait:

```
local db = assert(babet.sqlite.open("state.db", { busy_timeout = 2500 }))
```

Request WAL without changing the default lock wait:

```
local db = assert(babet.sqlite.open("state.db", { wal = true }))
```

Open for reads only:

```
local db = assert(babet.sqlite.open("state.db", { readonly = true }))
```

Enable foreign keys on a writer connection:

```
local db = assert(babet.sqlite.open("state.db", { foreign_keys = true }))
```

Combine WAL, lock waiting, and foreign keys:

```
local db = assert(babet.sqlite.open("state.db", {
    wal = true,
    busy_timeout = 2500,
    foreign_keys = true,
}))
```

The `opts` table is strict: unknown fields, non-string option keys, wrong types, and out-of-range timeouts raise a Lua error. Option lookup reads raw table entries; an `__index` metamethod cannot supply or hide an option. Types and incompatible combinations are checked before a native handle is acquired. A contract error therefore creates neither a file nor a connection.

Combine all compatible reader options:

```
local db = assert(babet.sqlite.open("state.db", {
    readonly = true,
    busy_timeout = 2500,
    foreign_keys = true,
}))
```

`db:in_transaction()` returns `true` whenever the connection is inside a transaction, whether it was opened by `db:transaction()`, by an outermost `db:savepoint()`, or by manual SQL.

Connection counters

The following methods read the connection state directly. They execute no SQL, always return a Lua integer while the connection is open, and are available in workers.

`db:last_insert_rowid()` Returns the ROWID from the latest successful `INSERT` on this connection, or `0` until a ROWID has been inserted. The value remains the latest relevant `INSERT` value after a `SELECT`, `UPDATE`, or `DELETE`, and even after rolling back an `INSERT` that had succeeded before the rollback.

```lua
local db = assert(babet.sqlite.open(":memory:"))
assert(db:exec("CREATE TABLE messages(id INTEGER PRIMARY KEY, body TEXT)"))
assert(db:exec("INSERT INTO messages(body) VALUES(?)", { "Hello" }))
local id = assert(db:last_insert_rowid())
print(id) -- 1
```

A successful statement did not necessarily insert a row. With `INSERT OR IGNORE`, a constraint can be ignored and `last_insert_rowid()` then retains the preceding ROWID. For a one-row insert that may be ignored, check `changes() == 1` before using the ROWID:

```lua
assert(db:exec("INSERT OR IGNORE INTO messages(body) VALUES(?)",
    { "unique message" }))
assert(db:changes() == 1, "message was not inserted")
local id = assert(db:last_insert_rowid())
```

`db:changes()` Returns the number of rows changed by the most recently completed `INSERT`, `UPDATE`, or `DELETE` on this connection. Auxiliary changes from triggers, foreign-key actions, or `REPLACE` conflict resolution are not included in this latest-statement counter.

```lua
assert(db:exec(
    "UPDATE messages SET body = ? WHERE id IN (?, ?)",
    { "archived", 1, 2 }
))
print(assert(db:changes())) -- 0, 1, or 2 depending on existing rows
```

`db:total_changes()` Returns the cumulative number of changed rows since this connection opened. The total includes changes made by triggers and foreign-key actions, but not internal `REPLACE` deletions. Each new connection starts from zero.

```lua
local before = assert(db:total_changes())
assert(db:exec("INSERT INTO messages(body) VALUES ('one'), ('two')"))
local delta = assert(db:total_changes()) - before
assert(delta == 2)
```

Use all three counters together after an insertion:

```lua
assert(db:exec("INSERT INTO messages(body) VALUES(?)", { "new" }))
local id = assert(db:last_insert_rowid())
local statement_rows = assert(db:changes())
local connection_rows = assert(db:total_changes())
print(id, statement_rows, connection_rows)
```

These values belong only to the current `db` handle: another connection, including one in a different worker, has independent counters. After `db:close()`, all three methods return `(nil, "sqlite: connection closed")`.

Coherent backups

```lua
local ok, err = db:backup(path, opts?)
```

`db:backup()` creates a coherent backup of the open connection's main database. It uses the native `sqlite3_backup_init` / `sqlite3_backup_step` / `sqlite3_backup_finish` API; it never copies the SQLite file directly with a filesystem copy primitive.

The method is synchronous and returns:

- `(true, nil)` after the backup has been copied, closed, synchronized, and published;
- `(nil, err)` for operational failures, persistent locks, timeouts, or an invalid destination;
- a Lua error for invalid arity, types, or options.

The destination must be a file inside an existing directory. It must be a non-empty strict Lua string without a NUL byte or a `..` component. `":memory:"` is not accepted as a destination. To persist an in-memory database, call `backup()` on that connection with a real filesystem path.

Options

Field	Type	Default	Contract
`timeout`	finite number from `0` to `86400` seconds	`5.0`	one global monotonic deadline
`pages_per_step`	integer from `1` to `INT_MAX`	`128`	maximum pages copied by each SQLite call
`sleep`	finite number from `0` to `60` seconds	`0.01`	pause between steps or retries
`overwrite`	boolean	`false`	atomically replaces an existing regular destination

The options table is strict and read with raw accesses. Unknown fields, non-string keys, non-finite numbers, and out-of-range values raise a Lua error. An `__index` metamethod cannot supply an option.

The deadline is computed once with a monotonic clock. It is never reset after a successful step, `SQLITE_BUSY`, or `SQLITE_LOCKED`. The source connection's `busy_timeout` is temporarily disabled during the backup and restored afterwards, preventing SQLite's busy handler from silently exceeding the global deadline.

`timeout = 0` performs a genuine non-blocking attempt: Babet calls `sqlite3_backup_step()` once, with no sleep or retry. A small database may therefore complete immediately; a database requiring another step returns a timeout and publishes no partial file.

The deadline is checked **between** `sqlite3_backup_step()` calls. SQLite does not provide a safe interruption point in the middle of one call. A large `pages_per_step` value reduces call count but also reduces deadline check frequency. Keep it moderate when responsiveness matters.

Atomic publication and an existing destination The database is never written directly to `path`. Babet:

1. opens the parent directory without following symbolic links;
2. creates a private neighboring `.babet-sqlite-backup-*` file;
3. runs the native backup into that file;
4. always calls `sqlite3_backup_finish()`;
5. closes SQLite, synchronizes the file, and atomically publishes it.

The published file keeps the private `0600` mode used by the temporary. This mode does not depend on the process `umask` and also replaces the permissions of an older destination when `overwrite = true`. A script may explicitly broaden permissions afterwards with `babet.setMode()` when its sharing model requires it.

On failure or timeout, the temporary file and any `-journal`, `-wal`, or `-shm` companions are removed. An existing destination remains unchanged.

An existing destination is refused by default. With `overwrite = true`, it is atomically replaced only when it is a regular file that is neither the source database, a symbolic link, nor a hard link to the source. Publication is also refused when `path-journal`, `path-wal`, or `path-shm` exists: those files may indicate an open connection or WAL state that must not be replaced underneath its users. Close and checkpoint the destination before retrying.

Symbolic-link directory components are refused. Missing directories are not created. A directory, FIFO, or other special file is never replaced.

If publication succeeded but the final directory synchronization fails, the error says so explicitly. In that exceptional case the published database is complete and readable, but persistence across an immediate system crash is not confirmed.

WAL and concurrent writes A WAL source is supported without special Lua-side handling. SQLite holds the source read lock only while reading the required pages, so other connections may continue to read or write. If another connection changes the source during the copy, SQLite automatically restarts the backup on a later step to produce a coherent database. The final backup represents one complete transactional state; it does not mix pages from two commits.

The published destination is self-contained. Open it normally with `babet.sqlite.open()`; there is no need to copy the source `-wal` file.

Examples Simple backup without replacement:

```
local db = assert(babet.sqlite.open("state.db"))
assert(db:backup("state-backup.db"))
assert(db:close())
```

Bounded WAL backup with small steps:

```
local db = assert(babet.sqlite.open("state.db", {
    wal = true,
    busy_timeout = 2000,
}))

local ok, err = db:backup("state-backup.db", {
    timeout = 10,
    pages_per_step = 64,
    sleep = 0.005,
})
assert(ok, err)
```

Explicitly replace a closed older backup:

```
assert(db:backup("state-backup.db", {
    overwrite = true,
    timeout = 5,
}))
```

Restore into a new file by opening the backup as a source and backing it up again:

```
local snapshot = assert(babet.sqlite.open("state-backup.db", {
    readonly = true,
}))
assert(snapshot:backup("state-restored.db"))
assert(snapshot:close())

local restored = assert(babet.sqlite.open("state-restored.db"))
```

Direct execution

Function	Returns
`db:exec(sql)`	`(true, nil)` | `(nil, err)` — multiple statements allowed
`db:exec(sql, params)`	`(true, nil)` | `(nil, err)` — **one** statement

Function	Returns
`db:query(sql, params?)`	`stmt, nil, nil, stmt | (nil, err)`
`stmt:close()`	`(true, nil)` — idempotent

`db:exec` Without `params`, `exec` accepts multiple semicolon-separated statements. They are prepared and executed in order. Execution stops on the first error; earlier statements remain committed unless they were inside an explicit transaction.

With `params`, exactly one statement is accepted. Trailing whitespace, semicolons, and SQL comments do not count as another statement.

A `SELECT` passed to `exec` is stepped to completion and its rows are ignored. Empty or comment-only SQL is a successful no-op with `exec(sql)` or `exec(sql, {})`.

`db:query` `query` prepares one statement immediately and returns a **single-use callable iterator**. Execution starts on the first iterator call:

```
for row in db:query("SELECT id, name FROM users ORDER BY id") do
    print(row.id, row.name)
end
```

Each row is a table keyed by column name. The iterator returns `nil` when it is exhausted and automatically finalizes its statement. On success `query` returns `stmt, nil, nil, stmt`: the fourth value lets Lua close the cursor immediately on every generic-for exit (`break`, `return`, `goto`, error or coroutine closure). Failures still return exactly `(nil, err)`; `local iter, err = db:query(...)` therefore keeps its usual meaning.

If you retain only the first result, `for row in iter do` has no fourth closing value. Use `local iter <close> = assert(db:query(...))` to close it at scope exit, or call `iter:close()` explicitly. Parenthesizing `(db:query(...))` also keeps only the first result. `__close` and explicit close are idempotent; surviving references remain valid exhausted iterators. `__gc` is the fallback for cursors abandoned without deterministic closure. Reusable `prepared:query` statements retain their existing explicit reset/reuse contract.

`query` also accepts DDL and DML. Such a statement runs on the first call and then ends without yielding a row.

Multiple statements are rejected. Empty or comment-only SQL returns an already exhausted iterator.

Reusable prepared statements

Function	Returns
`db:prepare(sql)`	`prepared | (nil, err)`
`prepared:exec(params?)`	`(true, nil) | (nil, err)`
`prepared:query(params?)`	the same `prepared` userdata `| (nil, err)`
`prepared:reset()`	`(true, nil) | (nil, err)`
`prepared:close()`	`(true, nil)` — idempotent
`prepared:finalize()`	alias of `close()`

`db:prepare()` accepts exactly **one non-empty SQL statement**. Preparation is immediate, so an unknown table or column is reported before the first run.

A statement can then be reused as many times as needed:

```lua
local insert = assert(db:prepare(
    "INSERT INTO files(path, digest) VALUES(?, ?)"
))

for _, file in ipairs(files) do
    assert(insert:exec({ file.path, file.digest }))
end

assert(insert:finalize())
```

`prepared:exec()` resets the statement before binding, steps it to `SQLITE_DONE`, then clears its bindings. A `SELECT` is fully consumed and its rows are ignored.

`prepared:query()` resets and binds the statement, then returns **the same userdata**, whose metatable makes it callable:

```lua
local select_by_age = assert(db:prepare([[
    SELECT id, name FROM users
    WHERE age >= ?
    ORDER BY id
]]))

for row in select_by_age:query({ 18 }) do
    print(row.id, row.name)
end

for row in select_by_age:query({ 65 }) do
    print("senior", row.name)
end
```

Natural exhaustion automatically resets the statement and clears all bindings. After breaking early, either call `prepared:reset()`, start another `prepared:query(...)`, or call `prepared:exec(...)`. The latter two operations also reset the previous partial iteration and discard its remaining rows. A prepared statement can have only one active iteration: reusing the same userdata from a nested loop therefore resets the outer loop.

Calling `prepared()` directly without an active `prepared:query()` simply returns `nil`; it never executes SQL with cleared bindings.

`prepared:reset()` aborts the current run and clears bindings. `close()` and `finalize()` permanently finalize the statement. Calling a closed prepared userdata directly returns `nil`; execution methods return `(nil, "sqlite: statement closed")`.

Managed transactions

```lua
local ok, result = db:transaction(function(tx)
    assert(tx:exec(
        "INSERT INTO accounts(name, balance) VALUES(?, ?)",
        { "alice", 100 }
    ))

    assert(tx:exec(
        "INSERT INTO audit(message) VALUES(?)",
        { "account created" }
    ))

    return "created"
end, "immediate")
```

```
assert(ok, result)
print(result) -- "created"
```

Signature:

```
ok, ...callback_results = db:transaction(callback [, mode])
-- or
nil, err = db:transaction(callback [, mode])
```

The callback receives the database connection as its only argument. `mode` is one of:

- `"deferred"` — default;
- `"immediate"`;
- `"exclusive"`.

When the callback returns normally, Babet commits and returns `true` followed by every callback result. A normal `nil` or `false` result **does not request a rollback**. Only a Lua error triggers automatic rollback.

SQLite operations usually return `(nil, err)`, so use `assert` inside the callback to convert operational failures into Lua errors:

```
local ok, err = db:transaction(function(tx)
    assert(tx:exec("INSERT INTO unique_names VALUES(?)", { "alice" }))
    assert(tx:exec("INSERT INTO unique_names VALUES(?)", { "alice" }))
end)

-- The second INSERT fails, assert raises, and both writes are rolled back.
```

Callback errors are caught: Babet attempts `ROLLBACK` and returns `(nil, "sqlite: transaction callback failed: ...")` rather than rethrowing. A failed `COMMIT` also returns `(nil, err)` after a best-effort rollback. Any values returned by the callback are discarded in that case.

After a successful `BEGIN`, Babet explicitly owns the transaction until `COMMIT` or `ROLLBACK`. The Lua stack space required to call the callback is reserved before `BEGIN`; an internal C++ exception also triggers an immediate rollback attempt. Callback diagnostics are formatted only after that attempt, so an unusual Lua error object cannot leave the connection in an intermediate state.

If the normal rollback fails, the returned message contains both the original failure and the rollback failure, followed by one final emergency rollback attempt. After an error, `db:in_transaction()` can be used to verify explicitly that the connection has returned to autocommit mode before continuing.

Example: failure during commit A deferred constraint may be checked only by `COMMIT`:

```
local db = assert(babet.sqlite.open(
    ":memory:", { foreign_keys = true }))
assert(db:exec([[
    CREATE TABLE parent(id INTEGER PRIMARY KEY);
    CREATE TABLE child(
        id INTEGER PRIMARY KEY,
        parent_id INTEGER NOT NULL,
        FOREIGN KEY(parent_id) REFERENCES parent(id)
            DEFERRABLE INITIALLY DEFERRED
    )
]]))

local ok, err = db:transaction(function(tx)
    assert(tx:exec(
        "INSERT INTO child(id, parent_id) VALUES(?, ?)",
        { 1, 999 }))
    return "this result will not be forwarded"
end)
```

```
assert(ok == nil)
assert(type(err) == "string")
assert(db:in_transaction() == false)
```

The `INSERT` is rolled back and the connection remains reusable when rollback succeeds.

Example: reusing a prepared statement after rollback

```
local insert = assert(db:prepare(
    "INSERT INTO log(id, value) VALUES(?, ?)"))

local ok, err = db:transaction(function()
    assert(insert:exec({ 1, "rolled back" }))
    error("intentional failure")
end)

assert(ok == nil and type(err) == "string")
assert(insert:exec({ 2, "kept" }))
insert:finalize()
```

Rollback resets SQLite's transactional state and the prepared statement can be reused normally afterwards.

Intentional limitations:

- nested `db:transaction()` helpers are rejected;
- the helper is rejected while a manual SQL transaction is already active;
- `db:close()` is rejected during the callback;
- the callback must not issue its own `BEGIN`, `COMMIT`, or `ROLLBACK`.

Use `db:savepoint()` for managed nested scopes. Raw SQL savepoints remain available when explicit SQL identifiers or manual control are required.

Nested savepoints

Signature:

```
true, ...callback_results = db:savepoint(callback)
-- or
nil, err = db:savepoint(callback)
```

The callback receives the same open connection as its only argument. Babet generates the SQL savepoint identifier internally; the API accepts no name and therefore cannot inject caller-controlled text into `SAVEPOINT`, `ROLLBACK TO`, or `RELEASE`.

On a normal callback return, Babet executes `RELEASE` and returns `true` followed by every callback value. Explicit `nil` and `false` values are normal results and do not request rollback. On a Lua error, Babet executes `ROLLBACK TO` and then `RELEASE`, catches the error, and returns `(nil, "sqlite: savepoint callback failed: ...")`.

Example: standalone savepoint Outside a transaction, the outermost savepoint starts a transaction. Releasing it commits the isolated work and restores autocommit:

```
local ok, id = db:savepoint(function(tx)
    assert(tx:exec(
        "INSERT INTO jobs(name) VALUES(?)",
        { "index" }))
    return assert(tx:last_insert_rowid())
end)

assert(ok, id)
assert(db:in_transaction() == false)
```

Example: rollback on callback error Use `assert` on SQLite operations whose failure must cancel the scope:

```
local ok, err = db:savepoint(function(tx)
    assert(tx:exec(
        "INSERT INTO unique_names(name) VALUES(?)",
        { "alice" }))
    assert(tx:exec(
        "INSERT INTO unique_names(name) VALUES(?)",
        { "alice" }))
end)

assert(ok == nil)
assert(type(err) == "string")
-- Neither INSERT remains, and the connection is reusable.
```

Example: recover from a failed inner scope Savepoint helpers may be nested. A failed inner scope rolls back only to its own marker; the outer callback can inspect the error and continue:

```
local ok, inner_err = db:savepoint(function(tx)
    assert(tx:exec("INSERT INTO audit(message) VALUES('before')"))

    local inner_ok, err = tx:savepoint(function(inner)
        assert(inner:exec("INSERT INTO audit(message) VALUES('optional')"))
        error("discard optional work")
    end)
    assert(inner_ok == nil)

    assert(tx:exec("INSERT INTO audit(message) VALUES('after')"))
    return err
end)

assert(ok, inner_err)
-- "before" and "after" remain; "optional" was rolled back.
```

A later error in the outer callback also rolls back any inner savepoint that was already released. `RELEASE` merges inner work into the surrounding scope; it never makes that work independent of the outer transaction.

Example: inside a managed transaction `db:savepoint()` is allowed inside `db:transaction()` even though a second transaction helper is not:

```
local ok, err = db:transaction(function(tx)
    assert(tx:exec("INSERT INTO audit(message) VALUES('required')"))

    local optional_ok = tx:savepoint(function(inner)
        assert(inner:exec("INSERT INTO audit(message) VALUES('optional')"))
    end)
    if not optional_ok then
        assert(tx:exec("INSERT INTO audit(message) VALUES('optional failed')"))
    end

    assert(tx:in_transaction() == true)
end, "immediate")

assert(ok, err)
```

Releasing the inner savepoint does **not** commit the managed transaction. If the outer callback subsequently fails, `db:transaction()` rolls back both the required write and every released inner write.

Example: inside a manual transaction

```
assert(db:exec("BEGIN"))

local ok, err = db:savepoint(function(tx)
    assert(tx:exec("UPDATE accounts SET balance = balance - 10 WHERE id = 1"))
end)
assert(ok, err)
assert(db:in_transaction() == true)

assert(db:exec("ROLLBACK")) -- also undoes the released savepoint work
```

Example: failure while releasing the outermost savepoint A deferred constraint may be checked only when the outermost `RELEASE` tries to commit. Callback results are discarded, Babet rolls back to the savepoint, removes it, and returns the release error:

```
local db = assert(babet.sqlite.open(
    ":memory:", { foreign_keys = true }))
assert(db:exec([[
    CREATE TABLE parent(id INTEGER PRIMARY KEY);
    CREATE TABLE child(
        id INTEGER PRIMARY KEY,
        parent_id INTEGER REFERENCES parent(id)
            DEFERRABLE INITIALLY DEFERRED
    )
]]))

local ok, err, leaked = db:savepoint(function(tx)
    assert(tx:exec("INSERT INTO child VALUES(1, 999)"))
    return "must not be forwarded"
end)

assert(ok == nil and type(err) == "string")
assert(leaked == nil)
assert(db:in_transaction() == false)
```

Inside an existing transaction, releasing an inner savepoint does not perform the outer commit, so the deferred error remains the responsibility of that transaction's later `COMMIT`.

The helper tracks nesting depth per connection and is available inside workers. `db:close()` is rejected while any savepoint callback is active. The Lua stack is reserved before opening the savepoint, and an allocation-free C++ guard attempts emergency `ROLLBACK TO` plus `RELEASE` if an internal exception occurs.

Do not mix manual transaction or savepoint-control statements inside the callback. In particular, a full `ROLLBACK` or `COMMIT` destroys the managed savepoint. Babet detects the restored autocommit state, discards callback results, skips redundant cleanup statements, and returns one stable error without exposing the generated identifier. A `ROLLBACK` leaves the connection reusable with the callback writes undone; a `COMMIT` may already have persisted them before Babet can diagnose the contract violation. Use nested `db:savepoint()` calls for recoverable inner scopes.

SQL parameters

The following forms are supported by direct and prepared execution:

Placeholder	Lua value
`?`	`params[1]`, `params[2]`, and so on in `?` order
`:name`, `@name`, `$name`	`params.name` with the prefix removed

Positional and named parameters may be mixed:

```
assert(db:exec(
    "INSERT INTO events VALUES (?, :kind, ?)",
    { 42, 1700000000, kind = "start" }
))
```

The table must match all placeholders exactly. Missing or extra values, sparse numeric indices, non-integer numeric indices, unsupported key types, and unsupported value types raise a Lua error. Explicit `nil` as the params argument is the same as omitting it; it is not a bind value.

Named values are read from raw table entries. A metatable `__index` result does not count as a supplied value, and an `__index` error is never executed during binding. Numbered `?NNN` placeholders are explicitly rejected; use anonymous `?` placeholders in their natural order instead.

Accepted bind values:

Lua type	SQLite value
boolean	INTEGER `0` or `1`
integer	INTEGER, including the full signed 64-bit range
finite non-integer number	REAL
string	TEXT, including embedded NUL bytes
`babet.sqlite.blob(data)`	binary-safe BLOB
`babet.sqlite.NULL`	SQL `NULL`

NaN and positive or negative infinity are rejected instead of being handed to SQLite with platform-dependent results.

Explicit NULL values

Lua cannot store `nil` inside a parameter table: assigning `nil` removes the key. Use the exact singleton `babet.sqlite.NULL` when a placeholder must be bound to SQL `NULL`.

Named optional value:

```lua
local NULL = babet.sqlite.NULL

assert(db:exec([[
    INSERT INTO users(name, nickname)
    VALUES(:name, :nickname)
]], {
    name = "Ada",
    nickname = NULL,
}))
```

Positional `NULL` in the middle of a parameter list:

```lua
assert(db:exec(
    "INSERT INTO events(id, payload, created_at) VALUES(?, ?, ?)",
    { 17, babet.sqlite.NULL, 1700000000 }
))
```

The sentinel works the same way with reusable statements and may alternate between ordinary values and `NULL` on successive runs:

```lua
local update = assert(db:prepare(
    "UPDATE users SET nickname = ? WHERE name = ?"
))

assert(update:exec({ "Countess", "Ada" }))
assert(update:exec({ babet.sqlite.NULL, "Ada" }))
assert(update:finalize())
```

Binding and reading are deliberately asymmetric:

Lua bind value	SQLite storage	Lua row value
`true`	INTEGER `1`	integer `1`
`false`	INTEGER `0`	integer `0`
`babet.sqlite.NULL`	`NULL`	`nil` / absent key

SQLite has no boolean storage class, so integers read back as integers; Babet does not infer booleans from `0` or `1`. A `NULL` result produces no key in the row table, which means both `row.column == nil` and absence from `pairs(row)`.

`babet.sqlite.NULL` is accepted only as a SQLite bind value. Other lightuserdata values are rejected. JSON encoding and values transferred to workers or channels also reject this sentinel with an explicit diagnostic, preventing an opaque pointer-shaped value from silently crossing subsystem boundaries.

Explicit BLOB values

A normal Lua string is **always** bound with `sqlite3_bind_text`, even for a column with BLOB affinity. `babet.sqlite.blob(data)` explicitly marks the same bytes as a BLOB:

```
local insert = assert(db:prepare(
    "INSERT INTO assets(name, payload) VALUES(?, ?)"
))

assert(insert:exec({
    "icon",
    babet.sqlite.blob("\x89PNG\r\n\x1a\n...")
}))
```

The constructor accepts exactly one Lua string and returns an opaque userdata. Embedded NUL bytes are preserved. `babet.sqlite.blob("")` creates a real zero-byte BLOB rather than `NULL`.

The wrapper is only used for binding. Reading a BLOB column still returns a binary-safe Lua string.

SQL text and NUL bytes

SQL text must be a string without a NUL byte. SQLite would treat a NUL as the end of the SQL string and could execute only a prefix, so Babet rejects it before preparation. SQL text is also limited to `INT_MAX` bytes to match `sqlite3_prepare_v2`.

This restriction applies only to **SQL text**. Bound strings and BLOB values are binary-safe.

Row reading and type mapping

SQLite type	Lua type
INTEGER	integer
REAL	number (float)
TEXT	string
BLOB	binary-safe string, including an empty BLOB
NULL	absent table key (`row.col == nil` and absent from `pairs`)

When multiple columns use the same name, the last column wins. Use aliases to preserve every value.

SQLite `length()` counts characters for TEXT and may stop at an embedded NUL. For diagnostic byte counts, use for example `length(CAST(col AS BLOB))`.

Lifetime

`db:close()` is idempotent. After closing, new connection operations such as `exec`, `query`, `prepare`, `transaction`, `savepoint`, and `in_transaction` return `(nil, "sqlite: connection closed")`.

An iterator or prepared statement created **before** `db:close()` remains usable. Babet uses `sqlite3_close_v2`, so SQLite keeps a zombie connection until the last active statement is finalized.

```lua
local stmt = assert(db:prepare("SELECT id FROM users ORDER BY id"))
assert(db:close())

for row in stmt:query() do
    print(row.id)
end

stmt:finalize()
```

Temporary `db:query` iterators and prepared statements both have a `__gc` fallback. Direct `for row in db:query(...)` loops and `<close>` locals finalize temporary cursors deterministically. For separately retained iterators and prepared statements, use explicit close/finalize to release resources promptly.

Statements deliberately hold no Lua reference and no raw `Db*` pointer to their parent userdata. SQLite itself keeps the native zombie connection alive, which avoids a dangling C++ owner pointer and lets the Lua `db` userdata be collected independently.

Error contract

Errors fall into two groups:

- operational failures (open, preparation, step, transaction, closed handle) generally return `(nil, "sqlite: <description>")`;
- wrong types, invalid arity, and invalid parameter tables raise a Lua error.

All public functions validate their exact arity. Parameter-contract errors also include numbered `?NNN`, non-finite REAL values, foreign lightuserdata, and option/parameter tables with unsupported keys.

`opts.readonly`, `opts.foreign_keys`, and `opts.wal` require strict Lua booleans. Combining `readonly = true` with `wal = true` raises a contract error. Opening a missing database read-only, writing through a read-only connection, or violating a foreign key are operational failures that return `(nil, err)`.

Errors raised while calling a query iterator (`db:query` or `prepared:query`) are Lua errors because the iterator protocol cannot also return a separate `(nil, err)` result.

When no params table is provided, placeholders are detected before execution. Babet never lets SQLite silently bind them as `NULL`.

`db:transaction()` and `db:savepoint()` are controlled exceptions: they catch callback Lua errors, roll back their owned scope, and convert the error to `(nil, err)`.

Complete examples

Prepared inserts, optional NULL, and BLOB values

```lua
local db = assert(babet.sqlite.open("state.db", {
    wal = true,
    busy_timeout = 2000,
}))

assert(db:exec([[
    CREATE TABLE IF NOT EXISTS files (
        path TEXT PRIMARY KEY,
        digest BLOB NOT NULL,
        media_type TEXT
    )
]]))
```

```
local insert = assert(db:prepare([[
    INSERT INTO files(path, digest, media_type)
    VALUES(?, ?, ?)
    ON CONFLICT(path) DO UPDATE SET
        digest = excluded.digest,
        media_type = excluded.media_type
]]))

for _, file in ipairs(files) do
    assert(insert:exec({
        file.path,
        babet.sqlite.blob(file.digest),
        file.media_type or babet.sqlite.NULL,
    }))
end

insert:finalize()
```

Reusable query and managed transaction

```
local by_prefix = assert(db:prepare([[
    SELECT path, digest
    FROM files
    WHERE path LIKE ?
    ORDER BY path
]]))

local ok, count = db:transaction(function(tx)
    local n = 0
    for row in by_prefix:query({ "images/%" }) do
        assert(tx:exec(
            "INSERT INTO audit(path) VALUES(?)",
            { row.path }
        ))
        n = n + 1
    end
    return n
end, "immediate")

assert(ok, count)
print(count, "rows audited")

by_prefix:finalize()
assert(db:close())
```

Functions not exposed

The following are intentionally not implemented:

- URI mode and other advanced `sqlite3_open_v2` flags;
- progress-handler and interrupt APIs;
- the `sqlite3_blob_open` streaming BLOB API;

Raw SQL savepoints and `VACUUM INTO` remain available. FTS5 and R-Tree are not enabled in the current embedded build.

babet strings — splitting strings

Babet exposes one string-manipulation function: `babet.split`. It is registered directly on the `babet` table, without a sub-namespace, to preserve the historical API.

Module contents

- API
- Separator mode
  - Limiting the number of cuts
- Byte mode
- Binary strings and NUL bytes
- Empty subject
- Error contract
- Limits and design choices

API

```
local parts = babet.split(s [, sep [, max_splits]])
```

The function requires **one to three arguments** and returns exactly one value: a dense table indexed from `1` to `n`.

Argument	Type and behavior
`s`	Required Lua string. Numbers are not converted implicitly. Embedded NUL bytes are preserved.
`sep`	Optional Lua string. Omitted or empty: byte mode. Otherwise it must contain exactly **one byte**, used literally.
`max_splits`	Optional Lua integer greater than or equal to `-1`. `-1` means unlimited and `0` prevents every cut. Used only when `sep` contains one byte.

`nil` is not a placeholder for an optional argument: `babet.split("abc", nil)` raises an error. To supply `max_splits` in byte mode, pass `""` explicitly as the second argument.

Separator mode

When `sep` contains exactly one byte, every occurrence of that byte produces a cut. The separator is **literal**: it is neither a Lua pattern nor a regular expression.

```
babet.split("a,b,c", ",")
-- { "a", "b", "c" }

babet.split("a.b.c", ".")
-- { "a", "b", "c" }  -- the dot is not a wildcard
```

Empty fields are preserved at the beginning, between adjacent separators, and at the end:

```
babet.split(",a", ",")
-- { "", "a" }

babet.split("a,,b", ",")
-- { "a", "", "b" }
```

```lua
babet.split("a,", ",")
-- { "a", "" }
```

If the separator does not occur, the result contains the whole subject:

```lua
babet.split("hello", ",")
-- { "hello" }
```

Limiting the number of cuts

`max_splits` counts **cuts**, not produced elements. The uncut remainder is stored verbatim in the last element.

```lua
babet.split("a,b,c,d", ",", 2)
-- { "a", "b", "c,d" }

babet.split("a,b,c", ",", 0)
-- { "a,b,c" }

babet.split("a,b,c", ",", -1)
-- { "a", "b", "c" }
```

Any value greater than the number of available separators has the same effect as `-1`.

Byte mode

If `sep` is omitted or is `""`, the subject is split byte by byte. There is therefore **no default space separator**.

```lua
babet.split("ab c")
-- { "a", "b", " ", "c" }

babet.split("abc", "")
-- { "a", "b", "c" }
```

When supplied in this mode, `max_splits` is still validated but is not used:

```lua
babet.split("abc", "", 0)
-- { "a", "b", "c" }
```

This mode works on **bytes**, not Unicode code points. A multi-byte UTF-8 character is therefore returned as several one-byte strings:

```lua
local bytes = babet.split("é")
-- #bytes == 2 in UTF-8
-- table.concat(bytes) == "é"
```

Likewise, a multi-byte UTF-8 separator such as `"é"` is rejected because `sep` must contain zero or one byte. Use Lua code (`string.find`, `string.gmatch`, and so on) or a dedicated function to split on a multi-byte string.

Binary strings and NUL bytes

The subject, separator, and output elements are handled with their exact Lua length. A NUL byte never truncates the data:

```lua
local parts = babet.split("a\0b,c", ",")
-- { "a\0b", "c" }

local parts2 = babet.split("a\0b", "\0")
-- { "a", "b" }
```

UTF-8 content also stays intact when split with an ASCII byte:

```
babet.split("été:ok", ":")
-- { "été", "ok" }
```

Empty subject

Two different historical behaviors are preserved:

```
babet.split("", ",")
-- { "" }

babet.split("")
-- {}

babet.split("", "")
-- {}
```

In separator mode, finding zero separators means returning one element with the whole subject, even when it is empty. In byte mode, a zero-byte subject produces no elements.

Error contract

Every failure is a raised Lua error; `split` has no `(nil, err)` return path.

The function raises in particular when:

- fewer than one or more than three arguments are supplied;
- `s` is not an actual Lua string;
- `sep` is present but is not an actual Lua string;
- `sep` contains more than one byte;
- `max_splits` is not a Lua integer;
- `max_splits` is less than `-1`.

Implicit number-to-string conversions are not accepted:

```
babet.split(123, ",")          -- error
babet.split("123", 2)          -- error
babet.split("a,b", ",", "1") -- error
```

Limits and design choices

- This is not a CSV parser: quoting, escaping, and records are not handled.
- Multi-byte separators and Lua patterns are not supported.
- Empty fields are intentionally preserved.
- No `join` counterpart is exposed because Lua's standard `table.concat` already provides it.

SYS - process, host, environment, executables, version, and Lua memory

The functions in this chapter are exposed directly on the global `babet` table. There is no `babet.sys` subtable: this flat namespace is historical and is part of the stable API.

SYS covers:

- the identity of the Babet process;
- the hostname and kernel information;
- reading and changing environment variables;
- locating an executable program;

- runtime version constants;
- memory used by the current Lua state.

The current working directory is also process-global state, but `currentDir` and `chdir` are documented under FS - Current working directory, alongside path operations.

Module contents

- General conventions
- API overview
- Version constants
- Identify the process and host
  - `pid`
  - `hostname`
  - `uname`
- Locate an executable
  - `PATH` lookup
  - Direct path
  - Missing or non-executable program
- Environment variables
  - `env`
  - `setenv`
  - Empty value versus missing variable
  - Interaction with workers
- Lua VM memory
  - `getMemoryUsage`
  - `getDetailedMemoryUsage`
  - What is and is not measured
- Error contract
- Design decisions and limits

General conventions

Flat functions

All functions are called directly from `babet`:

```
local pid = babet.pid()
local home = babet.env("HOME")
```

The following forms do not exist:

```
-- Incorrect: there is no babet.sys subtable
-- babet.sys.pid()
-- babet.sys.env("HOME")
```

Strict strings

`env`, `setenv`, and `which` require actual Lua strings. Numbers are not converted automatically.

```
local ok, err = pcall(function()
    return babet.env(42)
end)

-- ok == false; err describes an invalid call
```

An embedded NUL byte is always rejected. This prevents a Lua string from being silently truncated when passed to a C API.

Process-global state

The environment and current working directory belong to the **whole process**, not to a single script or worker.

- `setenv` changes the environment seen by the runtime and future programs launched through `babet.exec`;
- `chdir` changes how all relative paths are resolved;
- both mutations are permanently forbidden after the first `babet.workers.spawn()`.

This restriction prevents data races between threads.

API overview

Function or constant	Result
`babet.VERSION`	runtime `major.minor.patch` string
`babet.VERSION_MAJOR`	integer major component
`babet.VERSION_MINOR`	integer minor component
`babet.VERSION_PATCH`	integer patch component
`babet.pid()`	process PID as an integer
`babet.hostname()`	hostname, or `(nil, err)`
`babet.uname()`	system table, or `(nil, err)`
`babet.which(name)`	executable path, or `(nil, err)`
`babet.env(name)`	variable value, or `nil` when missing
`babet.setenv(name, value)`	`(true, nil)` or `(nil, err)`
`babet.getMemoryUsage()`	current Lua-state memory in bytes
`babet.getDetailedMemoryUsage()`	two currently identical integers

Version constants

The four constants describe the Babet binary running the script.

```lua
print(babet.VERSION)       -- for example "2.22.0"
print(babet.VERSION_MAJOR) -- for example 2
print(babet.VERSION_MINOR) -- for example 15
print(babet.VERSION_PATCH) -- for example 0
```

`babet.VERSION` matches the output of:

```
babet --version
```

The integer components let code compare versions without parsing a string.

```lua
local function version_at_least(major, minor, patch)
    local current = {
        babet.VERSION_MAJOR,
        babet.VERSION_MINOR,
        babet.VERSION_PATCH,
    }
    local wanted = { major, minor, patch }

    for i = 1, 3 do
        if current[i] ~= wanted[i] then
            return current[i] > wanted[i]
        end
    end
    return true
end
```

```lua
assert(version_at_least(2, 9, 0), "Babet >= 2.9.0 is required")
```

The following consistency is covered by tests:

```lua
assert(babet.VERSION == string.format(
    "%d.%d.%d",
    babet.VERSION_MAJOR,
    babet.VERSION_MINOR,
    babet.VERSION_PATCH
))
```

Identify the process and host

`babet.pid()`

Returns the Babet process ID as a strictly positive integer. The call cannot fail on a supported POSIX system.

```lua
local pid = babet.pid()
print("Babet PID:", pid)
```

Workers are threads in the same process. They therefore see the same PID as the main thread; `pid()` cannot identify an individual worker.

```lua
local main_pid = babet.pid()
local worker = assert(babet.workers.spawn([[
    return babet.pid()
]]))

local joined, worker_pid = worker:join()
assert(joined and worker_pid == main_pid)
```

`babet.hostname()`

Returns the hostname configured for the machine.

```lua
local host, err = babet.hostname()
assert(host, err)
print("Host:", host)
```

Signature:

```lua
local host, err = babet.hostname()
```

Results:

- success: `host` is a non-empty string and `err` is `nil`;
- system failure: `(nil, "hostname: ...")`.

The hostname is not necessarily a fully qualified DNS name. It may be a short locally configured name.

`babet.uname()`

Returns the five main POSIX `uname(2)` fields in a table.

```lua
local info, err = babet.uname()
assert(info, err)

print("System :", info.sysname)
print("Node   :", info.nodename)
print("Kernel :", info.release)
```

```lua
print("Version:", info.version)
print("Machine:", info.machine)
```

Returned table:

```lua
{
    sysname  = "Linux",
    nodename = "workstation",
    release  = "6.12.0-arch1-1",
    version  = "#1 SMP PREEMPT_DYNAMIC ...",
    machine  = "x86_64",
}
```

Field	Meaning
`sysname`	operating system or kernel name
`nodename`	network node name, often close to the hostname
`release`	kernel release string
`version`	detailed kernel build string
`machine`	hardware architecture reported by the kernel

If `uname(2)` fails, the function returns `(nil, "uname: ...")`. This is very rare, but it is part of the real contract.

Locate an executable

Signature:

```lua
local path, err = babet.which(name)
```

`name` must be a string without an embedded NUL byte.

`PATH` lookup

When `name` contains no `/`, Babet scans the `PATH` environment variable from left to right and returns the first candidate that is:

- a regular file;
- executable by the current process.

```lua
local shell, err = babet.which("sh")
assert(shell, err)
print(shell) -- usually /usr/bin/sh or /bin/sh
```

The returned path is normalized to an absolute path whenever the system allows it.

To perform a simple availability check:

```lua
if babet.which("ffmpeg") then
    print("ffmpeg is available")
end
```

`PATH` is read at call time. A change made with `setenv` before workers and GTK loading is therefore visible to later calls.

```lua
local old_path = babet.env("PATH")
assert(babet.setenv("PATH", "/opt/my-app/bin:/usr/bin"))
local tool = babet.which("my-tool")
```

An empty `PATH` component, as in `:/usr/bin` or `/bin::/usr/bin`, represents the current working directory, following the historical Unix convention.

Direct path

When the argument contains `/`, `PATH` is not consulted. The path is tested directly.

```
local sh, err = babet.which("/bin/sh")
assert(sh, err)
```

A relative path containing `/` is accepted too:

```
local tool, err = babet.which("./bin/my-tool")
assert(tool, err)
```

A valid symlink to an executable regular file is accepted because inspection follows the target.

Missing or non-executable program

When no valid executable is found:

```
local path, err = babet.which("missing-program")
-- path == nil
-- err  == "which: 'missing-program' not found in PATH"
```

A file that exists but is not executable is treated as not found:

```
local path, err = babet.which("./script-without-x-bit")
assert(path == nil)
print(err)
```

The following are rejected too:

- directories;
- broken symlinks;
- non-regular files;
- files the current process cannot execute.

Environment variables

`babet.env(name)`

Reads a process environment variable.

```
local home = babet.env("HOME")
if home then
    print("HOME:", home)
end
```

Signature:

```
local value = babet.env(name)
```

Results:

- defined variable: its value as a string;
- missing variable: `nil` only, with no error message.

This enables the common idiom:

```
local port = babet.env("APP_PORT") or "8080"
```

`name` must be a string. The following forms raise a Lua error:

```
-- babet.env()       -- missing argument
-- babet.env(42)     -- no automatic conversion
-- babet.env({})     -- wrong type
-- babet.env("A\0B") -- embedded NUL
```

babet.setenv(name, value)

Creates an environment variable or replaces its existing value.

```
local ok, err = babet.setenv("APP_MODE", "production")
assert(ok, err)
assert(babet.env("APP_MODE") == "production")
```

A later call overwrites the value:

```
assert(babet.setenv("APP_MODE", "test"))
assert(babet.env("APP_MODE") == "test")
```

`name` and `value` must both be actual Lua strings.

Caller errors, which raise a Lua error:

```
-- babet.setenv()
-- babet.setenv("APP_MODE")
-- babet.setenv(42, "test")
-- babet.setenv("APP_MODE", 42)
```

Runtime validation errors return `(nil, err)`:

```
local ok, err = babet.setenv("BAD=NAME", "x")
-- ok == nil; err reports an invalid name
```

An empty name, a name containing `=`, or an embedded NUL byte is rejected. A NUL byte in the value is rejected as well.

Empty value versus missing variable

An empty value is still defined:

```
assert(babet.setenv("APP_OPTION", ""))
assert(babet.env("APP_OPTION") == "")
```

It is therefore different from a missing variable:

```
local value = babet.env("MISSING_VARIABLE")
assert(value == nil)
```

Babet does not currently expose `unsetenv`. A variable cannot be removed with `setenv(name, nil)`: `nil` is a wrong type and the call raises a Lua error.

Interaction with workers

`setenv` is allowed only before the first `workers.spawn()` and before the first GTK loading attempt (`gui.available()` or `gui.init()`).

```
assert(babet.setenv("APP_MODE", "production"))

local worker = assert(babet.workers.spawn([[
    return babet.env("APP_MODE")
]]))

local joined, value = worker:join()
assert(joined and value == "production")
```

After the first spawn, even when the worker has already finished:

```
local ok, err = babet.setenv("APP_MODE", "test")
-- ok == nil
-- err explains that the shared environment can no longer be changed
```

The restriction lasts for the whole process, including when the first spawn failed. Prepare `PATH`, application variables, and the current directory before starting workers or loading GTK. A failed GTK load or initialization also retains the freeze. Reading values through `babet.env` remains possible; see the GUI contract.

Lua VM memory

Both functions first perform a full Lua garbage-collection cycle. The result therefore reflects memory still in use after collection, and the call may cause a pause proportional to the size of the Lua state.

`babet.getMemoryUsage()`

Returns the number of bytes currently accounted for by the memory manager of the current Lua state.

```lua
local bytes = babet.getMemoryUsage()
print(string.format("Lua uses %d bytes", bytes))
```

The result is a positive integer.

Example before/after comparison:

```lua
local before = babet.getMemoryUsage()

local values = {}
for i = 1, 10000 do
    values[i] = string.rep("x", 100)
end

local after = babet.getMemoryUsage()
print("Lua change:", after - before, "bytes")
```

This is not a precise profiling tool: allocator behavior, string interning, and GC optimizations may change the result.

`babet.getDetailedMemoryUsage()`

Returns two integers:

```lua
local used, total = babet.getDetailedMemoryUsage()
```

In the current version, both values are **strictly identical**:

```lua
assert(used == total)
```

The second return value is kept only for API stability. Lua does not expose a separate reliable measurement of fragmentation or reserved-but-unused memory.

New code that needs one measurement should prefer:

```lua
local bytes = babet.getMemoryUsage()
```

What is and is not measured

The measurement covers memory managed by **the Lua state performing the call**.

It does not represent:

- total process RSS;
- the native stack;
- C++ runtime allocations;
- OpenSSL, SQLite, HTTP, or socket buffers;

- executable code and shared libraries;
- Lua states belonging to other workers.

Inside a worker, the function measures only that worker's isolated Lua state. To measure the whole process, use a system facility such as `/proc`, `ps`, `smem`, Valgrind, or an appropriate profiler.

Error contract

Situation	Behavior
missing argument or wrong type	Lua error, recoverable with `pcall`
NUL in `env`, `setenv`, or `which`	Lua error for `env`, `(nil, err)` for `setenv` and `which`
missing variable in `env`	`nil` only
executable not found	`(nil, err)`
`hostname` or `uname` system error	`(nil, err)`
invalid `setenv` or mutation after a worker	`(nil, err)`
`pid`	always an integer
memory functions	always one or two integers after a full GC

An unexpected C++ exception in `which`, `env`, `setenv`, `hostname`, `uname`, or `pid` is stopped by the shared boundary and becomes `(nil, "sys: out of memory")`, `(nil, "sys: internal failure")`, or `(nil, "sys: unknown internal failure")`.

This is an internal safety boundary, not a new expected runtime state. The ordinary `env` missing-variable result remains one `nil`, and `pid` remains one integer.

Handling a runtime failure:

```
local tool, err = babet.which("optional-tool")
if not tool then
    print("Feature disabled:", err)
end
```

Handling an invalid call:

```
local ok, err = pcall(function()
    return babet.env(42)
end)

if not ok then
    print("Programming error:", err)
end
```

Design decisions and limits

- Historical functions remain directly under `babet` to avoid breaking existing scripts.
- `which` returns the first executable found, not every candidate.
- `which` observes the process `PATH` and current-process permissions; its result may differ between users or environments.
- `env` distinguishes a missing variable (`nil`) from a variable defined as an empty string (`""`).
- `setenv` always overwrites an existing value.
- No `unsetenv` function or complete environment dump is exposed.
- `pid` is the process PID shared by all workers.
- Memory helpers deliberately run a full GC and do not measure the whole process.
- For system accounts, see User - system users.

babet tables — merging and copying Lua tables

Two historical helpers live directly on the `babet` table:

- `babet.mergeTables(...)` builds a new table from several sources;
- `babet.deepCopyTable(t)` recursively copies **values** that are tables.

They operate on entries actually stored in Lua tables. The `__pairs` and `__index` metamethods therefore do not provide virtual fields to merge or copy.

Module contents

- API
- `babet.mergeTables(t1, t2, ...)`
  - Positive integer keys: concatenation
  - Every other key: last writer wins
  - Shallow merge
- `babet.deepCopyTable(t)`
  - Values and shared graph structure
  - Keys are not copied
  - Metatables and raw traversal
  - Maximum depth
- Error contract
- Limits and design choices
- Not in v1

API

Function	Returns
`babet.mergeTables(t1, t2, ...)`	one new table
`babet.deepCopyTable(t)`	one new table

Both functions return exactly one value on success. They never return `(nil, err)`: call errors and depth errors are raised.

`babet.mergeTables(t1, t2, ...)`

At least two arguments are required and every argument must be a table. The API sets no additional arity limit beyond Lua's general limits.

Sources are not mutated. The result is a plain table without a metatable, even when source tables have one.

Positive integer keys: concatenation

Every key represented by Lua as an integer `>= 1` is treated as a list position. This includes a key written as `2.0`, which Lua canonicalizes to an integer.

Within each source, such keys are visited in ascending numeric order, then their values are appended to the result. Holes and original indices are compacted:

```
local r = babet.mergeTables(
    { [4] = "d", [2] = "b" },
    { [7] = "g", [1] = "a" }
)

-- r == { "b", "d", "a", "g" }
```

The same list position appearing in several sources is not overwritten: every value is appended.

Every other key: last writer wins

String, boolean, table, function, thread, userdata and light userdata keys, along with floating, zero and negative numeric keys, keep their original identity. A later source overwrites an earlier value for the same key:

```
local key = {}
local r = babet.mergeTables(
    { mode = "safe", [key] = 1, [0] = "a" },
    { mode = "fast", [key] = 2, [0] = "b" }
)

-- r.mode == "fast"
-- r[key]  == 2
-- r[0]    == "b"
```

Shallow merge

Values are never copied. A subtable placed in the result is the same table as in the source:

```
local nested = { enabled = false }
local r = babet.mergeTables({ nested = nested }, {})

r.nested.enabled = true
-- nested.enabled == true
```

To obtain an independent graph for table values afterwards, use:

```
local isolated = babet.deepCopyTable(
    babet.mergeTables(defaults, overrides)
)
```

`babet.deepCopyTable(t)`

The function requires exactly one table. It creates a new table for the root and for every **value** that is itself a table.

Values and shared graph structure

Cycles reached through values are supported and graph identity is preserved:

```
local shared = { value = 42 }
local source = { a = shared, b = shared }
source.self = source

local copy = babet.deepCopyTable(source)

-- copy ~= source
-- copy.a ~= shared
-- copy.a == copy.b
-- copy.self == copy
```

Non-table values — numbers, strings, booleans, functions, threads, userdata and light userdata — are reused as-is.

Keys are not copied

Every key keeps its original value and identity. In particular, a table used as a key remains the source table:

```lua
local key = { id = 1 }
local source = {
    [key] = "value",
    key_as_value = key,
}
local copy = babet.deepCopyTable(source)

-- copy[key] == "value"          -- original key retained
-- copy.key_as_value ~= key      -- table value copied
-- copy[copy.key_as_value] == nil
```

The function is therefore a deep copy of **table values**, not a structural copy of table keys.

Metatables and raw traversal

The actual metatable of every copied table is attached to its copy **by reference**. It is not duplicated. Mutating that metatable therefore affects both source and copy.

Only actually stored pairs are traversed. A field returned through `__index` or invented by `__pairs` is not materialized in the copy. Once the shared metatable is attached, its behavior naturally remains active on the copy.

Maximum depth

The root is at depth `0`. Up to **75 descents into new table values** are accepted, for at most 76 tables along one root-inclusive path. A 76th descent into a previously unseen table raises:

`Table is too deep to copy (max depth 75 exceeded)`

A reference to an already copied table does not create a new level, so a cycle may close exactly at the boundary.

Error contract

Errors are raised through the Lua API for:

- fewer than two arguments to `mergeTables`;
- any non-table argument to `mergeTables`;
- an argument count other than one for `deepCopyTable`;
- a non-table argument to `deepCopyTable`;
- exceeding the maximum depth.

```lua
local ok, err = pcall(babet.deepCopyTable, 42)
-- ok == false; err contains "Argument must be a table"
```

Limits and design choices

- `mergeTables` is not a deep merge: a later subtable replaces the earlier reference wholesale.
- Source metatables are ignored by `mergeTables`; its result has none.
- `deepCopyTable` shares metatables and table keys with the source.
- A weak metatable (`__mode`) remains weak on the copy because it is shared.
- Map-key order is undefined. Only the positive-integer ordering of `mergeTables` is guaranteed.
- `mergeTables` copies dense positive integer keys `1..n` directly. Sparse positive keys are sorted in temporary storage owned by Lua, which remains reclaimable after an allocation error. For

a source with `m` entries and `k` positive integer keys, the traversal costs O(m) for dense keys and O(m + k log k) for sparse keys, with O(k) temporary storage in the latter case.

Not in v1

Features such as `deepMergeTables` or deep structural comparison could be added separately. They are not part of the current API.

babet time - clocks, sleep, dates and durations

Babet exposes two sub-second clocks, interruptible sleep, and small utilities for ISO 8601 timestamps and compact durations.

The historical functions remain available at the root:

- `babet.monotonic();`
- `babet.now();`
- `babet.sleep(amount [, unit])`.

The same functions are available under `babet.time`, together with four additional utilities: `iso`, `parse_iso`, `parse_duration`, and `format_duration`.

Module contents

- API
- `monotonic()` and `now()`
- `sleep(amount [, unit])`
- `babet.time.iso([ts])`
- `babet.time.parse_iso(text)`
- `babet.time.parse_duration(text)`
- `babet.time.format_duration(seconds)`
- Error contract summary
- Limits and design choices

API

Function	Result
`babet.monotonic()`	`number`, or exceptionally `(nil, err)` if `clock_gettime` fails
`babet.now()`	`number`, or exceptionally `(nil, err)` if `clock_gettime` fails
`babet.sleep(amount [, unit])`	`(true, nil)` or `(nil, err)`
`babet.time.iso([ts])`	UTC `string`
`babet.time.parse_iso(text)`	`(integer, nil)` or `(nil, "parse_iso: ...")`
`babet.time.parse_duration(text)`	`(integer, nil)` or `(nil, "parse_duration: ...")`
`babet.time.format_duration(seconds)`	compact `string`, for example `"1d2h3m4s"`

The `babet.time.now`, `babet.time.monotonic`, and `babet.time.sleep` aliases use exactly the same bindings and contract as their root names.

`monotonic()` and `now()`

```
local started = babet.monotonic()
do_something()
local elapsed = babet.monotonic() - started
print(string.format("elapsed: %.3f s", elapsed))

local timestamp = babet.now()
print(timestamp)
```

`monotonic()` uses `CLOCK_MONOTONIC`. Its value starts at an arbitrary origin, usually related to system boot. It is intended for measuring durations because it does not suffer wall-clock jumps.

`now()` uses `CLOCK_REALTIME`. It returns POSIX time in seconds since 1970-01-01 UTC, including a fractional part. It is appropriate for timestamps, but not for measuring durations because the wall clock may be adjusted.

Both functions:

- accept no arguments;
- return one `number` in normal operation;
- may theoretically return `(nil, err)` if the `clock_gettime` system call fails.

`sleep(amount [, unit])`

```
assert(babet.sleep(250, "ms"))
assert(babet.time.sleep(0.5))       -- 0.5 second
assert(babet.sleep(1000, "us"))
```

`amount` must be an actual Lua `number`, finite and non-negative. Floating-point values are accepted. A numeric string such as `"100"` is rejected.

`unit` must be an actual Lua `string`:

Unit	Meaning
`"s"`	seconds, the default
`"ms"`	milliseconds
`"us"`	microseconds

A zero duration succeeds immediately. An unknown textual unit keeps the historical behavior:

```
local ok, err = babet.sleep(1, "minutes")
-- ok == nil
-- err == "Invalid time unit"
```

If a signal managed by `babet.signal` arrives during the wait, its Lua callback is dispatched and `sleep` then returns `(nil, "interrupted")`. An unmanaged signal that causes `EINTR` does not shorten the wait: Babet resumes with the remaining time.

Any other `nanosleep` failure returns `(nil, "sleep: <description>")`.

`babet.time.iso([ts])`

`iso` formats a Unix second in UTC:

```
print(babet.time.iso(0))
-- 1970-01-01T00:00:00Z

print(babet.time.iso())
-- current time, at whole-second precision
```

Without an argument, `iso()` uses the current time. With an argument:

- `ts` must be an actual Lua `number`;
- an integer or finite float is accepted;
- a fractional value is rounded toward negative infinity;
- the value must fit in a signed 64-bit integer.

The rounding matters before the Unix epoch:

```
babet.time.iso(0.9)   -- 1970-01-01T00:00:00Z
babet.time.iso(-0.5)  -- 1969-12-31T23:59:59Z
```

For ordinary years, the result has the form `YYYY-MM-DDTHH:MM:SSZ`. Extreme valid `int64` timestamps may produce a signed year wider than four digits. That extended result is not necessarily accepted by `parse_iso`, whose grammar requires an exactly four-digit year.

`iso` applies neither locale nor local timezone and does not retain fractional seconds.

babet.time.parse_iso(text)

The parser intentionally accepts a small, deterministic subset:

```
YYYY-MM-DDTHH:MM:SSZ
YYYY-MM-DD HH:MM:SSZ
YYYY-MM-DDTHH:MM:SS.fraction+HH:MM
YYYY-MM-DDTHH:MM:SS.fraction-HH:MM
```

Exact rules:

- `text` must be an actual Lua `string`;
- the year contains exactly four digits and ranges from `0001` to `9999`;
- the Gregorian calendar is validated, including leap years;
- the separator is uppercase `T` or one space;
- hours `00..23`, minutes `00..59`, seconds `00..59`;
- leap seconds written as `:60` are rejected;
- a fraction is optional, but the dot must be followed by at least one digit; all fraction digits are ignored, without rounding;
- a timezone is required: uppercase `Z`, `+HH:MM`, or `-HH:MM`;
- accepted offsets range from `00:00` through `23:59`;
- `+0200`, `+02`, lowercase `z`, leading or trailing whitespace, and trailing data are rejected;
- an embedded NUL byte is never truncated: it makes the string invalid.

Example:

```
local ts = assert(babet.time.parse_iso(
    "2026-06-17T10:00:00+02:00"))

assert(ts == babet.time.parse_iso("2026-06-17T08:00:00Z"))
```

A syntactically or calendrically invalid string returns `(nil, "parse_iso: ...")`. A wrong type or argument count raises a Lua error.

babet.time.parse_duration(text)

The format is a whitespace-free sequence of `number + unit` chunks:

```
assert(babet.time.parse_duration("45s") == 45)
assert(babet.time.parse_duration("1h30m") == 5400)
assert(babet.time.parse_duration("2d12h") == 216000)
```

Available units:

Unit	Seconds
d	86400
h	3600
m	60
s	1

Each unit may occur at most once and units must be in strict order `d > h > m > s`. Signs, decimal values, whitespace, and the units `w`, `ms`, `us`, or `ns` are rejected. An embedded NUL byte is not truncated and also produces a parse error.

Components are not normalized: `"1h90m"` is valid and equals 9000 seconds. Leading zeroes and ordered zero-valued chunks are also accepted, for example `"0001s"` and `"0d0h0m0s"`.

The total must remain between `0` and `math.maxinteger` on Babet's 64-bit configuration. Overflow returns a parse error.

An invalid string returns `(nil, "parse_duration: ...")`. A wrong type or argument count raises a Lua error.

`babet.time.format_duration(seconds)`

```
babet.time.format_duration(0)      -- "0s"
babet.time.format_duration(90)     -- "1m30s"
babet.time.format_duration(90061)  -- "1d1h1m1s"
```

`seconds` must be an actual Lua `number` with a non-negative integer value. A floating-point value such as `3.0` is accepted; `3.7`, `"3"`, NaN, infinity, and negative values raise a Lua error.

The output is Babet's canonical representation:

- units in `d`, `h`, `m`, `s` order;
- zero components omitted;
- zero represented as `"0s"`.

For every valid integer `n >= 0`:

```
assert(babet.time.parse_duration(
    babet.time.format_duration(n)) == n)
```

The textual reverse is not guaranteed because the parser also accepts non-normalized forms such as `"90m"` or `"0h"`.

Error contract summary

Situation	Behavior
Wrong type or wrong argument count	Lua error
NaN, infinity, negative duration, or out-of-range numeric value	Lua error
Unknown textual unit in `sleep`	`(nil, "Invalid time unit")`
Managed signal during `sleep`	`(nil, "interrupted")` after callback dispatch
Invalid ISO or duration text	`(nil, "parse_iso: ...")` or `(nil, "parse_duration: ...")`
Clock or sleep system failure	`(nil, err)`

Limits and design choices

- Use `monotonic()` for durations and `now()` for timestamps.

- `iso` is a fixed UTC formatter, not a general replacement for `os.date`.
- No named timezone, daylight-saving rule, locale, or calendar arithmetic is provided.
- ISO fractions are discarded and are not emitted by `iso`.
- `parse_duration` works with whole seconds only.
- `CLOCK_BOOTTIME`, negative durations, and week/month/year units are not exposed.

TLS — direct encryption, STARTTLS, certificates, and SNI

Babet TLS uses the same userdata as `babet.socket`. After the handshake, `send`, `recv`, `recv_line`, `recv_all`, `set_timeout`, `peer`, `sockname`, and `close` behave like their plain-TCP versions.

Two modes are available:

- `babet.socket.connect_tls()` starts TLS on the first application byte;
- `sock:starttls()` upgrades an existing TCP connection after a plaintext protocol negotiation.

The implementation provides TLS 1.2 minimum, certificate verification by default, hostname checks, SNI independent from verification, system and per-call CA trust, shared TCP+handshake deadlines, and fail-closed STARTTLS. It does not expose TLS servers, client certificates, ALPN, public-key pinning, or peer-certificate inspection.

Module contents

- Core conventions
- API overview
- TLS options
- Direct TLS connection
- STARTTLS upgrade
- Verification, hostname, and SNI
- Certificate authorities
- TLS versions
- Timeouts
- Socket state after failure
- Complete examples
- Error contract
- Security and limitations

Core conventions

Verification is on by default

```
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
```

`verify = false` explicitly disables both chain and hostname verification. It may be useful in controlled tests but encryption without authentication does not prevent an active man-in-the-middle attack.

Same stream API after handshake

```
local sock = assert(babet.socket.connect_tls("irc.example.net", 6697, {
    timeout = 10,
}))
assert(sock:set_timeout(120))
assert(sock:send("PING :hello\r\n"))
local line = assert(sock:recv_line())
```

Handshake timeout and later I/O timeout are separate.

SNI is not certificate verification

SNI selects a TLS virtual host. Verification checks trust and reference identity. Babet sends DNS SNI whenever a DNS hostname is available, including when `verify = false`.

API overview

```
local sock, err = babet.socket.connect_tls(host, port, opts?)
local ok, err = sock:starttls(opts?)
```

```
{
    verify = true,
    hostname = "example.com",
    ca_cert = "/path/ca-bundle.pem",
    ca_path = "/path/certs",
    min_version = "1.2",
    timeout = 10,
}
```

Field	Type	Default	Meaning
`verify`	strict boolean	`true`	chain and hostname verification
`hostname`	string	`connect_tls` host	reference name and DNS SNI
`ca_cert`	string	no custom file	extra PEM trust file
`ca_path`	string	no custom directory	OpenSSL hashed CA directory
`min_version`	`"1.2"` or `"1.3"`	`"1.2"`	minimum protocol version
`timeout`	finite number `>= 0`	`0`	handshake budget in seconds

Unknown fields are currently ignored; misspellings do not fail.

TLS options

`verify` must be an actual boolean. `hostname`, `ca_cert`, `ca_path`, and `min_version` must be strings without NUL. `timeout` must be finite and non-negative; a positive value below 1 ms is rounded up.

`hostname` has two roles:

1. OpenSSL reference identity when verification is enabled;
2. SNI value when it is a DNS name.

Connect to an IP while validating a DNS certificate:

```
local sock = assert(babet.socket.connect_tls("203.0.113.10", 443, {
    hostname = "api.example.com",
    timeout = 10,
}))
```

An IP literal is not sent as SNI. `ca_cert` and `ca_path` augment trust for that connection; they are not exclusive pinning.

`min_version = "1.3"` raises the minimum. It does not cap the maximum.

babet.socket.connect_tls(host, port, opts?)

Establishes TCP then performs a client handshake:

```
local sock, err = babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
})
assert(sock, err)
```

Without `opts.hostname`, the `host` argument is used for verification and DNS SNI. For an IP connection to a DNS certificate, supply the name explicitly.

After synchronous DNS resolution, one deadline covers every TCP address attempt and the TLS handshake. The handshake does not receive a fresh budget. DNS itself remains outside this controlled deadline.

`opts.timeout` is not retained as the socket I/O default:

```
local sock = assert(babet.socket.connect_tls(host, port, { timeout = 10 }))
assert(sock:set_timeout(30))
```

sock:starttls(opts?)

Upgrades an existing connected plain TCP socket in place:

```
local ok, err = sock:starttls({
    hostname = "mail.example.com",
    timeout = 10,
})
```

On success it returns exactly `(true, nil)`.

The socket must be open, connected, not already TLS, and have no pending plaintext buffered by Babet. `starttls` does not remember the hostname passed to the earlier TCP constructor, so `hostname` is mandatory when `verify = true`.

Babet does not send a protocol `STARTTLS` command. The script must:

1. negotiate in plaintext;
2. request STARTTLS;
3. validate the positive protocol reply;
4. call `starttls`;
5. resume the protocol over TLS, often with a new greeting.

If a prior line/all read timed out after consuming plaintext, finish consuming or otherwise resolve that plaintext before upgrading. Babet rejects STARTTLS while the shared pending buffer is non-empty.

Verification, hostname, and SNI

Connection	`verify`	`hostname`	Identity check	SNI
DNS `example.com`	`true`	absent	`example.com`	`example.com`
IP	`true`	`api.example.com`	DNS name	DNS name
IP	`true`	absent	IP identity	none
DNS	`false`	absent	none	DNS host
IP	`false`	DNS name	none	DNS name
IP	`false`	absent	none	none

Providing `hostname` can therefore still be required by a virtual host even when verification is disabled.

Certificate authorities

The socket TLS context attempts OpenSSL defaults, OpenSSL environment variables, and several known Linux/BSD CA locations. Per-call `ca_cert` and `ca_path` are then added.

This applies to both `connect_tls` and `starttls`. An empty string adds no custom trust. In contrast, a non-empty `HTTP` `ca_cert` replaces the default authorities for that request; the identically named options therefore have different trust policies across these modules.

Custom trust is isolated per connection:

```
local a = assert(babet.socket.connect_tls("internal.example", 443, {
    ca_cert = "/tmp/test-root.pem",
    timeout = 5,
}))
a:close()
-- Later calls do not retain /tmp/test-root.pem.
```

`ca_path` must follow OpenSSL's hashed-directory conventions. A CA file trusts certificates issued by that CA; it is not a leaf-certificate/public-key pin. Babet does not expose the peer certificate for custom pinning.

TLS versions

Protocols older than TLS 1.2 are always rejected.

- default minimum: TLS 1.2;
- optional minimum: TLS 1.3;
- TLS 1.3 is negotiated automatically with the default when supported.

No maximum-version option is exposed.

Timeouts

For `connect_tls`, `opts.timeout` covers TCP plus handshake after DNS. For `starttls`, it covers only the upgrade handshake and does not inherit `sock:set_timeout()`.

After success, configure I/O separately:

```
assert(sock:set_timeout(30))
local line, err = sock:recv_line()
```

TLS sockets remain non-blocking internally and OpenSSL WANT_READ/WANT_WRITE is coordinated with one poll deadline.

Socket state after failure

`connect_tls` returns no socket and closes all intermediate resources.

Pre-handshake STARTTLS validation/configuration failures leave the plain TCP socket usable: invalid options, missing hostname, pending plaintext, context setup, or pre-handshake `fcntl` failure.

Once `SSL_connect` begins, any failure closes the socket: timeout, interruption, certificate failure, alert, protocol, or I/O error. A ClientHello may already have been sent and peer bytes consumed, so falling back to plaintext would be unsafe. This is intentionally fail-closed.

After the first `SSL_write` attempt in `send`, abandoning that call because of a timeout, interruption or error closes the transport. OpenSSL may have begun a record without reporting application bytes written, so checking a positive byte counter is insufficient. The original error is retained; subsequent I/O reports a closed socket, and `close()` stays idempotent. Closure happens before any signal callback is dispatched. A timeout before the first `SSL_write` attempt leaves the socket usable.

Complete examples

Raw HTTPS request

For normal HTTP use babet.http; this demonstrates stream use:

```
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
assert(sock:set_timeout(10))
assert(sock:send(
    "GET / HTTP/1.1\r\n" ..
    "Host: example.com\r\n" ..
    "Connection: close\r\n\r\n"
))
local response = assert(sock:recv_all(nil, 8 * 1024 * 1024))
print(response)
sock:close()
```

IP connection with DNS identity

```
local sock = assert(babet.socket.connect_tls("203.0.113.20", 443, {
    hostname = "api.example.com",
    ca_cert = "/etc/myapp/ca.pem",
    timeout = 5,
}))
```

Simplified SMTP STARTTLS

```
local sock = assert(babet.socket.connect("mail.example.com", 587, 5))
assert(sock:set_timeout(10))
assert(sock:recv_line():match("^220"))
assert(sock:send("EHLO client.example\r\n"))
-- Parse the full multiline EHLO response and require STARTTLS.
assert(sock:send("STARTTLS\r\n"))
assert(sock:recv_line():match("^220"))
assert(sock:starttls({
    hostname = "mail.example.com",
    timeout = 10,
}))
assert(sock:send("EHLO client.example\r\n"))
```

A complete SMTP client must correctly parse every multiline reply and prevent STARTTLS downgrade.

Controlled self-signed test

```
local sock = assert(babet.socket.connect_tls("127.0.0.1", 8443, {
    verify = false,
    hostname = "dev.local", -- SNI is still sent
    timeout = 3,
}))
```

Do not use this trust policy in production.

Error contract

Wrong positional types raise; invalid values and runtime failures return `(nil, err)`: empty host, bad port/options, CA loading, DNS/TCP, timeout, interruption, handshake, certificate/hostname ver-

ification, and STARTTLS preconditions.

OpenSSL wording varies. Compare stable states such as `"timeout"` and `"interrupted"`; log full messages for diagnosis.

OpenSSL writes are protected against `SIGPIPE`, including during handshake, read and shutdown. Transport failures are reported through the API instead of terminating the program; `close()` remains best-effort. The application's `babet.signal` handler is not replaced.

Security and limitations

- Keep `verify = true` in production.
- Validate against a trusted expected hostname, never one supplied by the peer.
- STARTTLS must be required by application policy to prevent stripping.
- No ALPN, client certificate, TLS server, peer-certificate access, custom cipher policy, or application OCSP/CRL policy.
- Protect and rotate private CA files.

`babet.toml` - TOML decoding

The `babet.toml` module uses **toml++ 3.4.0** and decodes documents that conform to **TOML 1.0.0**. Optional extensions outside the TOML 1.0 profile supported by this toml++ version are disabled.

The v1 API only decodes a Lua string. It does not read a file for you and it does not provide an encoder.

Module contents

- API
- Type mapping
  - Integers and floats
- Strings, UTF-8, and keys
- Arrays and tables
  - Empty-container ambiguity
- Dates and times
- Ordering and discarded information
- Error contract
- Limits
- Not exposed in v1

API

Function	Contract
`babet.toml.decode(text)`	Returns `table, nil` or `nil, err`

`decode` requires exactly **one argument**, and it must be an actual Lua string. A number is not implicitly converted to text, and extra arguments are rejected.

To read a file:

```lua
local file = assert(io.open("config.toml", "rb"))
local text = assert(file:read("*a"))
file:close()

local config, err = babet.toml.decode(text)
```

```lua
if not config then
    error(err)
end
```

A TOML document always has a table at its root. An empty string or a document containing comments only therefore returns an empty Lua table.

Type mapping

TOML	Lua
string	string
signed 64-bit integer	Lua integer
float	Lua number
boolean	boolean
array	`1..n` sequence table
table, inline table, or dotted table	table with string keys
array of tables	sequence table containing tables
local date/time and date-time	normalized string

TOML has no value equivalent to Lua `nil` or JSON `null`.

Integers and floats

TOML integers cover the full signed 64-bit range, from `math.mininteger` to `math.maxinteger`, and are preserved exactly. Decimal, hexadecimal, octal, and binary notation all produce a Lua integer. A value outside that range is a parse error.

Floats become Lua numbers. TOML also accepts these special values:

```lua
local values = assert(babet.toml.decode([[
positive = inf
negative = -inf
invalid  = nan
]]))

assert(values.positive == math.huge)
assert(values.negative == -math.huge)
assert(values.invalid ~= values.invalid) -- NaN property
```

As in Lua, `NaN` compares equal to no value, including itself.

Strings, UTF-8, and keys

The TOML document must contain valid UTF-8. Basic, literal, and multiline strings are decoded before they are returned to Lua: TOML escape sequences become their corresponding characters.

Lua strings are binary-safe. A TOML sequence such as `\u0000` may therefore produce a NUL byte in a value **or in a quoted key**, without truncation:

```lua
local value = assert(babet.toml.decode([[
"a\u0000b" = "x\u0000y"
]]))

assert(value["a\0b"] == "x\0y")
```

A raw NUL byte injected into the TOML text is not treated as the end of a C string: the parser receives the whole buffer and rejects the document. The same applies to invalid UTF-8.

Bare, quoted, and dotted keys follow TOML rules:

```
local value = assert(babet.toml.decode([[
"a.b" = 1
site."google.com" = true
physical.color = "orange"
]]))

assert(value["a.b"] == 1)              -- literal dot in a quoted key
assert(value.site["google.com"] == true)
assert(value.physical.color == "orange")
```

An empty quoted key (`""`) is valid TOML and becomes the Lua key `""`.

Arrays and tables

TOML arrays become Lua sequences indexed from 1. TOML 1.0 permits heterogeneous arrays:

```
local value = assert(babet.toml.decode([[
items = [1, "two", true, { name = "three" }]
]]))

assert(value.items[1] == 1)
assert(value.items[2] == "two")
assert(value.items[3] == true)
assert(value.items[4].name == "three")
```

Sections, inline tables, and dotted keys all become ordinary Lua tables. The original syntax is not retained.

Empty-container ambiguity

An empty array `[]` and an empty table `{}` both become an empty Lua table:

```
local value = assert(babet.toml.decode([[
a = []
b = {}
]]))

assert(type(value.a) == "table" and next(value.a) == nil)
assert(type(value.b) == "table" and next(value.b) == nil)
```

No metatable or sentinel remembers the original TOML type. This information loss is usually harmless when reading configuration, but it prevents a faithful round trip.

Dates and times

TOML defines four temporal types. Since Lua has no native date type, Babet converts them to strings:

TOML type	Lua form
local date	`YYYY-MM-DD`
local time	`HH:MM:SS[.fraction]`
local date-time	`YYYY-MM-DDTHH:MM:SS[.fraction]`
offset date-time	`YYYY-MM-DDTHH:MM:SS[.fraction]Z` or `+/-HH:MM`

```
local value = assert(babet.toml.decode([[
date = 1979-05-27
```

```
time = 07:32:00.1234
local_dt = 1979-05-27T07:32:00
utc_dt = 1979-05-27T07:32:00Z
]]))
```

The temporal value is retained, but its original **spelling** may be normalized by toml++: canonical separator, fractional part, or UTC notation. Comments and the exact source representation are not available. After conversion, a TOML date is also indistinguishable from a TOML string containing the same text.

Ordering and discarded information

A TOML table has no guaranteed semantic order, and Lua table iteration must not be used as a proxy for source order. Decoding does not preserve:

- comments;
- lexical key order;
- whitespace and line breaks;
- the choice between sections, inline tables, and dotted keys;
- the original spelling of numbers and dates;
- the type of an empty container.

The module is intended to **read values**, not to edit and reproduce a TOML document byte-for-byte.

Error contract

A TOML syntax or value error returns:

```lua
local value, err = babet.toml.decode("answer = ")
assert(value == nil)
print(err) -- toml: ... (line ..., col ...)
```

The message is prefixed with `toml:` and includes the line and column reported by toml++.

The following are returned as `(nil, err)`, among others:

- invalid syntax or a redefined key;
- integer outside the signed 64-bit range;
- invalid UTF-8 or a forbidden control character;
- an extension unavailable in the TOML 1.0 profile used by Babet.

API misuse raises a Lua error instead:

- missing argument;
- non-string argument;
- extra argument.

Limits

The whole document is parsed and converted in memory. There is no streaming API, `decode_file`, schema validation, or access to source positions for individual values. Recursive conversion protects the Lua stack; a depth that cannot be converted is returned as a `toml:` error.

Not exposed in v1

- `babet.toml.encode`;
- `babet.toml.decode_file`;
- parser options or extensions outside TOML 1.0;
- nodes retaining comments, order, and source positions;
- sentinels distinguishing empty arrays from empty tables;

- schema validation.

USER - look up system accounts by name or UID

The `babet.user` module queries the system user database through NSS. It can:

- look up an account by name;
- look up an account by UID;
- quickly test whether an account exists;
- retrieve the name, UID, primary GID, GECOS field, home directory, and shell.

It never parses `/etc/passwd` directly. It uses the same resolution mechanisms as `id` or `getent passwd`, so it can see accounts supplied by LDAP, SSSD, NIS, FreeIPA, or other sources configured in `/etc/nsswitch.conf`.

Module contents

- General conventions
- API overview
- Returned user table
- Look up an account with `get`
  - By name
  - By UID
  - Missing account
- Test existence with `exists`
  - By name
  - By UID
  - When to prefer `get`
- Argument validation
- NSS, workers, and security
- Error contract
- Design decisions and limits

General conventions

Dedicated subtable

Unlike the historical SYS or FS functions, user helpers live under `babet.user`:

```
local user = babet.user.get("root")
local exists = babet.user.exists("root")
```

Name or UID, without implicit conversion

The argument accepts exactly:

- a Lua string: lookup by **name**;
- a non-negative Lua integer in the `uid_t` range: lookup by **UID**.

A numeric string remains a name:

```
-- Looks for an account literally named "1000"
local by_name = babet.user.get("1000")

-- Looks up numeric UID 1000
local by_uid = babet.user.get(1000)
```

Floats, booleans, tables, `nil`, negative UIDs, and oversized UIDs raise a Lua error. No silent conversion or truncation is performed.

Fields are always present

When a user is found, the returned table always contains all six documented fields. Text fields may be empty strings, but never `nil`.

API overview

Function	Result
`babet.user.get(name_or_uid)`	table, `(nil, "user not found")`, or `(nil, "user: ...")`
`babet.user.exists(name_or_uid)`	strict boolean for NSS results; `(nil, err)` only for an unexpected C++ boundary failure

Returned user table

Typical example:

```
{
    name  = "www-data",
    uid   = 33,
    gid   = 33,
    gecos = "www-data",
    home  = "/var/www",
    shell = "/usr/sbin/nologin",
}
```

Field	Type	Meaning
`name`	string	canonical account name returned by NSS
`uid`	integer	numeric user identifier
`gid`	integer	account's **primary** GID
`gecos`	string	raw descriptive field, often the full name
`home`	string	configured home directory
`shell`	string	configured login shell

`gid` is not a list of supplementary groups. The module does not perform group lookups and does not expose additional memberships.

The `gecos` field is returned unchanged. Its historical format can contain comma-separated values, but many systems simply store a free-form name.

```
local u = assert(babet.user.get("root"))
print("Name :", u.name)
print("UID  :", u.uid)
print("GID  :", u.gid)
print("GECOS:", u.gecos)
print("Home :", u.home)
print("Shell:", u.shell)
```

Look up an account with get

Signature:

```
local info, err = babet.user.get(name_or_uid)
```

Use `get` when you need account details or must distinguish a missing account from an NSS failure.

By name

```
local root, err = babet.user.get("root")
assert(root, err)
assert(root.name == "root")
assert(root.uid == 0)
```

Preparing a directory for a service account:

```
local account, err = babet.user.get("my-service")
assert(account, err)

assert(babet.mkdir("/var/lib/my-service"))
assert(babet.setAttributes(
    "/var/lib/my-service",
    account.uid,
    account.gid,
    tonumber("750", 8)
))
```

The name is passed to NSS unchanged. It must be a string without an embedded NUL byte. An empty string is a valid API input, but normally matches no account and returns `user not found`.

By UID

```
local root, err = babet.user.get(0)
assert(root, err)
assert(root.name == "root")
```

Displaying the account associated with a file owner:

```
local attrs, err = babet.getAttributes("report.txt")
assert(attrs, err)

local owner, user_err = babet.user.get(attrs.owner)
if owner then
    print("Owner:", owner.name)
else
    print("Unresolved owner UID:", attrs.owner, user_err)
end
```

The UID must be a non-negative integer compatible with the system `uid_t` type. On Linux the limit is usually `2^32 - 1`, but the code uses the actual platform limit at compile time.

Missing account

A missing account is not a Lua error. The function returns:

```
local info, err = babet.user.get("missing-account")
-- info == nil
-- err  == "user not found"
```

Typical handling:

```
local account, err = babet.user.get("my-service")
if not account then
    if err == "user not found" then
        print("The account must be created")
    else
        print("NSS lookup failed:", err)
```

```
    end
end
```

Avoid this when absence is expected:

```
-- Wrong for an optional account: assert raises immediately
-- local account = assert(babet.user.get("optional-account"))
```

Test existence with `exists`

Signature:

```
local present = babet.user.exists(name_or_uid)
```

The function always returns a boolean for a valid argument.

By name

```
if babet.user.exists("www-data") then
    print("The www-data account exists")
end
```

Simple precondition:

```
if not babet.user.exists("my-service") then
    error("The my-service account must exist before startup")
end
```

By UID

```
assert(babet.user.exists(0)) -- root on a normal Unix system
```

A missing UID returns `false`:

```
local present = babet.user.exists(2000000000)
print(present)
```

When to prefer `get`

`exists` deliberately converts an NSS error to `false`.

This makes it convenient for a simple branch, but it cannot distinguish:

- an account that truly does not exist;
- a temporarily unavailable LDAP directory;
- an I/O error;
- an out-of-memory condition in the resolver.

For an important administrative or security decision, use `get`:

```
local account, err = babet.user.get("my-service")
if account then
    print("Account available")
elseif err == "user not found" then
    print("Account missing")
else
    error("Unable to query NSS: " .. err)
end
```

Argument validation

The following cases raise a Lua error, recoverable with `pcall`:

```
local invalid_calls = {
    function() return babet.user.get() end,
    function() return babet.user.get(nil) end,
    function() return babet.user.get(true) end,
    function() return babet.user.get({}) end,
    function() return babet.user.get(1.5) end,
    function() return babet.user.get(-1) end,
    function() return babet.user.get(8589934592) end,
    function() return babet.user.get("root\0other") end,
}

for _, call in ipairs(invalid_calls) do
    local ok, err = pcall(call)
    assert(not ok)
    print(err)
end
```

`exists` applies exactly the same validation:

```
local ok = pcall(function()
    return babet.user.exists(-1)
end)
assert(not ok)
```

Rejecting NUL bytes prevents `"root\0other"` from being interpreted as `"root"` by `getpwnam_r`.

NSS, workers, and security

Account sources

The result depends on the machine's NSS configuration. According to `/etc/nsswitch.conf`, an account may come from:

- `/etc/passwd`;
- LDAP;
- SSSD;
- NIS;
- FreeIPA;
- systemd-userdb;
- another NSS module.

The same script may therefore see different accounts on different machines without any Babet change.

Calls from workers

The module uses the reentrant `getpwnam_r` and `getpwuid_r` functions. It can be used inside workers:

```
local worker = assert(babet.workers.spawn([[
    local root, err = babet.user.get("root")
    if not root then
        error(err)
    end
    return root.uid
]]))
```

```lua
local joined, uid = worker:join()
assert(joined and uid == 0)
```

Each call queries NSS. Babet does not maintain an application-level user cache.

Security-sensitive decisions

The `home` and `shell` fields are configuration data, not proof that a path exists or that a program is executable.

```lua
local u = assert(babet.user.get("my-service"))
local home_is_dir = assert(babet.isDir(u.home))
local shell_path = babet.which(u.shell)
```

Likewise, the presence of an account does not prove that it can log in, that a password is valid, or that it belongs to a particular supplementary group.

Error contract

Situation	`get`	`exists`
account found	user table	`true`
account missing	`(nil, "user not found")`	`false`
NSS error	`(nil, "user: ...")`	`false`
invalid argument	Lua error	Lua error
unexpected C++ exception	`(nil, "user: out of memory")`, `(nil, "user: internal failure")`, or `(nil, "user: unknown internal failure")`	same `(nil, err)` boundary result

The final row is distinct from an NSS error. It exists only to prevent an unexpected C++ exception from crossing Lua's C frames. Under every ordinary NSS outcome, `exists` still returns exactly one strict boolean as documented above.

Generic handling:

```lua
local ok, info, err = pcall(function()
    return babet.user.get("my-service")
end)

if not ok then
    print("Invalid call:", info)
elseif not info then
    print("Lookup failed:", err)
else
    print("UID:", info.uid)
end
```

Design decisions and limits

- NSS is used instead of parsing `/etc/passwd`.
- Reentrant `_r` variants are used for worker compatibility.
- Text fields are always present and replaced with `""` if NSS supplies a null pointer.
- The NSS buffer grows dynamically up to an internal 64 KiB limit; beyond that, `get` returns an NSS error.
- `exists` favors a simple boolean API and hides NSS errors; use `get` when a diagnosis is required. Only the internal C++ safety boundary can return its exceptional `(nil, err)` diagnostic.

- The module does not expose supplementary groups.
- It exposes neither `/etc/shadow`, passwords, nor account expiration data.
- It does not create, remove, or modify users.
- To create an account, a script can invoke a system utility through `babet.exec`, with appropriate privileges.

WORKERS — OS threads, isolated Lua states, queues, and channels

`babet.workers` executes Lua code in real POSIX threads. Each worker owns a separate `lua_State`, receives a serialized copy of its initial arguments, and communicates with the parent through two bounded message queues. Shared channels can also connect the parent and several workers, or workers directly, without relaying messages through the parent's inbox/outbox.

The module covers:

- starting a Lua chunk in an OS thread;
- passing an initial argument table;
- blocking or non-blocking retrieval of the first return value;
- a parent-to-worker inbox and a worker-to-parent outbox;
- direct bounded FIFO multi-producer/multi-consumer channels;
- timeouts and capacity-based backpressure;
- closing with drainage and wake-up of blocked calls;
- loading Babet, bundled, and user modules in each state;
- Lua-memory isolation between threads;
- a bounded pool of persistent workers that reuses pthreads and Lua states across several tasks.

It does not provide shared Lua memory, forced termination, or channels between distinct OS processes. Available cancellation is strictly cooperative.

Module contents

- Essential conventions
- API overview
- `spawn(code, args?, opts?)`
  - `code`
  - `args` and `worker.args`
  - Queue capacities
  - Passing channels through `opts.channels`
  - When `setenv` and `chdir` become locked
- Transferable values
  - Scalars and strings
  - List and object tables
  - Rejected values
  - Copies, identity, depth, and budgets
- Final worker result
  - `status()` — observe without consuming
  - `done()` — non-consuming boolean test
  - `join(timeout?)` — wait and consume
  - `poll()` — check and consume
  - Do not call `join()` after completed `poll()`
- Parent-to-worker messages
- Worker-to-parent messages
- `worker.send`, `worker.recv`, and `worker.cancelled`
- Direct shared channels
  - Creating a channel and choosing capacity

    - `send` and `recv`
    - `close` and `is_closed`
    - Concurrency, ordering, and lifetime
- Timeouts and failure reasons
- `close()` and queue lifecycle
- `cancel()` and cooperative cancellation
- Module loading and worker environment
- Available CPU count
- Reusable bounded pool
    - Creation and options
    - Submission and tasks
    - State isolation and reuse
    - Backpressure and timeouts
    - Close, cancellation, and join
    - Shared channels in tasks
- Complete examples
    - Simple calculation with `join`
    - Structured arguments
    - Non-blocking progress with `poll`
    - Persistent request/response worker
    - Sending and receiving `nil`
    - Close, then drain the outbox
    - Backpressure and timeout
    - Parent to worker through a channel
    - Worker to worker without parent relay
    - Multiple producers and consumers
    - Cooperative cancellation
    - Native pool for many tasks
- Deadlocks to avoid
- Error contract
- Garbage collection and destruction
- Signals, environment, and concurrency
- Features not provided

Essential conventions

Each worker owns a separate Lua state

A table, global, or module changed in the parent is not shared with a worker. Data crosses state boundaries only through serialization:

- `args` during `spawn`;
- `job:send` and `worker.recv`;
- `worker.send` and `job:recv`;
- shared channel `send` and `recv` methods;
- the chunk's first return value.

There can therefore be no data race on Lua objects themselves. External side effects — files, sockets, databases, and processes — remain system resources and must still be coordinated by the application.

Results use a `pcall`-like convention

`join(timeout?)` returns:

```
true, business_value
false, error_message
nil, "timeout"
```

The third state only means the worker is still active. The result is neither joined nor consumed.

This lets a worker legitimately return `nil`:

```
local ok, value = job:join(0.5)
if ok == true then
    -- value may be nil.
elseif ok == false then
    io.stderr:write(value, "\n")
else
    assert(value == "timeout")
end
```

Queue methods also return a boolean first, so a `nil` message is unambiguous.

The final result can be consumed once

As soon as `join()` retrieves the result, or `poll()` returns `"done"` or `"error"`, that result is consumed. A later `join()` or `poll()` reports `result already consumed`.

API overview

Parent side

```
local job, err = babet.workers.spawn(code, args?, opts?)
local channel, err = babet.workers.channel(opts?)
local cpu_count = babet.workers.cpu_count()
local pool, err = babet.workers.pool(opts?)

local state = job:status()
local finished = job:done()
local ok, value_or_reason = job:join(timeout?)
local state, value = job:poll()
local ok, err = job:cancel()

local ok, err = job:send(value, timeout?)
local ok, value_or_err = job:recv(timeout?)
local ok, err = job:close()

local ok, err = channel:send(value, timeout?)
local ok, value_or_reason = channel:recv(timeout?)
local ok, err = channel:close()
local closed = channel:is_closed()
```

API	Main result	Blocking
`spawn`	`job` or `(nil, err)`	creation only
`job:status()`	`"running"`, `"done"`, or `"error"`	no, never consumes
`job:done()`	boolean	no, never consumes
`job:join(t?)`	`(true, result)`, `(false, err)`, or `(nil, "timeout")`	depends on `t`
`job:poll()`	`("running", nil)`, `("done", result)`, or `("error", err)`	no
`job:cancel()`	`(true, nil)`	no
`job:send(v, t?)`	`(true, nil)`, `(false, reason)`, or `(nil, err)`	depends on `t`
`job:recv(t?)`	`(true, value)` or `(false, reason)`	depends on `t`

API	Main result	Blocking
`job:close()`	`(true, nil)`	no
`workers.channel(opts?)`	`channel` or `(nil, err)`	creation only
`channel:send(v, t?)`	`(true, nil)`, `(false, reason)`, or `(nil, err)`	depends on `t`
`channel:recv(t?)`	`(true, value)`, `(false, reason)`, or `(nil, err)`	depends on `t`
`channel:close()`	`(true, nil)`	no
`channel:is_closed()`	boolean	no
`workers.cpu_count()`	positive integer	no
`workers.pool(opts?)`	`pool` or `(nil, err)`	creates persistent workers
`pool:submit(code, args?, t?)`	`task` or `(nil, reason)`	depends on `t`
`task:done()`	boolean	no, never consumes
`task:status()`	`"running"`, `"done"`, or `"error"`	no
`task:join(t?)`	`(true, result)`, `(false, err)`, or `(nil, "timeout")`	depends on `t`
`task:poll()`	`("running", nil)`, `("done", result)`, or `("error", err)`	no
`pool:close(t?)`	`(true, nil)` or `(false, reason)`	depends on `t`
`pool:cancel()`	`(true, nil)`	no; cooperative cancellation
`pool:join(t?)`	`(true, nil)`, `(false, err)`, or `(nil, "timeout")`	depends on `t`
`pool:stats()`	`(table, nil)` or `(nil, err)`	no

Worker side

The chunk receives a global `worker` table:

```
worker.args
worker.channels
worker.send(value, timeout?)
worker.recv(timeout?)
worker.cancelled()
```

API	Result
`worker.args`	copy of `args`, or `nil`
`worker.channels`	table of handles passed through `opts.channels`
`worker.send(v, t?)`	`(true, nil)` or `(false, reason_or_error)`
`worker.recv(t?)`	`(true, value)` or `(false, reason)`
`worker.cancelled()`	boolean, no side effect

spawn(code, args?, opts?)

```
local job, err = babet.workers.spawn(code, args?, opts?)
```

On success, `spawn` returns only the `job` userdata. The thread may still be running, may already have completed, or may already have failed by the time the parent receives the userdata.

Chunk load or runtime errors occur **inside** the worker: `spawn` may succeed, then `join()` or `poll()` reports the error.

```
local job = assert(babet.workers.spawn("this is not valid Lua"))
local ok, err = job:join()
assert(not ok)
```

`code`

`code` is required and must be an actual Lua string. Numbers are not coerced.

```lua
local job = assert(babet.workers.spawn([[
    return 6 * 7
]]))
```

The source is loaded as a text chunk in a fresh state. It is not a closure that captures parent upvalues.

```lua
local factor = 3

-- factor does not exist in the worker: pass it through args.
local job = assert(babet.workers.spawn([[
    return worker.args.value * worker.args.factor
]], {
    value = 14,
    factor = factor,
}))
```

The binding does not truncate NUL bytes in `code`, but a NUL normally makes the source invalid for the Lua parser. Use ordinary Lua source text.

`args` and `worker.args`

`args` is optional. It must be a table or `nil`.

```lua
local job = assert(babet.workers.spawn([[
    return worker.args.left + worker.args.right
]], {
    left = 20,
    right = 22,
}))
```

Inside the worker:

- `worker.args` contains a deserialized copy of the table;
- without `args`, `worker.args == nil`;
- modifying `worker.args` never changes the parent's table;
- metatables and subtable identity do not cross the boundary;
- serialization reads raw entries and invokes neither `__len` nor `__index`.

A direct scalar is not accepted as the second argument:

```lua
babet.workers.spawn("return worker.args", 42) -- raises a Lua error
```

Wrap it in a table:

```lua
babet.workers.spawn("return worker.args.value", { value = 42 })
```

Queue capacities

`opts` can define:

```lua
{
    inbox_capacity = 64,
    outbox_capacity = 64,
}
```

- `inbox_capacity`: maximum queued parent-to-worker messages;
- `outbox_capacity`: maximum queued worker-to-parent messages;
- default: 64 messages for each queue;
- allowed range: integer from `1` to `1,000,000`;

- the value must be an actual Lua integer; the string `"64"` and the floating-point value `64.0` are rejected;
- capacity counts **messages**, not bytes.

```
local job = assert(babet.workers.spawn(code, nil, {
    inbox_capacity = 8,
    outbox_capacity = 32,
}))
```

Every unknown `opts` key raises a Lua error, catching typos immediately:

```
babet.workers.spawn("return 1", nil, {
    inbox_capcity = 16, -- error: unknown option
})
```

Option names must be real strings with no hidden NUL suffix; numeric keys are also rejected.

A larger capacity may consume more memory and does not replace a draining strategy. A small capacity applies backpressure earlier.

Passing channels through `opts.channels`

A channel is a special shared resource. It never crosses the JSON serialization used for `worker.args` and cannot be sent as a message. Pass it explicitly through the `channels` field:

```
local tasks = assert(babet.workers.channel({ capacity = 16 }))
local results = assert(babet.workers.channel({ capacity = 16 }))

local job = assert(babet.workers.spawn([[
    local ok, task = worker.channels.tasks:recv(2)
    if not ok then return task end

    assert(worker.channels.results:send({
        id = task.id,
        result = task.value * 2,
    }, 2))

    return true
]], nil, {
    channels = {
        tasks = tasks,
        results = results,
    },
}))
```

In the new Lua state:

- `worker.channels` always exists and is an empty table when no channel was passed;
- each `opts.channels` key becomes a field of `worker.channels`;
- several names may reference the same channel;
- the parent and all workers hold distinct userdata pointing to the same C++ queue;
- names must be non-empty UTF-8 strings containing no NUL;
- every value must be a channel created by `babet.workers.channel()`.

A channel placed in `args` remains non-serializable userdata and is rejected.

When `setenv`, `chdir` and `os.setlocale` become locked

After validating types and capacities, the first `spawn` marks the process as having used workers. From then on:

- `babet.setenv(...)` is permanently rejected;
- `babet.chdir(...)` is permanently rejected;
- `os.setlocale(locale, category)` raises a Lua error when `locale` is not `nil`, including `""` and the already active locale;
- the restriction remains after `join()`;
- it remains even if `args` serialization, queue initialization, or `pthread_create` later fails.

A call rejected **before** this mark — for example `spawn(42)` or a non-integer capacity — does not lock the environment.

The first GTK loading attempt through `gui.available()` or `gui.init()` triggers the same freeze before entering the native library, even if loading or initialization fails. This also covers GTK-created threads without a `workers.spawn` call; see the GUI contract.

Configure the current directory, environment and locale before the first worker or GUI loading call. `os.setlocale()` and `os.setlocale(nil, category)` remain available as queries in every state. Before the freeze, the standard Lua contract is preserved: a locale name on success, or one `nil` for an unavailable locale. Argument validation remains the same as in Lua.

The freeze also survives destruction and recreation of an embedding context. These rules guard Babet's Lua entry points; native plugins and embedding hosts remain responsible for their own process-wide mutations and threads.

Transferable values

The internal transport format is JSON. The same rules apply to:

- `args`;
- messages in either direction;
- messages sent through channels;
- the worker's first return value.

Scalars and strings

Lua value	Transferable	Note
`nil`	yes	becomes JSON `null`, then `nil` again
boolean	yes	identity preserved
integer	yes	normally restored as a Lua integer
finite floating-point number	yes	`NaN`, `+Inf`, and `-Inf` rejected
text string	yes	must pass the UTF-8 validator and contain no NUL

Unlike several filesystem and socket APIs, worker messages are **not binary-safe**. A string containing `\0` or bytes rejected by the UTF-8 validator cannot be transferred.

List and object tables

A table is transferable in one of two shapes.

Non-empty dense list

```
{ "a", "b", "c" }
```

All keys must be consecutive integers `1..n`, with no holes and no other key.

String-keyed object

```
{
    name = "alice",
    age = 30,
}
```

All keys must be strings accepted by the text validator.

An empty table is serialized as an **empty object** `{}`, not an empty array. There is no equivalent of `babet.json.empty_array` in the worker transport.

Sparse or mixed tables are rejected:

```
{ [1] = "a", [3] = "c" }      -- hole
{ [1] = "a", label = "x" }   -- list + map
{ [0] = "zero" }              -- numeric key outside 1..n
```

Rejected values

These values do not cross Lua states:

- functions and closures;
- userdata: channel, socket, SQLite statement, watcher, job, file, and so on;
- the SQLite-only lightuserdata sentinel `babet.sqlite.NULL`;
- Lua coroutines/threads;
- cyclic tables;
- sparse or mixed tables;
- non-string keys in object tables;
- `NaN` or infinite numbers;
- strings with NUL or bytes rejected as UTF-8;
- structures beyond the depth, node, or byte budgets.

Pass a path, URL, or serializable configuration instead of the system object itself. The worker then opens its own resource.

The NULL sentinel is rejected with a diagnostic that explicitly names `babet.sqlite.NULL`. Translate it to a domain value such as `{ kind = "sql-null" }` if that intent must cross a worker or channel boundary.

```
-- Wrong: socket is userdata.
babet.workers.spawn(code, { socket = peer })

-- Correct: pass connection parameters.
babet.workers.spawn(code, {
    host = "127.0.0.1",
    port = 9000,
})
```

Copies, identity, depth, and budgets

Transport copies values:

- no Lua reference is shared;
- metatables are lost;
- two references to the same subtable become two separate tables;
- cycles are rejected;
- maximum nesting is 32 levels according to the internal serialization counter;
- one operation may expand to at most **1,000,000 JSON values**;
- the estimated representation must remain within a conservative **64 MiB** budget.

```lua
local shared = { value = 1 }
local args = { a = shared, b = shared }

-- In the worker, worker.args.a and worker.args.b have equal content,
-- but are not the same table.
```

The depth limit protects the C++ and Lua stacks. The two additional budgets prevent a small Lua structure from expanding into an exponential amount of data when the same subtable or string is referenced repeatedly.

Every expanded value consumes exactly one node and a fixed cost of 32 bytes. Every string occurrence, including an object key, also consumes an upper bound of `6 × byte_length + 2`. The factor of six covers the worst JSON escaping of a control byte. The two ceilings deliberately overlap: together they bound both the number of objects in the JSON DOM and the expanded text payload.

The limits apply independently to each operation: `spawn` arguments, final result, `job:send`, `worker.send`, or `channel:send`. A rejected value is not added to the queue. The identical receive-side check validates message consistency; the primary memory bound is applied by the sender before strings are copied and before the internal JSON is published.

These ceilings are part of the public contract. Lowering them in a later release could reject transfers that used to succeed and should be announced as an incompatible change.

Final worker result

When the chunk completes normally, only its **first** return value is serialized.

```lua
return "first", "second", "third"
```

The parent receives only `"first"`.

Return a table to transport several fields:

```lua
return {
    value = 42,
    elapsed = 0.12,
}
```

With no `return`, or `return nil`, the business result is `nil`.

An uncaught Lua error becomes the `error` state. The transferred text is a useful diagnostic, but Babet does not automatically add a traceback. A worker can use `xpcall(..., debug.traceback)` when it wants to build one.

`status()` — observe without consuming

```lua
local state = job:status()
```

`status()` returns exactly one of these strings:

State	Meaning
`"running"`	the chunk or its cleanup is still running
`"done"`	the worker completed normally
`"error"`	the worker completed with an error

The method does not block, join the pthread, or consume the result. It remains usable before and after `join()`:

```lua
while job:status() == "running" do
    update_interface()
    babet.sleep(10, "ms")
```

```lua
end

local ok, result = job:join()
assert(ok, result)
assert(job:status() == "done")
```

`status()` intentionally does not expose a separate cancelling state: until the chunk ends, it remains `"running"`.

`done()` — non-consuming boolean test

```lua
if job:done() then
    print("the worker has finished")
end
```

`done()` returns `false` while the internal status is `"running"`, then `true` for both `"done"` and `"error"`. It does not join the pthread, deserialize the result, or consume it. It is suitable for event loops that only need to know whether a final `join()` or `poll()` can now be attempted.

`join(timeout?)` — wait and consume

```lua
local ok, value_or_reason = job:join(timeout?)
```

The timeout uses the same seconds contract as message queues:

- omitted or `nil`: wait indefinitely;
- `0`: immediate test;
- finite positive number up to `86400`: bounded wait;
- negative value, NaN, infinity, numeric string, or an extra argument: raises a Lua error.

There are three return states:

```lua
true, result       -- normal completion, result consumed
false, err         -- failed completion, error consumed
nil, "timeout"     -- worker still active, nothing consumed
```

Retry after an expiry:

```lua
local ok, value = job:join(0.05)

if ok == nil then
    assert(value == "timeout")
    print("worker is still running")

    -- The same job remains fully usable.
    ok, value = job:join(2)
end

if ok then
    print("result", value)
else
    io.stderr:write("worker: ", value, "\n")
end
```

A timeout:

- closes no queue;
- requests no cancellation;
- does not join the pthread;
- consumes neither result nor error;
- still allows `status`, `send`, `recv`, `close`, `cancel`, and another `join`.

The timeout bounds only this call to `join()`. If the worker remains active and its last reference is later collected, notably while the Lua state is closing, its `__gc` must still perform an unbounded `pthread_join()` and can therefore block program termination.

Even when the worker legitimately returns `nil`, success is distinguishable by `ok == true`.

`poll()` — check and consume

```
local state, value = job:poll()
```

`state`	`value`	Consumed
`"running"`	`nil`	no
`"done"`	result, possibly `nil`	yes
`"error"`	message	yes

```
while true do
    local state, value = job:poll()
    if state == "running" then
        babet.sleep(10, "ms")
    elseif state == "done" then
        print("completed", value)
        break
    else
        io.stderr:write("worker: ", value, "\n")
        break
    end
end
```

Once the state is no longer `running`, `poll()` quickly joins the already completed pthread and consumes the result.

Do not call `join()` after completed `poll()`

This sequence is wrong:

```
local state = job:poll()
if state ~= "running" then
    local ok, result = job:join() -- result already consumed
end
```

Use both values from `poll()` directly:

```
local state, value = job:poll()
if state == "done" then
    use_result(value)
elseif state == "error" then
    report_error(value)
end
```

Alternatively, use `poll()` only while it returns `running`, and design another mechanism that tells code when to call `join()` without first consuming the result. In most event loops, polling through the final result is simpler.

Parent-to-worker messages

```
local ok, err = job:send(value, timeout?)
```

`job:send` serializes `value`, then appends it to the worker inbox in FIFO order.

Results:

- `(true, nil)`: message accepted;
- `(false, "full")`: full queue with timeout `0`;
- `(false, "timeout")`: queue stayed full until the deadline;
- `(false, "closed")`: inbox closed;
- `(false, "cancelled")`: cancellation requested through `cancel()`;
- `(nil, err)`: non-transferable value or internal serialization failure.

```
local ok, err = job:send({ command = "scan", path = "/tmp" }, 1)
if not ok then
    io.stderr:write("send: ", err, "\n")
end
```

Success means the message entered the queue, not that the worker has already processed it.

Worker-to-parent messages

```
local ok, value_or_err = job:recv(timeout?)
```

`job:recv` removes the oldest message from the outbox.

Results:

- `(true, value)`: message received; `value` may be `nil`;
- `(false, "empty")`: empty queue with timeout `0`;
- `(false, "timeout")`: no message before the deadline;
- `(false, "closed")`: queue closed and fully drained.

When a queue is closed but still contains messages, those messages are returned before `"closed"`.

```
while true do
    local ok, message = job:recv(0)
    if ok then
        process(message)
    elseif message == "empty" then
        break
    elseif message == "closed" then
        break
    else
        error(message)
    end
end
```

`worker.send`, `worker.recv`, and `worker.cancelled`

Inside the chunk:

```
local ok, message_or_err = worker.recv(timeout?)
local ok, err = worker.send(value, timeout?)
local cancelled = worker.cancelled()
```

`worker.recv` reads the inbox filled by `job:send`. `worker.send` writes the outbox read by `job:recv`. `worker.cancelled()` reads, without blocking, the flag set by `job:cancel()`.

After cancellation:

- `worker.cancelled()` returns `true`;
- the next `worker.recv()` returns `(false, "cancelled")` and no longer delivers commands still waiting in the inbox; when cancellation becomes visible immediately after a successful queue pop, that extracted command is deliberately discarded rather than delivered after the protocol boundary; a command already returned to Lua may naturally still be processing;

- `worker.send()` remains available for a final result, diagnostic, or stop acknowledgement when the outbox already has space; a send blocked on a full outbox is awakened and returns `(false, "cancelled")`;
- the chunk decides when and how to finish.

FIFO ordering, capacities, timeouts, and closing remain symmetric outside this cancellation case.

Serialization-error convention differs slightly:

- `job:send` returns `(nil, err)`;
- `worker.send` returns `(false, err)`.

This keeps a boolean status convention inside worker code.

```
local job = assert(babet.workers.spawn([[
    while not worker.cancelled() do
        local ok, message = worker.recv(0.2)

        if ok then
            process(message)
        elseif message == "cancelled" then
            break
        elseif message ~= "timeout" then
            return { error = message }
        end
    end

    worker.send({ stopped = true })
    return "cancelled"
]]))
```

Direct shared channels

A channel is a bounded queue shared by the parent and any number of workers. Unlike a job inbox or outbox, it is not owned by one worker: every handle for the same channel may produce or consume messages.

Channels are thread-safe, bounded by message count, multi-producer, multi-consumer, FIFO by effective insertion order, and explicitly closed with draining of already queued messages.

Creating a channel and choosing capacity

```
local channel, err = babet.workers.channel(opts?)
```

The only option is `capacity`, which defaults to 64 messages and must be a strict Lua integer between 1 and 1,000,000. Capacity counts messages, not bytes.

```
local tasks = assert(babet.workers.channel({ capacity = 16 }))
```

Unknown options, non-string keys, names containing NUL, and out-of-range capacities raise a Lua error. Allocation or initialization failures return `(nil, err)`.

`send` and `recv`

```
local ok, err = channel:send(value, timeout?)
local ok, value_or_reason = channel:recv(timeout?)
```

`send` returns `(true, nil)`, `(false, "full"|"timeout"|"closed"|"cancelled")`, or `(nil, err)` when serialization or an internal operation fails.

`recv` returns `(true, value)`, `(false, "empty"|"timeout"|"closed"|"cancelled")`, or `(nil, err)` for an internal deserialization anomaly. A `nil` message is unambiguous because the first result is `true`.

```lua
local channel = assert(babet.workers.channel({ capacity = 1 }))
assert(channel:send(nil, 0))

local ok, value = channel:recv(0)
assert(ok == true and value == nil)
```

Channels use the same JSON transfer rules as workers. Channel handles and other userdata cannot themselves be sent as messages; neither can `babet.sqlite.NULL`. A rejected send publishes nothing to the channel. For binary Selenium screenshots, prefer a file path, or keep Base64 text and decode it with `babet.base64.decode()` at the consumer.

Inside a worker, `job:cancel()` also wakes a blocked `channel:send()` or `channel:recv()` for **that worker only**. The call returns `(false, "cancelled")` without closing the shared channel or disturbing other participants. New channel sends and receives from that cancelled worker also return `"cancelled"`; use `worker.send()` for a final acknowledgement to the parent.

`close` and `is_closed`

`close()` is global and idempotent. It rejects future sends, makes `is_closed()` return `true`, wakes blocked senders and receivers, and preserves already queued messages. Once those messages are drained, `recv()` returns `(false, "closed")`.

```lua
local channel = assert(babet.workers.channel({ capacity = 2 }))
assert(channel:send("one"))
assert(channel:send("two"))
assert(channel:close())
assert(channel:is_closed())

assert(select(2, channel:recv()) == "one")
assert(select(2, channel:recv()) == "two")
local ok, reason = channel:recv()
assert(not ok and reason == "closed")
```

Concurrency, ordering, and lifetime

All handles for a channel reference the same C++ object, while each Lua state owns a separate userdata. Collecting one local handle does not close the channel for the others. Explicit `close()` affects every handle. When the last reference disappears, Babet closes the queue, wakes waiters, and destroys any remaining messages.

FIFO refers to effective insertion order. Concurrent sends from different producers have no deterministic relative order, while sequential sends from a single producer remain ordered.

Capacity does not limit individual message size. JSON serialization creates copies, so large messages can consume substantial memory. Publishing binary data atomically to a file and sending its path is usually preferable.

Timeouts and failure reasons

All four message methods and `join(timeout?)` share the same timeout validation rules:

Lua `timeout`	Behavior
omitted or `nil`	wait indefinitely
`0`	immediate, non-blocking attempt

Lua `timeout`	Behavior
`0 < t <= 86400`	wait at most `t` seconds
negative, NaN, Inf, or `> 86400`	raises a Lua error
other type, including `"1"`	raises a Lua error

Internal resolution is one millisecond. A strictly positive duration is rounded up: `0.0005` second waits at least 1 ms and is not turned into non-blocking mode.

Queue reasons are:

Reason	Meaning
`"full"`	immediate send, queue full
`"empty"`	immediate receive, queue empty
`"timeout"`	positive deadline reached
`"closed"`	send to closed queue, or receive from closed and empty queue
`"cancelled"`	`job:send`, `worker.recv`, or a channel method called in a cancelled worker

Internal diagnostics such as `"out of memory"` or `"internal mutex error"` may exceptionally appear instead.

`close()` and queue lifecycle

```lua
local ok, err = job:close()
```

`job:close()`:

- closes only the parent-to-worker **inbox**;
- is idempotent;
- returns `(true, nil)`;
- wakes a waiting `worker.recv()`, which gets `(false, "closed")` once already queued messages are drained;
- makes future `job:send()` calls fail with `(false, "closed")`;
- leaves the outbox open so the parent can still receive final worker messages.

It does not forcibly terminate the thread and does not consume its final result.

The worker automatically closes **both** queues when it completes, whether successfully or with an error. The parent can still drain messages already in the outbox before receiving `"closed"`.

To cleanly finish a worker that waits for commands:

```lua
assert(job:close())
local ok, result = job:join()
```

The worker must treat `worker.recv() == false, "closed"` as an exit condition.

`cancel()` and cooperative cancellation

```lua
local ok, err = job:cancel()
```

`job:cancel()`:

- sets an atomic flag visible through `worker.cancelled()`;
- closes the inbox to wake a blocked `worker.recv()` immediately;
- also wakes the channel `send` or `recv` currently blocked in that worker, without closing the global channel;

- makes future `job:send()` calls return `(false, "cancelled")`;
- leaves the outbox open, allows a final message when space is immediately available, and wakes a `worker.send()` blocked on a full outbox with `(false, "cancelled")`;
- does not join the thread;
- does not consume the final result;
- is idempotent and always returns `(true, nil)` for a valid job.

```
assert(job:cancel())

local got, final = job:recv(1)
if got then
    print("final message", final)
end

local ok, result = job:join(2)
if ok == nil then
    error("worker did not cooperate before the deadline")
end
assert(ok, result)
```

This mechanism never uses `pthread_cancel()` and never abruptly interrupts Lua, SQLite, a C++ lock, or a transaction. A worker must therefore check `worker.cancelled()` or regularly return to `worker.recv()`.

A worker blocked in a non-interruptible system call or in a loop that never checks the flag can remain active. Babet channel waits and a `worker.send()` waiting for space in the outbox are awakened by cancellation. `join(timeout)` keeps the parent bounded in every other case, but Babet does not force termination.

`close()` and `cancel()` differ:

- `close()` means “no more commands” and allows already queued commands to drain; `worker.recv()` eventually returns `"closed"`;
- `cancel()` means “abandon current work as soon as possible”; queued inbox commands are no longer delivered and `worker.recv()` returns `"cancelled"`. A command already removed before cancellation may still be processing: the stop remains cooperative.

Module loading and worker environment

Each thread creates a fresh Lua state and:

- opens the standard Lua libraries;
- registers the complete `babet` namespace;
- exposes bundled modules through `require`;
- configures user-module loading like the parent project, in directory or embedded mode;
- exposes `worker.args`, `worker.channels`, `worker.send`, `worker.recv`, and `worker.cancelled`;
- does not copy the parent’s global `arg` table: `arg == nil` in the worker.

```
local job = assert(babet.workers.spawn([[
    local inspect = require("inspect")
    local mymod = require("mymod")
    return inspect({ answer = mymod.answer() })
]]))
```

Lua states are isolated, but threads belong to the same process:

- same PID;
- same process-wide current directory;
- same process-wide environment;
- same process-wide C locale;
- same filesystem and accessible external resources;

- Lua memory measured separately in each state.

Babet rejects cwd, environment and locale mutations through these Lua APIs after the first `spawn` or GTK loading attempt.

`os.exit(...)` raises a Lua error inside every worker, including its coroutines and nested workers. An uncaught error makes `job:join()` return `(false, err)`; `pcall` can catch it without closing the worker's Lua state. Use `return` to finish a worker normally. This also applies to `require("os").exit` and saved aliases. In the CLI and generated executables, `os.exit` retains Lua's argument rules, restores the UI and terminates without native `atexit` callbacks. Without `close=true` it does not join workers; a truthy `close` requests Lua closure, which can wait for non-cooperative native calls in workers. See the exit contract, including the embedding policy.

Available CPU count

```
local count = babet.workers.cpu_count()
assert(math.type(count) == "integer" and count >= 1)
```

`cpu_count()` returns the number of CPUs available to the current process. On Linux, Babet first reads the effective affinity with `sched_getaffinity()`, so CPU sets imposed on the process are respected. If that information is unavailable, `_SC_NPROCESSORS_ONLN` is used, with a final fallback to `1`.

The function accepts no arguments. It is also the default size source for `workers.pool()`.

Reusable bounded pool

`workers.pool()` creates a fixed number of persistent workers. Each worker creates one pthread and one `lua_State`, then processes several tasks received through an internal channel. The pool avoids a complete `workers.spawn()` for every small operation while keeping strict bounds on concurrency and outstanding tasks.

The pool is implemented as embedded Lua on top of the native `workers.spawn()` and `workers.channel()` primitives. It therefore keeps the same serialization, depth, memory-budget, and cooperative-cancellation limits.

Creation and options

```
local pool, err = babet.workers.pool({
    size = math.min(babet.workers.cpu_count(), 8),
    queue_capacity = 64,
    channels = {
        progress = progress_channel,
    },
})
assert(pool, err)
```

Accepted options:

Option	Default	Contract
`size`	`min(cpu_count(), 1024)`	integer from `1` to `1024`
`queue_capacity`	`max(64, size * 4)`	integer from `1` to `1,000,000`
`channels`	none	name-to-channel table shared with every task

`__babet_pool_tasks` and `__babet_pool_results` are reserved for the two internal channels. Unknown options and invalid names are rejected immediately.

The maximum number of accepted but not yet collected tasks is:

`min(queue_capacity + size, 1,000,000)`

This bound includes running tasks and tasks still waiting in the queue.

Submission and tasks

```
local task, err = pool:submit([[
    return worker.args.left + worker.args.right
]], { left = 20, right = 22 }, 1.0)
assert(task, err)

local ok, result = task:join(2)
assert(ok and result == 42)
```

`submit(code, args?, timeout?)` accepts the same text chunk and serializable argument table as `workers.spawn()`. The timeout applies to admission into the pool: submission may need to wait for a slot in the outstanding-task bound or in the work channel.

A task exposes:

- `task:done()`: non-consuming boolean;
- `task:status()`: `"running"`, `"done"`, or `"error"`;
- `task:poll()`: non-blocking check that consumes a completed result;
- `task:join(timeout?)`: wait for and consume the result.

As with a normal job, only the first return value crosses the state boundary and the result can be consumed once. A task load error or Lua exception becomes an error for that task without stopping the persistent worker. An unserializable return value is also converted into a task error, after which the worker keeps processing later tasks.

State isolation and reuse

Each task receives a fresh global table whose `_G` points to itself. Ordinary global assignment therefore does not leak to the next task:

```
local first = assert(pool:submit("temporary = 42; return temporary"))
local second = assert(pool:submit("return temporary"))

assert(select(2, first:join()) == 42)
assert(select(2, second:join()) == nil)
```

Libraries and the `package.loaded` cache still belong to the persistent worker state. Explicitly mutating a shared module table, `package.loaded`, a C module's registry state, or an external resource can therefore be visible to a later task on the same worker. The pool isolates ordinary globals; it does not create a complete Lua state per task.

Inside a pool task, `worker` intentionally exposes only:

```
worker.args
worker.channels
worker.cancelled()
```

The private `worker.send()` and `worker.recv()` inbox/outbox APIs are not part of the pool-task contract. Use `opts.channels` for additional persistent communication.

Backpressure and timeouts

```
local pool = assert(babet.workers.pool({
    size = 1,
    queue_capacity = 1,
```

```
}))

local a = assert(pool:submit("babet.sleep(1); return 'a'"))
local b = assert(pool:submit("babet.sleep(1); return 'b'"))
local c, reason = pool:submit("return 'c'", nil, 0)
assert(c == nil and reason == "timeout")
```

With one worker and one queue slot, at most two tasks are outstanding: one running and one waiting. A zero timeout is non-blocking. A positive value up to `86400` uses a monotonic deadline and may return `"timeout"`. An omitted timeout waits while the workers remain healthy.

The pool collects results while waiting to submit or close. The result channel capacity matches the outstanding-task bound, so a worker does not remain blocked merely because the parent has not joined every task yet.

Close, cancellation, and `join`

`pool:close(timeout?)` rejects new submissions and enqueues one stop marker per worker **after** all accepted tasks. FIFO ordering therefore guarantees normal processing before thread shutdown.

`pool:join(timeout?)` automatically calls `close()` when needed, collects all pending results, and joins every persistent worker. A timeout consumes neither pending task results nor already completed worker joins; the same pool can be joined again to finish the cleanup:

```
local task = assert(pool:submit("return 42"))
assert(pool:join(5))
assert(select(2, task:join(0)) == 42)
```

`pool:cancel()` immediately closes admission, marks outstanding tasks as `"cancelled"`, closes the work channel, and requests cancellation of every worker. A running task can stop only by returning or checking `worker.cancelled()`. There is no `pthread_cancel()` and no forced shutdown.

A joined pool cannot be joined a second time. `pool:stats()` reports `size`, `queue_capacity`, `max_pending`, `pending`, `accepting`, `closing`, `cancelled`, and `joined`.

Shared channels in tasks

```
local progress = assert(babet.workers.channel({ capacity = 16 }))
local pool = assert(babet.workers.pool({
    size = 2,
    channels = { progress = progress },
}))

local task = assert(pool:submit([[
    assert(worker.channels.progress:send({ percent = 100 }))
    return "done"
]]))

local received, message = progress:recv(2)
assert(received and message.percent == 100)
assert(task:join(2))
assert(pool:join(2))
```

Handles are shared with all pool workers and keep the normal bounded FIFO multi-producer/multi-consumer channel contract. The pool never automatically closes user-provided channels.

Complete examples

Simple calculation with `join`

```
local job, err = babet.workers.spawn([[
    return 21 * 2
]])
assert(job, err)

local ok, result = job:join()
assert(ok, result)
print(result) -- 42
```

Structured arguments

```
local job = assert(babet.workers.spawn([[
    local total = 0
    for _, value in ipairs(worker.args.values) do
        total = total + value
    end
    return {
        name = worker.args.name,
        total = total,
    }
]], {
    name = "batch A",
    values = { 10, 20, 12 },
}))

local ok, result = job:join()
assert(ok, result)
print(result.name, result.total)
```

Non-blocking progress with `poll`

```
local job = assert(babet.workers.spawn([[
    babet.sleep(200, "ms")
    return "ready"
]]))

while true do
    local state, value = job:poll()

    if state == "running" then
        update_ui()
        babet.sleep(10, "ms")
    elseif state == "done" then
        print(value)
        break
    else
        error(value)
    end
end
```

Do not call `join()` after the `done` or `error` branch: `poll()` has already consumed the result.

Persistent request/response worker

```
local job = assert(babet.workers.spawn([[
    while true do
        local ok, request = worker.recv()
        if not ok then
            return "parent closed inbox"
        end

        if request.op == "stop" then
            return "stopped"
        elseif request.op == "square" then
            local sent, err = worker.send({
                id = request.id,
                value = request.value * request.value,
            })
            if not sent then
                return { error = err }
            end
        end
    end
]], nil, {
    inbox_capacity = 16,
    outbox_capacity = 16,
}))

for i = 1, 5 do
    assert(job:send({ id = i, op = "square", value = i }))
end

for _ = 1, 5 do
    local ok, response = job:recv(2)
    assert(ok, response)
    print(response.id, response.value)
end

assert(job:send({ op = "stop" }))
local ok, result = job:join()
assert(ok, result)
print(result)
```

Sending and receiving `nil`

The status boolean distinguishes a real `nil` message from an empty queue.

```
local job = assert(babet.workers.spawn([[
    local ok, value = worker.recv()
    assert(ok)
    assert(value == nil)

    assert(worker.send(nil))
    return "done"
]]))

assert(job:send(nil))

local got, value = job:recv()
```

```
assert(got == true)
assert(value == nil)

local ok, result = job:join()
assert(ok and result == "done")
```

Close, then drain the outbox

```
local job = assert(babet.workers.spawn([[
    assert(worker.send("ready"))

    local ok, reason = worker.recv()
    assert(not ok and reason == "closed")

    assert(worker.send("closing"))
    return "finished"
]]))

local ok, first = job:recv(1)
assert(ok and first == "ready")

assert(job:close()) -- closes the inbox only

local got, last = job:recv(1)
assert(got and last == "closing")

local joined, result = job:join()
assert(joined and result == "finished")
```

Backpressure and timeout

```
local job = assert(babet.workers.spawn([[
    babet.sleep(500, "ms")
    while true do
        local ok, message = worker.recv()
        if not ok then return "closed" end
        if message == "stop" then return "done" end
    end
]], nil, {
    inbox_capacity = 2,
}))

assert(job:send("one", 0))
assert(job:send("two", 0))

local ok, reason = job:send("three", 0)
assert(ok == false and reason == "full")

local waited, why = job:send("three", 0.1)
assert(waited == false and why == "timeout")

job:close()
job:join()
```

Parent to worker through a channel

```
local tasks = assert(babet.workers.channel({ capacity = 8 }))

local job = assert(babet.workers.spawn([[
    local ok, task = worker.channels.tasks:recv(2)
    assert(ok, task)
    return task.left + task.right
]], nil, {
    channels = { tasks = tasks },
}))

assert(tasks:send({ left = 20, right = 22 }, 2))
local ok, result = job:join(2)
assert(ok and result == 42)
```

Worker to worker without parent relay

```
local tasks = assert(babet.workers.channel({ capacity = 16 }))
local results = assert(babet.workers.channel({ capacity = 16 }))

local producer = assert(babet.workers.spawn([[
    for i = 1, 10 do
        assert(worker.channels.tasks:send({ id = i, value = i * 10 }, 2))
    end
    return "producer done"
]], nil, { channels = { tasks = tasks } }))

local consumer = assert(babet.workers.spawn([[
    for _ = 1, 10 do
        local ok, task = worker.channels.tasks:recv(2)
        assert(ok, task)
        assert(worker.channels.results:send({
            id = task.id,
            result = task.value * 2,
        }, 2))
    end
    return "consumer done"
]], nil, { channels = { tasks = tasks, results = results } }))

for expected = 1, 10 do
    local ok, item = results:recv(2)
    assert(ok and item.id == expected and item.result == expected * 20)
end

assert(producer:join(2))
assert(consumer:join(2))
```

Producer-to-consumer messages never pass through the parent.

Multiple producers and consumers

The same channel can be passed to several producers and consumers. Each message is removed by exactly one consumer, but distribution among consumers depends on thread scheduling and is not predictable. Close the task channel only after all producers have completed, then let consumers drain it until they receive `"closed"`.

Native cooperative cancellation

```
local job = assert(babet.workers.spawn([[
    local total = 0

    for i = 1, worker.args.limit do
        total = total + expensive_step(i)

        if i % 1000 == 0 and worker.cancelled() then
            worker.send({
                stopped = true,
                partial = total,
            })
            return {
                cancelled = true,
                partial = total,
            }
        end
    end

    return { cancelled = false, total = total }
]], { limit = 1000000 }))

-- Later:
assert(job:cancel())

-- The outbox remains drainable after cancel().
local got, progress = job:recv(0.5)
if got then
    print("partial", progress.partial)
end

local ok, result = job:join(2)
if ok == nil then
    error("cancellation was not observed within two seconds")
end
assert(ok, result)
```

Combined `worker.recv()` and flag example:

```
local job = assert(babet.workers.spawn([[
    while not worker.cancelled() do
        local ok, command = worker.recv(0.1)

        if ok then
            execute(command)
        elseif command == "cancelled" then
            break
        elseif command ~= "timeout" then
            return { error = command }
        end
    end

    worker.send({ stopped = true })
    return "cancelled"
]]))

assert(job:cancel())
```

```lua
local ok, result = job:join(2)
assert(ok and result == "cancelled")
```

Cancellation remains cooperative: it does not arbitrarily interrupt a system call or code that never checks the flag.

Native pool for many tasks

```lua
local pool = assert(babet.workers.pool({
    size = math.min(4, babet.workers.cpu_count()),
    queue_capacity = 16,
}))

local tasks = {}
for index = 1, 100 do
    tasks[index] = assert(pool:submit([[
        return worker.args.value * worker.args.value
    ]], { value = index }))
end

assert(pool:close(5))

local results = {}
for index, task in ipairs(tasks) do
    local ok, value = task:join(5)
    assert(ok, value)
    results[index] = value
end

assert(pool:join(5))
assert(results[10] == 100)
```

The pthread count remains fixed across all one hundred tasks. `close()` may be called before collecting results: the pool processes every accepted submission, while each `task:join()` retrieves its result by identifier.

Deadlocks to avoid

Worker waits for inbox while parent calls `join()`

```lua
-- Worker
local ok, message = worker.recv() -- waits indefinitely

-- Parent
job:join() -- waits for worker
```

Both sides wait. Close the inbox or send a command first:

```lua
job:close()
job:join()
```

Worker blocks on a full outbox

If the worker calls `worker.send()` with no timeout, the outbox is full, and the parent calls `join()` without draining it, neither side can progress.

Solutions:

- parent regularly calls `job:recv()`;
- worker uses a finite timeout;

- outbox capacity matches the protocol;
- message and join phases are clearly separated;
- during cancellation, call `job:cancel()` before the bounded `join()`: cancellation now wakes the blocked outbox send without discarding messages that were already queued.

Producers blocked on a full channel

A bounded channel can create the same wait cycle as a full outbox: producers wait for space while the parent calls `join()` before consumers have drained the queue. Define who consumes, who closes the channel, and when. Long-running protocols should use finite timeouts or a shutdown strategy so one failed participant cannot block all others.

Forgetting that `poll()` consumes

Code that waits for `poll() ~= "running"` and then calls `join()` does not deadlock, but loses the real result and gets `already consumed`. Use both values from `poll()`.

Potentially blocking GC

Dropping the last reference to an active worker can run `__gc`, which closes queues and then waits for the thread. Queue waits are released, but GC cannot interrupt infinite computation or an unbounded external syscall.

Error contract

Lua errors raised for invalid use

These include:

- missing or non-string `code`;
- `args` other than table or `nil`;
- `opts` other than table or `nil`;
- non-numeric, non-integer, or out-of-range capacities;
- non-numeric, negative, non-finite, or greater-than-86,400-second timeout;
- method call on a value that is not the expected job or channel;
- invalid `workers.channel()` arity, options, or capacity;
- invalid `opts.channels`, empty/non-UTF-8/NUL name, or non-channel value.

```
local ok, err = pcall(function()
    babet.workers.spawn(42)
end)
assert(not ok)
```

Failures returned by `spawn`

`spawn` returns `(nil, err)` for runtime failures before or during creation:

- non-transferable `args`;
- serialization-budget exhaustion;
- JSON serialization failure;
- queue or completion-signal initialization failure;
- `pthread_create` failure.

```
local job, err = babet.workers.spawn("return 1", {
    fn = function() end,
})
assert(job == nil)
```

Final worker errors

`join()` returns `(false, err)` and `poll()` returns `("error", err)` for:

- invalid Lua syntax;
- uncaught Lua error;
- non-transferable return value;
- caught internal C++ exception;
- internal result deserialization corruption or failure.

The diagnostic does not automatically contain a complete traceback.

Message serialization

- `job:send`: `(nil, err)` when the value cannot be serialized;
- `worker.send`: `(false, err)` for the same case;
- `channel:send`: `(nil, err)` for the same case;
- `job:recv` and `worker.recv`: `(false, err)` on an internal anomaly;
- `channel:recv`: `(nil, err)` on an internal anomaly.

Garbage collection and destruction

The userdata has a safety-net `__gc`. When collected:

1. the cancellation flag is set;
2. inbox and outbox are closed;
3. queue waiters are awakened;
4. Babet calls `pthread_join` when needed;
5. pthread primitives, the completion signal, and internal strings are destroyed.

This prevents freeing state still used by a thread. It can nevertheless block when the worker cannot terminate.

Do not use GC as the normal synchronization mechanism. Keep the job, finish the protocol, then call `join()` or consume the result through `poll()`.

A pool is a Lua object that owns several worker userdata values. Dropping the last pool and task references without `pool:join()` eventually delegates every thread to those userdata finalizers and can therefore block collection or Lua state shutdown. Normal code should explicitly `close()` or `cancel()`, then retry `pool:join()` until it completes.

Channel handles also have a `__gc`, but its scope is local: it only releases the reference owned by that Lua state. It never replaces `channel:close()` and does not close the resource while other handles remain.

`tostring(channel)` returns `WorkerChannel(open)` or `WorkerChannel(closed)` from the instantaneous global state.

`tostring(job)` returns an indication such as:

```
Worker(running)
Worker(done)
Worker(error)
```

This reflects instantaneous internal status, not whether the result has already been consumed.

Signals, environment, and concurrency

Signals

All six signals managed by `babet.signal` are blocked in worker pthreads. `babet.signal.handle`, `ignore`, and `default` raise inside a worker. The main thread remains the only owner of POSIX

callbacks.

`os.execute` and `io.popen` inside a worker

In worker states, both functions launch `/bin/sh -c` with an empty child signal mask. Signals blocked to protect worker threads therefore remain unblocked in the command even when the shell normally preserves its inherited mask. The worker's own mask is unchanged. Explicitly ignored dispositions remain inheritable under the ordinary rules for executing programs.

Launching and waiting for `os.execute` in a worker does not change the global `SIGINT` or `SIGQUIT` dispositions, so it does not temporarily disable the main thread's signal handler.

Lua contracts remain available: `os.execute()` returns a boolean indicating shell availability; a terminated command returns `true` or `nil`, followed by `"exit"` or `"signal"` and the corresponding code. A native launch or wait failure returns `nil, message, errno`. `io.popen` accepts `"r"` and `"w"` and returns an ordinary Lua file, or `nil, message, errno` on launch failure. Its methods, `io.type`, `close`, `<close>` and GC continue to work; closing waits for and reaps the child. Pipe ends retained by the parent are not inherited across another `exec`.

These calls remain blocking. Neither `job:cancel()` nor a `job:join(timeout)` timeout automatically terminates the command; closing an `io.popen` file, including during GC, can wait for the command to finish. The main Lua state's functions retain their standard implementation.

Environment and cwd

`setenv` and `chdir` change process-wide state that cannot safely be mutated while other threads use it. Babet therefore rejects both after the first `spawn` or GTK loading attempt.

External resources

Two workers can open distinct files, SQLite databases, or sockets. When they target the same external resource, worker serialization provides no automatic lock.

For SQLite, each worker should open its own connection. `wal = true` and a suitable `busy_timeout` can improve concurrency, but do not replace sound transaction design.

Cost

Each `spawn` creates:

- a pthread;
- a complete Lua state;
- Babet modules;
- two queues and their buffers;
- handles for explicitly passed channels;
- JSON copies of transferred data.

Avoid a worker for tiny operations. Batch work or use a few persistent workers when the protocol allows it.

Features not provided

The module currently does not provide:

- forced worker termination;
- individual forced cancellation of a pool task;
- dynamic resizing of a pool after creation;
- channels between distinct OS processes;
- shared Lua memory;
- transfer of functions, userdata, or coroutines;

- a binary message format;
- automatic transfer of multiple return values.

The 2.18 pool is local to the Lua state that created it. It is not a global scheduler shared by separate Babet processes, and it does not migrate a task that has already started from one worker to another.

Cookbook

Concrete recipes for common scripting tasks. Each recipe is short and self-contained ; copy, adapt, ship.

Watch a directory and email new files

A drop directory that mails out every file as it arrives. Uses `babet.inotify` for instant wakeup (no polling), and listens on `close_write` + `moved_to` so the file is guaranteed complete.

```
local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

babet.signal.handle("TERM", function()
    w:close()
    os.exit(0)
end)

while true do
    local events, err = w:read()
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            -- queue overflowed: rescan the directory
            -- (events were lost)
        else
            local path = "/srv/incoming/" .. ev.name
            send_email(path)
            os.remove(path)
        end
    end
end
```

Ensure a service user exists, then drop to it

A typical install-time check before launching a daemon as a non-root user.

```
local function ensure_user(name)
    if babet.user.exists(name) then return true end
    local ok, err = babet.exec("useradd", {
        "--system",
        "--home-dir", "/var/lib/" .. name,
        "--create-home",
        "--shell", "/usr/sbin/nologin",
        name,
    })
    if not ok then
        return nil, "useradd failed: " .. tostring(err)
```

```
    end
    return true
end

assert(ensure_user("myapp"))
local u = assert(babet.user.get("myapp"))
babet.chdir(u.home)
```

Fetch many URLs in parallel

`babet.workers` runs Lua on multiple cores. Total wall-clock is close to the slowest single fetch, not the sum.

```
local urls = { "https://a.example/", "https://b.example/",
               "https://c.example/", "https://d.example/" }

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { error = err } end
        return { status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    print(i, ok, result and result.status or result.error)
end
```

Use WebSocket as a WebDriver BiDi transport

Once a WebDriver session has returned its `webSocketUrl`, Babet can carry the BiDi JSON messages without knowing anything Selenium-specific. This minimal sequential dispatcher resolves one command at a time and surfaces unsolicited events separately:

```
local ws = assert(babet.websocket.connect(webSocketUrl, {
    timeout = 5,
    max_message_bytes = 8 * 1024 * 1024,
}))

local next_id = 0
local function bidi_command(method, params)
    next_id = next_id + 1
    local id = next_id
    assert(ws:send_text(assert(babet.json.encode({
        id = id,
        method = method,
        params = params or {},
    }))))

    while true do
        local message = assert(ws:recv())
        assert(message.type == "text", "BiDi expects text JSON messages")
        local packet = assert(babet.json.decode(message.data))
        if packet.id == id then
```

```
            return packet
        end
        if packet.method then
            print("BiDi event:", packet.method)
        end
    end
end

local status = bidi_command("session.status", {})
print(assert(babet.json.encode(status)))
assert(ws:close())
```

A complete Selenium binding should replace this sequential loop with its own pending-command table and event dispatcher. The WebSocket transport stays generic.

Graceful shutdown of a long-running script

Catch `SIGTERM` / `SIGINT` so the script can close resources properly. Works with `systemctl stop` and Ctrl-C.

```
local log = require("logging")
log.set_level("info")

local running = true
babet.signal.handle("TERM", function()
    log.info("SIGTERM, shutting down")
    running = false
end)
babet.signal.handle("INT", function()
    log.info("Ctrl-C, shutting down")
    running = false
end)

while running do
    -- main loop ; blocking calls return (nil, "interrupted") when
    -- a handled signal arrives, so the loop exits quickly.
    local line, err = sock:recv_line()
    if not line then
        if err == "interrupted" then break end
        log.error("recv: ", err); break
    end
    process(line)
end

sock:close()
log.info("clean shutdown")
```

Read a TOML config, validate, persist to SQLite

```
-- Load TOML
local f = assert(io.open("config.toml", "r"))
local body = f:read("a"); f:close()
local cfg, perr = babet.toml.decode(body)
if not cfg then error("config.toml: " .. perr) end

-- Validate (cheap manual checks ; for a real schema, build one in Lua)
assert(type(cfg.server) == "table", "missing [server]")
```

```
assert(type(cfg.server.host) == "string", "missing server.host")
assert(math.type(cfg.server.port) == "integer", "server.port must be integer")

-- Open db, persist
local db = assert(babet.sqlite.open("state.db", { wal = true }))
db:exec([[CREATE TABLE IF NOT EXISTS config (k TEXT PRIMARY KEY, v TEXT)]])
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
        { "host", cfg.server.host })
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
        { "port", tostring(cfg.server.port) })
db:close()
```

Cache with a human-readable TTL, expiry as ISO timestamps

Parse a TTL from config (`"15m"`, `"2h30m"`, `"1d"`), compute a deadline, and log it in a uniform UTC format that grep'ing across log files actually works on.

```
-- Config from TOML, CLI, env, anywhere.
local raw_ttl = "15m"

local ttl, err = babet.time.parse_duration(raw_ttl)
if not ttl then error("bad TTL '" .. raw_ttl .. "': " .. err) end

local deadline = babet.now() + ttl
print(string.format("[%s] cache entry created, expires at %s (TTL %s)",
    babet.time.iso(),
    babet.time.iso(deadline),
    babet.time.format_duration(ttl)))

-- ...later, somewhere else in the script:
if babet.now() >= deadline then
    print("[" .. babet.time.iso() .. "] cache miss: entry expired")
end
```

Useful properties of this pattern :

- `babet.time.iso()` is always UTC, always 20 characters, always the same shape. `grep` and `sort` work on the timestamps directly.
- `parse_duration("15m")` returns an integer, so `now() + ttl` is exact (no float drift over long durations).
- `format_duration(ttl)` round-trips : `parse_duration( format_duration(n)) == n` for every `n >= 0`. Safe to use as a canonical representation in logs or databases.

Pretty-print a Lua value

The bundled `inspect` module gives readable output for nested tables.

```
local inspect = require("inspect")
print(inspect({ a = 1, b = { c = 2, d = { 3, 4, 5 } } }))
-- { a = 1, b = { c = 2, d = { 3, 4, 5 } } }
```

Hello there

The minimal Babet script :

```
babet.helloThere()
```